

Паттерны проектирования

для C# и платформы .NET CORE

Гаурав Арораа, Джеффри Чилберто



Hands-On Design Patterns with C# and .NET Core

Write clean and maintainable code by using reusable solutions to common software design problems

Gaurav Aroraa
Jeffrey Chilberto

Packt>

BIRMINGHAM - MUMBAI

Паттерны проектирования

для C# и платформы .NET CORE

Гаурав Арораа
Джеффри Чилберто



Санкт-Петербург • Москва • Екатеринбург • Воронеж
Нижний Новгород • Ростов-на-Дону • Самара • Минск

2021

ББК 32.973.2-018-02
УДК 004.42
А84

Арораа Гаурав, Чилберто Джеффри

- А84 Паттерны проектирования для C# и платформы .NET Core. — СПб.: Питер, 2021. — 352 с.: ил. — (Серия «Для профессионалов»).
- ISBN 978-5-4461-1523-5

Паттерны проектирования — удобный прием программирования для решения рутинных задач разработки ПО. Грамотное использование паттернов позволяет добиться соответствия любым требованиям и снизить расходы. В этой книге описаны эффективные способы применения паттернов проектирования с учетом специфики языка C# и платформы .NET Core.

Кроме знакомых паттернов проектирования из книги «Банды четырех» вы изучите основы объектно-ориентированного программирования и принципов SOLID. Затем узнаете о функциональных, реактивных и конкурентных паттернах, с помощью которых будете работать с потоками и корутинами. Заключительная часть содержит паттерны для работы с микросервисными, бессерверными и облачно-ориентированными приложениями. Вы также узнаете, как сделать выбор архитектуры, например микросервисной или MVC.

16+ (В соответствии с Федеральным законом от 29 декабря 2010 г. № 436-ФЗ.)

ББК 32.973.2-018-02
УДК 004.42

Права на издание получены по соглашению с Packt Publishing. Все права защищены. Никакая часть данной книги не может быть воспроизведена в какой бы то ни было форме без письменного разрешения владельцев авторских прав.

Информация, содержащаяся в данной книге, получена из источников, рассматриваемых издательством как надежные. Тем не менее, имея в виду возможные человеческие или технические ошибки, издательство не может гарантировать абсолютную точность и полноту приводимых сведений и не несет ответственности за возможные ошибки, связанные с использованием книги. Издательство не несет ответственности за доступность материалов, ссылки на которые вы можете найти в этой книге. На момент подготовки книги к изданию все ссылки на интернет-ресурсы были действующими.

ISBN 978-1789133646 англ.

© Packt Publishing 2019.

First published in the English language under the title 'Hands-On Design Patterns with C# and .NET Core — (9781789133646)

ISBN 978-5-4461-1523-5

© Перевод на русский язык ООО Издательство «Питер», 2021

© Издание на русском языке, оформление ООО Издательство «Питер», 2021

© Серия «Для профессионалов», 2021

Краткое содержание

Предисловие	14
Об авторах	16
О научных редакторах.....	17
Введение.....	18

Часть I. Основы паттернов проектирования в C# и .NET Core

Глава 1. Обзор ООП в .NET Core и C#.....	24
Глава 2. Современные паттерны и принципы проектирования ПО	42

Часть II. Углубленное изучение утилит и паттернов .NET Core

Глава 3. Реализация паттернов проектирования — основы (часть 1).....	70
Глава 4. Реализация паттернов проектирования — основы (часть 2).....	100
Глава 5. Реализация паттернов проектирования в .NET Core.....	134
Глава 6. Реализация паттернов проектирования для веб-приложений (часть 1)	157
Глава 7. Реализация паттернов проектирования для веб-приложений (часть 2)	178

Часть III. Функциональное программирование, реактивное программирование и кодирование для облака

Глава 8. Конкурентное программирование в .NET Core	212
Глава 9. Функциональное программирование.....	229
Глава 10. Модели и методы реактивного программирования	245
Глава 11. Усовершенствованные методы проектирования и применения баз данных	284
Глава 12. Разработка облачных приложений	309

Приложения

Приложение А. Дополнительные практические рекомендации	336
Приложение Б. Ответы на вопросы	343

Оглавление

Предисловие	14
Об авторах	16
О научных редакторах.....	17
Введение.....	18
Кому подойдет эта книга	18
Структура издания.....	19
Как получить максимум от книги.....	20
Загрузка файлов с примерами	21
Код в действии.....	21
Полноцветные изображения	21
Условные обозначения.....	21
От издательства.....	22

Часть I. Основы паттернов проектирования в C# и .NET Core

Глава 1. Обзор ООП в .NET Core и C#.....	24
Технические требования.....	25
Установка Visual Studio.....	25
Установка .NET Core.....	25
Используемые в книге модели	26
Определение ООП. Как работают классы и объекты.....	28
Определение ООП	28
Класс.....	29
Объект.....	30
Интерфейс	33
Наследование	34
Инкапсуляция	36

Полиморфизм	36
Статический полиморфизм.....	37
Динамический полиморфизм	38
Резюме	40
Вопросы	41
Глава 2. Современные паттерны и принципы проектирования ПО	42
Технические требования.....	43
Установка Visual Studio.....	43
Установка .NET Core	43
Принципы проектирования.....	44
DRY — «Не повторяйся».....	44
KISS — «Делай проще, тупица»	44
YAGNI — «Вам это не понадобится»	45
MVP — «Продукт с минимальным функционалом»	45
SOLID	45
Паттерны программного обеспечения	47
Паттерны «Банды четырех»	47
Паттерны интеграции корпоративных приложений.....	59
Паттерны жизненного цикла разработки программного обеспечения	65
Waterfall SDLC	65
Agile SDLC	66
Резюме	67
Вопросы	68

Часть II. Углубленное изучение утилит и паттернов .NET Core

Глава 3. Реализация паттернов проектирования — основы (часть 1).....	70
Технические требования.....	71
Установка Visual Studio.....	71
Установка .NET Core	71
Продукт с минимальным функционалом	71
Требования.....	72
Каковы преимущества MVP в дальнейшей разработке.....	74
Разработка через тестирование	75
Почему разработчики выбирают TDD.....	76
Настройка проектов.....	77
Определения начальных модульных тестов.....	80
Паттерн проектирования «Абстрактная фабрика»	82

Принципы SOLID	87
Принцип единственной ответственности (SRP)	87
Принцип открытости/закрытости (ОСР)	88
Принцип замещения Лисков (LSP)	89
Принцип сегрегации интерфейса (ISP)	90
Принцип инверсии зависимостей	91
Модульные тесты для InventoryCommand	93
Резюме	98
Вопросы	99
Глава 4. Реализация паттернов проектирования — основы (часть 2)	100
Технические требования	101
Установка Visual Studio	101
Установка .NET Core	101
Паттерн «Синглтон»	101
Процессы и потоки	102
Паттерн «Репозиторий»	104
Модульные тесты	105
Иллюстрация состояния гонки	112
Класс AddInventoryCommand	115
Класс UpdateQuantityCommand	119
Команда GetInventoryCommand	121
Паттерн «Фабрика»	123
Модульные тесты	124
Функциональность .NET Core	128
Интерфейс IServiceCollection	128
Интерфейс CatalogService	129
IServiceProvider	130
Консольное приложение	131
Резюме	133
Вопросы	133
Глава 5. Реализация паттернов проектирования в .NET Core	134
Технические требования	134
Установка Visual Studio	135
Установка .NET Core	135
Время жизни сервисов в .NET Core	135
Временная зависимость (Transient)	136
Время жизни области применения (Scoped)	136

«Одиночка» в .NET Core	136
Вернемся к FlixOne	136
Что такое время жизни области применения	142
Реализующая фабрика.....	143
Интерфейс InventoryContext.....	144
Интерфейс InventoryReadContext	144
Интерфейс InventoryWriteContext	145
Класс InventoryCommandFactory	146
Класс InventoryCommand	147
Резюме	155
Вопросы	156
Глава 6. Реализация паттернов проектирования для веб-приложений (часть 1)	157
Технические требования.....	157
Установка Visual Studio.....	158
Установка .NET Core	158
Установка SQL Server	158
Создание веб-приложения .NET Core.....	159
Запуск проекта.....	159
Разработка веб-приложения	160
Реализация CRUD-страниц	166
Резюме	176
Вопросы	177
Дальнейшее чтение	177
Глава 7. Реализация паттернов проектирования для веб-приложений (часть 2)	178
Технические требования.....	178
Установка Visual Studio.....	179
Установка .NET Core	179
Установка SQL Server	179
Расширение веб-приложения .NET Core	180
Начало проекта.....	180
Аутентификация и авторизация	182
Аутентификация в действии.....	187
Авторизация в действии	197
Создание тестового проекта веб-приложения	205
Резюме	209
Вопросы	209
Дальнейшее чтение	210

Часть III. Функциональное программирование, реактивное программирование и кодирование для облака

Глава 8. Конкурентное программирование в .NET Core	212
Технические требования	212
Установка Visual Studio	213
Установка .NET Core	213
Установка SQL Server	213
Конкурентность в реальном мире	214
Многопоточное и асинхронное программирование	216
async/await — чем плоха блокировка	219
Конкурентная коллекция	220
Паттерны и рекомендации — TDD и параллельный LINQ	221
Резюме	227
Вопросы	228
Дальнейшее чтение	228
Глава 9. Функциональное программирование	229
Технические требования	229
Установка Visual Studio	230
Установка .NET Core	230
Установка SQL Server	230
Основы функционального программирования	231
Совершенствование приложения FlixOne	235
Требования	236
Вернемся к FlixOne	237
Паттерн «Стратегия» и функциональное программирование	242
Резюме	243
Вопросы	244
Глава 10. Модели и методы реактивного программирования	245
Технические требования	245
Установка Visual Studio	246
Установка .NET Core	246
Установка SQL Server	246
Принципы реактивного программирования	247
Будьте реактивны с реактивным программированием	249
Реактивность и интерфейс IObservable	257
Паттерн «Наблюдатель» — реализация с помощью IObservable<T>	257

Реактивные расширения: .NET Rx Extensions	264
Настройка приложения FlixOne	266
Начало проекта	266
Применение в приложении FlixOne фильтрации, пагинации и сортировки....	267
Паттерны и практики — MVVM	275
Реализация MVVM	277
Резюме	282
Вопросы	283
Дальнейшее чтение	283

Глава 11. Усовершенствованные методы проектирования и применения

баз данных	284
Технические требования	284
Установка Visual Studio	285
Установка .NET Core	285
Установка SQL Server	285
Обсуждение примера использования	286
Начало проекта	286
Требования	287
Поговорим о базах данных	289
Обработка баз данных	289
OLTP	290
OLAP	291
Базы данных в стиле бухгалтерской книги	291
Реализация паттерна CQRS	293
Резюме	307
Вопросы	308

Глава 12. Разработка облачных приложений

Технические требования	310
Ключевые моменты, которые следует учитывать при разработке облачных решений	310
Масштабируемость	311
Рабочая нагрузка	311
Паттерны	312
Устойчивость/доступность	319
Паттерны решений	320
Безопасность	323
Паттерны решений	323

Проектирование приложения	325
Паттерны решений	325
DevOps	330
Паттерны решений	330
Резюме	333
Вопросы	334
Дальнейшее чтение	334

Приложения

Приложение А. Дополнительные практические рекомендации	336
Технические требования.....	336
Обсуждение примера использования.....	337
UML-диаграмма	338
Практические рекомендации	339
Другие паттерны проектирования.....	340
Резюме.....	341
Вопросы	342
Дальнейшее чтение	342
Приложение Б. Ответы на вопросы	343
Глава 1. Обзор ООП в .NET Core и C#.....	343
Глава 2. Современные паттерны и принципы проектирования ПО.....	343
Глава 3. Реализация паттернов проектирования — основы (часть 1).....	344
Глава 4. Реализация паттернов проектирования — основы (часть 2).....	345
Глава 5. Реализация паттернов проектирования в .NET Core	345
Глава 6. Реализация паттернов проектирования для веб-приложений (часть 1)....	346
Глава 7. Реализация паттернов проектирования для веб-приложений (часть 2)....	346
Глава 8. Конкурентное программирование в .NET Core	347
Глава 9. Функциональное программирование	348
Глава 10. Модели и методы реактивного программирования	348
Глава 11. Усовершенствованные методы проектирования и применения баз данных	350
Глава 12. Разработка облачных приложений	350
Приложение А. Практические рекомендации.....	351

*Моей матери Лате Шримати Сантош
и в память о моем отце, покойном
Ш. Рамкришане, за их жертвенность и твердость
духа. Моей младшей сестренке, также покойной
крошке Канчан, за ее любовь и за то, что была
моим маленьким талисманом.*

Гаурав Арораа

*Моим родителям Френсис и Джойс которые
неустанно поднимали на ноги своих детей
с любовью, заботой и добротой. Моим братьям:
Джеку — за мотивацию сохранять спокойствие
в череде испытаний и Майку — за напоминание
остановиться и насладиться жизнью.*

Джеффри Чилберто

Предисловие

При разработке качественного ПО программисты стремятся избегать дублирования кода. Мы весьма часто применяем прием DRY — Don't Repeat Yourself («Не повторяйся») — даже не задумываясь! Разработчики обычно разделяют функциональность, создают многократно используемые методы и пишут вспомогательные классы. Паттерны проектирования создавались и обновлялись годами. Это полезные, общепринятые и пригодные к многократному применению решения повседневных проблем.

Гаурав и Джеффри собрали лучшие и наиболее широко используемые паттерны и применили их в мире открытого C# и .NET Core. Вы начнете с ООП, классов, объектов и будете продвигаться далее к наследованию, инкапсуляции и полиморфизму. Авторы описали такие подходы, как DRY, KISS и SOLID (вы очень скоро поймете, что все это значит!), и применили их к классическим паттернам, которые помогут вам разрабатывать чистое и надежное программное обеспечение.

Книга содержит примеры рабочего кода, которые покажут, как использовать получаемые знания в создании ПО на .NET Core и C# в наши дни. Вы освоите паттерны «Строитель», «Декоратор», «Фабрика», «Посетитель», «Стратегия» и многие другие.

Эти методики будут применены сначала к простой программе, а затем к веб-приложениям. Далее будут рассмотрены более сложные темы, включая конкурентность и параллелизм. Наконец, вы сможете воспользоваться паттернами на принципиально более высоком уровне, применяя решения, которые помогут вам переместить проекты в облако, где их можно будет масштабировать и легко обслуживать.

Надеюсь, что, как и я, вы по достоинству оцените эту книгу. Желаю получать удовольствие, работая с .NET Core, так же, как получали его мы, когда разрабатывали эту среду!

*Скотт Хансельман,
руководитель партнерской программы
Microsoft .NET и Open Source Community*

Эта книга — библия для каждого разработчика, желающего не только улучшить свои навыки программирования, но и, самое главное, научиться создавать надежные, масштабируемые и удобные в сопровождении решения. Здесь описана большая часть практических рекомендаций, дополненных наглядными примерами.

Помимо паттернов проектирования, книга затрагивает архитектурные принципы и ключевые моменты работы в облаке, такие как безопасность и масштабирование.

Будучи архитектором решений, я ежедневно проектирую комплексные приложения, уделяя особое внимание разработке инфраструктур и инструментов, и все же поражаюсь качеству контента и охвату тем, представленных в этой книге. В ней содержится исчерпывающий список паттернов, рекомендуемых к ознакомлению всем, кто связан с разработкой и развертыванием ПО.

Такое полное и глубокое погружение в мир объектно-ориентированного программирования (ООП) и .NET Core полезно всем, кто заинтересован в разработке лучших приложений.

Стефани Айскенс, Azure MVP

Об авторах

Гаурав Арора получил степень магистра в области компьютерных наук, имеет сертификаты Microsoft MVP, Alibaba Cloud MVP, тренера/коуча по Scrum. Член *Компьютерного сообщества Индии (CSI)*, сотрудник IndiaMentor, сертифицированный специалист ITIL Foundation, APMG PRINCE-F и APMG PRINCE-P. Гаурав — разработчик открытого программного обеспечения, внесший вклад в развитие TechNet Wiki и основавший компанию Ovatic Systems Private Limited. За свою более чем 20-летнюю карьеру он обучил тысячи студентов. Вы можете написать Гаураву в Twitter по адресу @g_arora.

Спасибо моей жене Шуби Арора и моему ангелу (дочери) Аачи Арора, которые вытерпели мое отсутствие, пока я писал эту книгу. Спасибо всей команде Packt, особенно Чайтаньи, Акиште и Нее, координационная деятельность и коммуникативность которых были так необходимы, а также Дениму Пинто, порекомендовавшему меня в качестве потенциального автора.

Джефффри Чилберто — консультант в сфере разработки программного обеспечения, специализирующийся на технологическом стеке Microsoft, включая Azure, BizTalk, ASP.NET, MVC, WCF и SQL Server, с опытом работы в различных сферах. Он участвовал в разработке банковских, телекоммуникационных и медицинских систем в Новой Зеландии, Австралии и США. Имеет степень бакалавра в области информационных и компьютерных наук, а также степень магистра по информационным технологиям и вычислительной технике.

Хотел бы поблагодарить мою семью за любовь, поддержку и вдохновение.

О научных редакторах

Сьеки Цааль — консультант по управлению, архитектор облачных систем Microsoft и Microsoft Azure MVP с более чем 15-летним стажем в области создания архитектур, разработки, консультирования и системного дизайна. Она работает в Capgemini — одной из крупнейших в мире консалтинговых компаний в сфере менеджмента и информационных технологий. Сьеки любит делиться знаниями и принимает очень активное участие в сообществе Microsoft как одна из основателей нидерландских пользовательских групп SP&C NL и MixUG. Кроме того, является членом правления Azure Thursdays. Сьеки часто выступает с лекциями и участвует в организации мероприятий. Она написала несколько книг, ведет блоги и принимает активное участие в сообществе Microsoft Tech Community (<https://techcommunity.microsoft.com/>).

У *Эфраима Кирякидиса* за плечами более 20 лет опыта в разработке ПО. Он получил диплом инженера в Университете имени Аристотеля в Салониках (Греция). Эфраим пользуется .NET с момента создания, с версии 1.0. В своей работе он в целом сфокусирован на технологиях Microsoft. В данный момент занимает пост старшего инженера в компании Siemens AG в Германии.

Введение

В этой книге на конкретных примерах описываются способы использования паттернов в современной разработке приложений. Количество паттернов, применяемых для поиска решений, огромно. Чаще всего разработчики пользуются ими, не понимая в полной мере, что и как эти паттерны делают. В издании рассматриваются паттерны от низкоуровневого кода до высокоуровневых концептуальных решений, которые работают в облачных системах.

Несмотря на то что большинство паттернов не требуют применения конкретного языка, для иллюстрации многих из них мы используем C# и .NET Core. Эти технологии были выбраны из-за их популярности и удобства архитектуры, на основе которой можно создавать различные решения — от простых консольных приложений до крупных корпоративных распределенных систем.

Описывая большое количество паттернов, эта книга знакомит со многими из них, позволяет глубже понять их на практике. Представленные паттерны были выбраны, чтобы проиллюстрировать определенные аспекты проектирования. Кроме того, добавлены ссылки на дополнительные материалы, которые помогут читателю подробнее изучить конкретный интересующий его паттерн.

И в простых сайтах, и в огромных корпоративных системах правильно подобранный паттерн может в корне изменить заведомо неверное решение, ведущее к краху проекта из-за его высокой стоимости и низкой производительности, и превратить это решение в выгодное, долгоживущее и успешное. Паттерны, описанные в книге, призваны преодолеть неизбежные проблемы, которые иначе не позволили бы вам оставаться конкурентоспособными. Они также позволят вашим приложениям достичь устойчивости и надежности, которыми должны обладать современные проекты.

Кому подойдет эта книга

Целевая аудитория — разработчики современных приложений. Поскольку книга содержит много кода, чтобы объяснить, как и где используются паттерны, подразумевается наличие у читателя опыта в разработке ПО. Не стоит рассматривать данную книгу как «программирование для чайников», скорее она отвечает на во-

прос «Как программировать лучше». Поэтому издание будет полезно начинающим и опытным программистам, архитекторам приложений и проектировщикам.

Структура издания

Глава 1 описывает объектно-ориентированное программирование и его применение в среде C#. Эта глава служит напоминанием о важных конструкциях и функциях ООП и C#, включающих наследование, инкапсуляцию и полиморфизм.

Глава 2 каталогизирует и представляет различные паттерны, используемые в современной разработке программного обеспечения. Эта глава исследует ряд паттернов и их каталогов, таких как SOLID, паттерны «Банды четырех» и паттерны корпоративной интеграции. Вдобавок в ней обсуждаются жизненный цикл и другие методики разработки ПО.

Глава 3 погрузит в паттерны проектирования, используемые для создания приложений на C#. На примере приложения-образца будут показаны разработка через тестирование, концепция продукта с минимальным функционалом и некоторые паттерны «Банды четырех».

Глава 4 продолжит знакомить с паттернами, используемыми в приложениях на C#. Будут введены такие понятия, как внедрение зависимости и инверсия управления, а также продолжится изучение паттернов проектирования, включая «Одиночку» и «Фабрику».

Глава 5 основана на главах 3 и 4 и описывает паттерны в .NET Core. Некоторые паттерны, включая «Внедрение зависимости» и «Фабрику», будут также пересмотрены с учетом среды .NET Core.

Глава 6 продолжает знакомить с .NET Core, описывая функции, поддерживаемые в разработке веб-приложений, в процессе создания тестового приложения. Эта глава содержит руководство по созданию базового веб-приложения, представляет важные характеристики, которыми оно должно обладать, и рассказывает, как создавать веб-страницы с поддержкой CRUD.

Глава 7 тоже посвящена разработке веб-приложений с помощью .NET Core и представляет различные архитектурные паттерны, а также варианты решений вопроса безопасности. Кроме того, описывает аутентификацию и авторизацию, а также модульные тесты с использованием Moq — фреймворка для создания имитаций реальных объектов.

Глава 8 углубляется в разработку веб-приложений и описывает понятие «конкурентность» при разработке приложений на C# и .NET Core. Будет рассмотрен паттерн `async/await`, а также многопоточность и конкурентность, которым посвящен

отдельный раздел. Кроме того, речь пойдет о Parallel LINQ с отложенным запуском и приоритетом потоков.

Глава 9 описывает функциональное программирование в .NET Core. Сюда входят функции языка C#, поддерживающие функциональное программирование, и их применение в тестовом приложении вместе с паттерном «Стратегия».

Глава 10 продолжает тему разработки веб-приложений .NET Core с помощью паттернов реактивного программирования и методик, используемых для создания адаптивных и масштабируемых сайтов. Глава описывает принципы реактивного программирования, включая паттерны *Reactive* и *IObservable*. Кроме того, рассматривает различные фреймворки, включая популярный проект .NET Rx Extensions, а также демонстрирует паттерн *«Модель — представление — модель представления»* (Model — View — View Model, MVVM).

Глава 11 освещает паттерны, используемые в проектировании баз данных, а также описывает сами БД. Будет показан практический пример применения паттерна CQRS, в том числе с помощью проектирования базы данных в виде журнала учета.

Глава 12 представляет разработку приложений в контексте облачных решений, включая пять ключевых аспектов: масштабирование, доступность, безопасность, проектирование приложений и DevOps. Будут описаны важнейшие паттерны, используемые в облачно-ориентированных решениях, включая различные типы масштабирования, и представлены паттерны, применяемые для событийно-ориентированных архитектур, интегрированной безопасности, кэша и телеметрии.

Приложение А содержит обсуждение дополнительных паттернов и практические рекомендации. Сюда включены разделы о моделировании примеров использования, рекомендованных методик и дополнительных паттернов, таких как пространственная архитектура и контейнерные приложения.

Приложение Б дает ответы на вопросы, размещенные в конце каждой главы.

Как получить максимум от книги

Предполагается, что вы уже немного знакомы с принципами ООП и языком C#. В книге описываются продвинутые темы, однако материал нельзя воспринимать как пошаговую инструкцию к применению. Цель издания — улучшить навыки разработчиков и проектировщиков с помощью широкого спектра паттернов, методик и принципов. По аналогии с ящиком для инструментов книга предлагает современным разработчикам эти самые инструменты для перехода от низкоуровневого проектирования к созданию более высокоуровневых архитектур, а также важные паттерны и принципы, широко используемые в настоящий момент.

Вдобавок эта книга обращает внимание на следующие моменты, призванные дополнить знания читателя:

- ❑ знание принципов SOLID, практические рекомендации с примерами кода на C#7.x и NET Core 2.2;
- ❑ углубленное понимание классических паттернов проектирования (от «Банды четырех»);
- ❑ принципы функционального программирования и рабочие примеры на языке C#;
- ❑ реальные примеры архитектурных паттернов (MVC, MVVM);
- ❑ понимание облачной ориентированности, микросервисов и пр.

Загрузка файлов с примерами

Вы можете скачать примеры из этой книги на GitHub по ссылке github.com/PacktPublishing/Hands-On-Design-Patterns-with-C-and-.NET-Core. В случае обновления кода примеры также будут обновлены в репозитории.

Код в действии

Перейдите по ссылке bit.ly/2KuuNgQ, чтобы просмотреть видео с примерами кода.

Полноцветные изображения

Мы также предоставляем PDF-файл с цветными изображениями схем и рисунков, используемыми в оригинале данной книги: static.packt-cdn.com/downloads/9781789133646_ColorImages.pdf.

Условные обозначения

В книге используется ряд условных обозначений.

Моноширинным шрифтом обозначены код в тексте, имена таблиц баз данных и т. п. Например: «В данном классе есть три метода: `CounterA()`, `CounterB()` и `CounterC()`, представляющих отдельную стойку для получения билетов».

Блок кода (листинг) оформляется так:

```
3-counters are serving...
Next person from row
Person A is collecting ticket from Counter A
Person B is collecting ticket from Counter B
Person C is collecting ticket from Counter C
```

Когда мы хотим обратить ваше внимание на конкретную часть листинга, то необходимые и ключевые строки выделяем **полужирным моноширинным** шрифтом:

```
public bool UpdateQuantity(string name, int quantity)
{
    lock (_lock)
    {
        _books[name].Quantity += quantity;
    }

    return true;
}
```

Любой ввод и вывод в оболочке командной строки выглядит следующим образом:

```
dotnet new sln
```

Курсивным шрифтом обозначены новые термины и важные слова.

URL, команды меню, элементы управления и заголовки окон выделяются **шрифтом без засечек**, например: «Команда **Create New Product** (**Создать новый продукт**) позволяет добавить новый продукт, а **Edit** (**Редактировать**) — изменять и обновлять уже имеющиеся продукты».



Важные замечания и предупреждения обозначены так.



Советы и подсказки обозначены таким образом.

От издательства

Ваши замечания, предложения, вопросы отправляйте по адресу comp@piter.com (издательство «Питер», компьютерная редакция).

Мы будем рады узнать ваше мнение!

На веб-сайте издательства www.piter.com вы найдете подробную информацию о наших книгах.

Часть I

Основы паттернов проектирования в C# и .NET Core

В этой части книги вы по-новому взглянете на паттерны проектирования. Мы изучим такие темы, как ООП, паттерны, методики и принципы SOLID. К концу части вы сможете самостоятельно создавать собственные паттерны.

Часть I состоит из таких глав:

- ❑ глава 1 «Обзор ООП в .NET Core и C#»;
- ❑ глава 2 «Современные паттерны и принципы проектирования ПО».

1

Обзор ООП в .NET Core и C#

Самые популярные за последние более чем 20 лет языки программирования были основаны на принципах *объектно-ориентированного программирования (ООП)*. Популярность языков с поддержкой ООП объясняется возможностью абстрагировать сложную логику программы в структуру, называемую объектом, которую намного проще объяснить и понять, а также легче использовать повторно. По сути, ООП есть подход к проектированию программного обеспечения. Точнее, это паттерн разработки ПО, который задействует концепцию объектов, включающих в себя данные и функциональность. По мере развития индустрии программного обеспечения в ООП появились паттерны для часто встречающихся задач, поскольку эффективно решались одни и те же вопросы, но в разных контекстах и отраслях. По мере продвижения ПО от мейнфреймов к клиентам и серверам, а затем к облакам появлялись новые паттерны, помогающие уменьшить стоимость разработки и повысить надежность программного обеспечения. В этой книге мы исследуем паттерны проектирования, начиная с их появления в ООП и заканчивая созданием паттернов проектирования облачного ПО.



ООП основано на концепции объекта. Объект обычно содержит данные, известные как свойства или поля, а также код или поведение, известное как методы.

Паттерны проектирования — решения распространенных проблем, с которыми программисты сталкиваются в процессе создания ПО. Эти решения построены на опыте того, что работает, а что — нет, и опробованы и протестированы многими разработчиками в различных ситуациях. Преимущество использования паттернов, основанных на предыдущем опыте, заключается в том, что они избавляют от необходимости повторять одни и те же действия снова и снова. Вдобавок применение паттернов дает ощущение уверенности в том, что при решении проблемы не появятся новые ошибки или затруднения.

Эта глава описывает ООП и его применение в C#. Обратите внимание: данная информация — просто краткое введение в тему, она не является полноценным руководством по ООП или языку C#. Напротив, глава рассказывает об ООП и C# достаточно подробно для того, чтобы познакомить вас с паттернами проектирования, которые будут обсуждаться в других главах.

В этой главе будут рассмотрены следующие темы:

- ❑ дискуссия об ООП, о работе классов и объектов;
- ❑ наследование;
- ❑ инкапсуляция;
- ❑ полиморфизм.

Технические требования

В этой главе содержатся примеры кода, объясняющие принципы качественной разработки. Код упрощен и предназначен лишь для демонстрации. В большинстве примеров используется консольное приложение на основе .NET Core, написанное на C#.

Чтобы запустить и выполнить код, вам потребуется следующее:

- ❑ Visual Studio 2019 (вы также можете запустить приложение, используя Visual Studio 2017 версии 3 и новее);
- ❑ .NET Core;
- ❑ SQL Server (в этой главе используется Express Edition).

Установка Visual Studio

Чтобы запустить примеры кода, вам необходимо установить Visual Studio. Для этого выполните следующие шаги.

1. Скачайте Visual Studio по ссылке docs.microsoft.com/ru-ru/visualstudio/install/install-visual-studio.
2. Следуйте инструкциям по установке. Доступны различные версии Visual Studio; в этой главе мы используем Visual Studio для Windows.

Вы также можете применить другую интегрированную среду разработки на ваше усмотрение.

Установка .NET Core

Если вы еще не установили .NET Core, то следуйте инструкции ниже.

1. Скачайте .NET Core по ссылке dotnet.microsoft.com/download.
2. Далее следуйте указаниям по установке соответствующей библиотеки: dotnet.microsoft.com/download/dotnet-core/2.2.



Полная версия исходного кода доступна на GitHub. Код, представленный в данной главе, может быть неполным, поэтому мы рекомендуем вам скачать код для запуска примеров (github.com/PacktPublishing/Hands-On-Design-Patterns-with-C-and-.NET-Core/tree/master/Chapter1).

Используемые в книге модели

Как практическое руководство, книга содержит много примеров кода на C# наряду с различными схемами и рисунками, призванными объяснить те или иные концепции. Это не книга по *унифицированному языку моделирования* (Unified Modeling Language, UML), однако тем, кто знаком с UML, многие схемы будут известны. В этом подразделе приводятся примеры схем классов, используемых в издании.

При обозначении класса указываются его поля и методы, разделенные пунктирными линиями. В случаях, важных для обсуждения, типы доступа будут обозначены так: – для приватных (закрытых), + для публичных (открытых), # для защищенных и ~ для внутренних. На рис. 1.1 это показано на примере класса `Car` с приватной переменной `_name` и публичным методом `GetName()`.

Когда описываются отношения между объектами, ассоциация показана сплошной линией, агрегация — в виде ромба, а композиция — в виде закрашенного ромба. При необходимости рядом с классом указывается количество его экземпляров. На рис. 1.2 показано, что у класса `Car` есть один водитель — `Owner` и до трех пассажиров — `Passengers`; кроме того, он имеет четыре колеса — `Wheels`.

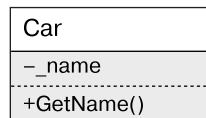


Рис. 1.1

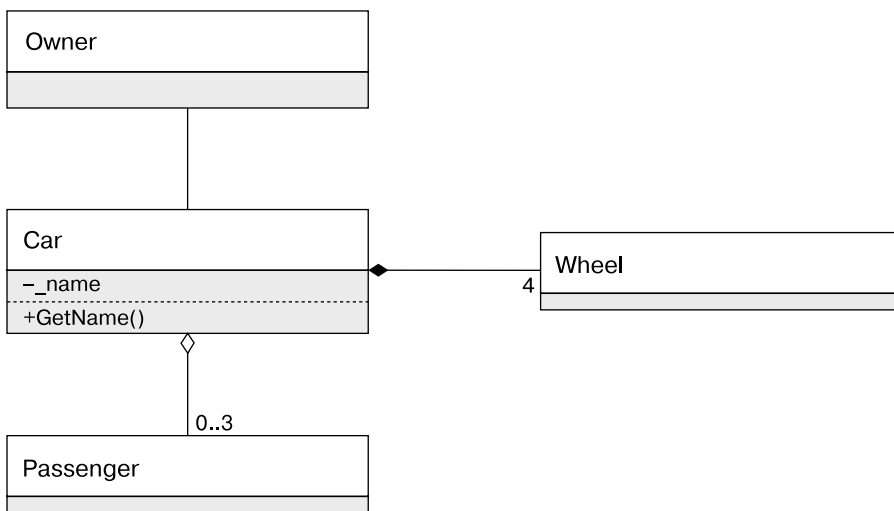


Рис. 1.2

Наследование обозначено с помощью пустого треугольника и сплошной линии, которые ведут к базовому классу. На рис. 1.3 отображены отношения между базовым классом `Account` и расширяющими его классами `SavingsAccount` и `CheckingAccount`.

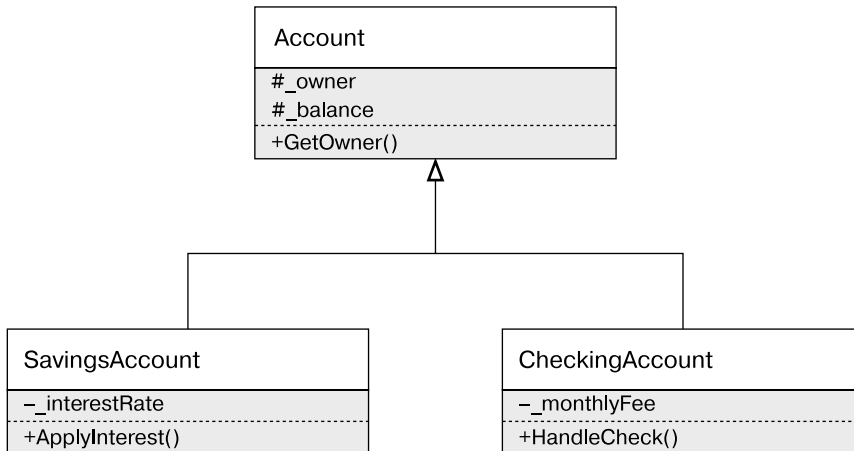


Рис. 1.3

Интерфейсы обозначаются похожим образом, но с использованием пунктирной линии и дополнительной метки `<<interface>>` (рис. 1.4).

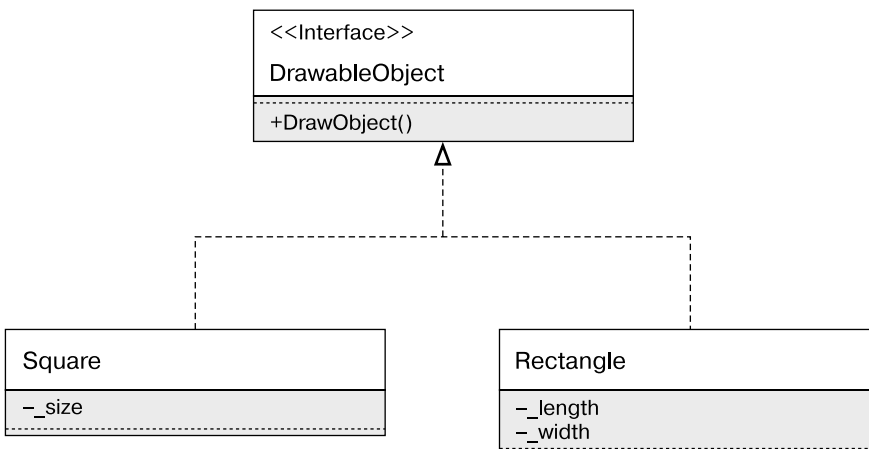


Рис. 1.4

В этом подразделе мы рассматриваем модели, используемые в книге. Такой стиль/подход выбран в расчете на то, что он знаком большинству читателей.

Определение ООП. Как работают классы и объекты

ООП — подход к разработке ПО, использующий объекты, определяемые в виде классов. Эти определения содержат поля, иногда называемые атрибутами, для сохранения данных, и методы для поддержки функциональности. Первый ООП-язык был симуляцией реальных систем. Он известен как Simula (en.wikipedia.org/wiki/Simula) и разработан в Норвежском вычислительном центре в 1960-е годы. Первый полноценный ООП-язык появился на свет в 1970-е и назывался Smalltalk (<https://ru.wikipedia.org/wiki/Smalltalk>). Он был разработан для устройства Dynabook (history-computer.com/ModernComputer/Personal/Dynabook.html), персонального компьютера, созданного Алланом Кеем. Smalltalk в какой-то степени стал прародителем некоторых других известных ООП-языков, таких как Java, C++, Python и C#.



ООП базируется на объектах, содержащих данные. Эта парадигма программирования позволяет разработчикам организовывать код в абстрактную или логическую структуру, называемую объектом. Он может содержать как данные, так и поведение.

ООП позволяет:

- ❑ *применить модуляризацию* — приложение разбивается на несколько разных модулей;
- ❑ *повторно использовать ПО* — приложение собирается из уже имеющихся или новых модулей. В следующих разделах мы подробно обсудим концепции ООП.

Определение ООП

Ранее подходы к программированию имели определенные ограничения и часто были сложными в сопровождении. ООП предлагает новую парадигму в разработке приложений, которая более выгодна по сравнению с другими подходами. Концепция организации кода в объекты проста для понимания и дает большое преимущество при освоении нового паттерна. Объяснить концепцию можно на примерах из реального мира. Сложные системы могут быть описаны с помощью более мелких блоков (то есть *объектов*). Это позволяет разработчикам анализировать отдельные компоненты и понимать их роль в самом решении.

Памятуя об этом, определим программу следующим образом: *«Программа — это список инструкций, указывающих компилятору языка, что ему делать»*.

Как видите, объект предоставляет способ организации списка инструкций логическим образом. Возвращаясь к примеру с домом, инструкции архитектора помогают нам построить дом, но сами они домом не являются. Инструкции — абстрактное представление дома. Определение класса — нечто подобное, так как описывает

характеристики объекта. Объект создается на основе определения класса, это часто называется *инстанцированием* (то есть *созданием экземпляра класса*).

Чтобы лучше понять ООП, следует отметить два других важных подхода к программированию.

- ❑ *Структурное программирование.* Это понятие было введено Эдсгером Дейкстрой в 1966 году. Структурное программирование — парадигма программирования, которая решает проблему управления 1000 строк кода и разделяет их на меньшие фрагменты. Их обычно называют *подпрограммами*, *блочными структурами*, циклами `for`, `while` и пр. К языкам программирования, использующим структурные методики, относятся ALGOL, Pascal, PL/I и др.
- ❑ *Процедурное программирование.* Эта парадигма появилась из структурного программирования и базируется на том, как выполняется вызов (или *процедурный вызов*). К языкам, использующим процедурные методы программирования, относятся COBOL, Pascal и C. Последний пример подобного языка — Go, выпущенный в 2009 году.



Процедурные вызовы

Процедурный вызов происходит, когда выполняется набор операторов, то есть процедура.

Главная проблема указанных подходов в том, что чем больше становятся программы, тем сложнее их обслуживать. Программы с более сложными и громоздкими кодовыми базами затрудняют применение этих двух парадигм; как следствие, усложняются анализ и сопровождение кода. Чтобы можно было решить эти проблемы, ООП предоставляет определенные механизмы, такие как:

- ❑ наследование;
- ❑ инкапсуляция;
- ❑ полиморфизм.

В следующих разделах мы подробно обсудим эти понятия.



Наследование, инкапсуляцию и полиморфизм иногда называют тремя столпами ООП.

Прежде чем начнем, обсудим некоторые структуры, присутствующие в ООП.

Класс

Класс — это групповое или шаблонное определение методов и переменных, которые описывают объект. Другими словами, класс — это образец, содержащий определение переменных и методов, общих для всех экземпляров класса. Экземпляр класса называется объектом.

Посмотрим на следующий пример кода:

```
public class PetAnimal
{
    private readonly string PetName;
    private readonly PetColor PetColor;

    public PetAnimal(string petName, PetColor petColor)
    {
        PetName = petName;
        PetColor = petColor;
    }

    public string MyPet() => $"My pet is {PetName}
        and its color is {PetColor}.";
}
```

В коде мы имеем класс `PetAnimal`, содержащий два приватных поля — `PetName` и `PetColor`, а также метод `MyPet()`.

Объект

В реальном мире объектам присущи две характеристики: состояние и поведение. Эти характеристики — не что иное, как состояние объекта. Возьмем для примера любое животное: у собаки и у кошки есть кличка. Так, собаку одного из авторов зовут Эйс, а кошку — Клементина. Им также присуще разное поведение: к примеру, собаки лают, а кошки мяукают.

В подразделе «Определение ООП» мы обсудили, что ООП — это модель программирования, призванная объединить состояние или структуру (данные) и поведение (методы), чтобы предоставить программный функционал. В предыдущем примере различные состояния животных являются данными, а их поведение — это метод.



Объект хранит информацию (обычно данные) в атрибутах и задает поведение через методы.

В терминах ООП-языка, такого как C#, объект — это экземпляр класса. В нашем предыдущем примере реальный объект `Dog` мог бы быть объектом класса `PetAnimal`.



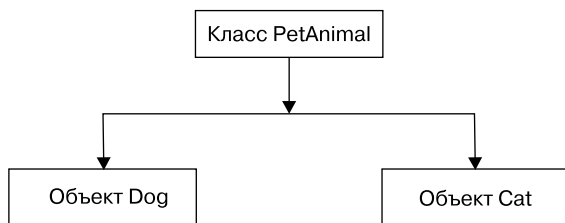
Объекты могут быть либо конкретными (то есть реальными, например собака или кошка, или любой тип файлов, скажем физический или электронный файл), либо же концептуальными, такими как схемы баз данных или образцы кода.

В следующем фрагменте кода показан объект, объединяющий в себе данные и метод, и пример его использования:

```
namespace OOPExample
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("OOP example");
            PetAnimal dog = new PetAnimal("Ace", PetColor.Black);
            Console.WriteLine(dog.MyPet());
            Console.ReadLine();
            PetAnimal cat = new PetAnimal("Clementine", PetColor.Brown);
            Console.WriteLine(cat.MyPet());
            Console.ReadLine();
        }
    }
}
```

В этом фрагменте кода мы создали два объекта: `dog` и `cat`. Они являются двумя разными экземплярами класса `PetAnimal`. Вы могли заметить, что поля или описания, содержащие данные о животных, получают свои значения из метода-конструктора. Это специальный метод, используемый для создания экземпляра класса.

Визуализируем данный пример — взгляните на рис. 1.5.



Наглядная иллюстрация объекта класса

Рис. 1.5

Это наглядное представление нашего предыдущего примера кода, в котором мы создали два разных объекта, `Dog` и `Cat`, класса `PetAnimal`. Схема интуитивно понятна: объект класса `Dog`, равно как и `Cat`, — экземпляр класса `PetAnimal`.

Ассоциации

Ассоциации объекта — важная особенность ООП. Подобно тому как в реальном мире объекты связаны друг с другом отношениями, в ООП ассоциации позволяют определять отношения между объектами типа *has-a* («имеет»). Например, велосипедист *имеет* велосипед, а кот *имеет* нос.

Существует несколько типов отношений *has-a*.

- ❑ *Ассоциация* — используется для описания отношений между объектами без описания собственности, как, например, отношения между машиной и человеком (в этом случае отношения могут быть представлены вождением). Человек может водить несколько машин, и одним автомобилем может управлять несколько человек.
- ❑ *Агрегация* — специальная форма ассоциации. Объекты также имеют жизненный цикл, но в данном случае предусматривается владение. Это значит, что дочерний объект не может принадлежать другому родительскому объекту. Агрегация — однонаправленный тип отношений, в которых объекты существуют независимо друг от друга. Например, отношения ребенка и родителей — это агрегация, так как каждый ребенок имеет родителей, однако не у всяких родителей есть ребенок.
- ❑ *Композиция* — определяет отношение удаления. Она описывает отношения между двумя объектами, где один (дочерний) зависит от другого (родительского). Если родитель удален, то все его дочерние объекты удаляются автоматически. Рассмотрим это на примере дома и комнаты. В одном доме может быть множество комнат, но одна комната не может быть одновременно во множестве домов. Если мы разрушим дом, то комната тоже будет уничтожена.

Посмотрим, как эти принципы применяются в C#, немного расширив предыдущий пример с животными за счет добавления класса `PetOwner`. Он может быть ассоциирован с одним или несколькими экземплярами `PetAnimal`. Поскольку у данного класса может не быть владельца, такое отношение является агрегацией. `PetAnimal` относится к `PetColor`, и в этой системе `PetColor` существует, только если относится к `PetAnimal`, делая ассоциацию композицией.

На рис. 1.6 показана как агрегация, так и композиция.

Данная модель основана на UML и может быть вам незнакома, поэтому выделим в ней ключевые моменты. Класс отображен как блок, состоящий из имени, атрибутов и методов, разделенных линиями. Пока проигнорируем символы перед именем метода, например + и -; мы рассмотрим эти модификаторы доступа чуть позже, при обсуждении инкапсуляции. Ассоциации обозначены с помощью линий, соединяющих классы. В случае композиции используется сплошной ромб, направленный к родителю; пустой ромб служит для обозначения агрегации. Обратите внимание: представленная схема позволяет показать количество возможных потомков. На ней класс `PetOwner` может иметь 0 или более классов `PetAnimal` (знак * говорит о том, что количество ассоциаций не ограничено).

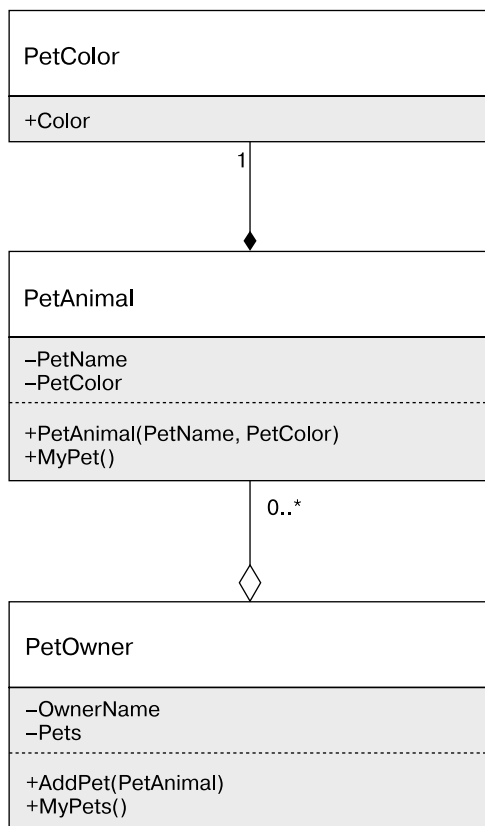


Рис. 1.6

**UML**

UML — язык моделирования, служащий для разработки программного обеспечения. Он был создан более 20 лет назад и поддерживается Группой управления объектами (Object Management Group, OMG). Пройдя по ссылке www.uml.org, можно узнать подробности.

Интерфейс

Интерфейс в C# определяет содержимое объекта или его контракт: в частности, методы, свойства, события или индексы объекта. Однако интерфейсы не предоставляют реализацию. В отличие от базового класса, предоставляющего и контракт, и реализацию, у них не может быть атрибутов. Класс, который реализует интерфейс, должен реализовывать все, что определено в интерфейсе.



Абстрактный класс

Абстрактный класс — это гибрид интерфейса и базового класса, предоставляющий и реализацию, и атрибуты, а также методы, которые должны быть определены в дочерних классах.

Сигнатура

Термин, которым тоже можно описать контракт объекта.

Наследование

Наследование — один из самых важных принципов ООП. Наследование между классами позволяет определить *тип* отношения: например, машина — *тип* транспорта. Важность принципа в том, что он позволяет объектам одного типа иметь схожие функции и признаки. Допустим, у нас есть система управления книжным интернет-магазином. Мы можем иметь один класс для хранения информации о бумажной копии книги, другой — для хранения данных о цифровой копии. Схожие признаки этих классов, такие как название, издатель и автор, могут храниться в каком-то третьем классе. Тогда классы бумажного и цифрового изданий должны наследоваться от него.



Для описания классов в наследовании есть разные термины: дочерние, или производные, классы наследуются от другого класса, в то время как классы, от которых наследуют, называются базовыми, или родительскими.

Далее мы подробно обсудим наследование.

Типы наследования. Наследование помогает определить дочерний класс. Этот класс наследует поведение родительского, или базового.



Наследование в C# обозначается двоеточием (:).

Посмотрим на различные типы наследования.

- ❑ *Одинокое наследование* — часто встречающийся тип наследования, описывает один класс, унаследованный от другого.

Вернемся к упомянутому ранее классу `PetAnimal` и используем наследование для определения классов `Dog` и `Cat`. С его помощью мы можем определить некоторые атрибуты, общие для обоих классов. Так, кличка животного и цвет могут быть общими, поэтому располагаются в базовом классе. Специфические свойства кота или собаки, напротив, должны быть определены в отдельных классах: например, звуки, которые издают собаки и кошки. На рис. 1.7 показан класс `PetAnimal` с двумя дочерними классами.

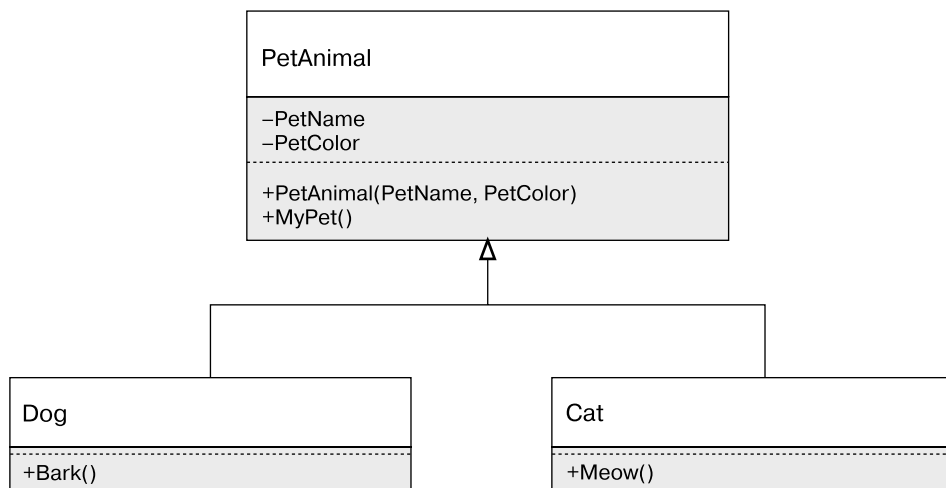


Рис. 1.7



C# поддерживает только одиночное наследование.

- ❑ *Множественное наследование* происходит, когда класс наследует несколько базовых. Такие языки, как C++, поддерживают множественное наследование. C# не поддерживает этот тип наследования, но мы можем достичь подобного эффекта с помощью интерфейсов.



Пройдя по ссылке blogs.msdn.microsoft.com/csharpfaq/2004/03/07/why-doesnt-c-support-multiple-inheritance/, можно получить больше информации о C# и множественном наследовании.

- ❑ *Иерархическое наследование* происходит, когда несколько классов наследуются от другого класса.
- ❑ *Многоуровневое наследование* — это наследование одного класса от другого, который сам является производным.
- ❑ *Гибридное наследование* — это сочетание нескольких типов наследования.



C# не поддерживает гибридное наследование.

- ❑ *Неявное наследование* — все типы в .NET Core неявно наследуются от класса `System.Object` и его производных классов.

Инкапсуляция

Инкапсуляция — фундаментальный принцип ООП, согласно которому детали класса, такие как его атрибуты и методы, могут быть видимыми или невидимыми вне объекта. С помощью инкапсуляции разработчик задает способы использования класса, а также помогает предотвратить его неправильное применение. К примеру, мы хотим разрешить добавление объектов `PetAnimal` только через метод `AddPet(PetAnimal)`. Мы можем сделать это с помощью метода `AddPet(PetAnimal)` класса `PetOwner`, притом атрибут `Pets` будет скрыт от любого кода за пределами класса `PetAnimal`. В C# это позволяет реализовать приватный атрибут `Pets`. Одна из причин тому — при добавлении класса `PetAnimal` требуется дополнительная логика, такая как ведение журнала или проверка того, может ли у владельца (класса `PetOwner`) быть домашнее животное.

C# поддерживает различные уровни доступа к элементам. Элемент может быть классом, его атрибутом, или методом, или перечислением. Значения модификаторов:

- ❑ `Public` — доступ к элементу снаружи;
- ❑ `Private` — доступ только из текущего объекта;
- ❑ `Protected` — доступ из текущего объекта и его наследников;
- ❑ `Internal` — доступ объектов только из той же сборки;
- ❑ `Protected Internal` — доступ объектов только из той же сборки и объектов, расширяющих текущий класс.

На рис. 1.8 показано, как модификаторы доступа можно применить к `PetAnimal`.

Как пример, кличка животного и его цвет сделаны приватными для предотвращения доступа вне класса `PetAnimal`. Здесь мы разрешаем доступ к значениям `PetName` и `PetColor` только из класса `PetAnimal`, чтобы их мог изменять лишь базовый класс. Конструктор `PetAnimal` был защищен — к нему может обращаться только класс-потомок. В данном приложении лишь классы той же библиотеки, такие как `Dog`, имеют доступ к методу `RegisterInObedienceSchool()`.

Полиморфизм

Способность управлять различными объектами, используя один и тот же интерфейс, называется полиморфизмом. Он позволяет разработчикам делать приложения гибкими, прописав лишь небольшой кусочек функционала, который может применяться к различным формам — главное, чтобы они имели общий интерфейс. Есть разные определения полиморфизма в ООП, и мы остановимся на двух главных его типах:

- ❑ *статическое, или раннее, связывание* — форма полиморфизма, проявляющаяся при компиляции;
- ❑ *динамическое, или позднее, связывание* — форма полиморфизма, проявляющаяся при работе приложения.

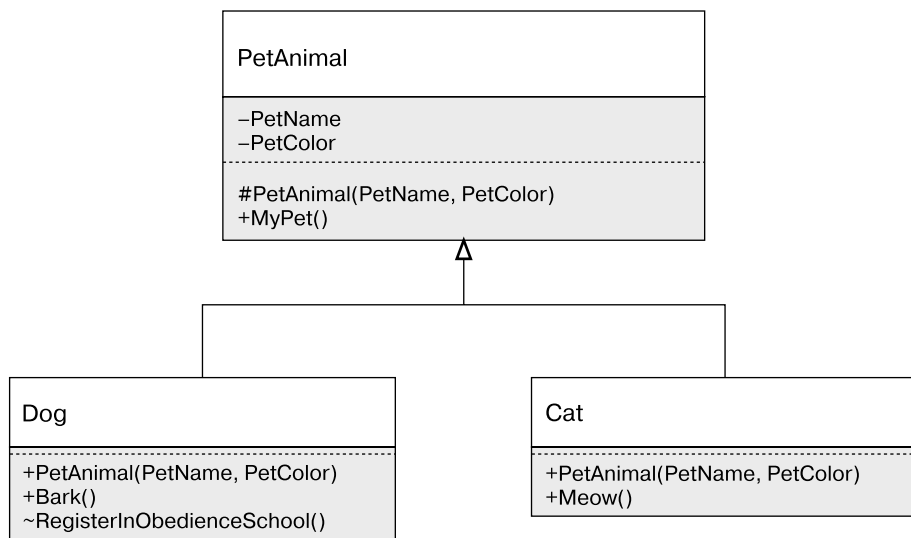


Рис. 1.8

Статический полиморфизм

Статический полиморфизм, или полиморфизм раннего связывания, происходит при компиляции и главным образом состоит из перегрузки методов, где класс имеет множество методов с одним именем, но разными параметрами. Это можно использовать для передачи смысла метода с помощью его имени или для упрощения кода. Так, применительно к калькулятору код будет намного понятней, если для добавления различных типов чисел использовать много одноименных методов, нежели иметь методы с разными именами для каждой отдельной ситуации. Сравним два фрагмента кода:

```
int Add(int a, int b) => a + b;
float Add(float a, float b) => a + b;
decimal Add(decimal a, decimal b) => a + b;
```

В коде ниже показан тот же функционал, но без перегрузки метода `Add()`:

```
int AddTwoIntegers(int a, int b) => a + b;
float AddTwoFloats(float a, float b) => a + b;
decimal AddTwoDecimals(decimal a, decimal b) => a + b;
```

В примере с животными владелец будет использовать разную еду для кормления объектов класса `cat` и `dog`. Мы можем реализовать это в виде класса `PetOwner` с двумя методами `Feed()`, вот так:

```
public void Feed(PetDog dog)
{
    PetFeeder.FeedPet(dog, new Kibble());
}

public void Feed(PetCat cat)
{
    PetFeeder.FeedPet(cat, new Fish());
}
```

Оба метода используют класс `PetFeeder` для кормления животного, в то время как класс `dog` получает `Kibble` (сухой корм), а класс `cat` получает `Fish`. Класс `PetFeeder` описывается в пункте «Дженерики» далее.

Динамический полиморфизм

Динамический полиморфизм, или полиморфизм позднего связывания, происходит во время работы приложения. При этом возникают разные ситуации, и мы рассмотрим три общие формы в C#: полиморфизм через интерфейс, наследование и дженерики.

Полиморфизм через интерфейс

Интерфейс определяет, какую сигнатуру должен реализовывать класс. На примере `PetAnimal` представьте, что мы определяем еду для животных через то количество энергии, которое эта еда предоставляет:

```
public interface IPetFood
{
    int Energy { get; }
}
```

Сам по себе интерфейс не может быть вызван, однако описывает, какой экземпляр `IPetFood` нужно реализовать. Например, `Kibble` и `Fish` могут давать разное количество энергии, как показано ниже:

```
public class Kibble : IPetFood
{
    public int Energy => 7;
}

public class Fish : IPetFood
{
    int IPetFood.Energy => 8;
}
```

В этом фрагменте кода `Kibble` дает меньше энергии, чем `Fish`.

Полиморфизм через наследование

Полиморфизм через наследование позволяет определять функционал во время работы приложения в схожей с интерфейсом манере, но в контексте наследования классов. В нашем примере животное можно накормить. Мы определяем это введением метода `Feed(IpetFood)`. Он использует представленный ранее интерфейс таким образом:

```
public virtual void Feed(IpetFood food)
{
    Eat(food);
}

protected void Eat(IpetFood food)
{
    _hunger -= food.Energy;
}
```

Этот код означает, что все реализации `PetAnimal` будут иметь метод `Feed(IpetFood)`, и дочерние классы могут предоставить другую реализацию. Метод `Eat(IpetFood food)` не отмечен как виртуальный, так как по замыслу все объекты `PetAnimal` будут использовать метод, не прибегая к переопределению его поведения. Вдобавок он помечен как защищенный, чтобы предотвратить доступ к нему вне объекта.



Виртуальный метод не обязательно объявлять в дочернем классе; это отличает его от интерфейса, все методы которого должны быть реализованы.

Элемент `PetDog` не будет переопределять поведение базового класса, так как собака будет есть и `Kibble`, и `Fish`. Кошка более прихотлива, что и показано в следующем фрагменте кода:

```
public override void Feed(IpetFood food)
{
    if (food is Fish)
    {
        Eat(food);
    }
    else
    {
        Meow();
    }
}
```

С помощью ключевого слова `override` элемент `PetCat` изменяет поведение базового класса: кошка будет есть только рыбу.

Дженерики

Дженерик (обобщенный класс) определяет поведение, которое можно применить к классу. Распространенная разновидность дженериков — коллекции, в которых объекты различных типов обрабатываются одним и тем же способом. Например, списки текстовых строк или целочисленных значений можно обработать одной и той же логикой, не разделяя на разные типы.

Возвращаясь к животным, мы могли бы определить обобщенный класс для кормления животных. Он просто будет обозначать прием пищи и котом, и собакой, как показано в следующем фрагменте кода:

```
public static class PetFeeder
{
    public static void FeedPet<TP, TF>(TP pet, TF food) where TP : PetAnimal
        where TF : IPetFood
    {
        pet.Feed(food);
    }
}
```

Здесь можно выделить парочку интересных моментов. Прежде всего, нам не обязательно создавать экземпляр этого класса, поскольку он, как и его методы, является статическим. Метод дженерика описывается с помощью сигнатуры, `FeedPet<TP, TF>`. Ключевое слово `where` используется для обозначения дополнительных требований о том, чем должны быть `TP` и `TF`. В этом примере `where` определяет принадлежность `TP` к типу `PetAnimal`, тогда как `TF` должен реализовывать интерфейс `IPetFood`.

Резюме

В этой главе мы обсудили ООП и основные особенности данной парадигмы: наследование, инкапсуляцию и полиморфизм. С их помощью классы приложения можно абстрагировать и предоставить их четкие определения. В этом заключается главное отличие языков с поддержкой ООП от некоторых ранних парадигм, таких как структурное и процедурное программирование. Возможность абстрагировать функциональность облегчает повторное применение и сопровождение кода.

В следующей главе мы обсудим различные паттерны, используемые в разработке корпоративного ПО. Рассмотрим паттерны проектирования, а также принципы и приемы разработки программного обеспечения *в жизненном цикле разработки ПО* (software development life cycle, SDLC).

Вопросы

Ответив на следующие вопросы, вы сможете закрепить информацию, представленную в этой главе.

1. Что означают термины «позднее» и «раннее» связывание?
2. Поддерживает ли C# множественное наследование?
3. Применительно к C# какой уровень инкапсуляции можно использовать для предотвращения доступа к классу вне библиотеки?
4. Чем различаются агрегация и композиция?
5. Могут ли интерфейсы содержать свойства? (Вопрос с подвохом.)
6. Едят ли собаки рыбу?

2

Современные паттерны и принципы проектирования ПО

В предыдущей главе мы обсудили объектно-ориентированное программирование (ООП), чтобы подготовиться к рассмотрению паттернов. Поскольку множество из них полагаются на принципы ООП, очень важно знать и помнить эти принципы. Наследование позволяет определить *тип отношения*, что, в свою очередь, дает возможность использовать более высокий уровень абстракции. Например, с помощью наследования можно проводить сравнения: *кошка* — это тип *животного*, как и *собака*. Инкапсуляция обеспечивает способ контроля видимости и доступа к деталям класса. Полиморфизм предоставляет возможность управлять различными объектами с помощью одного и того же интерфейса. ООП позволяет достичь высокого уровня абстрагирования, обеспечивая более управляемый и понятный способ работы с большими системами.

Эта глава представляет различные виды паттернов, применяемых в современной разработке программного обеспечения. В книге используется широкое определение того, что такое паттерн. Таковым в разработке ПО является любое решение обыденной проблемы, с которой рядовые программисты сталкиваются в процессе работы. Решения строятся на опыте о том, что действует, а что — нет. Вдобавок их опробовали и протестировали множество разработчиков в различных ситуациях. Использование паттерна позволяет избежать повторения уже выполненных действий, а также гарантировать, что решение проблемы не вызовет новых дефектов.

В частности, когда речь идет о паттернах, относящихся к определенным технологиям, следует понимать, что их сегодня огромное количество, слишком большое, чтобы вместить все в книгу. Поэтому в данной главе мы отметим некоторые специальные паттерны с целью показать их различия по типам. Мы постарались выбрать самые общие и самые полезные паттерны, исходя из нашего опыта. В последующих главах мы изучим часть из них более подробно.

В этой главе будут рассмотрены следующие темы:

- ❑ принципы проектирования, включая SOLID;
- ❑ перечень паттернов, включая паттерны «Банды четырех» (Gang of Four, GoF) и *паттерн интеграции корпоративных приложений* (Enterprise Integration Pattern, EIP);

- ❑ паттерны жизненного цикла разработки программного обеспечения;
- ❑ паттерны и методы разработки решений, облачной разработки и разработки сервисов.

Технические требования

В этой главе содержатся примеры кода, объясняющие принципы качественной разработки. Код упрощен и предназначен лишь для демонстрации. В большинстве примеров используется консольное приложение на основе .NET Core, написанное на C#.

Чтобы запустить и выполнить код, вам потребуется следующее:

- ❑ Visual Studio 2019 (вы также можете запустить приложение, используя Visual Studio 2017 версии 3 и новее);
- ❑ .NET Core;
- ❑ SQL Server (в этой главе применяется Express Edition).

Установка Visual Studio

Чтобы запустить примеры кода, вам необходимо установить Visual Studio. Для этого выполните следующие шаги.

1. Скачайте Visual Studio по ссылке docs.microsoft.com/ru-ru/visualstudio/install/install-visual-studio.
2. Следуйте инструкциям по установке. Доступны различные версии Visual Studio; в этой главе мы используем Visual Studio для Windows.

Вы также можете применять другую интегрированную среду разработки по вашему усмотрению.

Установка .NET Core

Если вы еще не установили .NET Core, то следуйте инструкции ниже.

1. Скачайте .NET Core по ссылке dotnet.microsoft.com/download.
2. Далее следуйте указаниям по установке соответствующей библиотеки: dotnet.microsoft.com/download/dotnet-core/2.2.



Полная версия исходного кода доступна на GitHub. Код, представленный в данной главе, может быть неполным, поэтому мы рекомендуем вам скачать код для запуска примеров (github.com/PacktPublishing/Hands-On-Design-Patterns-with-C-and-.NET-Core/tree/master/Chapter2).

Принципы проектирования

Пожалуй, важнейший аспект разработки качественного ПО — его проектирование. Создание решений, функционально точных и удобных в сопровождении, — непростая задача, которая в значительной мере зависит от того, какие принципы используются в разработке. Со временем многие решения, принятые на ранних стадиях проекта, могут привести к повышению стоимости и сложности сопровождения, и придется переписывать системы. В то же время продуманную архитектуру можно дополнить и адаптировать к изменениям в технологиях и бизнесе.

Существует множество принципов проектирования и разработки. В этой главе мы отметим несколько самых важных и популярных из них, чтобы вы могли познакомиться с ними поближе.

DRY — «Не повторяйся»

Главный посыл принципа *«Не повторяйся»* (Don't Repeat Yourself, DRY) в том, что дублирование — это просто потеря времени и сил. Дублирование может проявляться в процессах или коде. Множественное выполнение одних тех же требований приводит к выгоранию и хаосу. При первом знакомстве с принципом может быть непонятно, как системе удастся дублировать процесс или код. К примеру, однажды кто-то определил порядок выполнения требования, так зачем тратить время на дублирование уже имеющегося функционала? В разработке ПО дублирование происходит часто. Понимание причины случившегося — ключ к осознанию ценности DRY.

Ниже приведены некоторые распространенные причины дублирования кода:

- ❑ *недопонимание* — в больших решениях разработчик может не до конца понимать суть и/или не знать, как применять абстрагирование для решения проблемы, используя имеющийся функционал;
- ❑ *копирование-вставка* — проще говоря, общая функциональность реализуется с помощью дублирования одного и того же кода вместо проведения рефакторинга во множестве классов.

KISS — «Делай проще, тупица»

Аналогично DRY *«Делай проще, тупица»* (Keep It Simple Stupid, KISS) — важный принцип разработки, существующий уже много лет. KISS делает акцент на том, что главной целью должна быть простота и нужно избегать усложнений. Цель принципа — не допустить ненужных сложностей и уменьшить вероятность ошибок в будущем.

YAGNI — «Вам это не понадобится»

«Вам это не понадобится» (You aren't Gonna Need It, YAGNI) подразумевает, что функционал должен добавляться только при необходимости. Иногда в разработке ПО есть тенденция проектировать с «заделом на будущее», на случай каких-либо изменений. Это может привести к появлению требований, которые не нужны на данный момент или в ближайшем будущем. «Всегда реализуйте функции, которые вам действительно нужны, и никогда, если всего лишь предполагаете, что они понадобятся» (Рон Джеффрис).

MVP — «Продукт с минимальным функционалом»

При использовании подхода «Продукт с минимальным функционалом» (Minimum Viable Product, MVP) объем работы ограничивается минимальным набором требований для получения действующего результата. MVP часто комбинируется с другой методологией разработки — Agile (см. раздел «Паттерны жизненного цикла разработки программного обеспечения» далее в этой главе). Количество требований сокращается до какого-то разумного предела, то есть до состояния, когда продукт может быть спроектирован, разработан, протестирован и выпущен. Этот подход уместен при разработке сайтов или приложений, где набор функций может быть введен в эксплуатацию за один цикл разработки.



В главе 3 MVP будет описан на примере гипотетического сценария, в котором этот подход послужит для ограничения охвата изменений, а также для того, чтобы помочь разработчикам сосредоточиться на проектировании и составлении требований.

SOLID

SOLID — один из важнейших принципов проектирования, и мы подробно рассмотрим его в главе 3. Фактически составленный из пяти принципов проектирования, SOLID преследует целью создание более удобных в сопровождении и понятных решений. Эти принципы позволяют легче модифицировать код и уменьшают риски возникновения различных проблем.



В главе 3 мы познакомимся с принципами SOLID подробнее, применив их в приложении на C#.

Принцип единственной ответственности

Каждый класс должен нести только одну ответственность. Цель этого принципа — упростить классы и логически их структурировать, так как многозадачные классы слишком сложны для понимания и дальнейшей разработки. Множественная ответственность в данной ситуации — уже причина для изменения. Ответственность

также можно считать одним из аспектов функциональности: «класс... должен иметь одну — и только одну — причину для изменения»¹ (Роберт С. Мартин).

Принцип открытости/закрытости

Данный принцип проектирования лучше всего описать с точки зрения ООП. Средством расширения возможностей класса должно служить наследование. Другими словами, возможность изменения должна быть запланирована и рассмотрена на стадии проектирования класса. Определение и использование интерфейса, реализуемого классом, является применением принципа открытости/закрытости. Класс *открыт* для модификации, в то время как его описание, интерфейс, *закрыто* для нее.

Принцип замещения Лисков

Основной смысл данного принципа состоит в возможности подменять объекты во время работы приложения. В ООП, если класс наследуется от базового класса или реализует интерфейс, на данный класс можно ссылаться как на объект базового класса или интерфейса. Проще будет объяснить это на примере.

Мы определим интерфейс для животного и создадим две его реализации, `Cat` и `Dog`, как показано ниже:

```
interface IAnimal
{
    string MakeNoise();
}
class Dog : IAnimal
{
    public string MakeNoise()
    {
        return "Woof";
    }
}
class Cat : IAnimal
{
    public string MakeNoise()
    {
        return "Meouw";
    }
}
```

Теперь мы можем ссылаться на `Cat` и `Dog`, как на животных:

```
var animals = new List<IAnimal> { new Cat(), new Dog() };
foreach(var animal in animals)
{
    Console.WriteLine(animal.MakeNoise());
}
```

¹ Мартин Р. Чистый код: создание, анализ и рефакторинг. — СПб.: Питер, 2021. — С. 167.

Принцип разделения интерфейса

Подобно принципу единственной ответственности, принцип разделения интерфейса гласит: в интерфейсе должны находиться только те методы, которые соблюдают принцип единственной ответственности. Упрощение интерфейса делает код более понятным и облегчает рефакторинг. Ключевая выгода принципа в том, что он помогает изолировать систему от избыточных зависимостей.

Принцип инверсии зависимости

Принцип инверсии зависимости, также известный как принцип внедрения зависимости, гласит: модули должны зависеть от абстракций, а не от деталей реализации. Этот принцип поощряет написание слабо связанного кода, чтобы повысить его читабельность и удобство сопровождения, особенно в крупных и сложных проектах.

Паттерны программного обеспечения

Со временем многие паттерны были сгруппированы в каталоги. В этом разделе в качестве наглядного примера используется два каталога. Первый — ООП-ориентированные паттерны «Банды четырех». Второй относится к интеграции систем и остается технологически независимым. В конце главы будут даны несколько отсылок к дополнительным каталогам и материалам.

Паттерны «Банды четырех»

Возможно, самые значимые и широко известные ООП-коллекции паттернов собраны в книге «Приемы объектно-ориентированного проектирования. Паттерны проектирования» «Банды четырех»¹. В ней в основном представлены низкоуровневые паттерны, относящиеся к созданию и взаимодействию объектов, а не к более общим архитектурным аспектам. Коллекция состоит из шаблонов, которые можно применять в различных сценариях для создания цельных строительных блоков, в то же время избегая ловушек, распространенных в объектно-ориентированной разработке.



Эрих Гамма, Джон Влиссидес, Ричард Хелм и Ральф Джонсон известны как «Банда четырех» благодаря своим публикациям в 1990-х годах. Книга «Паттерны объектно-ориентированного проектирования» была переведена на несколько языков и содержит примеры на C++ и Smalltalk.

¹ Гамма Э., Хелм Р., Джонсон Р., Влиссидес Дж. Паттерны объектно-ориентированного проектирования. — СПб.: Питер, 2020.

Коллекция разбита на три категории: порождающие паттерны, структурные паттерны и поведенческие паттерны, о которых мы и будем говорить ниже.

Порождающие паттерны

С инстанцированием (созданием экземпляров) объектов связаны следующие пять паттернов:

- ❑ «*Абстрактная фабрика*» — паттерн для создания объектов, принадлежащих к семейству классов. Конкретный объект определяется во время выполнения;
- ❑ «*Строитель*» — полезный паттерн для более сложных объектов, в котором создание экземпляров контролируется вне созданного класса;
- ❑ «*Фабричный метод*» — паттерн для создания объектов, наследованных от класса, в котором конкретный класс определяется во время работы;
- ❑ «*Прототип*» — паттерн для копирования или клонирования объекта;
- ❑ «*Одиночка*» — паттерн, позволяющий создавать только один экземпляр заданного класса.



В главе 3 мы подробно рассмотрим паттерн «Абстрактная фабрика». В главе 4 обсудим паттерны «Фабричный метод» и «Одиночка», включая использование поддерживающего их фреймворка .NET Core.

Структурные паттерны

Со взаимоотношениями между объектами и классами связаны следующие паттерны:

- ❑ «*Адаптер*» — паттерн проведения соответствия между двумя разными классами;
- ❑ «*Мост*» — паттерн, позволяющий заменять реализацию деталей класса, не прибегая к его модификации;
- ❑ «*Компоновщик*» — используется для создания иерархии классов с древовидной структурой;
- ❑ «*Декоратор*» — паттерн для замены функционала класса во время выполнения;
- ❑ «*Фасад*» — применяется для упрощения сложных систем;
- ❑ «*Приспособленец*» — паттерн, используемый для уменьшения расхода ресурсов в сложных моделях;
- ❑ «*Заместитель*» — паттерн, используемый для представления другого объекта и обеспечивающий дополнительный уровень контроля между вызывающим и вызываемым объектами.

Паттерн «Декоратор»

Чтобы продемонстрировать структурный паттерн, рассмотрим на примере паттерн «Декоратор». В этом примере будут выдаваться сообщения в консольном приложении. Для начала приведем определение базового сообщения с соответствующим интерфейсом:

```
interface IMessage
{
    void PrintMessage();
}

abstract class Message : IMessage
{
    protected string _text;
    public Message(string text)
    {
        _text = text;
    }
    abstract public void PrintMessage();
}
```

Базовый класс позволяет хранить текстовые фразы и требует, чтобы дочерние классы реализовывали метод `PrintMessage()`. Этот код мы дополним двумя классами.

Первый класс, `SimpleMessage`, выводит текст в консоль:

```
class SimpleMessage : Message
{
    public SimpleMessage(string text) : base(text) { }

    public override void PrintMessage()
    {
        Console.WriteLine(_text);
    }
}
```

Второй класс, `AlertMessage`, тоже выводит текст в консоль, но, помимо этого, выдается короткий звуковой сигнал:

```
class AlertMessage : Message
{
    public AlertMessage(string text) : base(text) { }
    public override void PrintMessage()
    {
        Console.Beep();
        Console.WriteLine(_text);
    }
}
```


Разница между этими классами в том, что в `AlertMessage` предусмотрен звуковой сигнал, а не только вывод текста, как в `SimpleMessage`.

Далее определим базовый класс-декоратор для хранения ссылки на объект `Message`:

```
abstract class MessageDecorator : IMessage
{
    protected Message _message;
    public MessageDecorator(Message message)
    {
        _message = message;
    }

    public abstract void PrintMessage();
}
```

Следующие два класса демонстрируют паттерн «Декоратор», добавляя функционал к нашей уже существующей реализации `Message`.

Первый класс, `NormalDecorator`, выделяет сообщение зеленым цветом:

```
class NormalDecorator : MessageDecorator
{
    public NormalDecorator(Message message) : base(message) { }

    public override void PrintMessage()
    {
        Console.ForegroundColor = ConsoleColor.Green;
        _message.PrintMessage();
        Console.ForegroundColor = ConsoleColor.White;
    }
}
```

Класс `ErrorDecorator` выделяет сообщение красным цветом:

```
class ErrorDecorator : MessageDecorator
{
    public ErrorDecorator(Message message) : base(message) { }

    public override void PrintMessage()
    {
        Console.ForegroundColor = ConsoleColor.Red;
        _message.PrintMessage();
        Console.ForegroundColor = ConsoleColor.White;
    }
}
```

Класс `NormalDecorator` будет выводить текст зеленого цвета, а класс `ErrorDecorator` — красного. Самое важное в данном примере то, что декоратор дополняет поведение объекта `Message`, на который мы ссылаемся.

В завершение примера следующий код показывает, как могут использоваться новые сообщения:

```
static void Main(string[] args)
{
    var messages = new List<IMessage>
    {
        new NormalDecorator(new SimpleMessage("First Message!")),
        new NormalDecorator(new AlertMessage("Second Message with a beep!")),
        new ErrorDecorator(new AlertMessage("Third Message with a beep
                                     and in red!")),
        new SimpleMessage("Not Decorated...")
    };
    foreach (var message in messages)
    {
        message.PrintMessage();
    }
    Console.Read();
}
```

Запуск примера покажет, как разные виды паттерна «Декоратор» можно использовать для изменения заданного функционала (рис. 2.1).

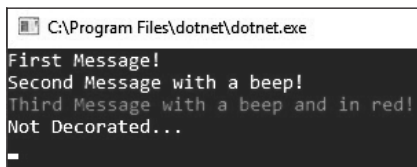


Рис. 2.1

Это упрощенный пример. Но представьте ситуацию, когда у проекта появляется новое требование: знак восклицания должен сопровождаться не коротким звуковым сигналом, а конкретным системным звуком.

```
class AlertMessage : Message
{
    public AlertMessage(string text) : base(text) { }

    public override void PrintMessage()
    {
        System.Media.SystemSounds.Exclamation.Play();
        Console.WriteLine(_text);
    }
}
```

У нас для этого уже есть структура, следовательно, нам достаточно было изменить всего одну строчку, как показано в данном блоке кода.

Поведенческие паттерны

Для определения взаимодействия между классами и объектами могут быть использованы следующие поведенческие паттерны:

- ❑ «*Цепочка ответственности*» — паттерн для обработки запросов объектами в коллекции;
- ❑ «*Команда*» — паттерн для представления запроса;
- ❑ «*Интерпретатор*» — паттерн, определяющий синтаксис или язык для написания инструкций в программе;
- ❑ «*Итератор*» — паттерн для обхода коллекции элементов в отсутствие знания подробностей об этих элементах;
- ❑ «*Посредник*» — паттерн для упрощения взаимодействия между классами;
- ❑ «*Хранитель*» — паттерн для фиксации и сохранения состояния объекта;
- ❑ «*Наблюдатель*» — паттерн, позволяющий объектам уведомлять друг друга об изменении своего состояния;
- ❑ «*Состояние*» — паттерн для изменения поведения объекта при изменении его состояния;
- ❑ «*Стратегия*» — паттерн для реализации коллекции алгоритмов, один из которых может быть применен во время исполнения;
- ❑ «*Шаблонный метод*» — паттерн, который определяет этапы алгоритма, делегируя детали реализации подклассу;
- ❑ «*Посетитель*» — паттерн, способствующий ослаблению связи между данными и функционалом; позволяет добавлять новые операции, не внося изменений в классы данных.

Паттерн «Цепочка ответственности»

Полезный паттерн, с которым вам следует познакомиться, — это «Цепочка ответственности», поэтому мы и берем его для примера. С его помощью мы создадим коллекцию, или цепочку, классов для обработки запроса. Идея в том, что в ходе обработки запрос должен пройти через каждый класс. Для демонстрации мы возьмем автомобильный сервисный центр, где каждый автомобиль проедет через различные отделы этого предприятия до полного завершения обслуживания.

Начнем с определения коллекции флагов, которые будут использованы для обозначения необходимых сервисов:

```
[Flags]
enum ServiceRequirements
{
```

```

None = 0,
WheelAlignment = 1,
Dirty = 2,
EngineTune = 4,
TestDrive = 8
}

```



Атрибут `FlagsAttribute` в C# предоставляет отличный способ использовать битовое поле для хранения коллекции флагов. Для указания значений перечисления, которые включаются с помощью побитовых операций, будет применяться всего одно поле.

Класс `Car` содержит два поля: одно описывает необходимые услуги, а другое определяет, проведено ли обслуживание:

```

class Car
{
    public ServiceRequirements Requirements { get; set; }

    public bool IsServiceComplete
    {
        get
        {
            return Requirements == ServiceRequirements.None;
        }
    }
}

```

Главное, что можно выделить здесь, — `Car` считает обслуживание завершенным, как только удовлетворяются все требования. Об этом сигнализирует свойство `IsServiceComplete`.

Мы используем базовый абстрактный класс для отображения каждого сервис-техника таким образом:

```

abstract class ServiceHandler
{
    protected ServiceHandler _nextServiceHandler;
    protected ServiceRequirements _servicesProvided;

    public ServiceHandler(ServiceRequirements servicesProvided)
    {
        _servicesProvided = servicesProvided;
    }
}

```

Обратите внимание: услуга, предоставляемая классом, который наследует `ServiceHandler` (то есть сервис-техником), должна быть передана в качестве параметра.

Обслуживание будет выполнено с помощью побитовой операции NOT (~) для *отключения* бита, показывающего необходимость сервиса, в заданном экземпляре Car в методе Service:

```
public void Service(Car car)
{
    if (_servicesProvided == (car.Requirements & _servicesProvided))
    {
        Console.WriteLine($"{this.GetType().Name} providing
            {this._servicesProvided} services.");
        car.Requirements &= ~_servicesProvided;
    }

    if (car.IsServiceComplete || _nextServiceHandler == null)
        return;
    else
        _nextServiceHandler.Service(car);
}
```

Когда все работы по обслуживанию автомобиля завершены и/или других услуг больше не предусмотрено, цепочка останавливается. Если же остаются другие услуги, а автомобиль еще не готов, то вызывается следующий обработчик.

Данный подход требует установки цепочки, и следующий пример показывает это в действии, используя метод SetNextServiceHandler() для запуска в работу очередного сервиса:

```
public void SetNextServiceHandler(ServiceHandler handler)
{
    _nextServiceHandler = handler;
}
```

Сервисные специалисты — это Detailer, Mechanic, WheelSpecialist и инженер QualityControl. Класс ServiceHandler, представляющий собой Detailer, показан в следующем фрагменте кода:

```
class Detailer : ServiceHandler
{
    public Detailer() : base(ServiceRequirements.Dirty) { }
}
```

Механик, в чью специализацию входят настройка и ремонт двигателя, показан в следующем коде:

```
class Mechanic : ServiceHandler
{
    public Mechanic() : base(ServiceRequirements.EngineTune) { }
}
```

Специалист по ходовой части показан в следующем фрагменте кода:

```
class WheelSpecialist : ServiceHandler
{
    public WheelSpecialist() : base(ServiceRequirements.WheelAlignment) { }
}
```

И наконец, контролер качества, который проводит тест-драйв автомобиля:

```
class QualityControl : ServiceHandler
{
    public QualityControl() : base(ServiceRequirements.TestDrive) { }
}
```

Техники сервисного центра определены. Теперь можно обслужить парочку автомобилей. Процесс будет показан в блоке кода `Main`, начиная с создания необходимых объектов:

```
static void Main(string[] args)
{
    var mechanic = new Mechanic();
    var detailer = new Detailer();
    var wheels = new WheelSpecialist();
    var qa = new QualityControl();
}
```

Следующим шагом будет настройка порядка выполнения разных сервисных работ:

```
qa.SetNextServiceHandler(detailer);
wheels.SetNextServiceHandler(qa);
mechanic.SetNextServiceHandler(wheels);
```

Далее мы сделаем два запроса к механику, который и начнет цепочку ответственности:

```
Console.WriteLine("Car 1 is dirty");
mechanic.Service(new Car { Requirements = ServiceRequirements.Dirty });

Console.WriteLine();

Console.WriteLine("Car 2 requires full service");
mechanic.Service(new Car { Requirements = ServiceRequirements.Dirty |
    ServiceRequirements.EngineTune |
    ServiceRequirements.TestDrive |
    ServiceRequirements.WheelAlignment });

Console.Read();
}
```

Стоит обратить внимание на порядок, в котором оформлена цепочка. На примере сервисного центра сначала механик проводит работы по двигателю, затем за дело

берется специалист по ходовой части. И уже после проводится тест-драйв и машина объявляется прошедшей обслуживание. Изначально тест-драйв был последним этапом, но сервис-центр определил, что, например, в дождливую погоду этот процесс нужно повторить. Может, это немного глупый пример, но он демонстрирует преимущества гибкого определения цепочки ответственности.

На рис. 2.2 показан экран после того, как два автомобиля прошли обслуживание.



```
C:\Program Files\dotnet\dotnet.exe
Car 1 is dirty
Detailer providing Dirty services.

Car 2 requires full service
Mechanic providing EngineTune services.
WheelSpecialist providing WheelAlignment services.
QualityControl providing TestDrive services.
Detailer providing Dirty services.
```

Рис. 2.2

Паттерн «Наблюдатель»

Довольно интересен для детального изучения паттерн «Наблюдатель». Он позволяет информировать одни экземпляры о том, что происходит в других. Таким образом, за одним экземпляром может наблюдать множество объектов. На рис. 2.3 показан этот паттерн.

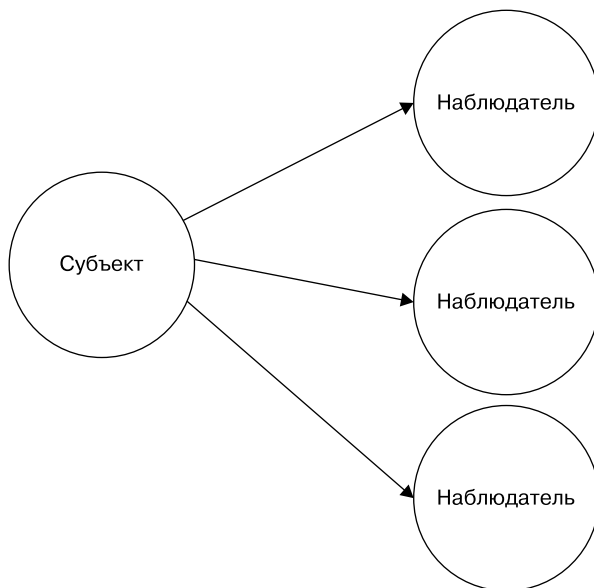


Рис. 2.3

Приведем пример, написав простое консольное приложение на C#, которое создаст один экземпляр класса `Subject` и много экземпляров класса `Observer`. Когда в классе `Subject` меняется значение `quantity`, мы хотим, чтобы каждый экземпляр класса `Observer` получил уведомление об этом.

Класс `Subject` содержит приватное поле `quantity`, которое обновляется публичным методом `UpdateQuantity`:

```
class Subject
{
    private int _quantity = 0;

    public void UpdateQuantity(int value)
    {
        _quantity += value;

        // оповещение наблюдателей
    }
}
```

Для оповещения любых наблюдателей в языке C# предназначены ключевые слова `delegate` и `event`. Ключевое слово `delegate` задает формат или обработчик, который будет вызван. Делегат используется при обновлении `quantity`, как показано в следующем коде:

```
public delegate void QuantityUpdated(int quantity);
```

Делегат определяет `QuantityUpdated` как метод, который получает целочисленное значение и ничего не возвращает. Затем к классу `Subject` добавляется событие:

```
public event QuantityUpdated OnQuantityUpdated;
```

И затем вызывается в методе `UpdateQuantity`, как показано в фрагменте ниже:

```
public void UpdateQuantity(int value)
{
    _quantity += value;

    // оповещение наблюдателей
    OnQuantityUpdated?.Invoke(_quantity);
}
```

В этом примере мы определяем в классе `Observer` метод, который имеет ту же сигнатуру, что и делегат `QuantityUpdated`:

```
class Observer
{
    ConsoleColor _color;
    public Observer(ConsoleColor color)
    {
        _color = color;
    }
}
```



```

internal void ObserverQuantity(int quantity)
{
    Console.ForegroundColor = _color;
    Console.WriteLine($"I observer the new quantity value of {quantity}.");
    Console.ForegroundColor = ConsoleColor.White;
}
}

```

Данная реализация будет оповещена, когда количество в экземпляре `Subject` изменится, и она выдаст в консоли сообщение соответствующего цвета.

Объединим это все в простом приложении. В начале приложения создаются один объект `Subject` и три объекта `Observer`:

```

var subject = new Subject();
var greenObserver = new Observer(ConsoleColor.Green);
var redObserver = new Observer(ConsoleColor.Red);
var yellowObserver = new Observer(ConsoleColor.Yellow);

```

Далее каждый экземпляр `Observer` подписывается на получение уведомлений об изменении `quantity`, которые рассылает `Subject`.

```

subject.OnQuantityUpdated += greenObserver.ObserverQuantity;
subject.OnQuantityUpdated += redObserver.ObserverQuantity;
subject.OnQuantityUpdated += yellowObserver.ObserverQuantity;

```

А затем мы дважды обновляем `quantity`, как показано ниже:

```

subject.UpdateQuantity(12);
subject.UpdateQuantity(5);

```

Когда приложение запустится, мы получим три разноцветных сообщения, по одному для каждого обновления состояния, как показано на рис. 2.4.

```

C:\Program Files\dotnet\dotnet.exe
Hello World!
I observer the new quantity value of 12.
I observer the new quantity value of 12.
I observer the new quantity value of 12.
I observer the new quantity value of 17.
I observer the new quantity value of 17.
I observer the new quantity value of 17.
Enter a key to quit.

```

Рис. 2.4

Это был простой пример применения ключевого слова `event` в C#, однако, надеемся, вы теперь понимаете, как можно использовать данный паттерн. Преимуществом

здесь будет уменьшение связи между субъектом и наблюдателями. Субъект не обязан знать о различных наблюдателях.

Паттерны интеграции корпоративных приложений

Интеграция — это область разработки программного обеспечения, преимущества которой основаны на использовании чужих знаний и опыта. В связи с этим существует множество каталогов с паттернами интеграции корпоративных приложений; одни из них не зависят от технологий, в то время как другие рассчитаны на определенный технологический стек. В этом подразделе будут описаны некоторые популярные паттерны интеграции.



В книге «Паттерны интеграции корпоративных приложений»¹ Грегора Хопа и Бобби Вульфа представлена обширная информация о многих паттернах интеграции применительно к различным технологиям. При обсуждении этих паттернов данная книга упоминается чаще всего. Ее оригинал доступен по ссылке www.enterpriseintegrationpatterns.com.

Топология

При интеграции корпоративных приложений важно учитывать топологию систем, между которыми устанавливается связь. В частности, существует две отдельные топологии: веерная система и сервисная шина предприятия.

Топология «*веерная система*» (хаб) описывает паттерн интеграции, где один компонент, хаб, централизован и напрямую общается с каждым приложением. Такой подход централизует взаимодействие настолько, что хабу необходимо лишь знать о существовании других приложений, как показано на рис. 2.5.

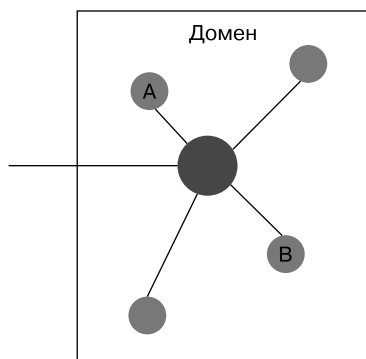


Рис. 2.5

¹ Хоп Г., Вульф Б. Шаблоны интеграции корпоративных приложений. — М.: Вильямс, 2016.

Хаб находится в центре. Он знает, как обращаться с различными приложениями. Это значит, что сообщение, переданное от *A* к *B*, отправляется из *A* в хаб, а затем перенаправляется к *B*. Преимущество данного подхода для бизнеса состоит в том, что соединение с *B* должно определяться и сопровождаться только в одном месте — в хабе. Важная особенность здесь заключается в том, что безопасность контролируется и сопровождается в едином центральном компоненте.

Сервисная шина предприятия (enterprise service bus, ESB) полагается на модель взаимодействия, состоящую из издателей и подписчиков (Pub-Sub). Издатель передает сообщения в шину, а подписчик регистрируется для получения сообщений от него. На рис. 2.6 показана эта топология.



Рис. 2.6

В данной схеме, если сообщение направлено от *A* к *B*, то *B* подписывается к шине для получения сообщений, опубликованных *A*. Когда *A* публикует новое сообщение, оно отправляется к *B*. На практике подписка может оказаться сложнее. Например, в системе запросов имеется два подписчика на приоритетные и обычные запросы. В этой ситуации приоритетные запросы могут обрабатываться одним образом, а обычные — другим.

Паттерны

Если мы определим интеграцию между двумя системами в виде последовательности шагов, то сможем определить паттерны в каждом шаге. Рассмотрим рис. 2.7 и обсудим конвейер интеграции.

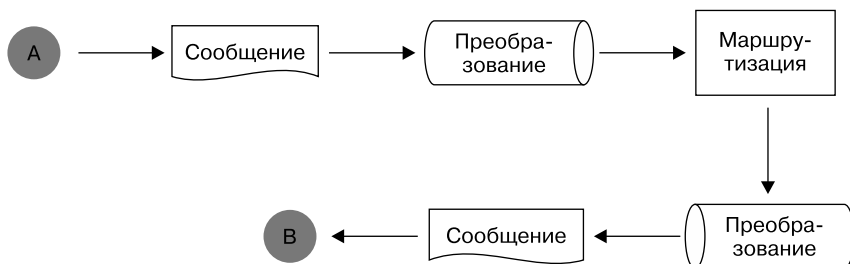


Рис. 2.7

Этот конвейер упрощен — в нем могло быть больше или меньше шагов, в зависимости от используемой технологии. Задача схемы — предоставить контекст при рассмотрении распространенных паттернов интеграции. Их же можно разделить на такие категории, как:

- ❑ *работа с сообщениями* — паттерны, относящиеся к управлению сообщениями;
- ❑ *преобразование* — паттерны, относящиеся к редактированию содержимого сообщений;
- ❑ *маршрутизация* — паттерны, относящиеся к обмену сообщениями.

Работа с сообщениями

Паттерны, относящиеся к работе с сообщениями, могут быть представлены в виде конструкции сообщений или каналов. Канал в этом контексте — конечная точка и/или то, как сообщение попадает на конвейер интеграции и выходит с него. Ниже представлены несколько примеров паттернов, относящихся к конструкции:

- ❑ *«Цепочка сообщений»* — сообщение содержит последовательность, указывающую на определенный порядок обработки сообщений;
- ❑ *«Идентификатор корреляции»* — сообщение содержит средство идентификации связанных сообщений;
- ❑ *«Обратный адрес»* — сообщение идентифицирует информацию об отправке ответного сообщения;
- ❑ *«Срок действия сообщения»* — сообщение актуально на протяжении ограниченного периода времени.

В пункте «Топология» выше мы рассмотрели паттерны, относящиеся к каналам, но следующие дополнительные паттерны тоже стоит взять на заметку при интеграции:

- ❑ *«Конкурирующие потребители»* — множественные процессы, которые могут работать с одним и тем же сообщением;
- ❑ *«Избирательный потребитель»* — потребители используют критерии для выбора сообщения, которое нужно обработать;
- ❑ *«Канал недоставленных сообщений»* — работает с сообщениями, которые не были успешно обработаны;
- ❑ *«Гарантированная доставка»* — обеспечивает надежное управление сообщениями без каких-либо потерь;
- ❑ *«Событийно-управляемый потребитель»* — управление сообщениями основано на опубликованных событиях;
- ❑ *«Опрашивающий потребитель»* — управляет сообщениями, напрямую полученными из системы-источника.

Преобразование

При интеграции сложных корпоративных систем паттерны преобразования обеспечивают гибкость управления сообщениями в системе. Преобразование позволяет изменять или дополнять сообщения между двумя приложениями. Ниже представлены несколько паттернов, относящихся к преобразованию:

- ❑ «Расширитель содержимого» — сообщение обогащается дополнительной информацией;
- ❑ «Каноническая модель» — сообщение преобразуется в независимый от приложения формат сообщений;
- ❑ «Транслятор сообщений» — паттерн для трансляции одного сообщения в другое.

Каноническая модель данных (canonical data model, CDM) заслуживает того, чтобы остановиться на ней отдельно. С помощью этого паттерна сообщение можно передать между множеством приложений, не приводя его к определенному типу. Лучше всего это показать на примере множества систем, обменивающихся сообщениями (рис. 2.8).

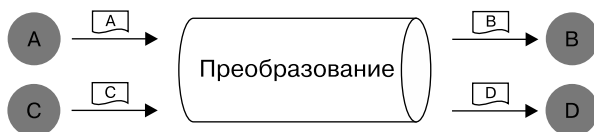


Рис. 2.8

В данной схеме приложения *A* и *C* хотят отправить свои сообщения в своем формате приложениям *B* и *D*. Если бы мы использовали паттерн «Транслятор сообщений», то лишь процессу, который управляет преобразованием, нужно было бы знать, как транслировать из *A* в *B* и из *A* в *D*, так же как из *C* в *B* и *C* в *D*. Эта конструкция усложняется с ростом количества приложений и при условии, что издатель не знает подробностей о подписчиках. С помощью канонической модели данных сообщения исходного приложения, направленные к *A* и *B*, транслируются в нейтральную схему *X*.



Каноническая схема

Каноническая схема (в некоторых источниках именуемая нейтральной) означает, что она не связана напрямую с исходной системой или системой назначения. Таким образом, эта схема подразумевается как «беспристрастная».

Сообщение в формате нейтральной схемы переводится в форматы сообщений для *B* и *D*, как показано на рис. 2.9.

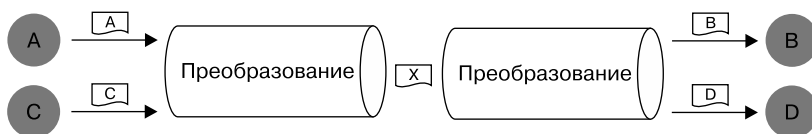


Рис. 2.9

В корпоративной среде этот механизм становится неуправляемым, если не соблюдать какие-либо стандарты. К счастью, во многих отраслях для разработки и урегулирования стандартов были созданы многочисленные организации. Вот лишь несколько примеров (на самом деле их намного больше!):

- ❑ «Электронный обмен данными в управлении, торговле и транспорте» (Electronic Data Interchange for Administration, Commerce and Transport, EDIFACT) — международный стандарт торговли;
- ❑ «Спецификация IMS “Унифицированные вопросы и тесты”» (Question and Test Interoperability, QTI) — стандарты для презентации оценки и результатов, проведенных информационной системой управления (Information Management System, IMS) глобального образовательного консорциума (Global Learning Consortium, GLC);
- ❑ «Стандарты интеграции технологий индустрии гостеприимства» (Hospitality Industry Technology Integration Standards, HITIS) — стандарты для управления системами собственности, поддерживаемые Американской ассоциацией отелей и мотелей;
- ❑ X12 EDI (X12) — коллекция схем для таких сфер, как здравоохранение, страхование, финансы, логистика и др. Поддерживается Уполномоченным комитетом по аккредитации X12;
- ❑ «Фреймворк корпоративного процесса» (eTOM) — модель обработки телекоммуникаций, поддерживаемая форумом ТМ.

Маршрутизация

Паттерны маршрутизации предлагают различные подходы к управлению сообщениями. Ниже представлены несколько примеров паттернов, попадающих в эту категорию:

- ❑ «Маршрутизация на основе контента» — маршрут или путь приложения определяется содержанием сообщения;
- ❑ «Фильтрация сообщений» — адресату (-ам) направляются только важные и необходимые сообщения;
- ❑ «Разветвитель» — генерация нескольких сообщений из одного;
- ❑ «Агрегатор» — одно сообщение генерируется из нескольких;
- ❑ «Рассылка — сборка» — паттерн для обработки рассылки множества сообщений и последующей агрегации ответов в одно сообщение.

Паттерн «Рассылка — сборка» весьма полезен и, так как включает в себе оба паттерна, «Разветвитель» и «Агрегатор», является просто отличным примером для исследования. С его помощью можно смоделировать более сложный корпоративный процесс.

Возьмем в качестве примера систему заказа виджетов. Итак, несколько поставщиков продают виджеты, но их цена слишком часто колеблется. Какой из поставщиков предлагает лучшую цену? Паттерн «Рассылка — сборка» позволяет системе обработки заказов опросить множество продавцов, выбрать лучшие цены и вернуть результат вызывающей системе. Мы используем паттерн «Разветвитель» для генерации нескольких сообщений продавцам, как показано на рис. 2.10.

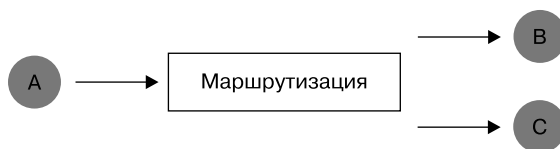


Рис. 2.10

Затем механизм маршрутизации ожидает ответа поставщика. Как только ответы получены, паттерн «Агрегатор» используется для объединения результатов в единое сообщение для вызывающей системы (рис. 2.11).

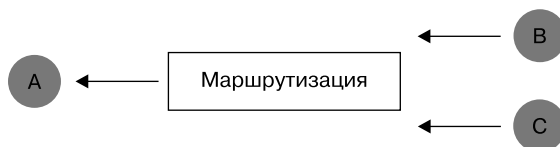


Рис. 2.11

Стоит заметить, что у этого паттерна существует множество разновидностей и сценариев применения. Например, он может требовать ответа от всех продавцов или некоторых из них или заставить ограничить время ожидания ответа длительностью обработки запроса. В одних случаях ответ на сообщения занимает лишь миллисекунды, в других — дни.



Движок интеграции — программное обеспечение, поддерживающее множество интеграционных паттернов. Это могут быть как локально установленные сервисы, так и облачные системы. Одними из самых популярных движков являются Microsoft BizTalk, Dell Boomi, MuleSoft Anypoint Platform, IBM WebSphere и SAS Business Intelligence.

Паттерны жизненного цикла разработки программного обеспечения

Существует множество подходов к управлению созданием ПО, и два самых главных паттерна жизненного цикла разработки ПО (software development life cycle, SDLC) — *Waterfall* и *Agile*. Имеется множество вариаций этих двух методологий, и компании часто адаптируют их к своим проектам и разработчикам, как и к своей внутренней культуре.



SDLC-паттерны *Waterfall* и *Agile* — лишь два примера, существует еще несколько паттернов разработки ПО, которые также могут подойти к культуре компании и индустрии лучше, чем другие.

Waterfall SDLC

Подход *Waterfall* состоит из определенных фаз, через которые постепенно проходят проект или его часть. Концептуально он проще в понимании и следует паттернам, используемым в других отраслях. Рассмотрим эти фазы:

- ❑ *фаза требований* — собираются и документируются все требования, которые необходимо реализовать;
- ❑ *фаза проектирования* — на основе документации, полученной на предыдущем шаге, создается проект, который должен быть реализован;
- ❑ *фаза разработки* — на основе проекта, созданного на предыдущем шаге, реализуются все изменения;
- ❑ *фаза тестирования* — изменения, реализованные на предыдущем шаге, проверяются на соответствие заявленным требованиям;
- ❑ *фаза развертывания системы* — система готовится к развертыванию, как только проведены все тесты и внесены изменения.

У модели *Waterfall* много преимуществ. Она проста для понимания и управления, поскольку каждая фаза имеет понятное определение того, что и когда должно быть сделано. Определив серию фаз, можно обозначить основные контрольные точки: так легче составить отчет по прогрессу. Вдобавок с четко прописанными фазами легче планировать и распределять необходимые роли и ресурсы.

Но если вдруг события развиваются не по плану или что-то меняется? *Waterfall SDLC* имеет свои минусы, и многие из них связаны с недостаточной устойчивостью к изменениям или с ситуациями, когда обнаруживается нечто новое, требующее информации из предыдущей фазы. В последнем случае предыдущую фазу

приходится повторять. Это влечет некоторые проблемы. Затрудняется создание отчетов о фазах, особенно после достижения контрольных точек: проект повторяет фазу заново. Это может привести компанию к культуре *охоты на ведьм*, когда все усилия направлены на поиск виноватого, а не на предотвращение подобных ситуаций. Кроме того, ресурсы могут быть недоступны, поскольку были перенаправлены на другие проекты и/или покинули компанию.

На рис. 2.12 показано, как меняется стоимость решения проблемы в зависимости от стадии, на которой она обнаружена.

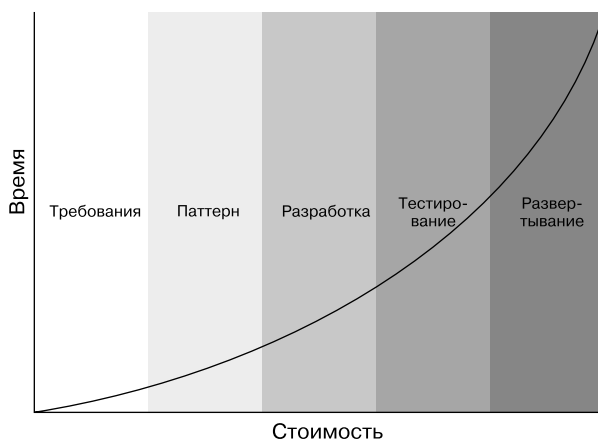


Рис. 2.12

Ввиду затрат, связанных с изменениями, Waterfall SDLC в основном применяется в проектах с малым риском изменений. Более крупные и сложные проекты увеличивают потенциал для изменений, так как требования могут варьироваться и всегда есть риск, что бизнес изменит ключевые направления деятельности еще на этапе проекта.

Agile SDLC

При подходе Agile SDLC в разработке программного обеспечения пытаются принять во внимание любые изменения и неточности. Это достигается с помощью паттерна, который позволяет изменения и/или допускает появление проблем во время жизненного цикла проекта или продукта. Ключевой принцип — разбиение проекта на малые итерации, чаще называемые циклами разработки. Базовые стадии Waterfall в каждом цикле Agile повторяются так, что он имеет свои фазы требований, проектирования, разработки, тестирования и развертывания.

Это упрощение, но стратегия разделения проекта на циклы имеет некоторые преимущества перед Waterfall:

- ❑ влияние изменений требований бизнеса уменьшается по мере того, как уменьшается сфера применения;
- ❑ заинтересованные стороны получают обозримую рабочую систему раньше, чем при Waterfall. Может, и не совсем полноценную, но это позволяет получать обратную связь на ранней стадии развития продукта;
- ❑ выделение ресурсов упрощается.

На рис. 2.13 показана сводка Agile и Waterfall.

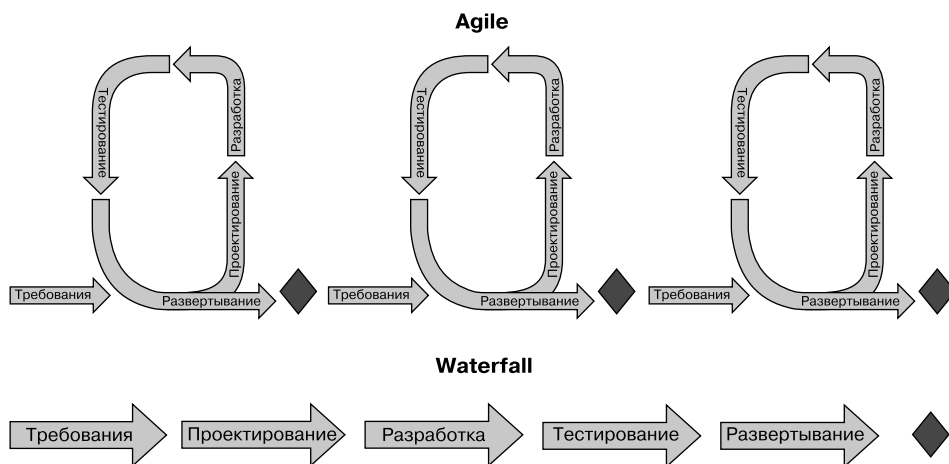


Рис. 2.13

Резюме

В этой главе мы обсудили важнейшие паттерны проектирования в современной разработке программного обеспечения, которые были представлены в предыдущей главе. Мы начали с обсуждения различных принципов создания ПО, таких как DRY, KISS, YAGNI, MVP и SOLID. Затем рассмотрели паттерны разработки программного обеспечения, в том числе паттерны «Банды четырех» и EIP, а также обсудили методологию SDLC, включающую Waterfall и Agile. Цель данной главы заключалась в том, чтобы показать, как паттерны используются на всех уровнях разработки программного обеспечения.

По мере развития индустрии ПО появляются новые паттерны как следствие эволюции опыта, техники и технологии. Некоторые паттерны были разработаны в помощь различным фазам SDLC. Например, в главе 3 мы рассмотрим технику «разработка через тестирование» (test-driven development, TDD), при которой тесты используются для определения прогресса относительно изложенных требований во время фазы разработки. По мере продвижения по главам мы обсудим

более высокие уровни абстракции в создании программного обеспечения, включая паттерны для веб-разработки, а также современные архитектурные паттерны в локальных и облачных средах.

Следующую главу начнем с построения вымышленного приложения на .NET Core. Кроме того, объясним различные паттерны, используемые в этой главе, включая принципы программирования SOLID, а также продемонстрируем несколько паттернов «Банды четырех».

Вопросы

Следующие вопросы помогут вам усвоить информацию, приведенную в этой главе.

1. Что означает буква S в аббревиатуре SOLID? Что подразумевается под ответственностью?
2. Какой метод основан на циклах — Waterfall или Agile?
3. Паттерн «Декоратор» порождающий или структурный?
4. За что отвечает паттерн интеграции «Издатель — подписчик»?

Часть II

Углубленное изучение утилит и паттернов .NET Core

В этой части вы получите практический опыт работы с различными паттернами проектирования. Конкретные паттерны будут рассмотрены в процессе построения приложения, используемого для поддержки программы инвентаризации. Мы выбрали приложение для инвентаризации, поскольку оно простое по своей сути, но при этом достаточно сложное для того, чтобы вы научились применять паттерны во время разработки. К некоторым паттернам мы будем возвращаться снова и снова; это относится к SOLID, *продукту с минимальным функционалом* (MVP) и *разработке через тестирование* (TDD). К концу части вы сможете писать аккуратный и понятный код с помощью разных паттернов.

Часть II состоит из таких глав:

- ❑ глава 3 «Реализация паттернов проектирования — основы (часть 1)»;
- ❑ глава 4 «Реализация паттернов проектирования — основы (часть 2)»;
- ❑ глава 5 «Реализация паттернов проектирования в .NET Core»;
- ❑ глава 6 «Реализация паттернов проектирования для веб-приложений (часть 1)»;
- ❑ глава 7 «Реализация паттернов проектирования для веб-приложений (часть 2)».

3 Реализация паттернов проектирования — основы (часть 1)

В предыдущих главах мы представили и определили широкий спектр современных паттернов и методик, относящихся к *жизненному циклу разработки ПО* (software development life cycle, SDLC), от низкоуровневых паттернов разработки до архитектурных паттернов высокоуровневых решений. В данной главе некоторые из этих паттернов применяются в демонстрационном сценарии с целью обеспечить контекст и более детальное понимание определений. Сценарий заключается в создании решения для инвентаризации книжного онлайн-магазина.

Мы выбрали именно этот сценарий, поскольку он достаточно сложен, чтобы можно было описать паттерны, тогда как принцип сам по себе достаточно прост. Компании нужно как-то управлять товарами и давать пользователям возможность заказывать ее продукцию. Как можно скорее необходимы работающее приложение для инвентаризации, а также многие дополнительные функции, включающие в себя доступ покупателей к заказу продукции и возможность написания отзывов. Количество требуемых функций растет, и в какой-то момент разработчики уже не знают, с чего начать. К счастью, опираясь на общепринятые рекомендации, которые помогают управлять ожиданиями и требованиями, разработчики получают возможность упростить поставку продукта и вернуться к процессу проектирования. Кроме того, с помощью паттернов разработчики способны построить надежный фундамент для возможного расширения функционала в будущем.

Эта глава будет посвящена началу работы над новым проектом и созданию первого релиза приложения. Будут рассмотрены:

- ❑ *продукт с минимальным функционалом (MVP)*;
- ❑ *разработка через тестирование (TDD)*;
- ❑ паттерн «Абстрактная фабрика» («Банда четырех»);
- ❑ принципы SOLID.

Технические требования

В этой главе приводятся примеры кода, объясняющие принципы качественной разработки. Код упрощен и предназначен лишь для демонстрации. Большинство примеров основаны на консольном приложении .NET Core, написанном на C#.

Чтобы запустить и выполнить код, вам потребуется следующее:

- ❑ Visual Studio 2019 (вы также можете запустить приложение, используя Visual Studio 2017 версии 3 и новее);
- ❑ .NET Core;
- ❑ SQL Server (в этой главе используется Express Edition).

Установка Visual Studio

Чтобы запустить примеры кода, вам необходимо установить Visual Studio. Для этого выполните следующие шаги.

1. Скачайте Visual Studio по ссылке docs.microsoft.com/ru-ru/visualstudio/install/install-visual-studio.
2. Следуйте инструкциям по установке. Доступны различные версии Visual Studio; в этой главе мы используем Visual Studio для Windows.

Вы также можете применить другую интегрированную среду разработки по вашему усмотрению.

Установка .NET Core

Если вы еще не установили .NET Core, то следуйте инструкции ниже.

1. Скачайте .NET Core по ссылке dotnet.microsoft.com/download.
2. Далее следуйте указаниям по установке соответствующей библиотеки: dotnet.microsoft.com/download/dotnet-core/2.2.



Полная версия исходного кода доступна на GitHub. Код, представленный в данной главе, может быть неполным, поэтому мы рекомендуем вам скачать полный код для запуска данных примеров (github.com/PacktPublishing/Hands-On-Design-Patterns-with-C-and-.NET-Core/tree/master/Chapter3).

Продукт с минимальным функционалом

Данный раздел описывает начальную фазу проекта по созданию программного приложения. Ее иногда называют стартом или запуском проекта; здесь планируются

начальные функции и возможности приложения (другими словами, происходит сбор требований).



Определить функции программного приложения позволяет множество подходов, которые можно рассматривать как паттерны. Рекомендации по эффективному моделированию, ведению интервью и практикумов, мозговой штурм и другие методики выходят за рамки этой книги. Вместо этого здесь описывается один подход: продукт с минимальным функционалом как пример того, что могут содержать паттерны.

Представим гипотетическую ситуацию: компания *FlixOne* хочет использовать приложение для работы с возрастающей коллекцией книг. Персонал будет применять его для управления книгами, а клиенты — для формирования новых заказов. Приложение должно быть масштабируемым, и его планируется использовать в обозримом будущем в качестве важной корпоративной системы.

Компания подразделяется на *бизнес-пользователей* и *команду разработчиков*, где первых прежде всего заботит функциональность системы, а вторых — удовлетворение требований, а также сопровождаемость системы. Это упрощенное описание; в реальности компании не всегда организованы подобным образом и отдельные люди могут выбиваться из той или иной классификации. Например, *бизнес-аналитик* (business analyst, BA) или *профильный эксперт* (subject matter expert, SME) зачастую могут представлять как корпоративного пользователя, так и члена команды разработки.

Поскольку эта книга по большей части техническая, нас в основном интересует взгляд со стороны команды разработки; мы обсудим паттерны и практики, используемые в реализации приложения для инвентаризации.

Требования

На протяжении нескольких встреч бизнес-пользователи и разработчики обсуждали требования к новой системе управления. Прогресс в определении четкого набора требований был медленным, дальнейшее видение продукта — неясным. Команда разработки решила сократить огромный список требований до минимального функционала, достаточного для того, чтобы отдельный пользователь мог начать работу. Такой подход позволит в минимальной степени заниматься инвентаризацией и предоставить фундамент для дальнейшего развития. Все последующие требования могут быть добавлены к начальному релизу.



Продукт с минимальным функционалом (Minimum Viable Product, MVP)

Продукт с минимальным функционалом — это приложение с минимальным набором функций, допущенное к релизу и имеющее основные возможности, достаточные для пользователя.

Превосходство этого подхода заключается в следующем: обе команды получают упрощенное видение того, что необходимо добавить в продукт, сокращая объем приложения. Уменьшение количества предоставляемых функций позволит команде сфокусироваться на определении дальнейших действий. В случае с FlixOne ценность встречи часто сводится к обсуждению деталей функции, которая, может, и важна для финальной версии продукта, но потребует добавления нескольких сопряженных с ней функций перед релизом. К примеру, команда занималась дизайном сайта и его пользовательского интерфейса, вместо того чтобы сфокусироваться на данных, хранящихся в системе инвентаризации.



MVP очень полезен в ситуациях, когда неясна сложность требований. Или нет видения конечного продукта. Но все же важно поддерживать видение желаемого продукта, чтобы не разработать в итоге функционал, ненужный для финальной версии продукта.

Бизнес-пользователи и разработчики определили следующие требования для начального приложения для инвентаризации.

- ❑ Приложение должно быть консольным:
 - должно выдавать сообщение приветствия с указанием версии сборки;
 - должно работать в цикле до поступления команды выхода;
 - если подобная команда не была выполнена или была неясна, то должно быть выдано сообщение с подсказками.
- ❑ Приложение должно откликаться на простые текстовые команды, не чувствительные к регистру.
- ❑ Каждая команда должна иметь короткую (односимвольную) и длинную формы.
- ❑ Если команда имеет дополнительные параметры, то:
 - каждая должна быть введена и подтверждена клавишей ввода;
 - каждая должна иметь строку ввода вида `Enter {parameter}:`, где `{parameter}` — имя параметра.
- ❑ Вспомогательная команда (?) также должна быть доступна для:
 - вывода списка команд;
 - вывода примера использования каждой из команд.
- ❑ Команда выхода (`q, quit`) должна быть доступна для:
 - вывода прощального сообщения;
 - завершения приложения.
- ❑ Команда добавления товаров (`a, addinventory`) должна быть доступна:
 - со строковым параметром `name`;
 - для добавления записи в базу данных с указанным названием и нулевым количеством.

- ❑ Команда обновления количества (`u, updatequantity`) должна быть доступна:
 - с параметром `name` в виде строки;
 - с параметром `quantity` с положительным или отрицательным целочисленным значением;
 - для обновления значения количества экземпляров книги с данным именем путем добавления этого значения.
- ❑ Команда получения товаров (`g, getinventory`) должна быть доступна для возвращения всех книг и их количества из базы данных.

Вдобавок были обозначены нефункциональные требования:

- ❑ никакой другой системы безопасности, кроме уже встроенной в операционную систему;
- ❑ краткая форма команд нужна для удобства использования, в то время как длинная — для удобства чтения.

Пример FlixOne — иллюстрация того, как MVP может использоваться для фокусирования и направления SDLC. Стоит отметить, что *проверка концепции* (proof of concept, PoC) и MVP в различных организациях будут различаться. В данной книге PoC отличается от MVP в том, что фактическое приложение не рассматривается как неполноценное. Для коммерческого продукта это означает, что конечный продукт может быть продан, а в случае внутреннего применения в бизнесе такое приложение могло бы принести пользу организации.

Каковы преимущества MVP в дальнейшей разработке

Очередное преимущество MVP с точки зрения фокусирования и ограничения требований заключается в его синергии с разработкой приложений по Agile. Разбиение циклов разработки на меньшие — это технология разработки программного обеспечения, ставшая более популярной по сравнению с традиционной разработкой методом Waterfall. Смысл принципа в том, что требования и решения развиваются на протяжении жизненного цикла приложения. Они включают в себя взаимодействие разработчиков и конечных пользователей. Обычно фреймворк разработки приложений Agile имеет короткий цикл релиза, в котором проектируется, разрабатывается, тестируется и выпускается новый функционал. Циклы релиза затем повторяются по мере включения в приложение новых функций. MVP весьма уместен в разработке Agile, когда спектр задач находится в рамках цикла релиза.



Scrum и Kanban — популярные фреймворки разработки приложений, основанные на Agile.

Спектр начальных требований MVP уменьшен: их достаточно для проектирования, разработки, тестирования и выпуска приложения за один цикл Agile. В следующем цикле добавятся новые требования к приложению. Главная цель — вместить множество новых функций в рамки одного цикла. Каждый новый релиз функционала ограничен естественными требованиями или его MVP. Дело в том, что благодаря итеративному подходу к разработке финальная версия приложения будет более полезной для конечного пользователя, чем выпуск приложения за один релиз, в котором все требования пришлось бы определять заранее.

На рис. 3.1 обобщены различия между методами разработки программного обеспечения Agile и Waterfall.

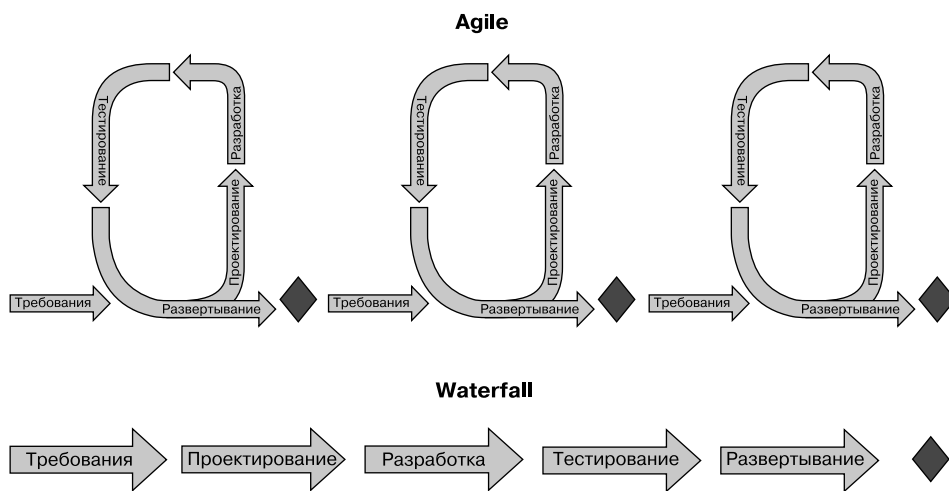


Рис. 3.1

Разработка через тестирование

Существуют различные подходы к *разработке через тестирование* (test-driven development, TDD), и смысл *теста* может варьироваться от модульных тестов, запускающихся по требованию во время разработки, модульных тестов, запускаемых во время сборки проекта, до тестовых скриптов, запускаемых при *приемочном пользовательском тестировании* (user acceptance testing, UAT). *Тест* может быть кодом или документом, описывающим необходимые шаги, выполняемые пользователем для подтверждения требования. Причина — различное видение того, что должно быть достигнуто с помощью TDD. Для одних разработчиков TDD — это методика доработки требований перед написанием кода, для других — способ оценки качества или проверки уже готового кода.

**UAT**

UAT — термин, применяемый для обозначения деятельности во время SDLC, при которой продукт верифицируется (проверяется на соответствие заявленным требованиям). Обычно UAT производится членами компании или специально отобранными пользователями. В зависимости от обстоятельств эта фаза может быть разбита на стадии альфа и бета, где альфа-тестирование проводится разработчиками, а бета — конечными пользователями.

Почему разработчики выбирают TDD

Разработчики решили использовать TDD по нескольким причинам. Во-первых, они стремились найти способ четко измерять прогресс во время разработки. Во-вторых, хотели иметь возможность повторять пройденные тесты в последующих циклах, чтобы проверять имеющийся функционал при добавлении новых свойств. По этим причинам разработчики будут применять модульные тесты для проверки того, удовлетворяет ли написанный функционал требованиям.

На рис. 3.2 показаны основы TDD.

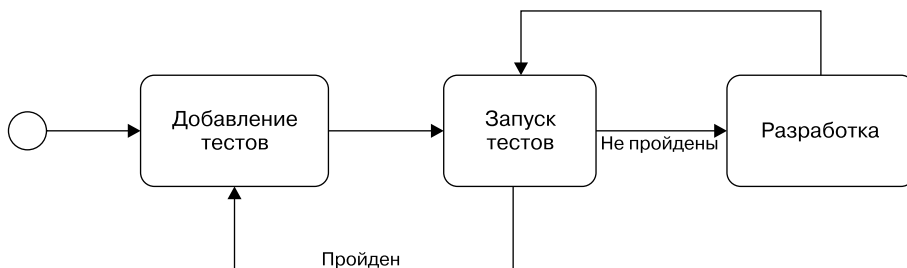


Рис. 3.2

Тесты добавляются и кодовая база обновляется до тех пор, пока они не будут пройдены. Важно заметить, что это циклический процесс. В каждой итерации добавляются новые тесты, и они не считаются успешными до тех пор, пока и новые, и существующие тесты не будут пройдены успешно.

Разработчики FlixOne решили объединить модульные тесты и UAT в один цикл Agile. В начале каждого цикла определяются новые критерии приемки. Сюда входит необходимый новый функционал, который также будет проверен и допущен в конце цикла разработки. Эти критерии в дальнейшем могут быть использованы для добавления тестов в проект. Разработчики будут создавать решение до тех пор, пока все новые и существующие тесты не будут пройдены, а затем подготовят сборку для приемочного тестирования. При нахождении каких-либо замечаний после его запуска разработчики напишут новые тесты либо откорректируют имеющиеся в зависимости от найденных проблем. Приложение будет разрабатываться до тех

пор, пока не пройдут все тесты и не будет готова новая сборка. Это повторяется до тех пор, пока не будет пройдено приемочное тестирование. Затем приложение переходит на этап развертывания, и начинается новый цикл.

На рис. 3.3 показан этот подход.

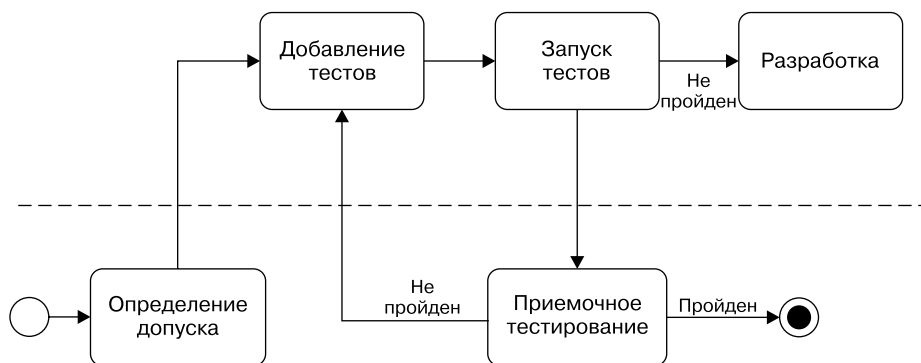


Рис. 3.3

Теперь у нас есть план, поэтому начнем писать код!

Настройка проектов

В данном сценарии мы используем фреймворк *модульных тестов Microsoft (MSTest)*. Данный подраздел содержит некоторые инструкции по созданию начального проекта с помощью интерфейса *командной строки* .NET Core (command-line interface, CLI). Эти шаги можно пройти, используя *интегрированную среду разработки* (integrated development environment, IDE), такую как Visual Studio или Visual Studio Code. Представленные здесь инструкции показывают, как можно использовать CLI вместе с IDE.



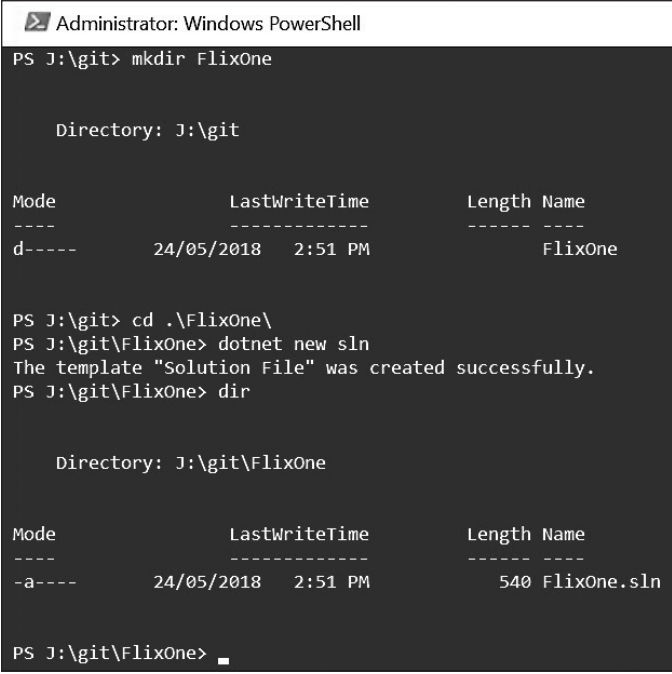
CLI

Инструменты .NET Core CLI — это кросс-платформенные утилиты для разработки .NET Core приложений. Они являются фундаментом для более сложных инструментов, таких как IDE. Пожалуйста, ознакомьтесь с документацией, чтобы узнать подробности: docs.microsoft.com/ru-ru/dotnet/core/tools/.

Решение, представленное в этой главе, состоит из трех проектов: консольного приложения, библиотеки классов и тестового проекта. Создадим рабочий каталог, `FlixOne`, и подкаталоги для трех наших проектов. Следующая команда создаст новый файл решения в текущем каталоге:

```
dotnet new sln
```

На рис. 3.4 показано создание каталога и решения (примечание: мы создали пока только пустой файл решения).



```

Administrator: Windows PowerShell

PS J:\git> mkdir FlixOne

Directory: J:\git

Mode                LastWriteTime         Length Name
----                -
d-----          24/05/2018   2:51 PM             FlixOne

PS J:\git> cd .\FlixOne\
PS J:\git\FlixOne> dotnet new sln
The template "Solution File" was created successfully.
PS J:\git\FlixOne> dir

Directory: J:\git\FlixOne

Mode                LastWriteTime         Length Name
----                -
-a----          24/05/2018   2:51 PM           540 FlixOne.sln

PS J:\git\FlixOne>
  
```

Рис. 3.4

Библиотека классов `FlixOne.InventoryManagement` будет содержать бизнес-сущности и бизнес-логику. В следующих главах мы разделим их на библиотеки, но пока наше приложение остается маленьким и упрощенным, оно будет содержаться в единственной сборке. Ниже показана консольная команда `dotnet` для создания проекта:

`dotnet new classlib --name FlixOne.InventoryManagement`

Обратите внимание на рис. 3.5: в этом новом каталоге содержится новый файл проекта библиотеки классов.

```

PS J:\git\FlixOne> dotnet new classlib --name FlixOne.InventoryManagement
The template "Class library" was created successfully.

Processing post-creation actions...
Running 'dotnet restore' on FlixOne.InventoryManagement\FlixOne.InventoryManagement.csproj...
  Restoring packages for J:\git\FlixOne\FlixOne.InventoryManagement\FlixOne.InventoryManagement.csproj...
  Generating MSBuild file J:\git\FlixOne\FlixOne.InventoryManagement\obj\FlixOne.InventoryManagement.csproj.nuget.g.props.
  Generating MSBuild file J:\git\FlixOne\FlixOne.InventoryManagement\obj\FlixOne.InventoryManagement.csproj.nuget.g.targets.
  Restore completed in 113.91 ms for J:\git\FlixOne\FlixOne.InventoryManagement\FlixOne.InventoryManagement.csproj.

Restore succeeded.
  
```

Рис. 3.5

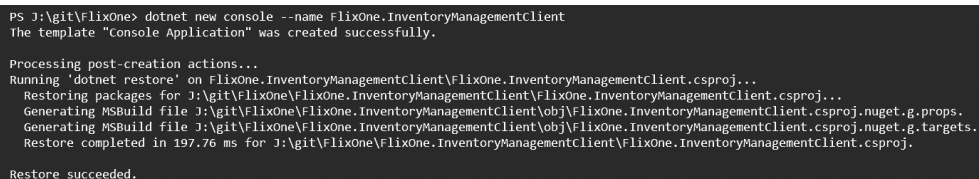
Необходимо установить ссылки от решения к новой библиотеке классов с помощью следующей команды:

```
dotnet sln add
..\FlixOne.InventoryManagement\FlixOne.InventoryManagement.csproj
```

Для создания нового проекта консольного приложения используется такая команда:

```
dotnet new console --name FlixOne.InventoryManagementClient
```

На рис. 3.6 видно, как разворачивается шаблон console.



```
PS J:\git\FlixOne> dotnet new console --name FlixOne.InventoryManagementClient
The template "Console Application" was created successfully.

Processing post-creation actions...
Running 'dotnet restore' on FlixOne.InventoryManagementClient\FlixOne.InventoryManagementClient.csproj...
  Restoring packages for J:\git\FlixOne\FlixOne.InventoryManagementClient\FlixOne.InventoryManagementClient.csproj...
  Generating MSBuild file J:\git\FlixOne\FlixOne.InventoryManagementClient\obj\FlixOne.InventoryManagementClient.csproj.nuget.g.props.
  Generating MSBuild file J:\git\FlixOne\FlixOne.InventoryManagementClient\obj\FlixOne.InventoryManagementClient.csproj.nuget.g.targets.
  Restore completed in 197.76 ms for J:\git\FlixOne\FlixOne.InventoryManagementClient\FlixOne.InventoryManagementClient.csproj.

Restore succeeded.
```

Рис. 3.6

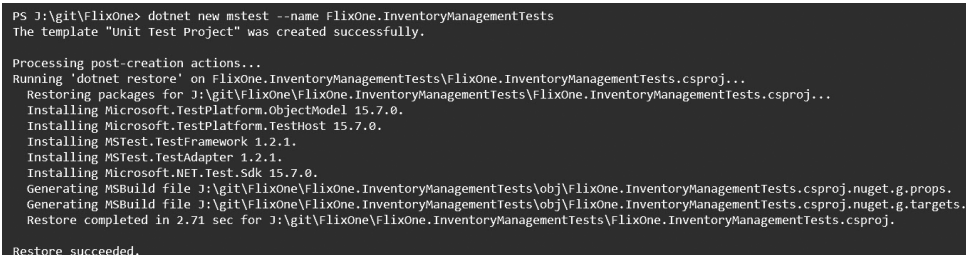
Консольное приложение также требует ссылки на библиотеку классов (замечание: команда должна быть запущена в каталоге проектного файла и содержать ссылку на него):

```
dotnet add reference
..\FlixOne.InventoryManagement\FlixOne.InventoryManagement.csproj
```

Новый проект *MSTest* будет создан с помощью такой команды:

```
dotnet new mstest --name FlixOne.InventoryManagementTests
```

На следующем снимке экрана показано создание проекта *MSTest*, который должен быть запущен в той же папке, где и решение: *FlixOne* (обратите внимание на пакеты, восстановленные в рамках команды и содержащие NuGet-пакеты, требуемые для *MSTest*) (рис. 3.7).



```
PS J:\git\FlixOne> dotnet new mstest --name FlixOne.InventoryManagementTests
The template "Unit Test Project" was created successfully.

Processing post-creation actions...
Running 'dotnet restore' on FlixOne.InventoryManagementTests\FlixOne.InventoryManagementTests.csproj...
  Restoring packages for J:\git\FlixOne\FlixOne.InventoryManagementTests\FlixOne.InventoryManagementTests.csproj...
  Installing Microsoft.TestPlatform.ObjectModel 15.7.0.
  Installing Microsoft.TestPlatform.TestHost 15.7.0.
  Installing MSTest.TestFramework 1.2.1.
  Installing MSTest.TestAdapter 1.2.1.
  Installing Microsoft.NET.Test.Sdk 15.7.0.
  Generating MSBuild file J:\git\FlixOne\FlixOne.InventoryManagementTests\obj\FlixOne.InventoryManagementTests.csproj.nuget.g.props.
  Generating MSBuild file J:\git\FlixOne\FlixOne.InventoryManagementTests\obj\FlixOne.InventoryManagementTests.csproj.nuget.g.targets.
  Restore completed in 2.71 sec for J:\git\FlixOne\FlixOne.InventoryManagementTests\FlixOne.InventoryManagementTests.csproj.

Restore succeeded.
```

Рис. 3.7

Тестовый проект также требует ссылку на библиотеку классов (эта команда должна быть запущена в том же каталоге, где находится проектный файл MSTest):

```
dotnet add reference
```

```
..\FlixOne.InventoryManagement\FlixOne.InventoryManagement.csproj
```

И наконец, приложение и проект MSTest должны быть добавлены к решению с помощью следующих команд в каталоге с файлом решения:

```
dotnet sln add
```

```
.\FlixOne.InventoryManagementClient\FlixOne.InventoryManagementClient.csproj
```

```
dotnet sln add
```

```
.\FlixOne.InventoryManagementTests\FlixOne.InventoryManagementTests.csproj
```

Решение выглядит так (рис. 3.8).

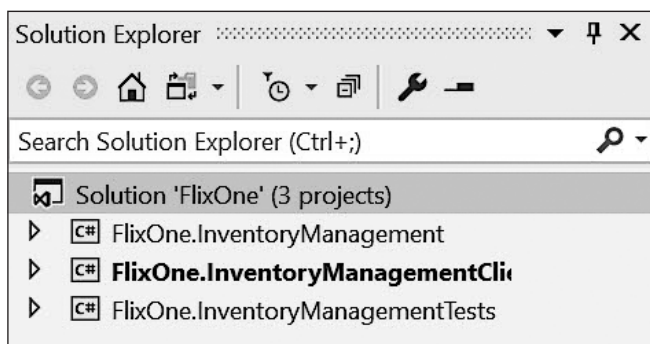


Рис. 3.8

Теперь, когда начальная структура решения готова, начнем с определения наших модульных тестов.

Определения начальных модульных тестов

Для начала разработчики преобразовали требования в некоторые базовые модульные тесты. Так как толком еще ничего не было спроектировано и написано, они не предоставляют проверку функционала. В ходе проектирования и разработки эти тесты будут совершенствоваться. Например, есть требование о добавлении товаров.



Доступна команда добавления товаров (а, addinventory):

- строковый параметр name;
- добавление записи в базу данных с заданным названием и нулевым количеством.

Для выполнения этого требования разработчики создали следующий модульный тест, служащий заглушкой:

```
[TestMethod]
private void AddInventoryCommand_Successful()
{
    // создаем экземпляр команды
    // добавляем новую книгу с параметром "name"
    // проверяем, была ли добавлена книга с данным названием и количеством 0

    Assert.Inconclusive("AddInventoryCommand_Successful has not been
    implemented.");
}
```

Когда архитектура приложения уже известна, а разработка началась, имеющиеся тесты расширяются и добавляются новые (рис. 3.9).

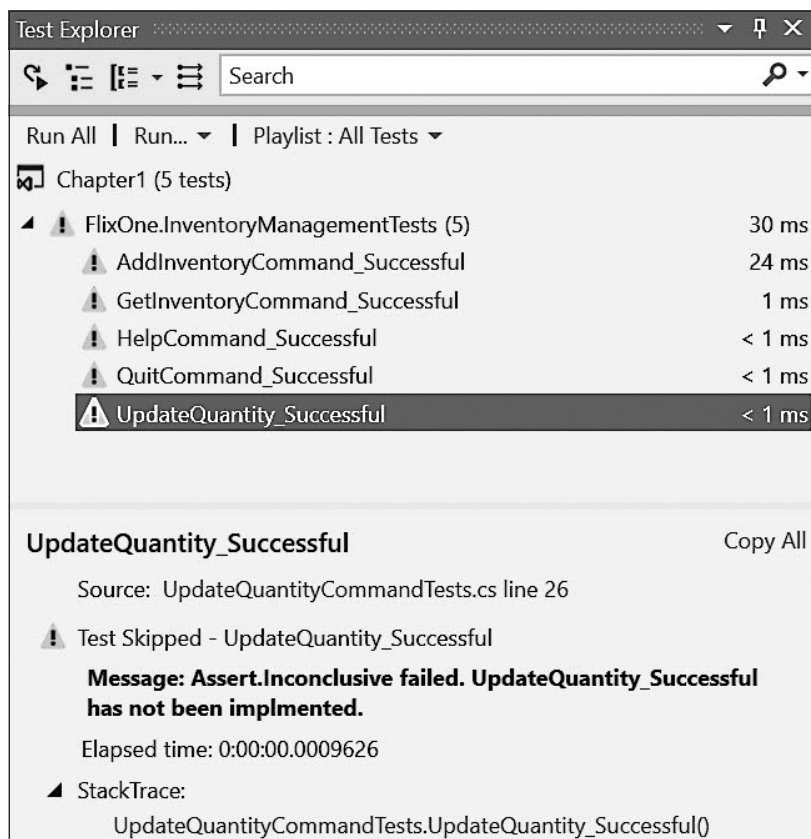


Рис. 3.9

Важность незавершенных тестов заключается в определении целей, которых должна достичь команда, а также в измерении прогресса разработки. Со временем незавершенные и проваленные тесты будут показывать, что необходимо доработать, а все успешные — сигнализировать о прогрессе выполнения текущих задач.

Паттерн проектирования «Абстрактная фабрика»

Чтобы показать вам наш первый паттерн, рассмотрим разработку команды `help` и начального консольного приложения. Первая версия приложения выглядит так:

```
private static void Main(string[] args)
{
    Greeting();
    // примечание: встроенная выходная переменная, введенная в C# 7.0
    GetCommand("?").RunCommand(out bool shouldQuit);

    while (!shouldQuit)
    {
        // обрабатываем команды
        ...
    }

    Console.WriteLine("CatalogService has completed.");
}
```

Когда приложение запущено, мы можем наблюдать как приветственное сообщение, так и результат вывода команды `help`. Затем приложение будет обрабатывать введенные команды до поступления команды выхода.

Следующий код занимается обработкой команд:

```
while (!shouldQuit)
{
    Console.WriteLine(" > ");
    var input = Console.ReadLine();
    var command = GetCommand(input);

    var wasSuccessful = command.RunCommand(out shouldQuit);

    if (!wasSuccessful)
    {
        Console.WriteLine("Enter ? to view options.");
    }
}
```

До тех пор пока приложение не закрыто, оно будет предлагать пользователю ввести команду. И если она не обработана должным образом, то отобразит справочный текст.

**RunCommand(out bool shouldQuit)**

C# 7.0 предоставляет более гибкий синтаксис для создания параметров `out`. Данный параметр объявляет переменные в области видимости блока команд. Это показано ниже: логическое выражение `shouldQuit` не объявлено заранее.

Абстрактный класс `InventoryCommand`. Для начала стоит отметить тот факт, что в этом начальном консольном приложении разработчики используют *объектно-ориентированное программирование (ООП)* для создания стандартного механизма обработки команд. В ходе создания этой начальной версии разработчики поняли, что все команды будут содержать метод `RunCommand()`, который станет возвращать две булевы переменные, сигнализирующие об успешности выполнения команды и о том, должна ли программа завершить свою работу. Например, команда `HelpCommand()` просто отобразит справочное сообщение в консоли и не вызовет завершения программы. Значение обоих логических переменных будет `true` для обозначения успешности обработки и `false` для обозначения того, что приложение не нужно завершать. Ниже показана начальная версия.



Троееточие (...) обозначает дополнительные инструкции, а в этом конкретном примере — дополнительные вызовы `Console.WriteLine()`.

```
public class HelpCommand
{
    public bool RunCommand(out bool shouldQuit)
    {
        Console.WriteLine("USAGE:");
        Console.WriteLine("\taddinventory (a)");
        ...
        Console.WriteLine("Examples:");
        ...

        shouldQuit = false;
        return true;
    }
}
```

Класс `QuitCommand` отображает сообщение и завершает программу. Так выглядит его первая версия:

```
public class QuitCommand
{
    public bool RunCommand(out bool shouldQuit)
    {
        Console.WriteLine("Thank you for using FlixOne Inventory
            Management System");
    }
}
```

```

        shouldQuit = true;
        return true;
    }
}

```

Разработчики решили создать либо интерфейс, реализуемый обоими классами, либо абстрактный класс, от которого эти классы могут наследоваться. Оба варианта могут помочь достичь динамического полиморфизма, но разработчики решают использовать абстрактный класс, так как все команды будут иметь общий функционал.



В ООП и, в частности, в С# полиморфизм реализуется тремя способами: перегрузкой функций, использованием дженериков, а также за счет подтипов (динамический полиморфизм).

Используя паттерн проектирования «Абстрактная фабрика», разработчики создали абстрактный класс для наследования команд, `InventoryCommand`. Класс `InventoryCommand` имеет один метод, `RunCommand`, который выполняет команду и возвращает значение того, была ли она успешно выполнена и необходимо ли завершить программу. Класс является абстрактным, это значит, что он содержит один абстрактный метод или более. В данном случае абстрактным выступает метод `InternalCommand()`. Цель состоит в том, что классы, наследующие от класса `InventoryCommand`, реализуют метод `InternalCommand` со специфичным функционалом. Например, `QuitCommand` расширяет `InventoryCommand` и предоставляет конкретную реализацию метода `InternalCommand()`. Следующий фрагмент кода показывает абстрактный класс `InventoryCommand` вместе с абстрактным методом `InternalCommand()`:

```

public abstract class InventoryCommand
{
    private readonly bool _isTerminatingCommand;
    internal InventoryCommand(bool commandIsTerminating)
    {
        _isTerminatingCommand = commandIsTerminating;
    }
    public bool RunCommand(out bool shouldQuit)
    {
        shouldQuit = _isTerminatingCommand;
        return InternalCommand();
    }

    internal abstract bool InternalCommand();
}

```

Абстрактный метод затем реализуется в каждом производном классе, как показано в `HelpCommand`. Этот класс просто выводит немного информации в консоль и возвращает `true`, показывая, что команда успешно выполнена:

```
public class HelpCommand : InventoryCommand
{
    public HelpCommand() : base(false) { }

    internal override bool InternalCommand()
    {
        Console.WriteLine("USAGE:");
        Console.WriteLine("\taddinventory (a)");
        ...
        Console.WriteLine("Examples:");
        ...
        return true;
    }
}
```

Разработчики решают внести два дополнительных изменения в `InventoryCommand`. Сначала им не понравилось, что булева переменная `shouldQuit` возвращается в виде переменной `out`. Поэтому они решили использовать новую функцию кортежей из C#7, чтобы вернуть один объект, `Tuple<bool, bool>`:

```
public (bool wasSuccessful, bool shouldQuit) RunCommand()
{
    /* добавленный код скрыт */

    return (InternalCommand(), _isTerminatingCommand);
}
```



Кортеж

Кортеж — это тип C#, предоставляющий простой синтаксис для упаковки множественных значений в одиночный объект. Минус подхода — потеря возможности наследования и других объектно-ориентированных функций. Подробную информацию см. на docs.microsoft.com/ru-ru/dotnet/csharp/tuples.

Второе изменение — объявление другого абстрактного класса для определения того, является ли команда непрерывающей, то есть командой, которая не вызывает завершения или закрытия программы.

Как показано в следующем коде, эта команда по-прежнему абстрактная: она не реализует метод `InternalCommand` класса `InventoryCommand`, но возвращает значение `false` для базового класса:

```
internal abstract class NonTerminatingCommand : InventoryCommand
{
    protected NonTerminatingCommand() : base(commandIsTerminating: false)
    {
    }
}
```

Преимущество решения в том, что теперь команды, которые не завершают приложение (другими словами, непрерывающие), имеют упрощенное определение:

```
internal class HelpCommand : NonTerminatingCommand
{
    internal override bool InternalCommand()
    {
        Interface.WriteMessage("USAGE:");
        /* остальной код скрыт */

        return true;
    }
}
```

На рис. 3.10 показано наследование абстрактного класса `InventoryCommand`:

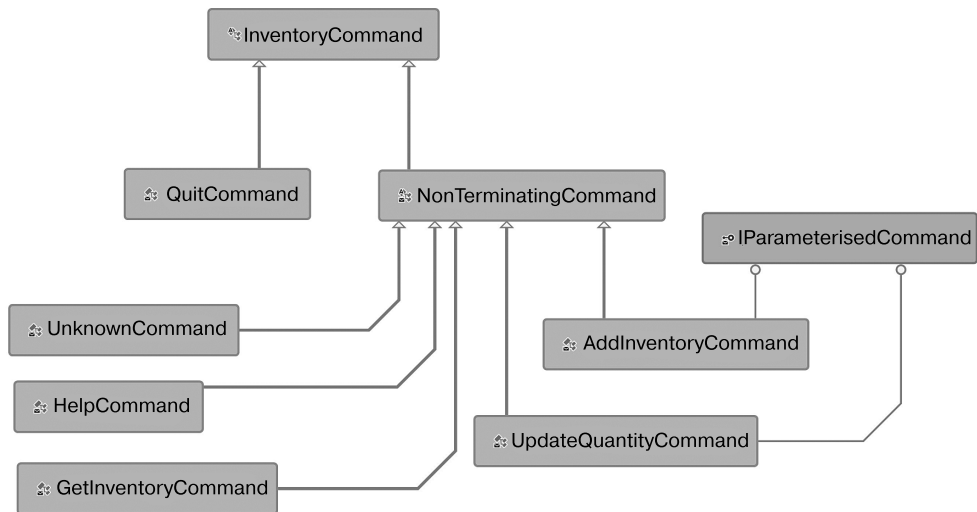


Рис. 3.10

Завершение программы вызывает только одна команда, `QuitCommand`. Остальные дополняют абстрактный класс `NonTerminatingCommand`. Стоит также заметить, что лишь `AddInventoryCommand` и `UpdateQuantityCommand` требуют параметров, а использование `IparameterisedCommand` мы разъясним позднее, в подразделе «Принцип замещения Лисков (LSP)» следующего раздела. Другая тонкость: все три типа, отличные от базового класса `InventoryCommand`, не являются публичными (видимыми внешними сборками). Мы рассмотрим это подробнее в пункте «Модификаторы доступа» подраздела «Модульные тесты для `InventoryCommand`» раздела «Принципы SOLID».

Принципы SOLID

Упрощая код с помощью паттернов, разработчики также используют принципы SOLID для определения проблем. Под упрощением имеется в виду, что разработчики ставят перед собой цель сделать код более удобным в сопровождении и понятным для новых членов команды. Такой подход к рассмотрению кода в соответствии с набором принципов очень полезен для написания лаконичных классов, выполняющих только нужные функции, а также для создания слоя абстракции, который тоже помогает писать понятный и легкий в редактировании код.

Принцип единственной ответственности (SRP)

Первый принцип, применяемый разработчиками, — это *принцип единственной ответственности* (single responsibility principle, SRP). Разработчики выяснили, что сейчас механизм вывода в консоль не является ответственностью классов `InventoryCommand`. Именно поэтому представлен класс `ConsoleUserInterface`, который отвечает за взаимодействие с пользователем. SRP поможет классам `InventoryCommand` оставаться небольшими по размеру и избежать ситуаций, где тот же самый код дублируется несколько раз. Например, приложение должно иметь универсальный способ уведомления пользователя о вводе информации и отображении сообщений и предупреждений. Вместо повторения этого в классах `InventoryCommand` данная логика инкапсулируется в классе `ConsoleUserInterface`.

Класс `ConsoleUserInterface` содержит три метода, как показано ниже:

```
public class ConsoleUserInterface
{
    // чтение значения из консоли

    // сообщение в консоль

    // вывод предупреждения в консоль
}
```

Первый метод используется для чтения ввода:

```
public string ReadValue(string message)
{
    Console.ForegroundColor = ConsoleColor.Green;
    Console.Write(message);
    return Console.ReadLine();
}
```

Второй метод выводит в консоль сообщение зеленого цвета:

```
public void WriteMessage(string message)
{
    Console.ForegroundColor = ConsoleColor.Green;
    Console.WriteLine(message);
}
```

Третий метод станет выводить в консоль сообщение темно-желтым цветом, то есть это будет предупреждение:

```
public void WriteWarning(string message)
{
    Console.ForegroundColor = ConsoleColor.DarkYellow;
    Console.WriteLine(message);
}
```

С помощью класса `ConsoleUserInterface` мы можем ограничить влияние изменений взаимодействием с пользователем. По мере разработки решения может оказаться, что наш интерфейс изменяется с консольного на веб-приложение. В теории мы могли бы заменить `ConsoleUserInterface` на `WebUserInterface`. Если бы мы не свели весь пользовательский интерфейс к одному классу, то влияние подобного изменения могло бы быть более деструктивным.

Принцип открытости/закрытости (OCP)

Принцип открытости/закрытости (open/closed principle), таящийся за буквой О в аббревиатуре SOLID, представлен различными классами `InventoryCommand`. Вместо реализации класса `InventoryCommand` для каждой команды разработчики могут определить один класс, содержащий множество операторов `if`. Каждый из них может определять, какую из необходимых функций следует запустить. К примеру, следующий фрагмент кода показывает, как команда могла бы нарушить этот принцип:

```
internal bool InternalCommand(string command)
{
    switch (command)
    {
        case "?":
        case "help":
            return RunHelpCommand();
        case "a":
        case "addinventory":
            return RunAddInventoryCommand();
        case "q":
        case "quit":
            return RunQuitCommand();
        case "u":
        case "updatequantity":
            return RunUpdateInventoryCommand();
    }
}
```

```

        case "g":
        case "getinventory":
            return RunGetInventoryCommand();
    }
    return false;
}

```

Метод, представленный выше, нарушает этот принцип, так как добавление новой команды может изменить поведение кода. Суть принципа в том, что он *закрит* для модификации, которая *изменила бы* его поведение, но *открыт* для расширения класса в целях поддержки дополнительного поведения. Все это достигается с помощью абстрактного `InventoryCommand` и производных классов (например, `QuitCommand`, `HelpCommand` и `AddInventoryCommand`). Уважительная причина тому, особенно в комбинации с другими принципами, заключается в том, что это ведет к лаконичному коду, который проще сопровождать и понимать.

Принцип замещения Лисков (LSP)

Команды выхода справки и запроса товаров не требуют параметров, в то время как `AddInventory` и `UpdateQuantityCommand` требуют их. Это можно реализовать разными способами, и разработчики решили создать интерфейс для определения таких команд:

```

public interface IParameterisedCommand
{
    bool GetParameters();
}

```

Применительно *принципа замещения Лисков* (Liskov substitution principle, LSP) лишь те команды, которым нужны параметры, должны реализовывать метод `GetParameters()`. Например, если взять команду `AddInventory`, интерфейс `IparameterisedCommand` реализован с помощью метода, определенного в базовом классе `InventoryCommand`:

```

public class AddInventoryCommand : InventoryCommand, IParameterisedCommand
{
    public string InventoryName { get; private set; }

    /// <summary>
    /// AddInventoryCommand requires name
    /// </summary>
    /// <returns></returns>
    public bool GetParameters()
    {
        if (string.IsNullOrEmpty(InventoryName))
            InventoryName = GetParameter("name");
        return !string.IsNullOrEmpty(InventoryName);
    }
}

```


Метод `GetParameter` класса `InventoryCommand` просто использует `ConsoleUserInterface` для чтения значений из консоли. Он будет показан чуть позже в этой главе. В C# существует удобный синтаксис, отлично показывающий, как LSP можно использовать для применения функционала к объектам определенного интерфейса. В первой строке метода `RunCommand` ключевое слово `is` используется как для проверки того, реализует ли текущий объект интерфейс `IparameterisedCommand`, так и для приведения объекта к новому типу: `parameterisedCommand`. Это выделено жирным шрифтом в следующем фрагменте:

```
public (bool wasSuccessful, bool shouldQuit) RunCommand()
{
    if (this is IParameterisedCommand parameterisedCommand)
    {
        var allParametersCompleted = false;

        while (allParametersCompleted == false)
        {
            allParametersCompleted = parameterisedCommand.GetParameters();
        }
    }

    return (InternalCommand(), _isTerminatingCommand);
}
```

Принцип сегрегации интерфейса (ISP)

Для обработки команд с параметрами и без можно определить еще один метод, `GetParameters`, в абстрактном классе `InventoryCommand`, а для тех команд, что не нуждаются в параметрах, просто возвращать `true`, сигнализируя о том, что все параметры были получены (в данном случае их число равно нулю). Например, `QuitCommand`, `HelpCommand` и `GetInventoryCommand` могут иметь реализации, подобные этой:

```
internal override bool GetParameters()
{
    return true;
}
```

Может, подобное и работает, но однозначно нарушает *принцип сегрегации интерфейса* (interface segregation principle, ISP), который гласит, что интерфейс должен содержать только требуемые методы и свойства. Подобно SRP, который применяется к классам, ISP применяется к интерфейсам и помогает поддерживать их лаконичными и сосредоточенными на какой-то одной задаче. В нашем примере лишь классы `AddInventoryCommand` и `UpdateQuantityCommand` будут реализовывать интерфейс `InventoryCommand`.

Принцип инверсии зависимостей

Принцип инверсии зависимостей (dependency inversion principle, DIP), также известный как *принцип внедрения зависимостей* (dependency injection principle, DIP), гласит, что модули должны зависеть от абстракций, а не от деталей. Этот принцип поощряет написание слабосвязанного кода для улучшения читабельности и сопровождения, особенно в больших и сложных кодовых базах.

Если вспомнить класс `ConsoleUserInterface`, который был представлен выше (в подразделе «Принцип единственной ответственности (SRP)»), то мы могли бы использовать его без `QuitCommand`, как показано здесь:

```
internal class QuitCommand : InventoryCommand
{
    internal override bool InternalCommand()
    {
        var console = new ConsoleUserInterface();
        console.WriteMessage("Thank you for using FlixOne
            Inventory Management System");
        return true;
    }
}
```

Это нарушает несколько принципов SOLID. Если же говорить о DIP, этот код крепко связывает `QuitCommand` и `ConsoleUserInterface`. Представьте ситуацию, в которой консоль больше не является средством отображения информации пользователю или конструктор `ConsoleUserInterface` требует дополнительных параметров.

Используя принцип DIP, мы делаем следующий рефакторинг. Сначала представляется новый интерфейс `IUserInterface`, заключающий в себе определения методов, реализованных в `ConsoleUserInterface`. Далее этот интерфейс (и ни один из конкретных классов) применяется в классах `InventoryCommand`. И наконец, ссылка на объект, реализующий `IUserInterface`, передается конструктору классов `InventoryCommand`. Этот подход защищает классы `InventoryCommand` от изменений деталей реализации классов `IUserInterface`, а также предоставляет механизм для более простой замены реализаций `IUserInterface`.

Принцип DIP показан с помощью следующей версии `QuitCommand`, которая в этой главе будет заключительной:

```
internal class QuitCommand : InventoryCommand
{
    public QuitCommand(IUserInterface userInterface) :
        base(commandIsTerminating: true, userInterface: userInterface)
    {
    }
}
```

```

        internal override bool InternalCommand()
        {
            Interface.WriteMessage("Thank you for using FlixOne
                                   Inventory Management System");
            return true;
        }
    }

```

Обратите внимание: `QuitCommand` расширяет абстрактный класс `InventoryCommand`, предоставляющий общий механизм для управления командами и общий функционал. Конструктор требует внедрения зависимости `IUserInterface`, когда объект инстанцируется. Кроме того, `QuitCommand` реализует единственный метод, `InternalCommand()`, благодаря чему `QuitCommand` можно легко прочитать и понять.

Для полноты картины рассмотрим финальный класс `InventoryCommand`. Следующий фрагмент показывает конструктор и свойства:

```

public abstract class InventoryCommand
{
    private readonly bool _isTerminatingCommand;
    protected IUserInterface Interface { get; }

    internal InventoryCommand(bool commandIsTerminating,
                               IUserInterface userInterface)
    {
        _isTerminatingCommand = commandIsTerminating;
        Interface = userInterface;
    }
    ...
}

```

Следует заметить, что в конструктор были переданы интерфейс `IUserInterface` и булева переменная, которая определяет, была ли команда завершена. Затем `IUserInterface` становится доступным всем реализациям `InventoryCommand` как свойство `Interface`.

Метод `RunCommand` — единственный публичный метод класса:

```

public (bool wasSuccessful, bool shouldQuit) RunCommand()
{
    if (this is IPParameterisedCommand parameterisedCommand)
    {
        var allParametersCompleted = false;

        while (allParametersCompleted == false)
        {
            allParametersCompleted = parameterisedCommand.GetParameters();
        }

        return (InternalCommand(), _isTerminatingCommand);
    }
}

internal abstract bool InternalCommand();

```

Более того, метод `GetParameter` является общим для реализации `InventoryCommand`, поэтому он внутренний:

```
internal string GetParameter(string parameterName)
{
    return Interface.ReadValue($"Enter {parameterName}:");
}
```



DIP и IoC

DIP и инверсия управления (Inversion of Control, IoC) очень близки между собой и созданы для одной цели, но немного различаются подходами. IoC и его специализированная форма, паттерн «Реестр сервисов» (Service Locator Pattern, SLP), используют механизм для предоставления реализации абстракций по запросу. Поэтому вместо внедрения реализации IoC действует как прокси для предоставления требуемых деталей. В следующей главе мы рассмотрим поддержку этих паттернов в .NET Core.

Модульные тесты для `InventoryCommand`

По мере обретения формы классов `InventoryCommand` вернемся к модульным тестам, чтобы проверить уже написанный код и определить еще не реализованные требования. Здесь принципы SOLID покажут свою истинную ценность. Поскольку мы сохранили малый размер наших классов (SRP) и интерфейсов (ISP) и фокусировались на минимальном наборе требуемого функционала (LSP), наши тесты тоже должны быть более простыми для написания и проверки. Например, тесту, который касается одной конкретной команды, не обязательно проверять вывод сообщений в консоль (например, цвет или размер шрифта), так как это является не ответственностью классов `InventoryCommand`, а реализацией `IUserInterface`. Вдобавок с помощью внедрения зависимости можно ограничить тест работой только с `InventoryCommand`.

На рис. 3.11 показано, как модульный тест проверяет лишь то, что содержится в зеленой зоне.

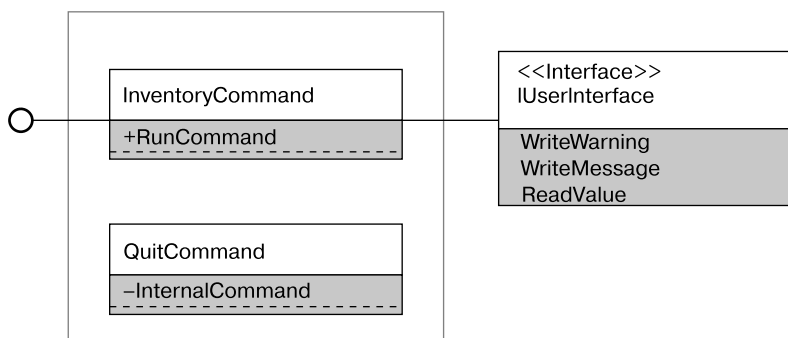


Рис. 3.11



Пока охват модульного теста ограничен, нам намного проще справляться с изменениями по мере развития приложения. В некоторых ситуациях, когда гораздо сложнее отделить функционал по причине взаимных зависимостей между классами (другими словами, когда мы не следуем SOLID), тест может охватывать более обширный участок приложения, включая репозитории. Такие тесты обычно называют интеграционными, а не модульными.

Модификаторы доступа

Модификаторы доступа — важная составляющая управления видимостью типов и членов типов с помощью инкапсуляции. С помощью грамотной стратегии доступа можно донести правила использования сборки и ее типов и заставить их применять. Например, в приложении FlixOne все типы, которые должны быть задействованы напрямую через консоль, отмечены как публичные. Это значит, что в консольном приложении должно быть доступно ограниченное количество типов и методов. Эти типы и методы следует отметить как публичные. Типы и методы, к которым нельзя обращаться из консоли, нужно отметить как внутренние (`internal`), приватные (`private`) или защищенные (`protected`).



Пожалуйста, ознакомьтесь с руководством по программированию от Microsoft, чтобы получить более подробную информацию по модификаторам доступа: docs.microsoft.com/ru-ru/dotnet/csharp/programming-guide/classes-and-structs/access-modifiers.

Абстрактный класс `InventoryCommand` сделан публичным, так как консольное приложение будет использовать метод `RunCommand` для управления.

В следующем фрагменте посмотрите, как конструктор и интерфейс становятся защищенными, чтобы предоставить доступ для подклассов:

```
public abstract class InventoryCommand
{
    private readonly bool _isTerminatingCommand;
    protected IUserInterface Interface { get; }

    protected InventoryCommand(bool commandIsTerminating,
        IUserInterface userInterface)
    {
        _isTerminatingCommand = commandIsTerminating;
        Interface = userInterface;
    }
    ...
}
```

В следующем фрагменте кода метод `RunCommand` публичный, а `InternalCommand` — внутренний:

```
public (bool wasSuccessful, bool shouldQuit) RunCommand()
{
    if (this is IParameterisedCommand parameterisedCommand)
    {
        var allParametersCompleted = false;

        while (allParametersCompleted == false)
        {
            allParametersCompleted = parameterisedCommand.GetParameters();
        }

        return (InternalCommand(), _isTerminatingCommand);
    }

    internal abstract bool InternalCommand();
}
```

Похожим образом реализации `InventoryCommand` отмечены как внутренние для предотвращения доступа к ним вне сборки. Здесь это показано на примере `QuitCommand`:

```
internal class QuitCommand : InventoryCommand
{
    internal QuitCommand(IUserInterface userInterface) :
        base(true, userInterface) { }

    protected override bool InternalCommand()
    {
        Interface.WriteMessage("Thank you for using FlixOne
            Inventory Management System");
        return true;
    }
}
```

Поскольку проект модульных тестов не будет иметь прямого доступа к разным реализациям, требуется дополнительный шаг для того, чтобы сделать внутренние типы видимыми. Директива `assembly` может быть помещен в любой скомпилированный файл. В случае с приложением `FlixOne` был добавлен файл `assembly.cs`, содержащий атрибут сборки:

```
using System.Runtime.CompilerServices;
[assembly: InternalsVisibleTo("FlixOne.InventoryManagementTests")]
```



В ситуациях, когда сборка имеет подпись, `InternalVisibleTo()` требует публичный ключ. Пожалуйста, ознакомьтесь с руководством по C# на сайте компании Microsoft, чтобы получить подробную информацию: docs.microsoft.com/ru-ru/dotnet/standard/assembly/create-signed-friend.

Класс-помощник TestUserInterface

В рамках модульного теста одной из реализаций `InventoryCommand` мы не хотим тестировать упоминаемые в ней зависимости. К счастью, поскольку команды твердо следуют принципу DIP, мы можем создать класс-помощник для проверки того, как реализация взаимодействует с зависимостями. Одна из зависимостей — `IUserInterface`, которая передается в реализацию через конструктор. Следующий фрагмент кода является напоминанием о методах интерфейса:

```
public interface IUserInterface : IReadUserInterface, IWriteUserInterface {  
}  
  
public interface IReadUserInterface  
{  
    string ReadValue(string message);  
}  
  
public interface IWriteUserInterface  
{  
    void WriteMessage(string message);  
    void WriteWarning(string message);  
}
```

Реализуя класс-помощник, мы можем предоставить информацию, которая нужна методу `ReadValue`, и проверить получение соответствующих сообщений методами `WriteMessage` и `WriteWarning`. В тестовом проекте мы создали новый класс `TestUserInterface`, реализующий интерфейс `IUserInterface`. Класс содержит три списка, в которых находятся ожидаемые вызовы `WriteMessage`, `WriteWarning` и `ReadValue`, а также счетчик того, сколько раз они вызывались.

Например, метод `WriteWarning` выглядит следующим образом:

```
public void WriteWarning(string message)  
{  
    Assert.IsTrue(_expectedWriteWarningRequestsIndex  
        < _expectedWriteWarningRequests.Count,  
        "Received too many command write warning requests.");  
    Assert.AreEqual(_expectedWriteWarningRequests  
        [_expectedWriteWarningRequestsIndex++], message,  
        "Received unexpected command write warning message");  
}
```

Метод `WriteWarning` делает два утверждения. Первое проверяет, что метод не вызывается большее количество раз, чем ожидается. Второе определяет, соответствует ли полученный ответ ожидаемому сообщению.

Метод `ReadValue` работает схожим образом, но дополнительно возвращает значение вызывающей его реализации `InventoryCommand`. Код ниже имитирует то, как пользователь вводит информацию в консоли:

```
public string ReadValue(string message)
{
    Assert.IsTrue(_expectedReadRequestsIndex < _expectedReadRequests.Count,
        "Received too many command read requests.");
    Assert.AreEqual(_expectedReadRequests[_expectedReadRequestsIndex].Item1,
        message, "Received unexpected command read message");
    return _expectedReadRequests[_expectedReadRequestsIndex++].Item2;
}
```

Как дополнительный шаг проверки в конце тестового метода вызывается `TestUserInterface` для определения того, что было получено ожидаемое количество запросов `ReadValue`, `WriteMessage` и `WriteWarning`:

```
public void Validate()
{
    Assert.IsTrue(_expectedReadRequestsIndex ==
        _expectedReadRequests.Count,
        "Not all read requests were performed.");
    Assert.IsTrue(_expectedWriteMessageRequestsIndex ==
        _expectedWriteMessageRequests.Count,
        "Not all write requests were performed.");
    Assert.IsTrue(_expectedWriteWarningRequestsIndex ==
        _expectedWriteWarningRequests.Count,
        "Not all warning requests were performed.");
}
```

Класс `TestUserInterface` показывает имитацию зависимости для подмены функционала и предоставления утверждений, которые помогают удостовериться в том, что класс демонстрирует ожидаемое поведение. В следующих главах мы используем сторонний фреймворк для более сложной имитации зависимостей.

Пример модульного теста для `QuitCommand`

Начиная с `QuitCommand`, требования были довольно прямолинейны: команда должна выводить прощальное сообщение и затем завершать программу. Мы сделали так, чтобы метод `InventoryCommand` возвращал две булевы переменные, сигнализирующие о завершении приложения и успешности выполнения команды:

```
[TestMethod]
public void QuitCommand_Successful()
{
    var expectedInterface = new Helpers.TestUserInterface(
        new List<Tuple<string, string>>(), // ReadValue()
        new List<string> // WriteMessage()
        {
            "Thank you for using FlixOne Inventory Management System"
        },
        new List<string>() // WriteWarning()
    );
};
```



```
// создание экземпляра команды
var command = new QuitCommand(expectedInterface);

var result = command.RunCommand();

expectedInterface.Validate();

Assert.IsTrue(result.shouldQuit, "Quit is a terminating command.");
Assert.IsTrue(result.wasSuccessful, "Quit did not complete
    Successfully.");
}
```

Тест использует `TestUserInterface` для проверки того, что текст "Thank you for using FlixOne Inventory Management System" отправлен методу `WriteMessage` и запросы `ReadValue` или `WriteWarning` не были получены. Последние два критерия проверяются с помощью вызова `expectedInterface.Validate()`. Чтобы проанализировать результат `QuitCommand`, мы убеждаемся в том, что `shouldQuit` и `wasSuccessful` равны `true`.



В случае с `FlixOne` отображаемый текст для простоты прописан прямо в коде решения. Лучшим подходом здесь было бы использование файлов ресурсов. Они позволяют отделить текст от функционала.

Резюме

В данной главе представлен сценарий книжного онлайн-магазина, `FlixOne`, для которого требуется создать приложение для инвентаризации. Здесь мы раскрыли широкий спектр паттернов и методик, которые разработчики могут задействовать при создании этого приложения. Разработчики использовали MVP, чтобы поддержать начальный набор функционала на должном уровне, а также для того, чтобы помочь сфокусировать бизнес на определенных требованиях, имеющих максимальную выгоду для организации. Разработчики решили применить TDD для проверки того, что написанный код соответствует требованиям, и для того, чтобы помочь команде измерить прогресс. Был создан базовый проект и фреймворк модульного тестирования `MSTest`. Кроме того, разработчики придерживались принципов SOLID, чтобы по мере улучшения и расширения приложения сохранять читабельность кода и удобство его сопровождения. Был использован первый паттерн «Банды четырех», «Абстрактная фабрика».

В следующей главе разработчики продолжат создание проекта для инвентаризации, удовлетворяя требованиям, определенным в MVP. Они будут использовать такие паттерны из набора «Банды четырех», как «Одиночка» и «Фабричный метод». Оба эти паттерна будут рассмотрены как в контексте соответствующих механизмов, поддерживаемых в .NET Core, так и без них.

Вопросы

Следующие вопросы позволят вам усвоить информацию, представленную в этой главе.

1. Почему иногда бывает затруднительно определить требования при разработке программного обеспечения для организаций?
2. В чем два главных преимущества и недостатка разработки ПО методами Waterfall и Agile?
3. Как внедрение зависимостей помогает при написании модульных тестов?
4. Почему следующее утверждение ложное? Используя TDD, вы не нуждаетесь в людях, тестирующих новое программное обеспечение.

4

Реализация паттернов проектирования — основы (часть 2)

В предыдущей главе мы представили FlixOne, начальную разработку нового приложения для инвентаризации. Группа разработчиков использовала несколько паттернов, начиная с призванных ограничить объем требований, таких как *минимально жизнеспособный продукт* (minimum viable product, MVP), и заканчивая паттернами, помогающими в создании проектов, например такими, как разработка через тестирование (test-driven development, TDD). Несколько паттернов «Банды четырех» (Gang of Four, GoF) также были применены, чтобы использовать опыт решения подобных проблем другими разработчиками и не повторять распространенные ошибки. Следующие принципы: единственной ответственности, открытости/закрытости, замещения Лисков, сегрегации интерфейсов и инверсии зависимостей (принципы SOLID) — были задействованы, чтобы обеспечить стабильность кодовой базы, которая поможет в управлении и дальнейшем развитии нашего приложения.

В этой главе мы продолжим объяснение того, как создавалось приложение FlixOne для инвентаризации с помощью включения в него дополнительных паттернов. Будут использоваться паттерны «Банды четырех», в том числе «Одиночка» и «Фабрика». «Одиночка» послужит для иллюстрации паттерна репозитория, который, в свою очередь, применяется для поддержания коллекции книг FlixOne. «Фабрика» будет способствовать пониманию *внедрения зависимости* (Dependency Injection, DI). Наконец, мы задействуем фреймворк .NET Core, чтобы облегчить работу с контейнером *инверсии управления* (Inversion of Control, IoC), который будет использоваться в целях завершения разработки первой версии консольного приложения для управления инвентаризацией.

В этой главе будут рассмотрены следующие темы:

- ❑ паттерн «Одиночка»;
- ❑ паттерн «Фабрика»;
- ❑ возможности .NET Core;
- ❑ консольные приложения.

Технические требования

В этой главе содержатся примеры кода, объясняющие принципы качественной разработки. Код упрощен и предназначен лишь для демонстрации. Большинство примеров основаны на консольном приложении .NET Core, написанном на C#.

Чтобы запустить и выполнить код, вам потребуется следующее:

- ❑ Visual Studio 2019 (вы также можете запустить приложение, используя Visual Studio 2017 версии 3 и новее);
- ❑ .NET Core;
- ❑ SQL Server (в этой главе применяется Express Edition).

Установка Visual Studio

Чтобы запустить примеры кода, вам необходимо установить Visual Studio. Для этого выполните следующие шаги.

1. Скачайте Visual Studio по ссылке docs.microsoft.com/ru-ru/visualstudio/install/install-visual-studio.
2. Следуйте инструкциям по установке. Доступны различные версии Visual Studio; в этой главе мы используем Visual Studio для Windows.

Вы также можете применять другую интегрированную среду разработки.

Установка .NET Core

Если вы еще не установили .NET Core, то следуйте инструкции ниже.

1. Скачайте .NET Core по ссылке dotnet.microsoft.com/download.
2. Далее следуйте указаниям по установке соответствующей библиотеки: dotnet.microsoft.com/download/dotnet-core/2.2.



Полная версия исходного кода доступна на GitHub. Код, представленный в данной главе, может быть неполным, поэтому мы рекомендуем вам скачать полный код для запуска примеров (github.com/PacktPublishing/Hands-On-Design-Patterns-with-C-and-.NET-Core/tree/master/Chapter4).

Паттерн «Одиночка»

«Одиночка» — паттерн проектирования «Банды четырех», который применяется, чтобы ограничить инстанцирование класса одним объектом. Он используется в ситуациях, когда действия в рамках системы должны быть скоординированы или доступ к данным следует ограничить. Например, если в приложении необходимо ограничить

доступ к файлу на запись так, чтобы это мог делать только один объект, то можно задействовать «Одиночку». Это позволит предотвратить одновременную попытку записи в файл со стороны нескольких объектов. В нашем сценарии мы будем использовать этот паттерн для поддержания коллекции книг и их инвентаризации.

Ценность «Одиночки» более очевидна, когда он показан на примерах. Этот раздел начнется с базового класса, а затем мы перейдем к определению различных проблем, которые решаются с помощью данного паттерна. Они будут выявлены, класс — обновлен, а затем проверен модульными тестами.

«Одиночку» следует использовать только при необходимости, поскольку он может создать в приложении потенциальное узкое место. Иногда он рассматривается как антипаттерн, вследствие того что вводит глобальное состояние, в котором, в свою очередь, вводятся неизвестные зависимости внутри приложения, после чего становится неясным, сколько типов может зависеть от некой сущности. Кроме того, многие фреймворки и репозитории уже ограничивают доступ при необходимости, поэтому внедрение дополнительного механизма может необоснованно снизить производительность.



.NET Core обеспечивает поддержку ряда обсуждаемых паттернов. В следующей главе мы воспользуемся тем, что класс `ServiceCollection` поддерживает как паттерн «Фабричный метод», так и паттерн «Одиночка».

В нашем сценарии «Одиночка» будет использоваться для хранения в памяти репозитория, содержащего коллекцию книг. Паттерн предотвращает обновление коллекции книг несколькими потоками одновременно. Для этого нам потребуется заблокировать часть кода, чтобы предотвратить непредсказуемые обновления.

Ввод «Одиночки» в приложение требует мастерства. Поэтому, чтобы получить четкое представление о паттерне, мы рассмотрим следующие темы:

- ❑ обработку процессов и потоков в .NET Framework;
- ❑ паттерн «Репозиторий»;
- ❑ состояние гонки;
- ❑ модульное тестирование для определения состояния гонки.

Процессы и потоки

Чтобы понять паттерн «Одиночка», нужно предоставить небольшой контекст. В .NET Framework приложение будет состоять из легких, управляемых подпроцессов, называемых доменами приложений, которые могут содержать один или несколько управляемых потоков. Чтобы понять «Одиночку», определим его как многопоточное приложение, которое включает один или несколько потоков, выполняющихся одновременно. Технически на самом деле потоки не работают

одновременно, но это достигается за счет разделения доступного процессорного времени между ними¹. Так что каждый поток выполняется в течение небольшого промежутка времени, а затем приостанавливает работу, позволяя выполнить другой поток².

Возвращаясь к «Одиночке», в многопоточном приложении нужно уделить особое внимание следующему аспекту: доступ к объекту должен быть ограничен так, чтобы только один поток попадал в определенные логические области одновременно. Благодаря подобной синхронизации один поток может получить значение и обновить его, а другой также обновляет это значение до того, как оно может быть сохранено.



Возможность получения несколькими потоками доступа к одним и тем же общим данным и их обновления с непредсказуемыми результатами можно назвать состоянием гонки.

Чтобы избежать некорректного обновления данных, необходимо некое ограничение, которое не позволит нескольким потокам выполнять один и тот же блок логики одновременно. Существует ряд механизмов обеспечения этого, поддерживаемых в .NET Framework. В «Одиночке» используется ключевое слово `lock`. В следующем коде показано это слово и то, что только один поток одновременно может выполнить выделенный код, тогда как все остальные потоки будут заблокированы:

```
public class Inventory
{
    int _quantity;
    private Object _lock = new Object();

    public void RemoveQuantity(int amount)
    {
        lock (_lock)
        {
            if (_quantity - amount < 0)
            {
                throw new Exception("Cannot remove more than we have!");
            }
            _quantity -= amount;
        }
    }
}
```

Блокировка — простой способ ограничить доступ к участку кода. Ее можно применить как к объектным экземплярам, что видно по нашему предыдущему примеру, так и к участкам кода, помеченным как статические.

¹ На многоядерных процессорах потоки могут выполняться строго одновременно. — *Примеч. науч. ред.*

² Синхронизация должна это предотвращать. — *Примеч. науч. ред.*

Паттерн «Репозиторий»

Паттерн «Одиночка», который вводится в проект, применяется к классу, используемому для обслуживания коллекции книг. Он предотвратит некорректный доступ из нескольких потоков, тогда как другой паттерн, «Репозиторий», послужит для создания фасада над данными, с которыми ведется работа.

Паттерн «Репозиторий» предоставляет абстракцию над хранилищем, обеспечивая тем самым слой между бизнес-логикой приложения и данными. Это дает ряд преимуществ. Благодаря чистому разделению наша бизнес-логика может поддерживаться и тестироваться независимо от данных. Часто один и тот же класс «Репозитория» повторно используют несколько бизнес-объектов. Примером тому могут быть объекты `GetInventoryCommand`, `AddInventoryCommand` и `UpdateInventoryCommand`. Все они задействуют один и тот же класс репозитория, что позволяет нам тестировать логику в этих командах, будучи в изоляции от репозитория. Еще одно преимущество паттерна таково: он позволяет более легко реализовать централизованные политики, связанные с данными, такие как кэширование.

Для начала рассмотрим интерфейс, описывающий методы, которые будет реализовывать «Репозиторий». Он содержит метод извлечения книг, добавления книги и обновления количества определенной книги:

```
internal interface IInventoryContext
{
    Book[] GetBooks();
    bool AddBook(string name);
    bool UpdateQuantity(string name, int quantity);
}
```

Исходная версия репозитория выглядит так:

```
internal class InventoryContext : IInventoryContext
{
    public InventoryContext()
    {
        _books = new Dictionary<string, Book>();
    }
    private readonly IDictionary<string, Book> _books;

    public Book[] GetBooks()
    {
        return _books.Values.ToArray();
    }

    public bool AddBook(string name)
    {
        _books.Add(name, new Book { Name = name });
        return true;
    }
}
```

```

public bool UpdateQuantity(string name, int quantity)
{
    _books[name].Quantity += quantity;
    return true;
}
}

```



В этой главе коллекция книг хранится в виде кэша в памяти, и в следующих главах она будет перемещена в хранилище, которое предоставляет возможность постоянного хранения данных. Конечно, текущая реализация неидеальна, поскольку по окончании работы приложения все данные будут потеряны. Однако она служит для иллюстрации паттерна «Одиночка».

Модульные тесты

Чтобы проиллюстрировать проблемы, решаемые паттерном «Одиночка», мы начнем с простого модульного теста, который добавляет 30 книг в хранилище, обновляет количество различных книг, а затем проверяет результат. Следующий код демонстрирует общую структуру модульного теста, а затем мы объясним каждый шаг по отдельности.

```

[TestClass]
public class InventoryContextTests
{
    [TestMethod]
    public void MaintainBooks_Successful()
    {
        var context = new InventoryContext();

        // добавление 30 книг
        ...

        // обновим количество книг, добавив 1, 2, 3, 4, 5...
        ...

        // обновим количество книг, отняв 1, 2, 3, 4, 5...
        ...

        // все величины должны быть равны 0
        ...
    }
}

```

Чтобы добавить 30 книг, экземпляр `context` используется для добавления книг от `Book_1` до `Book_30`:

```

// добавление 30 книг
foreach(var id in Enumerable.Range(1, 30))
{
    context.AddBook($"Book_{id}");
}

```


Следующий раздел обновляет количество книг, добавляя числа от 1 до 10 к количеству каждой из них:

```
// обновим количество книг, добавив 1, 2, 3, 4, 5...
foreach (var quantity in Enumerable.Range(1, 10))
{
    foreach (var id in Enumerable.Range(1, 30))
    {
        context.UpdateQuantity($"Book_{id}", quantity);
    }
}
```

Затем в следующем разделе мы вычтем числа от 1 до 10 из количества каждой книги:

```
foreach (var quantity in Enumerable.Range(1, 10))
{
    foreach (var id in Enumerable.Range(1, 30))
    {
        context.UpdateQuantity($"Book_{id}", -quantity);
    }
}
```

Поскольку мы добавили и удалили одно и то же количество для каждой книги, последняя часть нашего теста проверит, что конечное количество равно 0:

```
// все количества должны быть 0
foreach (var book in context.GetBooks())
{
    Assert.AreEqual(0, book.Quantity);
}
```

После запуска теста мы видим, что он проходит (рис. 4.1).

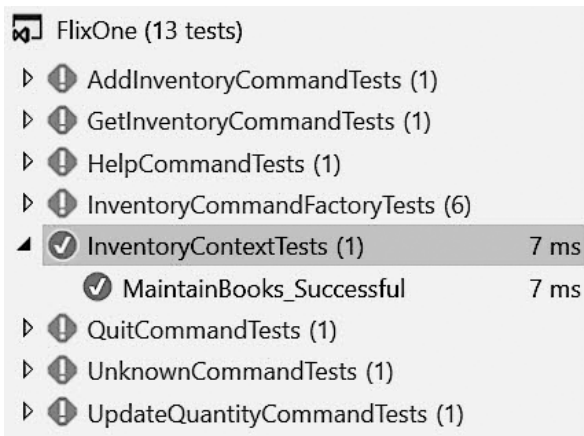


Рис. 4.1

Итак, когда тест выполняется в одном процессе, «Репозиторий» работает должным образом. Однако как быть, если запросы на обновление выполняются в отдельных потоках? В целях проверки этого модульный тест будет рефакторизован так, чтобы мог выполнить вызовы `InventoryContext` в отдельных потоках.

Добавление книг перемещается в метод, выполняющий действие как задачу (то есть в собственном потоке):

```
public Task AddBook(string book)
{
    return Task.Run(() =>
    {
        var context = new InventoryContext();
        Assert.IsTrue(context.AddBook(book));
    });
}
```

Кроме того, код, отвечающий за обновление количества книг, перенесен в другой метод с аналогичным подходом:

```
public Task UpdateQuantity(string book, int quantity)
{
    return Task.Run(() =>
    {
        var context = new InventoryContext();
        Assert.IsTrue(context.UpdateQuantity(book, quantity));
    });
}
```

Модульный тест затем обновляется для вызова новых методов. Стоит отметить, что тест станет ждать добавления всех книг, прежде чем обновить количество.

Раздел `add thirty books` теперь выглядит следующим образом:

```
// добавление 30 книг
foreach (var id in Enumerable.Range(1, 30))
{
    tasks.Add(AddBook($"Book_{id}"));
}

Task.WaitAll(tasks.ToArray());
tasks.Clear();
```

Аналогичным образом `UpdateQuantity` изменяется для вызова методов `Add` и `subtract` в задачах:

```
// обновим количество книг, добавив 1, 2, 3, 4, 5...
foreach (var quantity in Enumerable.Range(1, 10))
{
    foreach (var id in Enumerable.Range(1, 30))
    {
```

```

        tasks.Add(UpdateQuantity($"Book_{id}", quantity));
    }
}

// обновим количество книг, отняв 1, 2, 3, 4, 5...
foreach (var quantity in Enumerable.Range(1, 10))
{
    foreach (var id in Enumerable.Range(1, 30))
    {
        tasks.Add(UpdateQuantity($"Book_{id}", -quantity));
    }
}

// ждем, пока все сложения и вычитания не завершатся
Task.WaitAll(tasks.ToArray());

```

После рефакторинга модульный тест больше не завершается успешно, а при его запуске сейчас выдается сообщение об ошибке, указывающее на то, что книга не была найдена в коллекции: "The given key was not present in the dictionary.". Так происходит потому, что каждый раз при инстанцировании контекста создается новая коллекция книг. Первым шагом является ограничение создания контекста. Это делается с помощью изменения доступа конструктора таким образом, чтобы класс больше не мог быть инстанцирован напрямую. Вместо этого добавляется новое публичное статическое свойство, которое поддерживает только операцию `get`. Это свойство вернет статический экземпляр класса `InventoryContext` и, если экземпляра отсутствует, создаст его:

```

internal class InventoryContext : IInventoryContext
{
    protected InventoryContext()
    {
        _books = new Dictionary<string, Book>();
    }

    private static InventoryContext _context;
    public static InventoryContext Singleton
    {
        get
        {
            if (_context == null)
            {
                _context = new InventoryContext();
            }

            return _context;
        }
    }
    ...
}

```

Этого все еще недостаточно, чтобы исправить сломанный модульный тест, однако он не работает уже по другой причине. Для выявления проблемы мы выполняем тест в режиме отладки с точкой останова, установленной в методе `UpdateQuantity`. При первом запуске мы видим, что 28 книг были созданы и загружены в коллекцию книг, как показано на следующем снимке экрана (рис. 4.2).

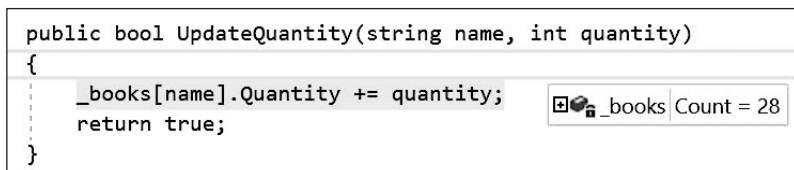


Рис. 4.2

На данном этапе модульного теста мы ожидаем 30 книг; однако прежде, чем начнем исследование, запустим тест во второй раз. На сей раз мы получаем ошибку `Object reference not set to an instance of an object` (Ссылка на объект не установлена на экземпляр объекта), когда пытаемся получить доступ к коллекции книг, чтобы добавить новую книгу, как показано на рис. 4.3.

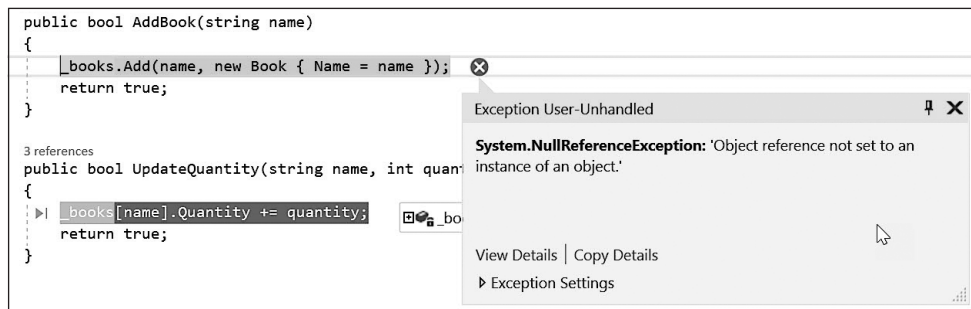


Рис. 4.3

Более того, когда модульный тест выполняется в третий раз, ошибка `Object reference not set to an instance of an object` (Ссылка на объект не установлена на экземпляр объекта) не встречается, однако в нашей коллекции есть только 27 книг, как показано на рис. 4.4.

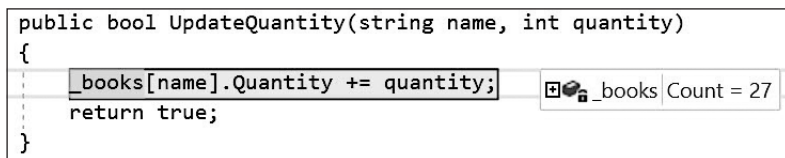


Рис. 4.4

Этот тип непредсказуемого поведения типичен для условий гонки и указывает на то, что общий ресурс, то есть одиночка `InventoryContext`, обрабатывается несколькими потоками в отсутствие синхронизации доступа. Сама по себе конструкция статического объекта по-прежнему позволяет создать более одного экземпляра `InventoryContext`:

```
public static InventoryContext Singleton
{
    get
    {
        if (_context == null)
        {
            _context = new InventoryContext();
        }

        return _context;
    }
}
```

Состояние гонки — это когда несколько потоков вычисляют выражение в `if` как `true` и все пытаются конструировать объект `_context`. Все они будут успешными, но станут перезаписывать ранее сконструированное значение. Конечно, это неэффективно, особенно когда работа конструктора — затратная операция. Однако проблема, обнаруженная с помощью модульного теста, заключается в том, что объект `_context` фактически создается потоком после того, как другой поток или потоки обновили коллекцию книг. Вот почему коллекция книг, `_books`, имеет разное количество элементов от запуска к запуску.

Предотвратить эту проблему в паттерне можно с помощью блокировки вокруг конструктора:

```
private static object _lock = new object();
public static InventoryContext Singleton
{
    get
    {
        if (_context == null)
        {
            lock (_lock)
            {
                _context = new InventoryContext();
            }
        }
        return _context;
    }
}
```

К сожалению, тесты все еще терпят неудачу. И вот почему. Несмотря на то что один поток одновременно может войти в блокировку, все заблокированные экземпляры все равно войдут в нее, как только блокирующий поток завершится. Паттерн об-

работывает эту ситуацию, имея дополнительную проверку внутри блокировки на случай, если конструирование уже завершено:

```
public static InventoryContext Singleton
{
    get
    {
        if (_context == null)
        {
            lock (_lock)
            {
                if (_context == null)
                {
                    _context = new InventoryContext();
                }
            }
        }
        return _context;
    }
}
```

Предшествующая блокировка важна, поскольку предотвращает многократное создание экземпляра статического `InventoryContext`. К сожалению, наш тест все еще иногда проваливается, но с каждым изменением он становится все ближе к успешному прохождению. Некоторые запуски модульного теста завершаются без ошибок, но иногда тест завершается неудачно, как показано на снимке экрана ниже (рис. 4.5).

Инстанцирование статического репозитория теперь потокобезопасно, но доступ к коллекции книг — нет. Одну деталь следует отметить: класс `Dictionary` используется не потокобезопасно. К счастью, существуют потокобезопасные коллекции, доступные как часть платформы .NET Framework. Эти классы гарантируют, что *добавления и удаления* из коллекции написаны для многопоточного процесса. Обратите внимание: только добавление и удаление потокобезопасны. Это станет важным немного позже. Обновленный конструктор показан в следующем коде:

```
protected InventoryContext()
{
    _books = new ConcurrentDictionary<string, Book>();
}
```



Корпорация Microsoft рекомендует использовать потокобезопасные коллекции в `System.Collections.Concurrent`, а не соответствующие коллекции в `System.Collections`, если только приложение не ориентировано на .NET Framework 1.1 или более раннюю версию.

После повторного запуска модульного теста введение класса `ConcurrentDictionary` все еще недостаточно для предотвращения неправильной поддержки книг. Модульный тест по-прежнему не проходит. Параллельный словарь защищает от непредсказуемого добавления и удаления нескольких потоков, однако не обеспечивает

никакой защиты для элементов в самой коллекции. Это значит, что обновления объектов в коллекции не являются потокобезопасными.

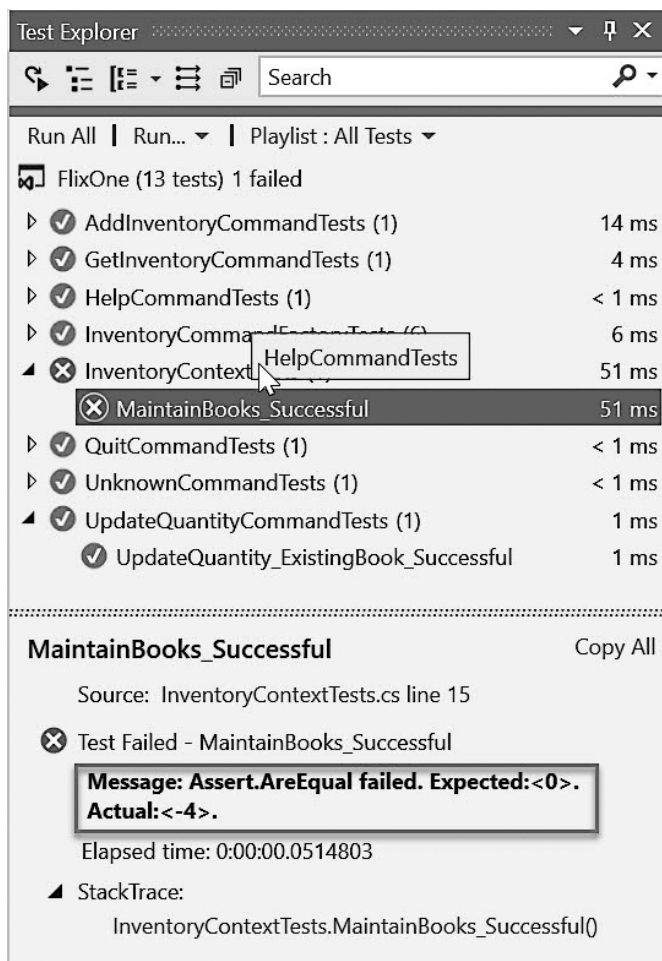


Рис. 4.5

Более подробно рассмотрим состояние гонки в многопоточной среде, чтобы понять, почему это так.

Иллюстрация состояния гонки

Следующая последовательность схем концептуально визуализирует то, что происходит между двумя потоками: ThreadA и ThreadB. На рис. 4.6 показаны оба потока без каких-либо значений из коллекции.

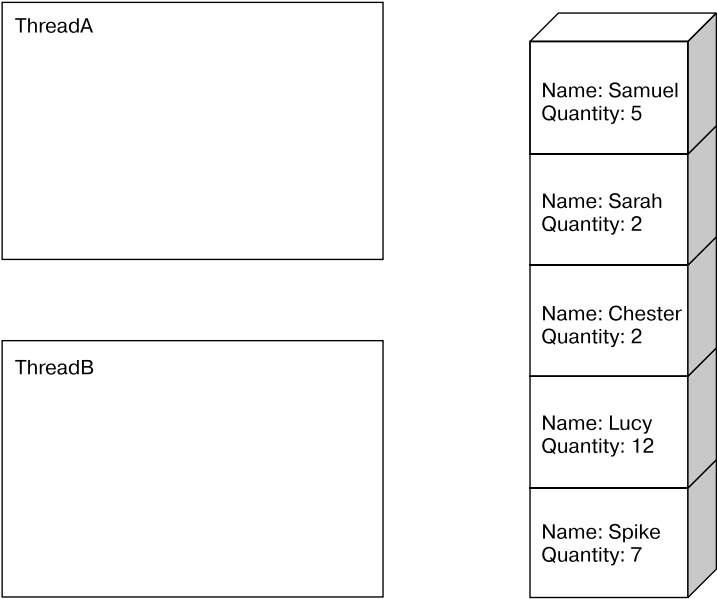


Рис. 4.6

На рис. 4.7 показано, что оба потока читают из коллекции книги с атрибутом `Name: Chester`.

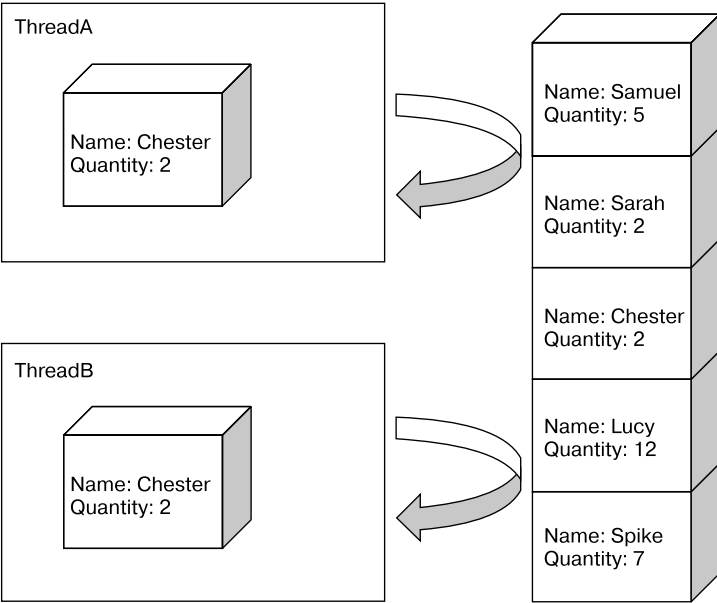


Рис. 4.7

На рис. 4.8 продемонстрировано, что ThreadA обновляет книгу, увеличивая количество (Quantity) на 4, в то время как ThreadB обновляет ее, увеличивая количество на 3.

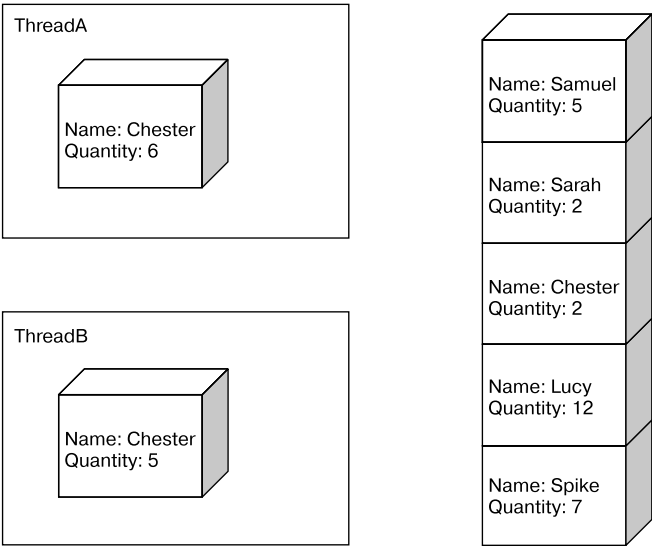


Рис. 4.8

Затем, когда обновленная книга сохраняется обратно в коллекцию, мы получаем неизвестное количество, как показано на рис. 4.9.

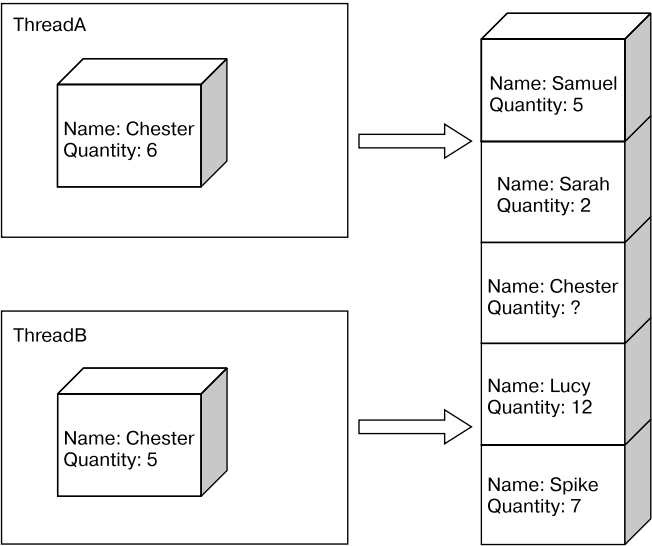


Рис. 4.9

Чтобы избежать этого состояния гонки, нам нужно блокировать другие потоки во время выполнения операции обновления. В классе `InventoryContext` блокировка других потоков принимает форму блокировки вокруг обновления количества книги:

```
public bool UpdateQuantity(string name, int quantity)
{
    lock (_lock)
    {
        _books[name].Quantity += quantity;
    }

    return true;
}
```

Модульное тестирование теперь завершается без ошибок, так как дополнительные блокировки предотвращают непредсказуемые состояния гонки.



Класс `InventoryContext` все еще не завершен. Он подходил нам только в смысле иллюстрации паттернов «Одиночка» и «Репозиторий». В следующих главах этот класс будет адаптирован для использования `Entity Framework` — библиотеки для объектно-реляционного отображения (`Object Relational Mapping, ORM`) данных. На этом этапе класс `InventoryContext` будет расширен для поддержки дополнительной функциональности.

Класс `AddInventoryCommand`

Теперь с помощью репозитория три класса `InventoryCommand` могут быть завершены. Первый, `AddInventoryCommand`, показан следующим образом:

```
internal class AddInventoryCommand : NonTerminatingCommand,
IParameterisedCommand
{
    private readonly IInventoryContext _context;

    internal AddInventoryCommand(IUserInterface userInterface,
        IInventoryContext context) : base(userInterface)
    {
        _context = context;
    }

    public string InventoryName { get; private set; }

    /// <summary>
    /// AddInventoryCommand требуется имя
    /// </summary>
    /// <returns></returns>
    public bool GetParameters()
```

```

{
    if (string.IsNullOrEmpty(InventoryName))
        InventoryName = GetParameter("name");

    return !string.IsNullOrEmpty(InventoryName);
}

protected override bool InternalCommand()
{
    return _context.AddBook(InventoryName);
}
}

```

Первое, что следует отметить, — репозиторий `IInventoryContext` вводится в конструктор вместе с интерфейсом `IUserInterface`, который описывается в предыдущей главе. Кроме того, команда требует, чтобы был указан один параметр — `name`. Он извлекается в методе `GetParameters`, реализующем интерфейс `IParameterisedCommand`, также рассматриваемый в предыдущей главе. Затем команда выполняется в методе `InternalCommand`, который просто выполняет `AddBook` в репозитории и возвращает логическое значение, указывающее, успешно ли выполнена команда.

TestInventoryContext

Подобно классу `TestUserInterface` в предыдущей главе, `TestInventoryContext` будет использоваться для моделирования поведения нашего репозитория через реализацию `IInventoryContext`. Этот класс будет поддерживать три метода интерфейса, а также два дополнительных метода для извлечения книг, добавленных в коллекцию во время модульного теста, и для извлечения книг, обновленных во время модульного теста.

Поддержка класса `TestInventoryContext` будет осуществляться с помощью двух коллекций:

```

private readonly IDictionary<string, Book> _seedDictionary;
private readonly IDictionary<string, Book> _books;

```

Первая используется для хранения начальной коллекции книг, вторая — для хранения конечной коллекции. Конструктор показан в следующем коде. Обратите внимание: словари являются копиями друг друга:

```

public TestInventoryContext(IDictionary<string, Book> books)
{
    _seedDictionary = books.ToDictionary(book => book.Key,
        book => new Book { Id = book.Value.Id,
            Name = book.Value.Name,
            Quantity = book.Value.Quantity });
    _books = books;
}

```

Методы `IInventoryContext` обновляют и возвращают только одну из коллекций, как показано ниже:

```
public Book[] GetBooks()
{
    return _books.Values.ToArray();
}

public bool AddBook(string name)
{
    _books.Add(name, new Book() { Name = name });

    return true;
}

public bool UpdateQuantity(string name, int quantity)
{
    _books[name].Quantity += quantity;

    return true;
}
```

В конце модульного теста можно использовать два оставшихся метода, чтобы определить разницу между начальной и конечной коллекциями:

```
public Book[] GetAddedBooks()
{
    return _books.Where(book => !_seedDictionary.ContainsKey(book.Key))
        .Select(book => book.Value).ToArray();
}

public Book[] GetUpdatedBooks()
{
    return _books.Where(book => _seedDictionary[book.Key].Quantity !=
        book.Value.Quantity)
        .Select(book => book.Value).ToArray();
}
```



Существует некоторая путаница между имитациями (mocks), заглушками (stubs), подделками (fakes) и другими терминами, используемыми для идентификации и/или классификации типов или сервисов, применяемых только при тестировании, но не в реальной работе и необходимых для модульного теста. Эти зависимости могут иметь функциональность, которая отличается от их реальных аналогов, отсутствует и/или совпадает с ними.

Например, класс `TestUserInterface` можно назвать имитацией ввиду того, что он обеспечивает некое ожидание (например, утверждения) модульного теста, в то время как класс `TestInventoryContext` был бы подделкой, поскольку предоставляет рабочую реализацию. В данной книге мы не будем следовать этим классификациям слишком строго.

AddInventoryCommandTest

`AddInventoryCommandTest` был обновлен разработчиками с целью проверить функциональность `AddInventoryCommand`. Этот тест проверит добавление одной книги в число товаров. Первая его часть заключается в определении того, что ожидается от интерфейса, который представляет собой только один запрос для получения нового названия книги (помните, что класс `TestUserInterface` принимает три параметра: ожидаемый ввод, ожидаемые сообщения и ожидаемые предупреждения):

```
const string expectedBookName = "AddInventoryUnitTest";
var expectedInterface = new Helpers.TestUserInterface(
    new List<Tuple<string, string>>
    {
        new Tuple<string, string>("Enter name:", expectedBookName)
    },
    new List<string>(),
    new List<string>()
);
```

Класс `TestInventoryContext` будет инициализирован одной книгой, которая имитирует существующую коллекцию книг:

```
var context = new TestInventoryContext(new Dictionary<string, Book>
{
    { "Gremlins", new Book { Id = 1, Name = "Gremlins", Quantity = 7 } }
});
```

В следующем фрагменте кода показано создание `AddInventoryCommand`, выполнение команды и инструкции `assert`, используемой для проверки успешного выполнения команды:

```
// создаем экземпляр команды
var command = new AddInventoryCommand(expectedInterface, context);

// добавляем новую книгу с параметром "name"
var result = command.RunCommand();

Assert.IsFalse(result.shouldQuit, "AddInventory is not a terminating command.");
Assert.IsTrue(result.wasSuccessful, "AddInventory did not complete
    Successfully.");

// убеждаемся, что была добавлена книга с заданным именем и количеством 0
Assert.AreEqual(1, context.GetAddedBooks().Length, "AddInventory should
have added one new book.");

var newBook = context.GetAddedBooks().First();
Assert.AreEqual(expectedBookName, newBook.Name, "AddInventory did not add
    book successfully.");
```

После запуска команды проверяется, что она выполнена без ошибок и не является завершающей командой. Остальные операторы `Assert` подтверждают предположение, что только одна книга была добавлена с ожидаемым именем.

Класс `UpdateQuantityCommand`

Класс `UpdateQuantityCommand` очень похож на `AddInventoryCommand`, и его код выглядит следующим образом:

```
internal class UpdateQuantityCommand : NonTerminatingCommand,
IParameterisedCommand
{
    private readonly IInventoryContext _context;
    internal UpdateQuantityCommand(IUserInterface userInterface,
        IInventoryContext context) : base(userInterface)
    {
        _context = context;
    }

    internal string InventoryName { get; private set; }

    private int _quantity;
    internal int Quantity { get => _quantity;
        private set => _quantity = value; }

    ...
}
```

Подобно `AddInventoryCommand`, `UpdateInventoryCommand` — незавершающая программу команда с параметрами. Поэтому она расширяет класс `NonTerminatingCommand` и реализует интерфейс `IParameterisedCommand`. Аналогичным образом в конструктор внедряются зависимости для `IUserInterface` и `IInventoryContext`:

```
/// <summary>
/// UpdateQuantity требуется название и целочисленное значение
/// </summary>
/// <returns></returns>
public bool GetParameters()
{
    if (string.IsNullOrEmpty(InventoryName))
        InventoryName = GetParameter("name");

    if (Quantity == 0)
        int.TryParse(GetParameter("quantity"), out _quantity);

    return !string.IsNullOrEmpty(InventoryName) && Quantity != 0;
}
```

Класс `UpdateQuantityCommand` имеет дополнительный параметр `quantity`, который определяется как часть `GetParameters`.

Наконец, количество экземпляров книги обновляется с помощью метода `UpdateQuantity` репозитория в переопределенном методе `InternalCommand`:

```
protected override bool InternalCommand()
{
    return _context.UpdateQuantity(InventoryName, Quantity);
}
```

Теперь, когда класс `UpdateQuantityCommand` определен, в следующем разделе будет добавлен модульный тест для проверки команды.

UpdateQuantityCommandTest

`UpdateQuantityCommandTest` содержит тест для проверки сценария, в котором книга обновляется в существующей коллекции. Создание ожидаемого интерфейса и существующей коллекции показано в коде ниже. Обратите внимание, что тест добавляет шесть книг к одной существующей:

```
const string expectedBookName = "UpdateQuantityUnitTest";
var expectedInterface = new Helpers.TestUserInterface(
    new List<Tuple<string, string>>
    {
        new Tuple<string, string>("Enter name:", expectedBookName),
        new Tuple<string, string>("Enter quantity:", "6")
    },
    new List<string>(),
    new List<string>()
);

var context = new TestInventoryContext(new Dictionary<string, Book>
{
    { "Beavers", new Book { Id = 1, Name = "Beavers", Quantity = 3 } },
    { expectedBookName, new Book { Id = 2, Name = expectedBookName,
        Quantity = 7 } },
    { "Ducks", new Book { Id = 3, Name = "Ducks", Quantity = 12 } }
});
```

Следующий блок кода показывает выполнение и первоначальную проверку успеха выполнения незавершающей команды:

```
// создание экземпляра команды
var command = new UpdateQuantityCommand(expectedInterface, context);
var result = command.RunCommand();

Assert.IsFalse(result.shouldQuit, "UpdateQuantity is not a terminating
    command.");
```

```
Assert.IsTrue(result.wasSuccessful, "UpdateQuantity did not complete  
Successfully.");
```

Ожидаемый результат теста — никакие новые книги не будут добавлены, а количество существующих книг — 7 — будет увеличено на 6, что приведет к новому количеству — 13:

```
Assert.AreEqual(0, context.GetAddedBooks().Length,  
    "UpdateQuantity should not have added one new book.");  
var updatedBooks = context.GetUpdatedBooks();  
Assert.AreEqual(1, updatedBooks.Length,  
    "UpdateQuantity should have updated one new book.");  
Assert.AreEqual(expectedBookName, updatedBooks.First().Name,  
    "UpdateQuantity did not update the correct book.");  
Assert.AreEqual(13, updatedBooks.First().Quantity,  
    "UpdateQuantity did not update book quantity successfully.");
```

Теперь, когда реализован класс `UpdateQuantityCommand`, можно добавить возможность извлечения сведений о товарах, что и будет сделано в следующем подразделе.

Команда `GetInventoryCommand`

Команда `GetInventoryCommand` отличается от двух предыдущих тем, что не требует никаких параметров. Она использует зависимости `IUserInterface` и `IInventoryContext` для записи содержимого коллекции. Это показано следующим образом:

```
internal class GetInventoryCommand : NonTerminatingCommand  
{  
    private readonly IInventoryContext _context;  
    internal GetInventoryCommand(IUserInterface userInterface,  
        IInventoryContext context) : base(userInterface)  
    {  
        _context = context;  
    }  
  
    protected override bool InternalCommand()  
    {  
        foreach (var book in _context.GetBooks())  
        {  
            Interface.WriteMessage($"{book.Name,-30}\tQuantity:{book.Quantity}");  
        }  
  
        return true;  
    }  
}
```

Следующий после реализации `GetInventoryCommand` шаг — добавление нового теста.

GetInventoryCommandTest

Тест `GetInventoryCommandTest` описывает сценарий, в котором с помощью `GetInventoryCommand` извлекается коллекция книг. Тест будет определять ожидаемые сообщения (помните, что первый параметр предназначен для параметров, второй — для сообщений, а третий — для предупреждений), которые будут возникать при тестировании пользовательского интерфейса:

```
var expectedInterface = new Helpers.TestUserInterface(
    new List<Tuple<string, string>>(),
    new List<string>
    {
        "Gremlins"           \tQuantity:7",
        "Willowsong"         \tQuantity:3",
    },
    new List<string>()
);
```

Эти сообщения будут соответствовать макету репозитория следующим образом:

```
var context = new TestInventoryContext(new Dictionary<string, Book>
{
    { "Gremlins", new Book { Id = 1, Name = "Gremlins", Quantity = 7 } },
    { "Willowsong", new Book { Id = 2, Name = "Willowsong", Quantity = 3 } },
},
});
```

Модульный тест запускает команду с имитацией зависимостей. Он убеждается, что команда выполнена без ошибок и не является завершающей:

```
// создание экземпляра команды
var command = new GetInventoryCommand(expectedInterface, context);
var result = command.RunCommand();
```

```
Assert.IsFalse(result.shouldQuit, "GetInventory is not
    a terminating command.");
```

Ожидаемые сообщения проверяются в `TestUserInterface`, и, следовательно, единственное, что остается сделать модульному тесту, — убедиться, что никакие книги не были случайно добавлены или обновлены командой:

```
Assert.AreEqual(0, context.GetAddedBooks().Length, "GetInventory should
    Not have added any books.");
Assert.AreEqual(0, context.GetUpdatedBooks().Length, "GetInventory should
    not have updated any books.");
```

Теперь, когда добавлены подходящие модульные тесты для команды `GetInventoryCommand`, мы введем паттерн «Фабрика» в целях управления созданием команд.

Паттерн «Фабрика»

Следующий паттерн, применяемый разработчиками, — «Фабрика». Паттерн вводит «создателя» — порождающий класс, в обязанности которого входит создание экземпляров реализаций определенного типа. Его цель — инкапсулировать сложность конструирования типов. «Фабрика» обеспечивает большую гибкость при изменении приложения, ограничивая количество необходимых изменений по сравнению с ситуацией, когда конструирование происходит в вызывающем классе. Это связано с тем, что сложность конструирования находится в одном месте, а не распределена в нескольких местах по всему приложению.

В примере FlixOne `InventoryCommandFactory` реализует паттерн и скрывает детали создания каждого экземпляра различных `InventoryCommand`. В этом сценарии входные данные, полученные от консольного приложения, будут использоваться при определении конкретной реализации `InventoryCommand` для возврата. Важно отметить, что возвращаемым типом является абстрактный класс `InventoryCommand`. Таким образом вызывающий класс отгорожен от деталей конкретного класса.

`InventoryCommandFactory` показан в следующем блоке кода. Но пока сосредоточьтесь на методе `GetCommand`, ведь именно он реализует паттерн «Фабрика»:

```
public class InventoryCommandFactory : IInventoryCommandFactory
{
    private readonly IUserInterface _userInterface;
    private readonly IInventoryContext _context = InventoryContext.Instance;

    public InventoryCommandFactory(IUserInterface userInterface)
    {
        _userInterface = userInterface;
    }

    ...
}
```

`GetCommand` использует заданную строку при определении конкретной реализации `InventoryCommand` для возврата:

```
public InventoryCommand GetCommand(string input)
{
    switch (input)
    {
        case "q":
        case "quit":
            return new QuitCommand(_userInterface);
        case "a":
        case "addinventory":
            return new AddInventoryCommand(_userInterface, _context);
        case "g":
        case "getinventory":
```

```

        return new GetInventoryCommand(_userInterface, _context);
    case "u":
    case "updatequantity":
        return new UpdateQuantityCommand(_userInterface, _context);
    case "?":
        return new HelpCommand(_userInterface);
    default:
        return new UnknownCommand(_userInterface);
    }
}

```

Все команды требуют предоставления `IUserInterface`, но некоторым также нужен доступ к репозиторию. Они будут снабжены экземпляром «Одиночки» `IInventoryContext`.



«Фабрика» часто используется с интерфейсом в качестве возвращаемого типа. Он показан здесь как базовый класс `InventoryCommand`.

Модульные тесты

На первый взгляд, идея создания модульных тестов для такого простого класса кажется пустой тратой времени разработчиков. Но при построении модульных тестов были выявлены две важные проблемы, которые могли остаться незамеченными.

Первая проблема: `UnknownCommand`

Первый вопрос в том, что делать, когда поступает команда, которая не соответствует ни одному из определенных `InventoryCommand`. После просмотра требований разработчики заметили, что они пропустили это требование, как показано на данном снимке экрана (рис. 4.10).

- The application is a console application
 - print a welcome message that includes the version of the assembly
 - loops until a quit command is given
 - if a given command is not successful or not understood, then print a helpful message
- The application is driven by simple case-insensitive text commands
- Each command has a short form of a single character and a long form

Рис. 4.10

Разработчики решили ввести новый класс `InventoryCommand` под названием `UnknownCommand`, чтобы справиться с этой ситуацией. Классу `UnknownCommand` следует вывести в консоль предупреждающее сообщение (с помощью метода `WriteWarning`

из `IUserInterface`). Он не должен приводить к завершению работы приложения и должен возвращать значение `false`, указывающее, что команда не была успешно выполнена. Подробности реализации приведены в следующем коде:

```
internal class UnknownCommand : NonTerminatingCommand
{
    internal UnknownCommand(IUserInterface userInterface) : base(userInterface)
    {
    }

    protected override bool InternalCommand()
    {
        Interface.WriteWarning("Unable to determine the desired command.");

        return false;
    }
}
```

Модульный тест, созданный для `UnknownCommand`, будет проверять наличие предупреждающего сообщения, а также двух логических значений, возвращаемых методом `InternalCommand`:

```
[TestClass]
public class UnknownCommandTests
{
    [TestMethod]
    public void UnknownCommand_Successful()
    {
        var expectedInterface = new Helpers.TestUserInterface(
            new List<Tuple<string, string>>(),
            new List<string>(),
            new List<string>
            {
                "Unable to determine the desired command."
            }
        );

        // создание экземпляра команды
        var command = new UnknownCommand(expectedInterface);
        var result = command.RunCommand();

        Assert.IsFalse(result.shouldQuit, "Unknown is not a terminating command.");
        Assert.IsFalse(result.wasSuccessful, "Unknown should not complete Successfully.");
    }
}
```

`UnknownCommandTests` покрывает команды, требующие проверки. Далее будут проведены тесты `InventoryCommandFactory`.

InventoryCommandFactoryTests

`InventoryCommandFactoryTests` содержит модульные тесты, связанные с `InventoryCommandFactory`. Поскольку каждый тест будет иметь схожую схему построения `InventoryCommandFactory` и его `IUserInterface` и затем запуск `GetCommand`, создается общий метод, который станет выполняться при инициализации теста:

```
[TestInitialize]
public void Initialize()
{
    var expectedInterface = new Helpers.TestUserInterface(
        new List<Tuple<string, string>>(),
        new List<string>(),
        new List<string>()
    );

    Factory = new InventoryCommandFactory(expectedInterface);
}
```

Метод `Initialize` создает заглушку `IUserInterface` и устанавливает свойство `Factory`. Затем отдельные модульные тесты принимают простую форму проверки того, что возвращаемый объект является правильным типом. Сначала должен быть возвращен экземпляр класса `QuitCommand`, когда пользователь вводит "q" или "quit", как показано ниже:

```
[TestMethod]
public void QuitCommand_Successful()
{
    Assert.IsInstanceOfType(Factory.GetCommand("q"), typeof(QuitCommand),
        "q should be QuitCommand");
    Assert.IsInstanceOfType(Factory.GetCommand("quit"), typeof(QuitCommand),
        "quit should be QuitCommand");
}
```

Тестовый метод `QuitCommand_Successful` проверяет, что при выполнении метода `InventoryCommandFactory` под названием `GetCommand` возвращаемый объект является конкретным экземпляром `QuitCommand`. `HelpCommand` доступен только тогда, когда отправлен запрос "?":

```
[TestMethod]
public void HelpCommand_Successful()
{
    Assert.IsInstanceOfType(Factory.GetCommand("?"), typeof(HelpCommand),
        "h should be HelpCommand");
}
```

Разработчики добавили тест для `UnknownCommand`, который проверяет, как `InventoryCommand` будет реагировать, если значение не соответствует существующей команде:

```
[TestMethod]
public void UnknownCommand_Successful()
```

```

{
    Assert.IsInstanceOfType(Factory.GetCommand("add"), typeof(UnknownCommand),
        "unmatched command should be UnknownCommand");
    Assert.IsInstanceOfType(Factory.GetCommand("addinventory"),
        typeof(UnknownCommand),
        "unmatched command should be UnknownCommand");
    Assert.IsInstanceOfType(Factory.GetCommand("h"), typeof(UnknownCommand),
        "unmatched command should be UnknownCommand");
    Assert.IsInstanceOfType(Factory.GetCommand("help"), typeof(UnknownCommand),
        "unmatched command should be UnknownCommand");
}

```

С помощью методов тестирования мы теперь можем охватить сценарий, в котором дается команда, не соответствующая известной команде в приложении.

Вторая проблема: регистрозависимость команд

Вторая проблема была обнаружена при повторном пересмотре требований: выяснилось, что команды не должны быть чувствительны к регистру (рис. 4.11).

- The application is a console application
 - print a welcome message that includes the version of the assembly
 - loops until a quit command is given
 - if a given command is not successful or not understood, then print a helpful message
- The application is driven by simple case-insensitive text commands
- Each command has a short form of a single character and a long form

Рис. 4.11

С помощью теста для `UpdateInventoryCommand` было обнаружено, что `InventoryCommandFactory` чувствительна к регистру:

```

[TestMethod]
public void UpdateQuantityCommand_Successful()
{
    Assert.IsInstanceOfType(Factory.GetCommand("u"),
        typeof(UpdateQuantityCommand),
        "u should be UpdateQuantityCommand");
    Assert.IsInstanceOfType(Factory.GetCommand("updatequantity"),
        typeof(UpdateQuantityCommand),
        "updatequantity should be UpdateQuantityCommand");
    Assert.IsInstanceOfType(Factory.GetCommand("UpdaTEQuantity"),
        typeof(UpdateQuantityCommand),
        "UpdaTEQuantity should be UpdateQuantityCommand");
}

```

К счастью, это было легко решить, применив метод `ToLower()` к входным данным перед определением команды:

```
public InventoryCommand GetCommand(string input)
{
    switch (input.ToLower())
    {
        ...
    }
}
```

Этот сценарий подчеркивает как значимость метода **Factory**, так и тот факт, что модульные тесты позволяют проверять выполнение требований во время разработки, не полагаясь на пользовательское тестирование.

Функциональность .NET Core

Глава 3, а также первая часть данной главы проиллюстрировали паттерны «Банды четырех», не прибегая к использованию каких-либо фреймворков. Стоит обратить на это внимание, поскольку иногда во фреймворке нет конкретного паттерна или он недоступен к применению в конкретном сценарии. Кроме того, важно понимать, какие функциональные возможности предоставляет фреймворк, чтобы знать, когда использовать паттерн. В оставшейся части главы рассмотрим несколько предоставляемых .NET Core функций, поддерживающих некоторые из описанных выше паттернов.

Интерфейс IServiceCollection

.NET Core разрабатывалась с *внедрением зависимостей*¹ (Dependency Injection, DI), встроенным в платформу. Как правило, запуск приложения .NET Core содержит настройку DI для приложения, которое в основном включает создание коллекции сервисов. Платформа использует их для предоставления зависимостей, когда они требуются приложению. Сервисы обеспечивают основу надежной платформы *инверсии управления* (Inversion of Control, IoC) и, возможно, являются одной из самых крутых функций .NET Core. В этом разделе мы завершим консольное приложение и продемонстрируем, как .NET Core поддерживает построение сложной платформы IoC на основе интерфейса `IServiceCollection`.

Данный интерфейс используется для определения сервисов, доступных контейнеру, реализующему интерфейс `IServiceProvider`. Сами сервисы — это типы, которые будут внедряться во время выполнения, когда это требуется приложению. Например, `ConsoleUserInterface`, определенный ранее, будет сервисом, внедренным во время выполнения. Это показано в следующем коде:

```
IServiceCollection services = new ServiceCollection();
services.AddTransient<IUserInterface, ConsoleUserInterface>();
```

¹ См.: *Симан М., Дерсен ван С.* Внедрение зависимостей на платформе .NET. 2-е изд. — СПб.: Питер, 2021.

В предыдущем коде `ConsoleUserInterface` добавляется как сервис, реализующий интерфейс `IUserInterface`. Если DI предоставляет другой тип, который требует зависимости `IUserInterface`, то будет использоваться интерфейс `ConsoleUserInterface`. Например, `InventoryCommandFactory` также добавляется к сервисам, как показано в коде ниже:

```
services.AddTransient<IInventoryCommandFactory, InventoryCommandFactory>();
```

`InventoryCommandFactory` имеет конструктор, требующий реализации `IUserInterface`:

```
public class InventoryCommandFactory : IInventoryCommandFactory
{
    private readonly IUserInterface _userInterface;

    public InventoryCommandFactory(IUserInterface userInterface)
    {
        _userInterface = userInterface;
    }
    ...
}
```

Позднее запрашивается экземпляр `InventoryCommandFactory`:

```
IServiceProvider serviceProvider = services.BuildServiceProvider();
var service = serviceProvider.GetService<IInventoryCommandFactory>();
service.GetCommand("a");
```

Затем экземпляр `IUserInterface` (в данном приложении это зарегистрированный `ConsoleUserInterface`) создается и передается конструктору `InventoryCommandFactory`.



Существуют различные типы времени жизни сервисов, которые можно указать при регистрации сервиса. Время жизни определяет, как будут создаваться экземпляры типов, включая временные, сервисы с областью применения и «Одиночка». Что такое временный сервис? Это такой сервис, который создается каждый раз в момент запроса. Области применения будут показаны позже, когда мы рассмотрим паттерны для создания сайтов и, в частности, ситуацию, где сервисы создаются по веб-запросу. Сервис-«одиночка» ведет себя аналогично паттерну «Одиночка», который описывался ранее. Он также будет рассмотрен позже в данной главе.

Интерфейс `CatalogService`

Данный интерфейс представляет консольное приложение, которое создает команда, и описывается как имеющий один метод запуска, что и показано в `ICatalogService`:

```
interface ICatalogService
{
    void Run();
}
```


Сервис имеет две зависимости: `IUserInterface` и `IInventoryCommandFactory`. Они будут введены в конструктор и сохранены в виде локальных переменных:

```
public class CatalogService : ICatalogService
{
    private readonly IUserInterface _userInterface;
    private readonly IInventoryCommandFactory _commandFactory;

    public CatalogService(IUserInterface userInterface,
        IInventoryCommandFactory commandFactory)
    {
        _userInterface = userInterface;
        _commandFactory = commandFactory;
    }
    ...
}
```

Метод `Run` основан на более раннем дизайне, показанном в главе 3. Он печатает приветствие, а затем закидывается до тех пор, пока пользователь не введет команду выхода. Каждый проход цикла выполнит команду, и если она не будет выполнена успешно, то он напечатает справочное сообщение:

```
public void Run()
{
    Greeting();

    var response = _commandFactory.GetCommand("?").RunCommand();

    while (!response.shouldQuit)
    {
        // посмотрите на ошибку с ToLower()
        var input = _userInterface.ReadValue("> ").ToLower();
        var command = _commandFactory.GetCommand(input);

        response = command.RunCommand();

        if (!response.wasSuccessful)
        {
            _userInterface.WriteMessage("Enter ? to view options.");
        }
    }
}
```

Теперь, когда интерфейс `CatalogService` готов, очередной шаг — собрать все вместе. В следующем подразделе это будет сделано с помощью .NET Core.

IServiceProvider

С определением `CatalogService` разработчики наконец могут объединить все. Запуск всех приложений, то есть исполняемых программ, прописывается в методе `Main`, и .NET Core не является исключением. Программа показана в следующем коде:

```

class Program
{
    private static void Main(string[] args)
    {
        IServiceCollection services = new ServiceCollection();
        ConfigureServices(services);
        IServiceProvider serviceProvider = services.BuildServiceProvider();

        var service = serviceProvider.GetService<ICatalogService>();
        service.Run();

        Console.WriteLine("CatalogService has completed.");
    }

    private static void ConfigureServices(IServiceCollection services)
    {
        // Добавляем сервисы приложения
        services.AddTransient<IUserInterface, ConsoleUserInterface>();
        services.AddTransient<ICatalogService, CatalogService>();
        services.AddTransient<IInventoryCommandFactory,
            InventoryCommandFactory>();
    }
}

```

В методе `ConfigureServices` в контейнер IoC добавляются различные типы, включая `ConsoleUserInterface`, `CatalogService` и `InventoryCommandFactory`. Классы `ConsoleUserInterface` и `InventoryCommandFactory` будут внедрены по мере необходимости. Класс `CatalogService` будет явно извлечен из интерфейса `IServiceProvider`, построенного из объекта `ServiceCollection`, содержащего добавленные типы. Программа *выполняется* до тех пор, пока не завершится работа метода `Run`, принадлежащего сервису `CatalogService`.



В главе 5 «Одиночка» будет рассмотрен повторно применительно к использованию встроенных возможностей .NET Core с помощью `IServiceCollection`, метода `AddSingleton` для управления экземпляром `InventoryContext`.

Консольное приложение

Консольное приложение, запускаемое из командной строки, простое, однако является основой для хорошо продуманного кода, который придерживается принципов SOLID, рассмотренных в главе 3. При запуске приложение выдает простое приветствие и выводит на экран справочное сообщение, включая поддерживаемые команды и примеры (рис. 4.12).

Затем приложение циклически перебирает команды до тех пор, пока не будет получена команда выхода. На рис. 4.13 показана его функциональность.

```

*****
*                                     *
*           Welcome to FlixOne Inventory Management System                 *
*                                     v1.0.0.0                             *
*****

USAGE:
    addinventory (a)
    getinventory (g)
    updatequantity (u)
    quit (q)
    ?
Examples:
    New Inventory
    > addinventory
    Enter name:The Meaning of Life

    Get Inventory
    > getinventory
    The Meaning of Life           Quantity:10
    The Life of a Ninja          Quantity:2

    Update Quantity (Increase)
    > updatequantity
    Enter name:The Meaning of Life
    11
    11 added to quantity

    Update Quantity (Decrease)
    > updatequantity
    Enter name:The Life of a Ninja
    -3
    3 removed from quantity

> _

```

Рис. 4.12

```

> addinventory
Enter name:Godfather
> u
Enter name:Godfather
Enter quantity:12
> g
Godfather                               Quantity:12
> notsurecommand
Unable to determine the desired command.
Enter ? to view options.
> q
Thank you for using FlixOne Inventory Management System
CatalogService has completed.

```

Рис. 4.13

Это не самое впечатляющее из консольных приложений, но оно послужило иллюстрацией многих принципов и паттернов.

Резюме

Как и в главе 3, в этой главе мы продолжили описывать построение консольного приложения для инвентаризации FlixOne, чтобы показать практические примеры использования паттернов проектирования в ООП. В центре внимания были паттерны «Одиночка» и «Фабрика» легендарной «Банды четырех». Они оба играют особенно важную роль в приложениях .NET Core и будут часто применяться в следующих главах. В текущей главе было также рассмотрено использование встроенных функций фреймворка для предоставления контейнера IoC.

В конце главы представлено консольное приложение для инвентаризации, основанное на требованиях, определенных в главе 3. Они легли в основу модульных тестов, созданных в обеих главах, и применялись для иллюстрации TDD. Разработчики достаточно уверены в том, что приложение пройдет *приемочное тестирование пользователей* (user acceptance testing, UAT) благодаря комплекту тестов, проверяющих функции, необходимые на данном этапе проектирования.

В следующей главе мы продолжим описывать разработку нашего консольного приложения. Основное внимание сместится с базовых паттернов ООП на использование фреймворка .NET Core для реализации различных паттернов. Так, паттерн «Одиночка», описанный в этой главе, будет рефакторизован, чтобы использовать возможность его создания средствами `IServiceCollection`. К тому же мы подробнее рассмотрим возможности DI. Кроме того, приложение будет расширено в целях поддержки логирования с помощью различных провайдеров.

Вопросы

Следующие вопросы позволят вам усвоить информацию, представленную в этой главе.

1. Приведите пример, почему использование «Одиночки» не подходит для ограничения доступа к общему ресурсу.
2. Верно ли следующее утверждение? Почему да или почему нет? `ConcurrentDictionary` предотвращает обновление элементов коллекции несколькими потоками одновременно.
3. Что такое состояние гонки и почему его следует избегать?
4. Как паттерн «Фабрика» помогает упростить код?
5. Требуют ли приложения .NET Core контейнеров IoC от сторонних производителей?

5

Реализация паттернов проектирования в .NET Core

В предыдущей главе мы продолжили создавать приложение FlixOne путем включения дополнительных паттернов. Было использовано больше паттернов «Банды четырех», в частности «Одиночка» и «Фабрика». Паттерн «Одиночка» служил для демонстрации репозитория, который, в свою очередь, применялся для обслуживания коллекции книг. Паттерн «Фабрика» использовался для дальнейшего изучения принципов *внедрения зависимости* (DI). Разработка первичного консольного приложения для инвентаризации была завершена с помощью фреймворка .NET Core с целью облегчить внедрение контейнера *инверсии управления* (IoC).

В этой главе мы продолжим работу над FlixOne, а также рассмотрим возможности .NET Core. Паттерн «Одиночка» из предыдущей главы будет повторно рассмотрен и создан как сервис с использованием встроенного в платформу .NET Core времени жизни «Синглтон»-сервиса. С помощью DI фреймворка будет показан паттерн «Конфигурация», а также объяснено *внедрение через конструктор* (constructor injection, CI).

В этой главе будут рассмотрены следующие темы:

- ❑ время жизни сервисов в .NET Core;
- ❑ реализующая фабрика.

Технические требования

В этой главе содержатся примеры кода, объясняющие принципы качественной разработки. Код упрощен и предназначен лишь для демонстрации. Большинство примеров основаны на консольном приложении .NET Core, написанном на C#.

Чтобы запустить и выполнить код, вам потребуется следующее:

- ❑ Visual Studio 2019 (вы также можете запустить приложение, используя Visual Studio 2017 версии 3 и новее);
- ❑ .NET Core;
- ❑ SQL Server (в этой главе применяется Express Edition).

Установка Visual Studio

Чтобы запустить примеры кода, вам необходимо установить Visual Studio. Для этого выполните следующие шаги.

1. Скачайте Visual Studio по ссылке docs.microsoft.com/ru-ru/visualstudio/install/install-visual-studio.
2. Следуйте инструкциям по установке. Доступны различные версии Visual Studio; в этой главе мы используем Visual Studio для Windows.

Вы также можете применять другую интегрированную среду разработки по вашему усмотрению.

Установка .NET Core

Если вы еще не установили .NET Core, то следуйте инструкции ниже.

1. Скачайте .NET Core по ссылке dotnet.microsoft.com/download.
2. Далее следуйте указаниям по установке соответствующей библиотеки: dotnet.microsoft.com/download/dotnet-core/2.2.



Полная версия исходного кода доступна на GitHub. Код, представленный в данной главе, может быть неполным, поэтому мы рекомендуем вам скачать полный код для запуска примеров (github.com/PacktPublishing/Hands-On-Design-Patterns-with-C-and-.NET-Core/tree/master/Chapter5).

Время жизни сервисов в .NET Core

Фундаментальная концепция, которую следует понимать при работе с DI .NET Core, — это время жизни сервиса. Оно определяет, как управляется зависимость, то есть как часто создается. В качестве иллюстрации рассмотрим DI как управление контейнером зависимостей. Зависимость — просто класс, о котором знает DI, поскольку класс был *зарегистрирован* с помощью DI. В DI, предоставляемом .NET Core, регистрация происходит благодаря использованию следующих трех методов `IServiceCollection`:

- `AddTransient<TService, TImplementation>();`
- `AddScoped<TService, TImplementation>();`
- `AddSingleton<TService, TImplementation>().`

Интерфейс `IServiceCollection` представляет собой набор зарегистрированных описаний сервисов, в основном содержащих зависимость, а также то, когда DI

должно предоставить зависимость. Например, при запросе `TService` поставляется (то есть внедряется) `TImplementation`.

В этом разделе мы рассмотрим три времени жизни сервиса и проиллюстрируем их с помощью модульных тестов. Мы также рассмотрим, как можно использовать «Фабрику» для создания экземпляров зависимостей.

Временная зависимость (Transient)

Transient означает, что каждый раз, когда DI получает запрос, создается новый экземпляр зависимости. Чаще всего именно это время жизни наиболее рационально, поскольку большинство классов должны быть спроектированы как легковесные сервисы без состояния. В ситуациях, когда состояние должно сохраняться между ссылками и/или когда для создания нового экземпляра требуются значительные усилия, другой срок жизни может подходить лучше.

Время жизни области применения (Scoped)

В .NET Core существует понятие области применения (действия), которую можно рассматривать как контекст или границу для выполняемого процесса. В ряде реализаций .NET Core данная область определяется неявно, поэтому вы можете не знать о ее создании. Так, в ASP.NET Core она создается для каждого принятого веб-запроса. Это значит, что если зависимость имеет время жизни области применения, то она будет создаваться только один раз для каждого веб-запроса. Как следствие, если одна и та же зависимость используется несколько раз для одного и того же веб-запроса, то будет применяться совместно.

Далее в этой главе мы явным образом создадим область действия, чтобы проиллюстрировать соответствующее время жизни. В приложении ASP.NET Core мы используем те же концепции, которые применимы и в модульном тесте.

«Одиночка» в .NET Core

В .NET Core «Одиночка» реализован таким образом, что зависимость инстанцируется единожды, то есть точно так же, как «Одиночка» из предыдущей главы. Как и в предыдущей главе, класс `singleton` должен быть потокобезопасным. «Фабричный метод», используемый для создания «Одиночки», гарантированно вызывается только один раз и одним потоком.

Вернемся к FlixOne

Чтобы проиллюстрировать DI в .NET Core, необходимо внести некоторые изменения в приложение FlixOne. Сначала следует обновить класс `InventoryContext`,

чтобы больше не реализовывать «Одиночку» (поскольку мы сделаем это с помощью DI в .NET Core):

```
public class InventoryContext : IInventoryContext
{
    public InventoryContext()
    {
        _books = new ConcurrentDictionary<string, Book>();
    }

    private readonly static object _lock = new object();

    private readonly IDictionary<string, Book> _books;

    public Book[] GetBooks()
    {
        return _books.Values.ToArray();
    }

    ...
}
```

Методы `AddBook` и `UpdateQuantity` показаны ниже:

```
public bool AddBook(string name)
{
    _books.Add(name, new Book {Name = name});
    return true;
}

public bool UpdateQuantity(string name, int quantity)
{
    lock (_lock)
    {
        _books[name].Quantity += quantity;
    }

    return true;
}
```

Есть пара особенностей, на которые стоит обратить внимание. Защищенный конструктор стал публичным. Это позволяет инстанцировать класс объектами вне класса. Кроме того, статическое свойство `Instance` и приватное статическое поле `_instance` удалены, в то время как приватное поле `_lock` осталось. Аналогично случаю с «Одиночкой» из предыдущей главы этим гарантируется только то, как инстанцируется класс. Это не мешает параллельному доступу к методам.



Интерфейсы `IInventoryContext` и `InventoryContext`, а также класс `Book` сделаны публичными, поскольку DI определяется во внешнем проекте.

Класс `InventoryCommandFactory`, используемый для возврата команд, обновлен, и в его конструктор вставлен экземпляр `InventoryContext`:

```
public class InventoryCommandFactory : IInventoryCommandFactory
{
    private readonly IUserInterface _userInterface;
    private readonly IInventoryContext _context;

    public InventoryCommandFactory(IUserInterface userInterface,
        IInventoryContext context)
    {
        _userInterface = userInterface;
        _context = context;
    }

    // GetCommand()
    ...
}
```

Метод `GetCommand` использует предоставленные входные данные, чтобы определить конкретную команду:

```
public InventoryCommand GetCommand(string input)
{
    switch (input.ToLower())
    {
        case "q":
        case "quit":
            return new QuitCommand(_userInterface);
        case "a":
        case "addinventory":
            return new AddInventoryCommand(_userInterface, _context);
        case "g":
        case "getinventory":
            return new GetInventoryCommand(_userInterface, _context);
        case "u":
        case "updatequantity":
            return new UpdateQuantityCommand(_userInterface, _context);
        case "?":
            return new HelpCommand(_userInterface);
        default:
            return new UnknownCommand(_userInterface);
    }
}
```

Как уже отмечалось, интерфейс `IInventoryContext` теперь будет поставляться контейнером DI, который определен в клиентском проекте. Консольное приложение теперь имеет дополнительную строку для создания «Одиночки» при запросе `IInventoryContext` с помощью класса `InventoryContext`:

```

class Program
{
    private static void Main(string[] args)
    {
        IServiceCollection services = new ServiceCollection();
        ConfigureServices(services);
        IServiceProvider serviceProvider = services.BuildServiceProvider();

        var service = serviceProvider.GetService<ICatalogService>();
        service.Run();

        Console.WriteLine("CatalogService has completed.");
        Console.ReadLine();
    }

    private static void ConfigureServices(IServiceCollection services)
    {
        // Добавляем сервисы приложения
        services.AddTransient<IUserInterface, ConsoleUserInterface>();
        services.AddTransient<ICatalogService, CatalogService>();
        services.AddTransient<IInventoryCommandFactory,
            InventoryCommandFactory>();

        services.AddSingleton<IInventoryContext, InventoryContext>();
    }
}

```

Теперь приложение работает с ручным тестом из предыдущей главы, но модульные тесты — это хороший способ понять, что достигается с помощью DI в .NET Core.



Примеры кода в данной главе представляют собой завершённый проект. Следующий раздел посвящён тестам `InventoryContext`. Тесты `IInventoryCommandFactory` тоже были модифицированы, но их изменения тривиальны, поэтому не освещаются здесь.

Модульные тесты

После изменений в классе `InventoryContext` у нас больше нет удобного свойства для получения единственного экземпляра класса. Это значит, что `InventoryContext.Instance` нужно заменить. В качестве первой попытки создадим метод, возвращающий новый экземпляр `InventoryContext`, используя `GetInventoryContext()` вместо `InventoryContext.Instance`:

```

private IInventoryContext GetInventoryContext()
{
    return new InventoryContext();
}

```

Как и ожидалось, модульный тест завершился с ошибкой: The given key was not present in the dictionary (Данный ключ отсутствовал в словаре) (рис. 5.1).

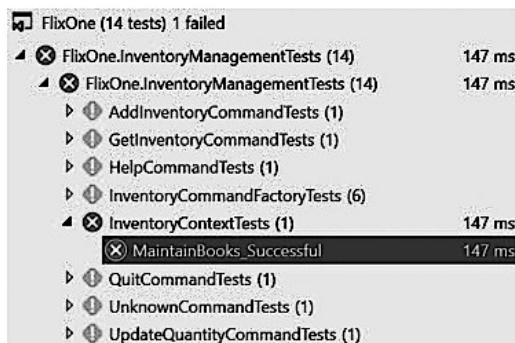


Рис. 5.1

Как мы видели в предыдущей главе, это происходит потому, что список книг `InventoryContext` пуст каждый раз при создании класса `InventoryContext`. Вот почему нам нужно создать контекст с помощью «Одиночки».

Обновим метод `GetInventoryContext()` так, чтобы он поставлял экземпляр интерфейса `IInventoryContext` с помощью DI .NET Core:

```
private IInventoryContext GetInventoryContext()
{
    IServiceCollection services = new ServiceCollection();
    services.AddSingleton<IInventoryContext, InventoryContext>();
    var provider = services.BuildServiceProvider();

    return provider.GetService<IInventoryContext>();
}
```

В обновленном методе создается экземпляр класса `ServiceCollection`, содержащий все зарегистрированные зависимости. Класс `InventoryContext` зарегистрирован как «Одиночка», поставляемый при запросе зависимости `IInventoryContext`. Затем генерируется экземпляр `ServiceProvider`, на самом деле выполняющий DI на основе регистраций в интерфейсе `IServiceCollection`. Последний шаг — поставка класса `InventoryContext` при запросе интерфейса `IInventoryContext`.



Нужно добавить библиотеку `Microsoft.Extensions.DependencyInjection` в проект `InventoryManagementTests`, чтобы иметь возможность ссылаться на DI-компоненты .NET Core.

К сожалению, модульный тест все равно не проходит и приводит к той же ошибке: The given key was not present in the dictionary (Данный ключ отсутствовал в словаре).

Это потому, что мы создаем новый экземпляр DI-фреймворка каждый раз, когда запрашивается `IInventoryContext`. То есть несмотря на то, что наша зависимость — «Одиночка», каждый экземпляр `ServiceProvider` предоставляет новый экземпляр класса `InventoryContext`. Чтобы обойти это, мы создадим `IServiceCollection` при первом запуске теста, а затем будем использовать ту же ссылку во время теста:

```
ServiceProvider Services { get; set; }

[TestInitialize]
public void Startup()
{
    IServiceCollection services = new ServiceCollection();
    services.AddSingleton<IInventoryContext, InventoryContext>();
    Services = services.BuildServiceProvider();
}
```



Использование атрибута `TestInitialize` — отличный способ разделить функциональность, которую требуют несколько `TestMethod` внутри класса `TestClass`. Метод будет запущен перед каждым тестом.

Теперь, когда есть ссылка на тот же экземпляр `ServiceProvider`, мы можем обновить его, чтобы получить зависимость. Ниже показано, как мы обновили метод `AddBook()`:

```
public Task AddBook(string book)
{
    return Task.Run(() =>
    {
        Assert.IsTrue(Services.GetService<IInventoryContext>().AddBook(book));
    });
}
```

Модульный тест теперь проходит успешно, так как во время его выполнения создается только один экземпляр класса `InventoryContext` (рис. 5.2).

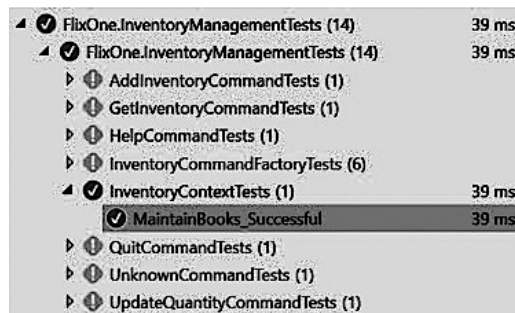


Рис. 5.2

В этом подразделе мы показали, как относительно легко реализовать «Одиночку» с помощью встроенного интерфейса DI. Важно понимать, когда следует использовать паттерн. В следующем подразделе мы подробнее рассмотрим ограниченную область применения, чтобы получить более полное представление о времени жизни сервисов.

Что такое время жизни области применения

В приложениях, в которых одновременно выполняется несколько процессов, понимание времени жизни сервисов очень важно как для функциональных, так и для нефункциональных требований. В предыдущем модульном тесте было показано, что без правильного срока сервиса `InventoryContext` работал нежелательным образом и приводил к ситуации, вызывающей ошибки. Точно так же неправильное использование времени жизни может привести к тому, что приложения будут плохо масштабироваться. В целом в многопроцессорных решениях следует избегать состояния взаимной блокировки.

Проиллюстрируем это. Допустим, приложение FlixOne предоставлено нескольким сотрудникам. Теперь проблема в том, как выполнить блокировку для нескольких приложений и иметь одно собранное состояние. С нашей точки зрения, это будет один класс `InventoryContext`, совместно используемый несколькими приложениями. Конечно, именно здесь имеет смысл изменить решение в пользу применения общего хранилища (скажем, базы данных) и/или преобразования в веб-приложение. Мы рассмотрим базы данных и паттерны веб-приложений в следующих главах, а сейчас, поскольку мы обсуждаем время жизни сервисов, имеет смысл описать их в терминах веб-приложений более подробно.

На рис. 5.3 изображено веб-приложение, принимающее два запроса.

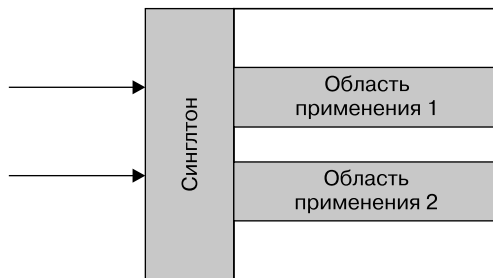


Рис. 5.3

С точки зрения времени жизни для обоих запросов будет доступно время жизни «Синглтон»-сервиса, в то время как для каждого запроса доступно свое время

жизни области применения. Главное, что здесь нужно отметить, — сборка мусора. Временные зависимости отмечаются как освобождаемые после того, как на объект больше нет ссылок. А вот зависимости, созданные со временем жизни области применения, не отмечаются как освобождаемые до тех пор, пока не завершится веб-запрос. Кроме того, зависимости, созданные со временем жизни «Одиночки», не отмечаются для освобождения до конца работы приложения. Вдобавок, как показано на рис. 5.4, важно помнить, что зависимости в .NET Core не являются общими для экземпляров сервера в веб-саду (web garden) или веб-ферме (web farm).

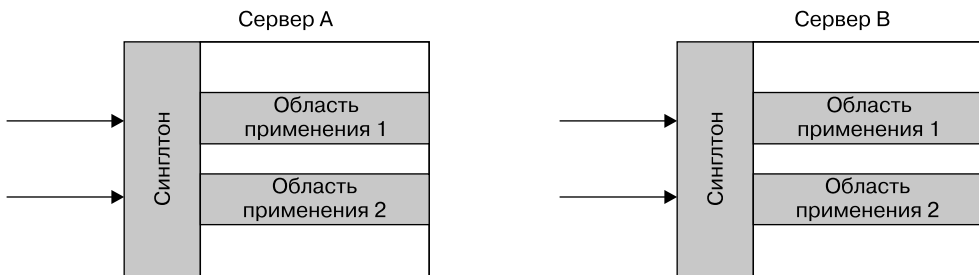


Рис. 5.4

В следующих главах мы показываем различные подходы к общему состоянию, включая использование общего кэша, баз данных и других видов хранилищ.

Реализующая фабрика

DI в .NET Core поддерживает возможность указать *реализующую фабрику* при регистрации зависимости. Это позволяет контролировать создание предоставляемой сервисом зависимости. Делается это при регистрации с помощью расширения интерфейса `IServiceCollection`:

```
public static IServiceCollection AddSingleton<TService,
TImplementation>(this IServiceCollection services,
    Func<IServiceProvider, TImplementation> implementationFactory)
    where TService : class
    where TImplementation : class, TService;
```

Когда требуется зависимость, расширение `AddSingleton` получает как класс, который должен быть зарегистрирован, так и класс, который должен быть поставлен. Интересно отметить, что DI в .NET Core будет поддерживать зарегистрированные сервисы и доставлять реализацию либо по запросу, либо в рамках инстанцирования одной из зависимостей. Это автоматическое инстанцирование называется *внедрением через конструктор* (constructor injection, CI). Мы рассмотрим оба примера в следующих подразделах.

Интерфейс IInventoryContext

В качестве примера вернемся к классу `InventoryContext`, который используется для управления инвентаризацией книг, и разделим его операции на операции чтения и записи, выполняемые с коллекцией книг. Итак, интерфейс `IInventoryContext` разбивается на `IInventoryReadContext` и `IInventoryWriteContext`:

```
using FlixOne.InventoryManagement.Models;

namespace FlixOne.InventoryManagement.Repository
{
    public interface IInventoryContext : IInventoryReadContext,
        IInventoryWriteContext { }

    public interface IInventoryReadContext
    {
        Book[] GetBooks();
    }

    public interface IInventoryWriteContext
    {
        bool AddBook(string name);
        bool UpdateQuantity(string name, int quantity);
    }
}
```

Интерфейс IInventoryReadContext

Интерфейс `IInventoryReadContext` содержит операцию чтения книг, а `IInventoryWriteContext` — операции изменения коллекции. Оригинальный интерфейс `IInventoryContext` создавался для удобства на случай, когда классу требуются оба типа зависимостей.



В следующих главах мы рассмотрим паттерны, использующие преимущества разделения контекста, включая паттерн «Разделение ответственности команд и запросов» (Command and Query Responsibility Segregation, CQRS).

Нам требуются некоторые изменения. В первых классах, требующих только чтения коллекции, конструкторы обновляются с помощью интерфейса `IInventoryReadContext`, как показано в классе `GetInventoryCommand`:

```
internal class GetInventoryCommand : NonTerminatingCommand
{
    private readonly IInventoryReadContext _context;
```

```

internal GetInventoryCommand(IUserInterface userInterface,
    IInventoryReadContext context) : base(userInterface)
{
    _context = context;
}

protected override bool InternalCommand()
{
    foreach (var book in _context.GetBooks())
    {
        Interface.WriteMessage($"{book.Name, -30}\tQuantity:{book.Quantity}");
    }

    return true;
}
}

```

Интерфейс IInventoryWriteContext

Аналогично классы для изменения коллекции книг обновляются до интерфейса **IInventoryWriteContext**, как показано на примере **AddInventoryCommand**:

```

internal class AddInventoryCommand : NonTerminatingCommand,
    IParameterisedCommand
{
    private readonly IInventoryWriteContext _context;

    internal AddInventoryCommand(IUserInterface userInterface,
        IInventoryWriteContext context) : base(userInterface)
    {
        _context = context;
    }

    public string InventoryName { get; private set; }

    ...
}

```

Ниже показан код методов **GetParameters** и **InternalCommand**:

```

/// <summary>
/// AddInventoryCommand requires name
/// </summary>
/// <returns></returns>
public bool GetParameters()
{
    if (string.IsNullOrEmpty(InventoryName))
        InventoryName = GetParameter("name");
}

```



```

        return !string.IsNullOrEmpty(InventoryName);
    }

    protected override bool InternalCommand()
    {
        return _context.AddBook(InventoryName);
    }

```

Обратите внимание на метод `InternalCommand`, в котором книга добавляется в число товаров с заданным названием, хранящимся в параметре `InventoryName`.

Посмотрим на «Фабрику» для команд.

Класс `InventoryCommandFactory`

Класс `InventoryCommandFactory` — это реализация «Фабрики» с помощью классов .NET. Он требует как чтения коллекции, так и записи в коллекцию книг:

```

public class InventoryCommandFactory : IInventoryCommandFactory
{
    private readonly IUserInterface _userInterface;
    private readonly IInventoryContext _context;

    public InventoryCommandFactory(IUserInterface userInterface,
        IInventoryContext context)
    {
        _userInterface = userInterface;
        _context = context;
    }

    public InventoryCommand GetCommand(string input)
    {
        switch (input.ToLower())
        {
            case "q":
            case "quit":
                return new QuitCommand(_userInterface);
            case "a":
            case "addinventory":
                return new AddInventoryCommand(_userInterface, _context);
            case "g":
            case "getinventory":
                return new GetInventoryCommand(_userInterface, _context);
            case "u":
            case "updatequantity":
                return new UpdateQuantityCommand(_userInterface, _context);
            case "?":

```

```

        return new HelpCommand(_userInterface);
    default:
        return new UnknownCommand(_userInterface);
    }
}

```

Интересно, что класс на самом деле не требовал модификации, так как полиморфизм обрабатывает приведение из `IInventoryContext` к `IInventoryReadContext` и `IInventoryWriteContext`.

Вместе с этим нужно изменить и регистрацию зависимостей, связанных с `InventoryContext`, используя реализующую фабрику:

```

private static void ConfigureServices(IServiceCollection services)
{
    // Добавляем сервисы приложения
    ...

    var context = new InventoryContext();
    services.AddSingleton<IInventoryReadContext,
        InventoryContext>(p => context);
    services.AddSingleton<IInventoryWriteContext,
        InventoryContext>(p => context);
    services.AddSingleton<IInventoryContext,
        InventoryContext>(p => context);
}

```

Для всех трех интерфейсов мы используем один и тот же экземпляр `InventoryContext`. Этот экземпляр создается один раз с помощью расширения с реализующей фабрикой. Он поставляется, когда запрашивается зависимость `IInventoryReadContext`, `IInventoryWriteContext` или `IInventoryContext`.

Класс `InventoryCommand`

Класс `InventoryCommandFactory` помогает показать, как паттерн «Фабрика» можно написать с помощью .NET, но вернемся к этому сейчас, когда мы используем платформу .NET Core. Наше требование — вернуть конкретную реализацию `InventoryCommand` на основе переданного строкового значения. Это можно сделать несколькими способами. В этом подразделе мы приведем три примера:

- ❑ реализующая фабрика, использующая функцию;
- ❑ применение сервисов;
- ❑ применение сторонних контейнеров.

Реализующая фабрику, использующая функцию

Реализующая фабрика из метода `GetService()` может использоваться для определения типа возвращаемого класса `InventoryCommand`. В этом же примере новый статический метод создается прямо в классе `InventoryCommand`:

```
public static Func<IServiceProvider, Func<string, InventoryCommand>>
GetInventoryCommand =>
provider => input =>
{
    switch (input.ToLower())
    {
        case "q":
        case "quit":
            return new QuitCommand(provider.GetService<IUserInterface>());
        case "a":
        case "addinventory":
            return new AddInventoryCommand(provider.
                GetService<IUserInterface>(),
                provider.GetService<IInventoryWriteContext>());
        case "g":
        case "getinventory":
            return new GetInventoryCommand(provider.
                GetService<IUserInterface>(),
                provider.GetService<IInventoryReadContext>());
        case "u":
        case "updatequantity":
            return new UpdateQuantityCommand(provider.
                GetService<IUserInterface>(),
                provider.GetService<IInventoryWriteContext>());
        case "?":
            return new HelpCommand(provider.GetService<IUserInterface>());
        default:
            return new UnknownCommand(provider.GetService<IUserInterface>());
    }
};
```

Это немного сложно прочитать, если вы не знакомы с лямбда-выражениями, так что объясним код немного подробнее. Прежде всего вернемся к синтаксису `AddSingleton`:

```
public static IServiceCollection AddSingleton<TService,
    TImplementation>(this IServiceCollection services,
    Func<IServiceProvider, TImplementation> implementationFactory)
    where TService : class
    where TImplementation : class, TService;
```

Здесь показано, что параметр расширения `AddSingleton` — это функция:

```
Func<IServiceProvider, TImplementation> implementationFactory
```

Это значит, что следующие примеры кода эквивалентны:

```
services.AddSingleton<IInventoryContext, InventoryContext>(provider =>
    new InventoryContext());

services.AddSingleton<IInventoryContext, InventoryContext>
(GetInventoryContext);
```

Посмотрите на код метода `GetInventoryContext` ниже:

```
static Func<IServiceProvider, InventoryContext> GetInventoryContext =>

provider =>
{
    return new InventoryContext();
};
```

В нашем примере конкретные типы `InventoryCommand` помечены как внутренние в проекте `FlixOne.InventoryManagement`, поэтому `FlixOne.InventoryManagementClient` не имеет к ним прямого доступа. Вот почему новый статический метод создавался в классе `FlixOne.InventoryManagement.InventoryCommand`, вернувшем этот тип:

```
Func<IServiceProvider, Func<string, InventoryCommand>>
```

Это значит, что при запросе сервиса для определения конкретного типа будет предоставлена строка. Изменившаяся зависимость означает, что конструктор `CatalogService` требует обновления:

```
public CatalogService(IUserInterface userInterface, Func<string,
    InventoryCommand> commandFactory)
{
    _userInterface = userInterface;
    _commandFactory = commandFactory;
}
```

Теперь, когда введенная пользователем строка попадает в зависимость `CommandFactory`, поступает корректная команда:

```
while (!response.shouldQuit)
{
    // посмотрите на ошибку с ToLower()
    var input = _userInterface.ReadValue("> ").ToLower();
    var command = _commandFactory(input);

    response = command.RunCommand();

    if (!response.wasSuccessful)
```

```

    {
        _userInterface.WriteMessage("Enter ? to view options.");
    }
}

```

Кроме того, были обновлены модульные тесты, связанные с «Фабрикой». Для сравнения из существующего класса `InventoryCommandFactoryTests` был создан новый класс `InventoryCommandFunctionTests`. Шаг инициализации показан в коде ниже, изменения выделены:

ServiceProvider Services { get; set; }

```

[TestInitialize]
public void Startup()
{
    var expectedInterface = new Helpers.TestUserInterface(
        new List<Tuple<string, string>>(),
        new List<string>(),
        new List<string>()
    );

    IServiceCollection services = new ServiceCollection();
    services.AddSingleton<IInventoryContext, InventoryContext>();
    services.AddTransient<Func<string,
        InventoryCommand>>>(InventoryCommand.GetInventoryCommand);

    Services = services.BuildServiceProvider();
}

```

Для получения сервиса по предоставленной строке были обновлены отдельные тесты. Это показано ниже с помощью `QuitCommand`:

```

[TestMethod]
public void QuitCommand_Successful()
{
    Assert.IsInstanceOfType(Services.GetService<Func<string,
        InventoryCommand>>>().Invoke("q"),
        typeof(QuitCommand),
        "q should be QuitCommand");

    Assert.IsInstanceOfType(Services.GetService<Func<string,
        InventoryCommand>>>().Invoke("quit"),
        typeof(QuitCommand),
        "quit should be QuitCommand");
}

```

Оба теста проверяют, что возвращаемые сервисы имеют тип `QuitCommand`, когда провайдеру подается "q" или "quit".

Применение сервисов

Класс `ServiceProvider` предоставляет метод `Services`, который можно использовать для определения соответствующего сервиса, когда для одного и того же типа зарегистрировано несколько зависимостей. В этом примере будет предложен другой подход с `InventoryCommands`. В связи с масштабом рефакторинга это будет сделано с помощью новых классов. Они создаются только в целях иллюстрации данного подхода.

В проекте модульных тестов была создана новая папка `ImplementationFactoryTests`, содержащая классы для данного раздела книги. В ней создан новый базовый класс для `InventoryCommand`:

```
public abstract class InventoryCommand
{
    protected abstract string[] CommandStrings { get; }
    public virtual bool IsCommandFor(string input)
    {
        return CommandStrings.Contains(input.ToLower());
    }
}
```

Концепция, лежащая в основе этого нового класса, заключается в том, что дочерние классы будут сами определять, на какие строки реагировать. Например, `QuitCommand` будет реагировать на строки "q" и "quit":

```
public class QuitCommand : InventoryCommand
{
    protected override string[] CommandStrings => new[] { "q", "quit" };
}
```

Ниже показаны классы `GetInventoryCommand`, `AddInventoryCommand`, `UpdateQuantityCommand` и `HelpCommand`, следующие этому подходу:

```
public class GetInventoryCommand : InventoryCommand
{
    protected override string[] CommandStrings =>
        new[] { "g", "getinventory" };
}

public class AddInventoryCommand : InventoryCommand
{
    protected override string[] CommandStrings =>
        new[] { "a", "addinventory" };
}

public class UpdateQuantityCommand : InventoryCommand
{
    protected override string[] CommandStrings =>
        new[] { "u", "updatequantity" };
}
```

```
public class HelpCommand : InventoryCommand
{
    protected override string[] CommandStrings => new[] { "?" };
}
```

Класс `UnknownCommand` будет использоваться по умолчанию, так что всегда станет возвращать `true`, переопределяя метод `IsCommandFor`:

```
public class UnknownCommand : InventoryCommand
{
    protected override string[] CommandStrings => new string[0];

    public override bool IsCommandFor(string input)
    {
        return true;
    }
}
```

Класс `UnknownCommand` рассматривается как класс по умолчанию, поэтому порядок регистрации важен и показан ниже в инициализации класса `test`:

```
[TestInitialize]
public void Startup()
{
    var expectedInterface = new Helpers.TestUserInterface(
        new List<Tuple<string, string>>(),
        new List<string>(),
        new List<string>()
    );

    IServiceCollection services = new ServiceCollection();
    services.AddTransient<InventoryCommand, QuitCommand>();
    services.AddTransient<InventoryCommand, HelpCommand>();
    services.AddTransient<InventoryCommand, AddInventoryCommand>();
    services.AddTransient<InventoryCommand, GetInventoryCommand>();
    services.AddTransient<InventoryCommand, UpdateQuantityCommand>();
    // UnknownCommand должна регистрироваться последней
    services.AddTransient<InventoryCommand, UnknownCommand>();

    Services = services.BuildServiceProvider();
}
```

Для удобства создан новый метод, который возвращает экземпляр класса `InventoryCommand`, когда задана соответствующая входная строка:

```
public InventoryCommand GetCommand(string input)
{
    return Services.GetServices<InventoryCommand>().First(svc =>
        svc.IsCommandFor(input));
}
```

Этот метод обходит коллекцию зависимостей, зарегистрированных для сервиса `InventoryCommand`, до тех пор, пока с помощью метода `IsCommandFor()` не будет найдено совпадение.

Затем модульный тест использует метод `GetCommand()` для определения зависимости, как показано ниже с помощью `UpdateQuantityCommand`:

```
[TestMethod]
public void UpdateQuantityCommand_Successful()
{
    Assert.IsInstanceOfType(GetCommand("u"),
                            typeof(UpdateQuantityCommand),
                            "u should be UpdateQuantityCommand");

    Assert.IsInstanceOfType(GetCommand("updatequantity"),
                            typeof(UpdateQuantityCommand),
                            "updatequantity should be UpdateQuantityCommand");

    Assert.IsInstanceOfType(GetCommand("UpdaTEQuantity"),
                            typeof(UpdateQuantityCommand),
                            "UpdaTEQuantity should be UpdateQuantityCommand");
}
```

Применение сторонних контейнеров

Фреймворк .NET Core обеспечивает большую гибкость и функциональность, однако некоторые функции могут не поддерживаться, поэтому сторонний контейнер может быть более подходящим выбором. К счастью, .NET Core расширяем и позволяет заменить встроенный сервисный контейнер контейнером стороннего производителя. Для примера мы используем `Autofac` как IoC-контейнер.



`Autofac` имеет множество отличных функций и показан здесь в качестве примера; но, конечно же, есть и другие IoC-контейнеры, которые можно было бы использовать. Например, `Castle Windsor` и `Unit` — отличные альтернативы, подходящие для рассмотрения.

Первый шаг — добавление в проект пакета `Autofac`. Используя консоль менеджера пакетов, добавьте пакет с помощью следующей команды (требуется только в тестовом проекте):

```
install-package autofac
```

Этот пример снова поддерживает нашу «Фабрику» `InventoryCommand` с помощью функции `Autofac` для именования зарегистрированных зависимостей. Эти зависимости будут использованы для получения корректного экземпляра `InventoryCommand` с помощью предоставленных входных данных.

Как и в предыдущем примере, зависимости регистрируются в методе `TestInitialize`. Регистрации будут именоваться на основе команды, которая, в свою очередь, послужит для определения команды. Ниже показана структура метода `Startup`, создающего объект `ContainerBuilder`, который, в свою очередь, строит экземпляр объекта `Container`.

```
[TestInitialize]
public void Startup()
{
    IServiceCollection services = new ServiceCollection();

    var builder = new ContainerBuilder();

    // команды
    ...

    Container = builder.Build();
}
```

Команды регистрируются так:

```
// команды
builder.RegisterType<QuitCommand>().Named<InventoryCommand>("q");
builder.RegisterType<QuitCommand>().Named<InventoryCommand>("quit");
builder.RegisterType<UpdateQuantityCommand>().Named<InventoryCommand>("u");
builder.RegisterType<UpdateQuantityCommand>().Named<InventoryCommand>
    ("updatequantity");
builder.RegisterType<HelpCommand>().Named<InventoryCommand>("?");
builder.RegisterType<AddInventoryCommand>().Named<InventoryCommand>("a");
builder.RegisterType<AddInventoryCommand>().Named<InventoryCommand>
    ("addinventory");
builder.RegisterType<GetInventoryCommand>().Named<InventoryCommand>("g");
builder.RegisterType<GetInventoryCommand>().Named<InventoryCommand>
    ("getinventory");
builder.RegisterType<UnknownCommand>().As<InventoryCommand>();
```

В отличие от предыдущего примера генерируемый контейнер — экземпляр `AutoFac.IContainer`. Это используется при получении каждой зарегистрированной зависимости. Так, `QuitCommand` будет вызываться с помощью двух команд, "q" и "quit", что можно применить для выполнения одной и той же команды. Вдобавок обратите внимание: последний зарегистрированный тип не имеет имени и принадлежит к `UnknownCommand`. Он работает по умолчанию, когда по имени не найдена ни одна команда. Определяем зависимость мы с помощью нового метода ее получения по имени:

```
public InventoryCommand GetCommand(string input)
{
    Return Container.ResolveOptionalNamed<InventoryCommand>(input.ToLower()) ??
        Container.Resolve<InventoryCommand>();
}
```

В интерфейсе `Autofac.IContainer` есть метод `ResolveOptionalNamed<T>(string)`, который вернет зависимость с заданным именем или `null` при отсутствии совпадений регистраций. Если зависимость с заданным именем не зарегистрирована, то возвращается экземпляр класса `UnknownCommand`. Это делается с помощью оператора объединения с `null` (`null-coalescing`), `??`, а также метода `IContainer.Resolve<T>`.



`Autofac.IContainer.ResolveNamed<T>(string)` выбрасывает исключение `ComponentNotRegisteredException`, если не удалось разрешить зависимость.

Для каждой команды написан метод тестирования с целью удостовериться, что команды разрешены правильно. Снова используя `QuitCommand` в качестве примера, мы видим следующее:

```
[TestMethod]
public void QuitCommand_Successful()
{
    Assert.IsInstanceOfType(GetCommand("q"), typeof(QuitCommand),
        "q should be QuitCommand");
    Assert.IsInstanceOfType(GetCommand("quit"), typeof(QuitCommand),
        "quit should be QuitCommand");
}
```

Пожалуйста, посмотрите класс `InventoryCommandAutofacTests` в исходном коде других примеров `InventoryCommand`.

Резюме

Целью этой главы было более детальное изучение фреймворка `.NET Core` и, в частности, `DI` в `.NET Core`. Поддерживаются три типа времени жизни сервиса, то есть существуют временный сервис, сервис с ограничением области применения и сервис-«одиночка». Временный сервис создает новый экземпляр зарегистрированной зависимости при каждом запросе. Сервис с ограничением области применения генерируется один раз для определенной области применения, а сервис-«одиночка» создается один раз в течение всего времени жизни коллекции сервисов `DI`.

Поскольку `DI` в `.NET Core` занимает центральное место в построении надежных приложений `.NET Core`, важно понимать его возможности и его ограничения. Следует эффективно использовать `DI`, а также избегать дублирования уже предоставляемой функциональности. Не менее важно знать ограничения фреймворка `DI` в `.NET Core`, а также сильные стороны других фреймворков `DI` в ситуациях, когда замена базового фреймворка `DI` в `.NET Core` на стороннюю реализацию может быть выгодна приложению. Следующая глава строится на основе предыдущих глав и посвящена изучению общеприменимых паттернов в веб-приложениях `.NET Core ASP.NET`.

Вопросы

Следующие вопросы позволят вам усвоить информацию, представленную в этой главе.

1. Если вы не уверены, какой тип срока жизни сервиса использовать, то с каким типом лучше всего зарегистрировать класс? Почему?
2. В решениях .NET Core ASP.NET область действия определяется для каждого веб-запроса или каждой сессии?
3. Сделает ли класс потокобезопасным его регистрация в качестве «Одиночки» во фреймворке DI в .NET Core?
4. Правда ли, что фреймворк DI в .NET Core можно заменить только другими фреймворками DI, поставляемыми компанией Microsoft?

6

Реализация паттернов проектирования для веб-приложений (часть 1)

В данной главе мы продолжим создание приложения *FlixOne* для инвентаризации (см. главу 3), а также обсудим преобразование консольного приложения в веб-приложение. Оно должно быть более привлекательным для пользователей, в отличие от консольного; здесь же мы поговорим о причинах таких перемен.

В этой главе будут рассмотрены следующие темы:

- ❑ проектирование веб-приложения .NET Core;
- ❑ реализация веб-приложения;
- ❑ реализация CRUD-страниц



Если вы еще не знакомы с предыдущими главами, то обратите внимание: веб-приложение FlixOne — воображаемый продукт. Мы создаем его для обсуждения различных паттернов проектирования, необходимых в веб-проектах.

Технические требования

В этой главе содержатся примеры кода, объясняющие принципы качественной разработки. Код упрощен и предназначен лишь для демонстрации. Большинство примеров основаны на консольном приложении .NET Core, написанном на C#.

Чтобы запустить и выполнить код, вам потребуется следующее:

- ❑ Visual Studio 2019 (вы также можете запустить приложение, используя Visual Studio 2017 версии 3 и новее);
- ❑ .NET Core;
- ❑ SQL Server (в этой главе применяется Express Edition).

Установка Visual Studio

Чтобы запустить примеры кода, вам необходимо установить Visual Studio 2017 (или более позднюю версию, например 2019). Для этого выполните следующие шаги.

1. Скачайте Visual Studio по ссылке docs.microsoft.com/ru-ru/visualstudio/install/install-visual-studio.
2. Следуйте инструкциям по установке. Доступны различные версии Visual Studio; в этой главе мы используем Visual Studio для Windows.

Вы также можете применять другую интегрированную среду разработки по вашему усмотрению.

Установка .NET Core

Если вы еще не установили .NET Core, то следуйте инструкции ниже.

1. Скачайте .NET Core по ссылке dotnet.microsoft.com/download.
2. Далее следуйте указаниям по установке соответствующей библиотеки: dotnet.microsoft.com/download/dotnet-core/2.2.

Установка SQL Server

Если у вас не установлен SQL Server, то вам необходимо выполнить следующие инструкции.

1. Скачайте SQL Server по ссылке www.microsoft.com/ru-RU/download/details.aspx?id=1695.
2. Инструкции по установке можно найти по ссылке docs.microsoft.com/ru-ru/sql/ssms/download-sql-server-management-studio-ssms?view=sql-server-2017.



Для устранения неполадок и получения дополнительной информации см. www.blackbaud.com/files/support/infinityinstaller/content/installermaster/tkinstallsqlserver2008r2.htm.

Данный раздел предназначен для предоставления необходимой информации о работе с веб-приложениями. Более подробно мы рассмотрим это в последующих разделах. В текущей главе мы будем использовать примеры кода для прояснения различных терминов и разделов.



Полная версия исходного кода доступна на GitHub. Код, представленный в этой главе, может быть неполным, поэтому мы рекомендуем вам скачать полный код для запуска примеров (github.com/PacktPublishing/Hands-On-Design-Patterns-with-C-and-.NET-Core/tree/master/Chapter6).

Создание веб-приложения .NET Core

В начале этой главы мы обсуждали наше консольное приложение FlixOne, и есть различные причины для перехода на веб-приложение, как это определено бизнес-пользователями. Теперь пришло время внести изменения в приложение. В данном разделе мы начнем создавать новый пользовательский интерфейс нашего существующего приложения FlixOne с новым внешним видом. Мы также обсудим все требования и инициализацию.

Запуск проекта

В продолжение нашего существующего консольного приложения FlixOne руководство решило обновить его. Оно пришло к выводу, что мы должны преобразовать консольное приложение в веб-решение.

Разработчики и бизнес-пользователи объединились и определили различные причины, по которым было принято решение отказаться от текущего консольного приложения:

- ☐ интерфейс не интерактивен;
- ☐ приложение доступно не везде;
- ☐ его сложно поддерживать;
- ☐ растущий бизнес нуждается в масштабируемой системе с более высокой производительностью и адаптируемостью.

Разработка требований

По итогам обсуждений был составлен следующий перечень требований. Выявленные потребности высокого уровня таковы:

- ☐ категоризация продуктов;
- ☐ добавление продукта;
- ☐ обновление продукта;
- ☐ удаление продукта.

Актуальные требования, которые предъявляет бизнес, лежат на разработчиках. Эти технические требования включают следующее:

- ☐ *целевая или главная страница* — это должна быть панель управления, содержащая различные виджеты и показывающая краткое описание магазина;
- ☐ *страница продукта* должна иметь возможность добавлять, обновлять и удалять продукты и категории.

Разработка веб-приложения

В соответствии с только что рассмотренными требованиями наша основная цель — преобразование существующего консольного приложения в веб-приложение. В ходе данной трансформации мы обсудим различные паттерны проектирования для веб-приложений, а также важность этих паттернов в контексте веб-приложений.

Веб-приложения и как они работают

Веб-приложения — одна из лучших реализаций клиент-серверной архитектуры. Веб-приложение может быть небольшим куском кода, программой, или полным решением проблемы, или бизнес-сценарием, в котором пользователи взаимодействуют друг с другом либо с сервером, применяя браузеры. Веб-приложение обслуживает запросы и ответы через браузеры в основном с помощью *протокола передачи гипертекста* (HyperText Transfer Protocol, HTTP).



Всякий раз при возникновении какой-либо коммуникации между клиентом и сервером происходят две вещи: клиент инициирует запрос, а сервер генерирует ответ. Это общение состоит из HTTP-запросов и HTTP-ответов. Для получения дополнительной информации см. документацию: www.w3schools.com/whatis/whatis_http.asp.

На рис. 6.1 представлен обзор веб-приложения и его работы.

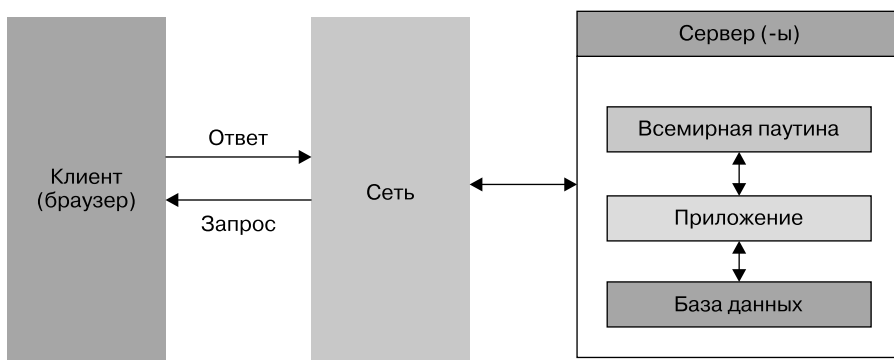


Рис. 6.1

Эта схема позволяет увидеть, что с помощью браузера (в качестве клиента) вы открываете двери для миллионов пользователей, которые могут получить доступ к сайту из любой точки мира и взаимодействовать с вами как с пользователем.

Веб-приложение позволяет вам и вашим клиентам легко взаимодействовать. Как правило, эффективное взаимодействие возможно, только когда вы собираете и храните всю информацию, необходимую для вашего бизнеса и пользователей. Затем она обрабатывается и результаты представляются пользователям.



Как правило, веб-приложения используют комбинацию кода на стороне сервера в целях обработки хранения и извлечения информации, а клиентские сценарии — для представления информации пользователям.

Веб-приложению требуется веб-сервер (например, *IIS* или *Apache*) для управления запросами, поступающими от клиента (из браузера, как показано на предыдущем рисунке). Для выполнения требуемых задач также нужен сервер приложений (например, *IIS* или *Apache Tomcat*). Иногда для хранения информации необходима база данных.



Проще говоря, и веб-сервер, и сервер приложений предназначены для выдачи HTTP-контента, но с определенными вариациями. Веб-серверы выдают статический HTTP-контент, например HTML-страницы. Серверы приложений могут выдавать не только статический HTTP-контент, но и динамический контент, используя различные языки программирования. Для получения дополнительной информации см. stackoverflow.com/questions/936197/what-is-the-difference-between-application-server-and-web-server.

Мы можем подробно описать рабочий процесс веб-приложения следующим образом. Ниже перечислены пять этапов, из которых обычно состоит рабочий процесс веб-приложения.

1. Запрос запускается клиентом (браузером) на веб-сервер с помощью HTTP (в большинстве случаев) через Интернет. Обычно это происходит через браузер или пользовательский интерфейс приложения.
2. Запрос возникает на веб-сервере, и тот пересылает запрос на сервер приложений (разные запросы отправляются на разные серверы приложений).
3. На сервере приложений выполняются запрошенные задачи. Это могут быть запрос к серверу баз данных, извлечение информации из базы данных, ее обработка и построение результатов.
4. Сгенерированные результаты (запрошенная информация или обработанные данные) отправляются на веб-сервер.
5. Наконец, ответ отправляется обратно запрашивающему лицу (клиенту) с веб-сервера вместе с запрошенной информацией. Страница отображается на экране пользователя.

На рис. 6.2 показан наглядный обзор всех пяти этапов.

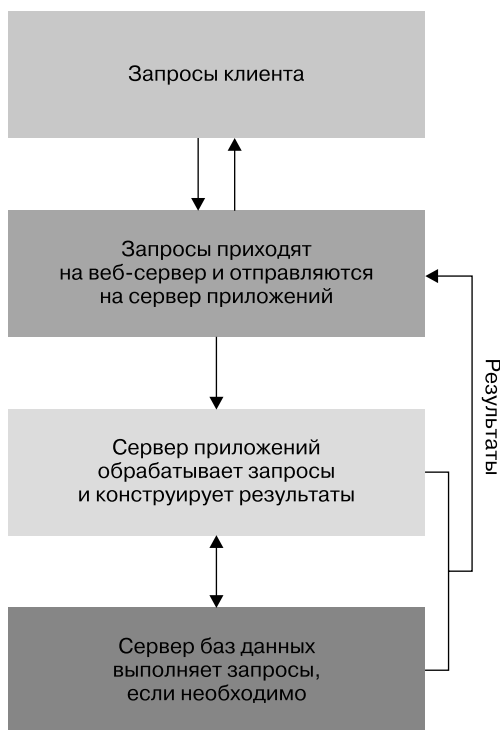


Рис. 6.2

Ниже мы опишем рабочий процесс веб-приложения с помощью паттерна «*Модель — представление — контроллер*» (Model — View — Controller, MVC).

Создание веб-приложения

До сих пор мы изучали требования и смотрели на нашу цель, которая заключается в преобразовании консольного приложения в веб-платформу или веб-приложение. В этом пункте мы разработаем собственно веб-приложение с использованием Visual Studio.

Чтобы создать веб-приложение с помощью Visual Studio, выполните следующие действия.

1. Запустите приложение Visual Studio.
2. Выберите команду меню File ► New ► Project (Файл ► Новый ► Проект) или нажмите сочетание клавиш Ctrl+Shift+N, как показано на рис. 6.3.
3. В окне New Project (Новый проект) выберите пункт Web ► .NET Core ► ASP.NET Core Web Application (Web ► .NET Core ► Веб-приложение ASP.NET Core).

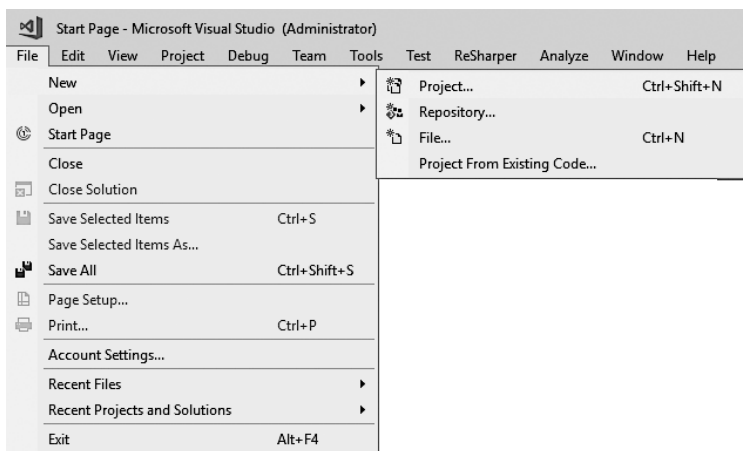


Рис. 6.3

4. Присвойте ему имя (например, `FlixOne.Web`), выберите расположение, далее сможете обновить имя решения. По умолчанию оно будет совпадать с именем проекта. Установите флажок `Create directory for solution` (Создать каталог для решения). Вы также можете установить флажок `Create new Git repository` (Создать новый репозиторий Git) (если хотите создать для этого проекта новый репозиторий, то вам нужен действующий аккаунт в Git).

На рис. 6.4 показан процесс создания нового проекта.

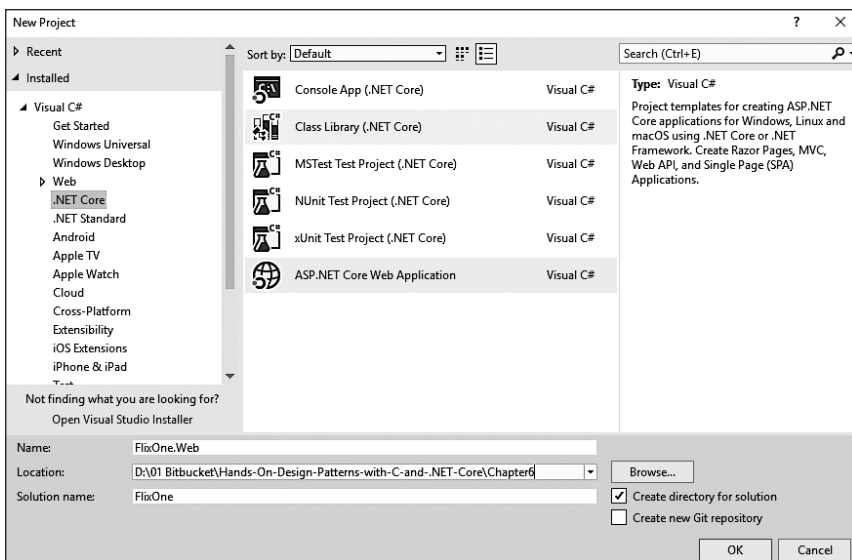


Рис. 6.4

5. Следующий шаг — выбор подходящего шаблона для вашего веб-приложения и версии *.NET Core*. Мы не собираемся включать поддержку Docker для этого проекта, так как не будем развертывать наше приложение, используя Docker в качестве контейнера. Мы будем использовать только протокол HTTP, а не HTTPS. Поэтому флажки **Enable Docker Support** (Включить поддержку Docker) и **Configure HTTPS** (Настройка HTTPS) должны быть сняты, как показано на снимке экрана ниже (рис. 6.5).

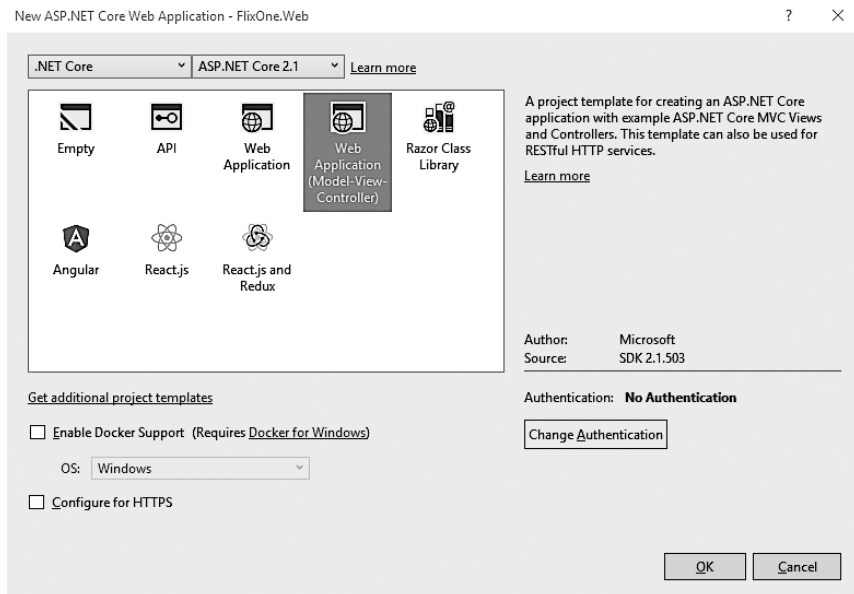


Рис. 6.5

Теперь у нас есть полный проект с нашим шаблоном и примером кода, использующий фреймворк MVC. На рис. 6.6 показано наше решение.



Архитектурные паттерны — способ внедрения практических рекомендаций в проектирование пользовательского интерфейса и самого приложения. Они предоставляют многообразные решения общих проблем. Эти схемы также позволяют легко разделить проблемы.

Наиболее популярные архитектурные паттерны:

- «Модель — представление — контроллер» (Model — View — Controller, MVC);
- «Модель — представление — представитель» (Model — View — Presenter, MVP);
- «Модель — представление — модель представления» (Model — View — View Model, MVVM).

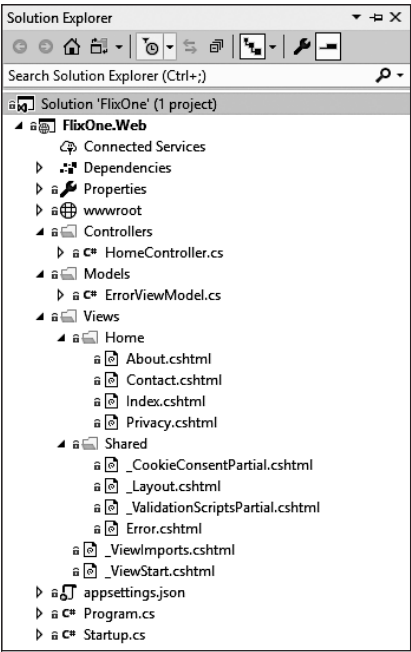


Рис. 6.6

Вы можете запустить приложение, нажав клавишу F5. На рис. 6.7 показана главная страница веб-приложения по умолчанию.

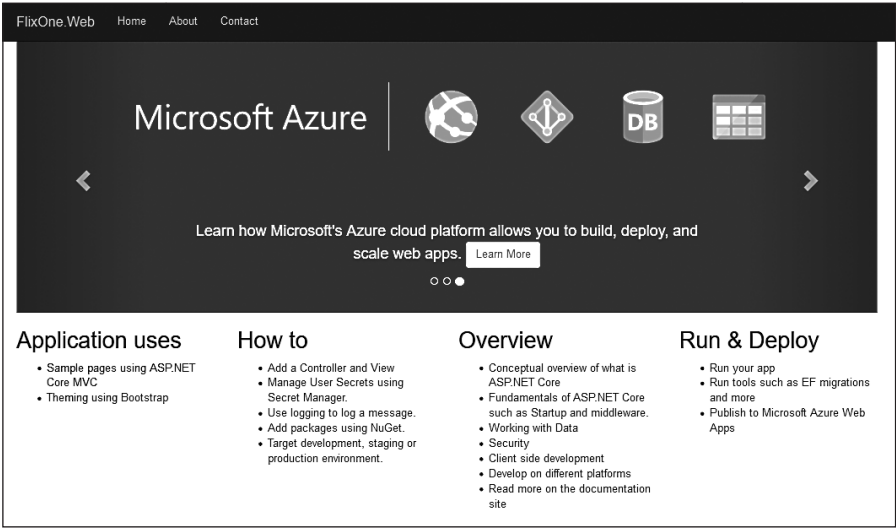


Рис. 6.7

В следующих разделах мы обсудим паттерны MVC и будем создавать *CRUD*-страницы (*Create, Update, Delete*), предназначенные для взаимодействия с пользователями.

Реализация CRUD-страниц

В этом разделе мы начнем воплощать функциональные страницы, цель которых — создание, обновление и удаление продуктов. Для начала откройте решение *FlixOne* и добавьте следующие классы в указанные папки.

Models — добавьте в папку **Models** следующие файлы.

❑ **Product.cs** — фрагмент кода класса **Product** выглядит следующим образом:

```
public class Product
{
    public Guid Id { get; set; }
    public string Name { get; set; }
    public string Description { get; set; }
    public string Image { get; set; }
    public decimal Price { get; set; }
    public Guid CategoryId { get; set; }
    public virtual Category Category { get; set; }
}
```

Класс **Product** представляет почти все элементы продукта. У него есть название, полное описание, изображение, цена и уникальный идентификатор, поэтому наша система распознает его. Кроме того, класс имеет идентификатор категории, к которой принадлежит данный продукт. Она также включает в себя полное определение категории.



Почему мы должны определять виртуальное (**virtual**) свойство?

В нашем классе **Product** мы определили свойство **virtual**. Мы сделали так потому, что в Entity Framework (EF) это свойство помогает создать прокси для виртуального свойства. Этот механизм позволяет свойству поддерживать ленивую загрузку и более эффективное отслеживание изменений. Это значит, что данные доступны по запросу. EF загружает данные при запросе на использование свойства **Category**.

❑ **Category.cs** — фрагмент кода из класса **Category** выглядит так:

```
public class Category
{
    public Category()
    {
        Products = new List<Product>();
    }
}
```

```
    }

    public Guid Id { get; set; }
    public string Name { get; set; }
    public string Description { get; set; }
    public virtual IEnumerable<Product> Products { get; set; }
}
```

Наш класс `Category` представляет фактическую категорию продукта. Категория имеет уникальный идентификатор, название, полное описание и набор относящихся к ней продуктов. Всякий раз, когда мы инициализируем класс `Category`, он инициализирует класс `Product`.

- `ProductViewModel.cs` — фрагмент кода класса `ProductViewModel` выглядит следующим образом:

```
public class ProductViewModel
{
    public Guid ProductId { get; set; }
    public string ProductName { get; set; }
    public string ProductDescription { get; set; }
    public string ProductImage { get; set; }
    public decimal ProductPrice { get; set; }
    public Guid CategoryId { get; set; }
    public string CategoryName { get; set; }
    public string CategoryDescription { get; set; }
}
```

Класс `ProductViewModel` представляет собой законченный продукт, который имеет атрибуты, такие как уникальный `ProductId`, `ProductName`, полное описание `ProductDescription`, `ProductImage`, `ProductPrice`, уникальный `CategoryId`, `CategoryName` и полное описание `CategoryDescription`.

Controllers — добавьте следующие файлы в папку `Controllers`.

- `ProductController` отвечает за все операции, связанные с продуктами. Посмотрим на код и операции, которые мы пытаемся выполнить в этом контроллере:

```
public class ProductController : Controller
{
    private readonly IInventoryRepository _repository;
    public ProductController(IInventoryRepository
        inventoryRepository) => _repository = inventoryRepository;

    ...
}
```

Здесь мы определили класс `ProductController`, который наследуется от класса `Controller`. Мы использовали *внедрение зависимостей*, поддержка которых встроена в фреймворк ASP.NET Core MVC.



Мы подробно обсудили инверсию управления в главе 5. Controller — это базовый класс контроллера MVC. Дополнительную информацию см. по ссылке docs.microsoft.com/en-us/dotnet/api/microsoft.aspnet-core.mvc.controller.

Итак, мы создали наш главный контроллер, `ProductController`. Теперь начнем добавлять функциональные возможности для наших операций CRUD.

Следующий код — это просто операция чтения или получения, которая запрашивает у репозитория (`_inventoryRepository`) список всех доступных продуктов, а затем переносит данный список в тип `ProductViewModel` и возвращает представление `Index`:

```
public IActionResult Index() =>
    View(_repository.GetProducts().ToProductvm());
public IActionResult Details(Guid id) =>
    View(_repository.GetProduct(id).ToProductvm());
```

В предыдущем фрагменте кода метод `Details` возвращает сведения о конкретном продукте на основе его уникального идентификатора. Это также операция `Get`, аналогичная нашему методу `Index`, но она предоставляет один объект вместо списка.



Методы контроллера MVC также называются action-методами и имеют возвращаемый тип `ActionResult`. В этом случае мы используем `IActionResult`. Можно сказать, что `IActionResult` — интерфейс класса `ActionResult`. Такой подход дает возможность возвращать многое, в том числе:

- `EmptyResult`;
- `FileResult`;
- `HttpStatusCodeResult`;
- `ContentResult`;
- `JsonResult`;
- `RedirectToRouteResult`;
- `RedirectResult`.

Мы не будем подробно обсуждать все эти вопросы, поскольку они выходят за рамки данной книги. Чтобы узнать больше о типах возвращаемых данных, посетите страницу <https://docs.microsoft.com/en-us/aspnet/core/web-api/action-returnntypes>.

В следующем коде мы создаем новый продукт. Фрагмент кода содержит два action-метода. Один из них имеет атрибут `[HttpPost]`, а другой не имеет его:

```
public IActionResult Create() => View();
[HttpPost]
[ValidateAntiForgeryToken]
public IActionResult Create([FromBody] Product product)
{
    try
    {
        _repository.AddProduct(product);
        return RedirectToAction(nameof(Index));
    }
    catch
    {
        return View();
    }
}
```

Первый метод просто возвращает `View`, то есть страницу `Create.cshtml`.



Если какой-либо из action-методов в MVC-фреймворке не имеет атрибута, то будет использоваться атрибут `[HttpGet]` по умолчанию. В других представлениях по умолчанию методами действия являются запросы GET. Всякий раз, когда пользователь просматривает страницу, мы применяем `[HttpGet]` или GET-запрос. Каждый раз, когда пользователь отправляет форму или выполняет действие, мы оперируем `[HttpPost]` или POST-запросом.

Если мы явно не упоминали имя представления в нашем action-методе, то платформа MVC ищет его по паттернам: `actionmethodname.cshtml` и `actionmethodname.vbhtml`. В нашем случае имя представления — `Create.cshtml`, поскольку мы используем язык C#. Имя было бы `vbhtml`, используй мы Visual Basic. Сначала фреймворк ищет в папке, имя которой похоже на имя папки контроллера. Не найдя файл в этой папке, ищет в общей папке `shared`.

Второй action-метод в предыдущем фрагменте кода использует атрибут `[HttpPost]`, то есть обрабатывает запросы POST. Данный action-метод просто добавляет продукт, вызывая `AddProduct` из объекта `_repository`. В этом action-методе мы использовали атрибут `[ValidateAntiForgeryToken]` и атрибут `[FromBody]`, который является связующим элементом модели.

Фреймворк MVC предоставляет много функций безопасности для защиты нашего приложения от *межсайтового скриптинга/подделки межсайтовых запросов* (Cross-Site Scripting/Cross-Site Request Forgery, XSS/CSRF), обеспечивая защиту с помощью атрибута `[ValidateAntiForgeryToken]`. Данный тип атак, как правило, включает некий опасный код сценария на стороне клиента.

Привязка модели в MVC отображает данные запросов HTTP на параметры action-метода. Часто используемые атрибуты привязки моделей с помощью action-методов выглядят следующим образом:

- ❑ [FromHeader];
- ❑ [FromQuery];
- ❑ [FromRoute];
- ❑ [FromForm].

Мы не будем обсуждать их более подробно, так как это выходит за рамки данной книги. Тем не менее вы можете найти полную информацию в официальной документации по адресу docs.microsoft.com/en-us/aspnet/core/mvc/models/model-binding.

В предыдущих фрагментах кода мы обсуждали операции создания и чтения. Пришло время написать код для операции обновления, **Update**. В следующем коде у нас есть два action-метода: один — для GET-запроса, другой — для POST.

```
public IActionResult Edit(Guid id) => View(_repository.GetProduct(id));

[HttpPost]
[ValidateAntiForgeryToken]
public IActionResult Edit(Guid id, [FromBody] Product product)
{
    try
    {
        _repository.UpdateProduct(product);
        return RedirectToAction(nameof(Index));
    }
    catch
    {
        return View();
    }
}
```

Первый action-метод из кода выше получает **Product** на основе идентификатора и возвращает **View**. Второй берет данные из представления и обновляет запрашиваемый **Product** на основе его идентификатора:

```
public IActionResult Delete(Guid id) => View(_repository.GetProduct(id));

[HttpPost]
[ValidateAntiForgeryToken]
public IActionResult Delete(Guid id, [FromBody] Product product)
{
    try
    {
        _repository.RemoveProduct(product);
        return RedirectToAction(nameof(Index));
    }
}
```

```
    }  
    catch  
    {  
        return View();  
    }  
}
```

Наконец, этот код предоставляет операцию **Delete** из CRUD. В нем также есть два action-метода: первый извлекает данные из репозитория и предоставляет их представлению, а другой принимает запрос данных и удаляет конкретный **Product** на основе его идентификатора.

Класс **CategoryController** несет ответственность за все операции категории продуктов. Добавьте следующий код к контроллеру, он представляет **CategoryController**, где мы внедрили зависимости, чтобы инициализировать **IInventoryRepository**:

```
public class CategoryController: Controller  
{  
    private readonly IInventoryRepository _inventoryRepository;  
    public CategoryController(IInventoryRepository inventoryRepository) =>  
        _inventoryRepository = inventoryRepository;  
    // код опущен  
}
```

Следующий код содержит два action-метода. Первый получает список категорий, второй — определенную категорию на основе ее уникального идентификатора:

```
public IActionResult Index() => View(_inventoryRepository.GetCategories());  
public IActionResult Details(Guid id) =>  
    View(_inventoryRepository.GetCategory(id));
```

В следующем коде используются запросы GET и POST для создания в системе новой категории:

```
public IActionResult Create() => View();  
[HttpPost]  
[ValidateAntiForgeryToken]  
public IActionResult Create([FromBody] Category category)  
{  
    try  
    {  
        _inventoryRepository.AddCategory(category);  
  
        return RedirectToAction(nameof(Index));  
    }  
    catch  
    {  
        return View();  
    }  
}
```

В коде ниже мы обновляем существующую категорию. Код содержит action-методы `Edit` с запросами `GET` и `POST`:

```
public IActionResult Edit(Guid id) =>
View(_inventoryRepository.GetCategory(id));
[HttpPost]
[ValidateAntiForgeryToken]
public IActionResult Edit(Guid id, [FromBody]Category category)
{
    try
    {
        _inventoryRepository.UpdateCategory(category);

        return RedirectToAction(nameof(Index));
    }
    catch
    {
        return View();
    }
}
```

Наконец, у нас есть метод `Delete`. Это последняя операция страниц `CRUD` для удаления категории, как показано ниже:

```
public IActionResult Delete(Guid id) =>
View(_inventoryRepository.GetCategory(id));

[HttpPost]
[ValidateAntiForgeryToken]
public IActionResult Delete(Guid id, [FromBody] Category category)
{
    try
    {
        _inventoryRepository.RemoveCategory(category);

        return RedirectToAction(nameof(Index));
    }
    catch
    {
        return View();
    }
}
```

Views — добавьте следующие представления в соответствующие папки:

- ☐ `Index.cshtml`;
- ☐ `Create.cshtml`;
- ☐ `Edit.cshtml`;
- ☐ `Delete.cshtml`;
- ☐ `Details.cshtml`.

Contexts — добавьте в папку Contexts файл InventoryContext.cs со следующим кодом:

```
public class InventoryContext : DbContext
{
    public InventoryContext(DbContextOptions<InventoryContext> options)
        : base(options)
    {
    }

    public InventoryContext()
    {
    }

    public DbSet<Product> Products { get; set; }
    public DbSet<Category> Categories { get; set; }
}
```

Предыдущий код предоставляет различные методы, необходимые для взаимодействия с базой данных с помощью EF. При его выполнении вы можете столкнуться со следующим исключением (рис. 6.8).

An unhandled exception occurred while processing the request.

InvalidOperationException: Unable to resolve service for type 'FlixOne.Web.Persistence.IInventoryRepository' while attempting to activate 'FlixOne.Web.Controllers.ProductController'.
Microsoft.Extensions.DependencyInjection.ActivatorUtilities.GetService(IServiceProvider sp, Type type, Type requiredBy, bool isDefaultParameterRequired)

Рис. 6.8

Чтобы исправить это исключение, вы должны отразить IInventoryRepository в Startup.cs, как показано на рис. 6.9.

```
// This method gets called by the runtime. Use this method to add services to the container.
public void ConfigureServices(IServiceCollection services)
{
    services.AddTransient<IInventoryRepository, InventoryRepository>();
    services.AddDbContext<InventoryContext>(o => o.UseSqlServer(Configuration.GetConnectionString("FlixOneDbConnection")));
    services.Configure<CookiePolicyOptions>(options =>
    {
        // This lambda determines whether user consent for non-essential cookies is needed for a given request.
        options.CheckConsentNeeded = context => true;
        options.MinimumSameSitePolicy = SameSiteMode.None;
    });

    services.AddMvc().SetCompatibilityVersion(CompatibilityVersion.Version_2_1);
}
```

Рис. 6.9

Мы добавили различные функциональные возможности веб-приложения, теперь решение выглядит так (рис. 6.10).

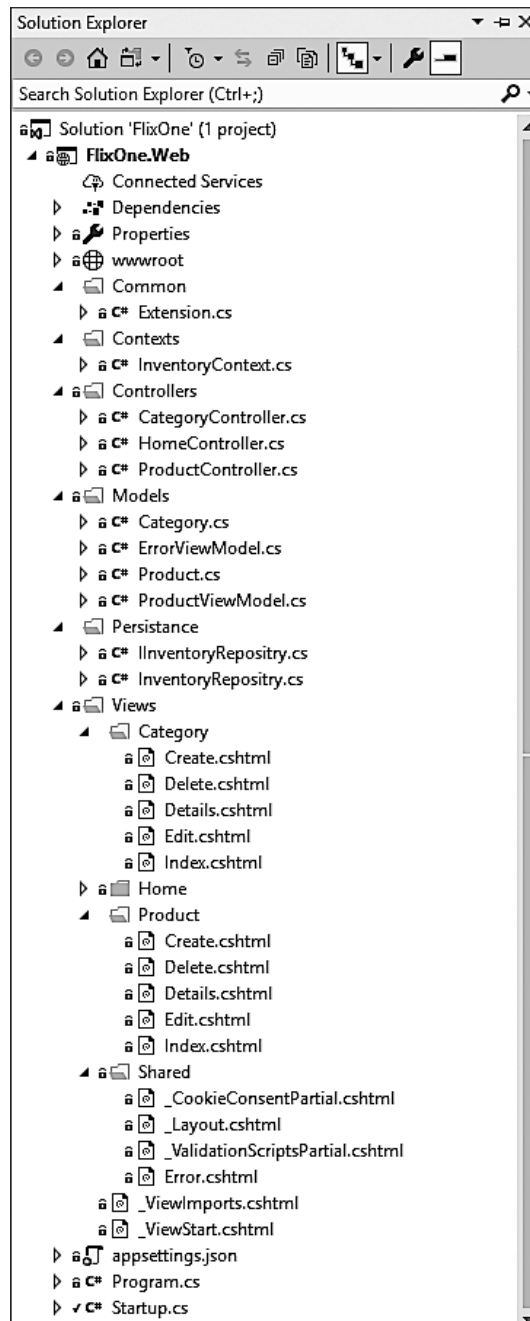


Рис. 6.10



Обратитесь к репозиторию GitHub для этой главы: <https://github.com/PacktPublishing/Hands-On-Design-Patterns-with-C-and-.NET-Core/tree/master/Chapter6>.

Если бы нам пришлось визуализировать модель MVC, то она бы работала так, как показано на рис. 6.11.

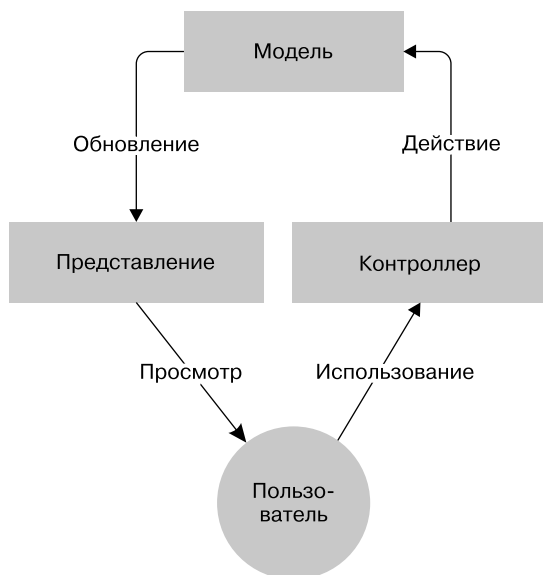


Рис. 6.11

Предыдущее изображение было адаптировано, источник commons.wikimedia.org/wiki/File:MVC-Process.svg.

Как показано на данной схеме, всякий раз, когда пользователь делает запрос, он приходит к контроллеру и запускает action-метод для дальнейшей манипуляции или обновления, (при необходимости до модели), а затем с его помощью пользователь получает представление.

В нашем случае каждый раз, когда пользователь запрашивает `/Product`, запрос переходит к action-методу `Index` в `ProductController` и выдает представление `Index.cshtml` после получения списка продуктов. Вы получите список товаров, как показано на рис. 6.12.

Здесь представлен простой список товаров, и это соответствует Read-части операций CRUD. На этом экране приложение показывает общее количество доступных

продуктов и их категории. На рис. 6.13 показаны интерактивные функции приложения.

FlixOne.Web

HomeProductsAboutContact

Product Listing

Create New

Product Name	Description	Image	Price (INR)	Category Name	Action
Mango	A juicy mango		40.00	Fruit	Edit Details Delete
Apple	Red apple		100.00	Fruit	Edit Details Delete
Orange	Fruity oranges		35.00	Fruit	Edit Details Delete

© 2019 - FlixOne.Web

Рис. 6.12

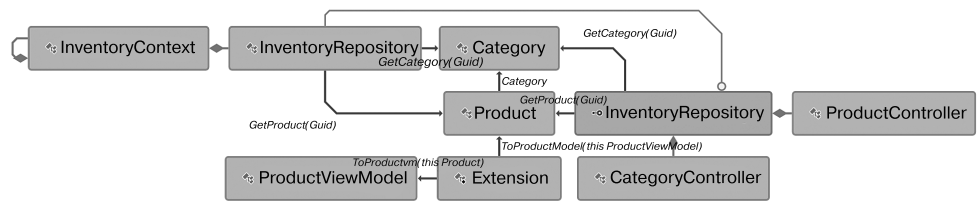


Рис. 6.13

Схема детально отображает процесс работы нашего приложения. Интерфейс `InventoryRepository` зависит от `InventoryContext` для работы с базами данных и взаимодействует с модельными классами, `Category` и `Product`. Контроллеры для продуктов и категорий используют интерфейс `IInventoryRepository`, чтобы взаимодействовать с репозиторием для CRUD-операций.

Резюме

Основная цель этой главы состояла в том, чтобы запустить базовое веб-приложение.

Мы начали главу с обсуждения бизнес-требований о том, зачем нам нужно веб-приложение и почему мы хотим обновить наше консольное приложение. Затем пошагово создали веб-приложения с помощью Visual Studio и паттерна MVC. Мы также обсудили, как веб-приложение может работать в качестве клиент-серверной модели, и рассмотрели паттерны пользовательского интерфейса. Наконец, мы начали создавать страницы CRUD.

В следующей главе мы продолжим работу с веб-приложением и обсудим другие паттерны проектирования для веб-приложений.

Вопросы

Следующие вопросы позволят вам усвоить информацию, представленную в этой главе.

1. Что такое веб-приложение?
2. Создайте веб-приложение по вашему выбору и покажите, как оно работает.
3. Что такое инверсия контроля?
4. Какие паттерны проектирования мы рассмотрели в этой главе? Какой из них вам нравится и почему?

Дальнейшее чтение

В книгах, указанных ниже, подробно рассматриваются RESTful-веб-сервисы и разработка на основе тестирования:

- ❑ *Arora G., Dash T.* Building RESTful Web services with .NET Core. — Packt. www.packtpub.com/application-development/building-restful-web-services-net-core;
- ❑ *Adewole A.* C# and .NET Core Test Driven Development. — Packt. www.packtpub.com/application-development/c-and-net-core-test-driven-development.

7

Реализация паттернов проектирования для веб-приложений (часть 2)

В предыдущей главе мы расширили наше консольное приложение FlixOne до веб-приложения, иллюстрируя различные паттерны. Кроме того, мы рассмотрели архитектурные паттерны *пользовательского интерфейса* (user interface, UI): «Модель — представление — контроллер» (Model — View — Controller, MVC), «Модель — представление — представитель» (Model — View — Presenter, MVP) и др. Предыдущая глава была посвящена обсуждению таких паттернов, как MVC. Теперь нам нужно расширить существующее приложение, чтобы включить больше паттернов.

В этой главе мы продолжим работу с существующим веб-приложением FlixOne и расширим его так, чтобы вы могли увидеть реализацию аутентификации и авторизации. Вдобавок мы обсудим *разработку через тестирование* (test-driven development, TDD).

В этой главе будут рассмотрены следующие темы:

- ❑ аутентификация и авторизация;
- ❑ создание проекта тестов в .NET Core для веб-приложения.

Технические требования

В этой главе содержатся примеры кода, объясняющие принципы качественной разработки. Код упрощен и предназначен лишь для демонстрации. Большинство примеров основаны на консольном приложении .NET Core, написанном на C#.

Чтобы запустить и выполнить код, вам потребуется Visual Studio 2019 (для запуска приложения также можно использовать Visual Studio 2017).

Установка Visual Studio

Чтобы запустить примеры кода, вам необходимо установить Visual Studio. Для этого выполните следующие шаги.

1. Скачайте Visual Studio по ссылке, которая содержит инструкции по установке: docs.microsoft.com/ru-ru/visualstudio/install/install-visual-studio.
2. Следуйте инструкциям по установке. Доступны различные версии Visual Studio; в этой главе мы используем Visual Studio для Windows.

Вы также можете применять другую интегрированную среду разработки по вашему усмотрению.

Установка .NET Core

Если вы еще не установили .NET Core, то следуйте инструкции ниже.

1. Скачайте .NET Core для Windows по ссылке dotnet.microsoft.com/download.
2. Для получения нескольких версий и соответствующей библиотеки перейдите по ссылке dotnet.microsoft.com/download/dotnet-core/2.2.

Установка SQL Server

Если у вас не установлен SQL Server, то вам необходимо выполнить следующие действия.

1. Скачайте SQL Server по ссылке www.microsoft.com/ru-RU/download/details.aspx?id=1695.
2. Инструкции по установке можно найти по ссылке docs.microsoft.com/ru-ru/sql/ssms/download-sql-server-management-studio-ssms?view=sql-server-2017.



Для устранения неполадок и получения дополнительной информации см. www.blackbaud.com/files/support/infinityinstaller/content/installermaster/tkinstallsqlserver2008r2.htm.

Полная версия исходного кода доступна на GitHub. Код, представленный в данной главе, может быть неполным, поэтому мы рекомендуем вам скачать полный код для запуска примеров (github.com/PacktPublishing/Hands-On-Design-Patterns-with-C-and-.NET-Core/tree/master/Chapter7).

Расширение веб-приложения .NET Core

В этой главе мы продолжим наше приложение для инвентаризации FlixOne. На протяжении всей главы мы будем обсуждать паттерны для веб-приложений и расширять приложение, разработанное в предыдущей главе.



В этой главе продолжается разработка веб-приложения из предыдущей главы. Если вы ее пропустили, то, пожалуйста, вернитесь и прочитайте.

В этом разделе мы рассмотрим процесс сбора требований, а затем обсудим различные проблемы ранее разработанного веб-приложения.

Начало проекта

В главе 6 мы расширили консольное приложение FlixOne, а также разработали веб-приложение. Мы расширяли приложение с учетом следующего:

- ❑ бизнес нуждается во многофункциональном пользовательском интерфейсе;
- ❑ новые возможности требуют адаптивного веб-приложения.

Требования

После нескольких встреч и обсуждений с руководством, *бизнес-аналитиками* (business analysts, BA) и специалистами по предпродажной подготовке руководство решило работать над следующими требованиями высокого уровня: *бизнес-требованиями* и *техническими требованиями*.

Бизнес-требования

Бизнес-пользователи в конечном итоге пришли к следующим требованиям.

- ❑ *Классификация продуктов.* Существует несколько продуктов, но если пользователь хочет найти определенный продукт, то может сделать это, отфильтровав все продукты по их категориям. Например, такие продукты, как манго, бананы и многие другие, должны быть в категории **Fruits**.
- ❑ *Добавление продукта.* Должен быть интерфейс, предоставляющий возможность добавлять новые продукты. Эту функцию следует сделать доступной только пользователям, имеющим привилегию **Add Products**.
- ❑ *Обновление продукта.* Должен существовать новый интерфейс, в котором возможны обновления продукта.
- ❑ *Удаление продукта.* Существует требование для администраторов удалять продукты.

Технические требования

Теперь актуальные требования для удовлетворения потребностей бизнеса готовы к разработке. Проведя несколько обсуждений с представителями бизнеса, мы пришли к выводу, что необходимы:

❑ *домашняя страница, которая должна:*

- иметь панель мониторинга, содержащую различные виджеты;
- показывать общее состояние магазина;

❑ *страница продукта, позволяющая:*

- добавлять, обновлять и удалять продукты;
- добавлять, обновлять и удалять категории продуктов.



Веб-приложение для инвентаризации FlixOne — наполовину воображаемый продукт. Мы создаем его, чтобы обсудить различные паттерны проектирования, необходимые и/или используемые в веб-проектах.

Проблемы

Несмотря на то что мы расширили наше существующее консольное приложение до нового веб-приложения, оно содержит различные проблемы как для разработчиков, так и для бизнеса. В этом разделе мы обсудим указанные проблемы, а затем найдем их решение.

Проблемы разработчиков

Ниже перечислены проблемы, которые возникли из-за масштабных изменений в приложении. Они также явились результатом основных расширений в целях обновления консольного приложения до веб-приложения.

- ❑ *Нет поддержки TDD* — в настоящее время в решение не включен тестовый проект. Это значит, что разработчики не могут следовать подходу TDD. Это, в свою очередь, может привести к большему количеству ошибок в приложении.
- ❑ *Безопасность* — сейчас в приложении отсутствует механизм ограничения или разрешения пользователю доступа к определенному экрану или модулю приложения. Кроме того, нет ничего связанного с аутентификацией и авторизацией.
- ❑ *UI и пользовательский опыт (user experience, UX)* — наше приложение началось как консольное, поэтому пользовательский интерфейс не очень функционален.

Проблемы бизнеса

Для достижения конечного результата требуется время, что задерживает выпуск продукта, приводя к убыткам бизнеса. При адаптации нового технологического стека возникают следующие проблемы и в коде происходит множество изменений:

- ❑ *потеря клиентов* — здесь мы все еще находимся в стадии разработки, но спрос на наш бизнес очень высок; однако у команды разработчиков на доставку продукта времени уходит больше, чем ожидалось;
- ❑ *задержка развертывания обновлений* — сейчас усилия по разработке отнимают много времени, что задерживает последующие действия и, наконец, приводит к задержке производства.

Поиск решения проблем

После нескольких встреч и мозговых штурмов команда разработчиков пришла к выводу: мы должны стабилизировать веб-решение. Чтобы преодолеть описанные выше проблемы, техническая команда и бизнес-команда объединились и собрали мнения и различные решения.

Итак, решение состоит из таких пунктов, как:

- ❑ реализация аутентификации и авторизации;
- ❑ следование TDD;
- ❑ редизайн пользовательского интерфейса для UX.

Аутентификация и авторизация

В предыдущей главе (в которой мы начали обновление консольного приложения до веб-приложения) мы добавили *операции создания, чтения, обновления и удаления* (Create, Read, Update, Delete, CRUD), доступные публично любому пользователю, способному их выполнять. Но мы не предусмотрели никакого кода, призванного ограничить права конкретного пользователя на выполнение этих операций. Риск состоит в том, что пользователи, которые не должны выполнять эти операции, могут легко сделать это. Последствия таковы:

- ❑ неконтролируемый доступ;
- ❑ незащищенность от злоумышленников;
- ❑ утечка данных.

Теперь, если мы стремимся защитить наше приложение и ограничить операции, сделав их доступными лишь для разрешенных пользователей, то должны реализовать дизайн, который позволяет выполнять операции только этим пользователям. Существуют сценарии, в которых мы могли бы оставить открытый доступ для не-

скольких операций. В нашем случае большинство операций предназначены лишь для ограниченного круга лиц. Если коротко, то мы можем реализовать нечто сообщающее приложению, что входящий пользователь способен выполнять указанную задачу.



Аутентификация — просто процесс, в котором система проверяет или идентифицирует входящие запросы с помощью учетных данных (обычно это идентификатор пользователя и пароль). Если система обнаруживает, что предоставленные учетные данные неверны, то уведомляет пользователя (обычно через сообщение на экране GUI) и обрывает процесс авторизации.

Авторизация всегда происходит после аутентификации. Это процесс, который позволяет аутентифицированному пользователю, отправившему запрос, получить доступ к определенным ресурсам или данным после проверки, есть ли у него доступ к ним.

Выше мы обсудили некоторые механизмы, предотвращающие неконтролируемый доступ к операциям в приложении. Обратимся к следующей схеме и обсудим, что она показывает (рис. 7.1).

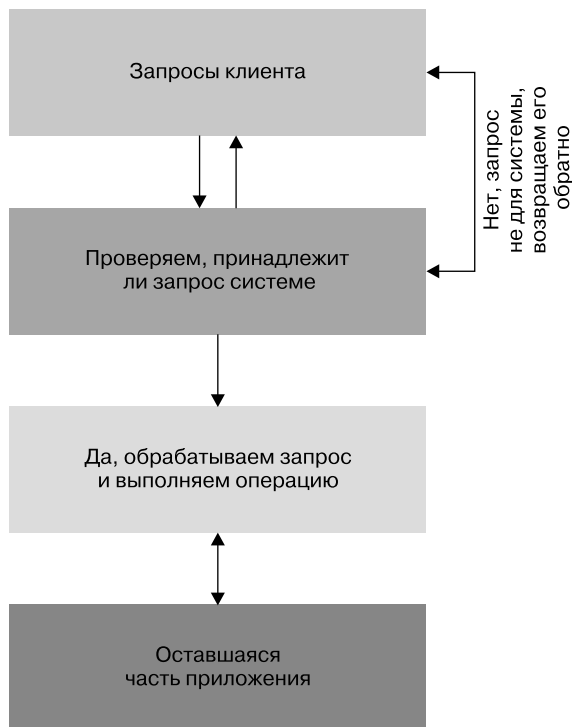


Рис. 7.1

Здесь показан сценарий, в котором система не разрешает неконтролируемый доступ. Это просто определяется следующим образом: входящий запрос принимается и внутренняя система (механизм аутентификации) проверяет, аутентифицирован ли запрос. Если да, то пользователю разрешается выполнять операции, для которых он авторизован. Это не единственная проверка, но в типичной системе авторизация происходит после аутентификации. Мы обсудим это в следующих разделах.

Чтобы лучше понять описанное, напомним простое приложение для входа в систему. Следуйте инструкциям, приведенным ниже.

1. Запустите приложение *Visual Studio 2019*.
2. Выберите команду меню **File** ▶ **New** ▶ **Project** (Файл ▶ Новый ▶ Проект) или нажмите сочетание клавиш **Ctrl+Shift+N**.
3. В окне **Project** (Проект) задайте имя своему проекту.
4. Выберите в раскрывающемся списке пункт **ASP.NET Core 2.2** и шаблон **Web Application (Model-View-Controller)** (Веб-приложение (Модель — представление — контроллер)) (рис. 7.2).

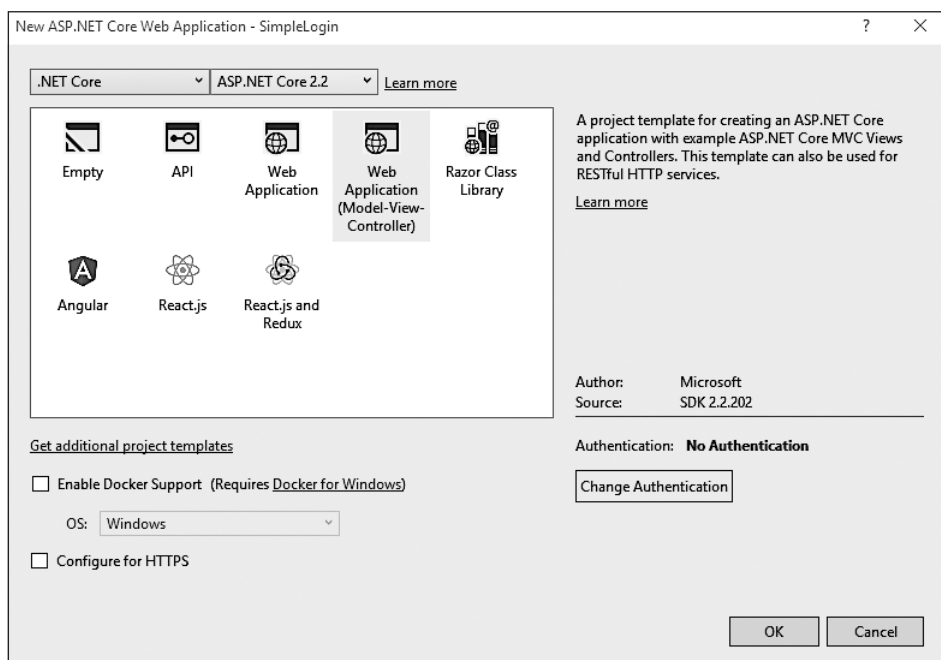
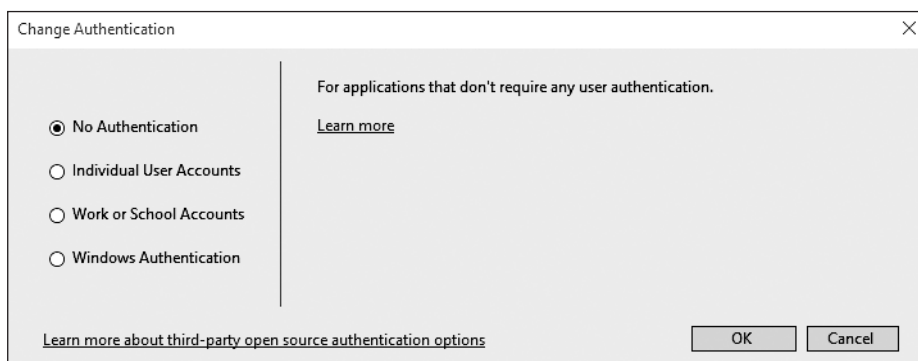
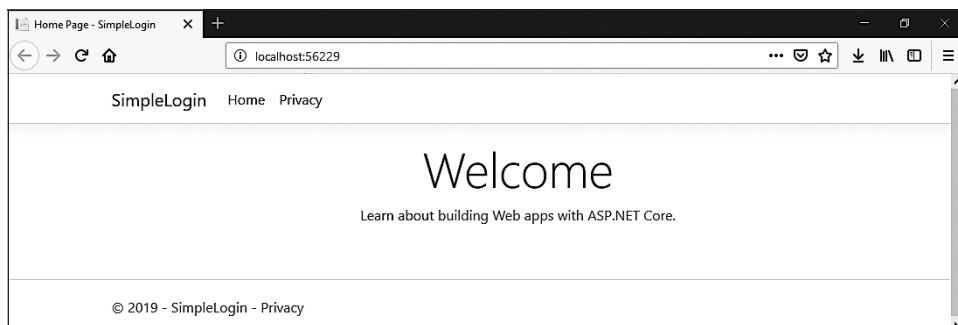


Рис. 7.2

5. Вы можете выбрать различные варианты аутентификации, доступные как часть выбранного шаблона.
6. По умолчанию шаблон предоставляет опцию No Authentication (Без аутентификации), как показано на рис. 7.3.

**Рис. 7.3**

7. Нажмите клавишу F5 и запустите приложение. Вы увидите главную страницу по умолчанию (рис. 7.4).

**Рис. 7.4**

Теперь вы можете перемещаться по каждой странице без каких-либо ограничений. Это имеет смысл, когда страницы должны находиться в открытом доступе. Страницы Home (Главная) и Privacy (Конфиденциальность) находятся в открытом доступе и не требуют аутентификации. Это значит, что любой человек может получить к ним доступ. С другой стороны, у нас может быть несколько страниц, не предназначенных для неконтролируемого доступа, например, User Profile (Профиль пользователя) и Admin (Администратор).



Обратитесь к репозиторию этой главы на GitHub по адресу github.com/PacktPublishing/Hands-On-Design-Patterns-with-C-and-.NET-Core/tree/master/Chapter7 и изучите приложение, которое мы написали с помощью ASP.NET Core MVC.

Чтобы продолжить работу с приложением (назовем его *SimpleLogin*), добавим экран с ограниченным доступом — **Products** (Продукты). В этой главе мы не будем обсуждать, как добавить новый контроллер или представления в существующий проект. Если вы хотите узнать об этом больше, то вернитесь к главе 6.

Мы добавили новую функциональность в наш проект, чтобы продемонстрировать продукты с CRUD-операциями. Теперь нажмите клавишу F5 и проверьте результат (рис. 7.5).

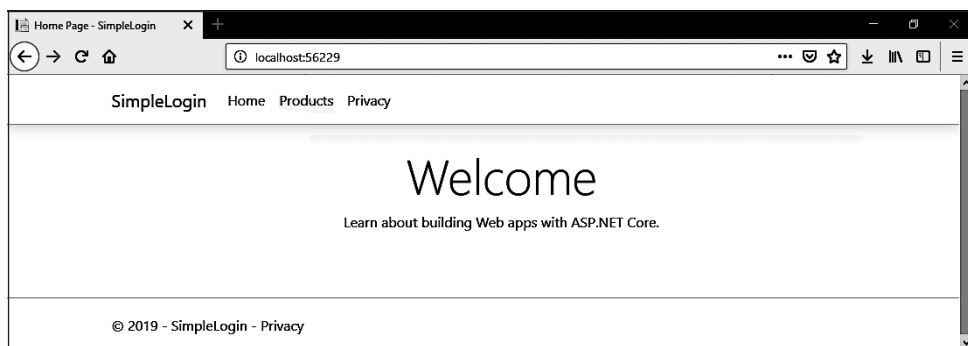


Рис. 7.5

Вы можете заметить, что теперь у нас есть новое меню под названием **Products** (Продукты).

Перейдем к новым опциям меню. Выберите меню **Products** (Продукты) (рис. 7.6).

На данном снимке экрана показана страница **Products** (Продукты). Она доступна всем, и любой желающий может просматривать ее без каких-либо ограничений. Вы можете посмотреть и заметить, что она позволяет создавать новые продукты (**Create New**), а также редактировать и удалять существующие продукты (**Edit**, **Delete**). Теперь представьте сценарий: неизвестный пользователь удалил определенный продукт, который очень важен и привлекает большой объем продаж. Только представьте, насколько это мешает бизнесу. Возможно, даже есть шанс потерять клиентов.

В нашем сценарии мы можем защитить страницу **Products** (Продукты) двумя способами.

- ❑ *Предварительная аутентификация* — на этой странице ссылка на продукты доступна не для всех, а лишь для аутентифицированных запросов/пользователей.

- ❑ *Пост-аутентификация* — на этой странице ссылка на продукты доступна для всех. Однако, как только кто-то запрашивает доступ к странице, система выполняет проверку подлинности.

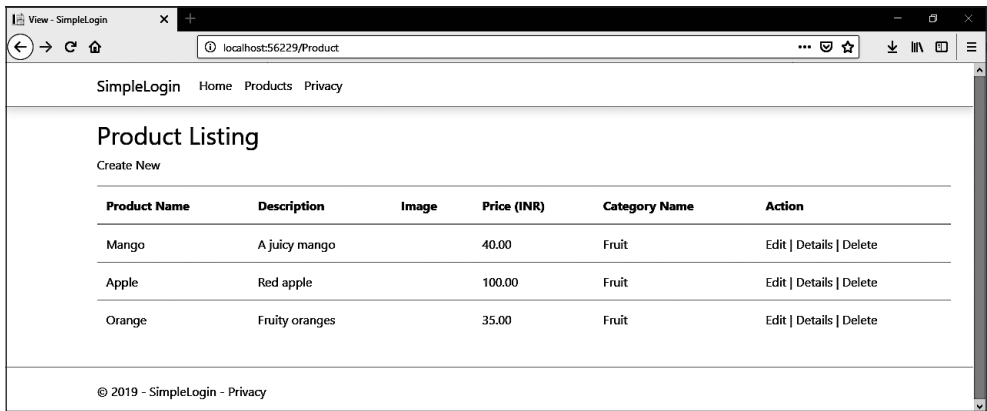


Рис. 7.6

Аутентификация в действии

В этом подразделе мы рассмотрим, как реализовать аутентификацию и сделать наши веб-страницы недоступными для запросов, не прошедших проверку подлинности.

Мы должны принять некий механизм, который предоставляет нам способ аутентификации пользователя. Как правило, если пользователь вошел в систему, то это означает, что он уже прошел проверку подлинности.

В нашем веб-приложении мы будем следовать тому же подходу и удостоверимся, что пользователь вошел в систему, прежде чем получить доступ к специальным страницам, представлениям и операциям:

```
public class User
{
    public Guid Id { get; set; }
    public string Username { get; set; }
    public string EmailId { get; set; }
    public string FirstName { get; set; }
    public string LastName { get; set; }
    public byte[] PasswordHash { get; set; }
    public byte[] PasswordSalt { get; set; }
    public string SecretKey { get; set; }
    public string Mobile { get; set; }
    public string EmailToken { get; set; }
```

```

    public DateTime EmailTokenDateTime { get; set; }
    public string OTP { get; set; }
    public DateTime OtpDateTime { get; set; }
    public bool IsMobileVerified { get; set; }
    public bool IsEmailVerified { get; set; }
    public bool IsActive { get; set; }
    public string Image { get; set; }
}

```

Класс выше — типичная модель (или сущность) *User*, представляющая соответствующую таблицу базы данных. В этой таблице будет сохранена вся информация о *User*. Вот как выглядит каждое поле:

- ❑ *Id* — *глобальный уникальный идентификатор* (Globally Unique Identifier, GUID) и первичный ключ в таблице;
- ❑ *UserName* обычно применяется во время входа в систему и других связанных с ним операций. Это программно генерируемое поле;
- ❑ *FirstName* и *LastName*, объединяясь, дают полное имя пользователя;
- ❑ *EmailId* — работающий адрес электронной почты пользователя. Он должен работать потому, что мы проверим его после/во время процесса регистрации;
- ❑ *PasswordHash* и *PasswordSalt* — массивы байтов, основанные на коде *аутентификации сообщений на основе хеша, защищенном алгоритме хеширования* (Hash-Based Message Authentication Code, Secure Hash Algorithm *HMACSHA*) 512. Значение атрибута *PasswordHash* — 64 байта, а *PasswordSalt* — 128 байт;
- ❑ *SecretKey* — строка в кодировке Base64;
- ❑ *Mobilie* — действительный номер мобильного телефона, который зависит от проверки валидности системой;
- ❑ *EmailToken* и *OTP* — *одноразовые пароли* (one-time passwords, OTP), которые генерируются случайным образом в целях проверки *emailId* и *Mobile number*;
- ❑ *EmailTokenDateTime* и *OtpDateTime* — свойства типа данных *datetime*; представляют дату и время, в котором *EmailToken* и *OTP* выдаются для пользователя;
- ❑ *IsMobileVerified* и *IsEmailverified* — логические значения (*true/false*), которые сообщают системе, проверены ли номер мобильного телефона и/или идентификатор электронной почты;
- ❑ *IsActive* — логическое значение (*true/false*), которое сообщает системе, активна ли модель *User*;
- ❑ *Image* — строка изображения в кодировке Base64; представляет фотографию профиля пользователя.

Нам нужно добавить новую сущность в класс *Context*. Добавим то, что мы видим на рис. 7.7.

```

public class InventoryContext : DbContext
{
    public InventoryContext(DbContextOptions<InventoryContext> options)
        : base(options)
    {
    }

    public InventoryContext()
    {
    }

    public DbSet<Product> Products { get; set; }
    public DbSet<Category> Categories { get; set; }
    public DbSet<User> Users { get; set; }
}

```

Рис. 7.7

Добавив предыдущую строку в класс `Context`, мы можем получить доступ к нашей пользовательской таблице напрямую, используя функциональность *Entity Framework (EF)*:

```

public class LoginViewModel
{
    [Required]
    public string Username { get; set; }
    [Required]
    [DataType(DataType.Password)]
    public string Password { get; set; }
    [Display(Name = "Remember Me")]
    public bool RememberMe { get; set; }
    public string returnUrl { get; set; }
}

```

Модель `LoginViewModel` используется для аутентификации пользователя. Ее значения приходят со страницы `Login` (Вход в систему) (которую мы обсудим и создадим в предстоящем подразделе). Она содержит следующие элементы:

- ❑ `UserName` — уникальное имя служит для идентификации пользователя. Это человекочитаемое значение, которое можно легко распознать. Не похоже на значение GUID;
- ❑ `Password` — секретное и чувствительное значение для любого пользователя;
- ❑ `RememberMe` — значение сообщает, хочет ли пользователь разрешить текущей системе сохранять в браузере клиента cookie-файлы для автоматического входа.

Для выполнения операций CRUD добавим в класс `UserManager` следующий код:

```

public class UserManager : IUserManager
{
    private readonly InventoryContext _context;
}

```

```

public UserManager(InventoryContext context) => _context = context;

public bool Add(User user, string userPassword)
{
    var newUser = CreateUser(user, userPassword);
    _context.Users.Add(newUser);
    return _context.SaveChanges() > 0;
}

public bool Login(LoginViewModel authRequest) => FindBy(authRequest) != null;

public User GetBy(string userId) => _context.Users.Find(userId);

```

Ниже приведен фрагмент кода из остальных методов класса `UserManager`:

```

public User FindBy(LoginViewModel authRequest)
{
    var user = Get(authRequest.Username).FirstOrDefault();
    if (user == null) throw new ArgumentException("You are
        not registered with us.");
    if (VerifyPasswordHash(authRequest.Password, user.PasswordHash,
        user.PasswordSalt)) return user;
    throw new ArgumentException("Incorrect username or password.");
}
public IEnumerable<User> Get(string searchTerm, bool isActive = true)
{
    return _context.Users.Where(x =>
        x.UserName == searchTerm.ToLower() || x.Mobile == searchTerm ||
        x.EmailId == searchTerm.ToLower() && x.IsActive == isActive);
}
...
}

```

Код выше — это класс `UserManager`, который позволяет взаимодействовать с таблицей `User` с помощью Entity Framework.

Код ниже отображает экран Login (Вход в систему):

```

<form asp-action="Login" asp-route-returnurl="@Model.ReturnUrl">
    <div asp-validation-summary="ModelOnly" class="text-danger"></div>

    <div class="form-group">
        <label asp-for="Username" class="control-label"></label>
        <input asp-for="Username" class="form-control" />
        <span asp-validation-for="Username" class="text-danger"></span>
    </div>

    <div class="form-group">
        <label asp-for="Password" class="control-label"></label>
        <input asp-for="Password" class="form-control" />
        <span asp-validation-for="Password" class="text-danger"></span>
    </div>

```

```
<div class="form-group">
  <label asp-for="RememberMe" ></label>
  <input asp-for="RememberMe" />
  <span asp-validation-for="RememberMe"></span>
</div>
<div class="form-group">
  <input type="submit" value="Login" class="btn btn-primary" />
</div>
</form>
```

Это код со страницы (представления) `Login.cshtml`. На данной странице представлена форма для ввода данных при входе в систему. Они поступают в контроллер `Account` и затем проверяются на валидность для аутентификации пользователя.

Ниже показан код метода действия `Login`:

```
[HttpGet]
public IActionResult Login(string returnUrl = "")
{
    var model = new LoginViewModel { ReturnUrl = returnUrl };
    return View(model);
}
```

Предыдущий фрагмент кода — это запрос `Get /Account/Login`, который отображает пустую страницу входа в систему, показанную на рис. 7.8.

Рис. 7.8

Данную страницу пользователь видит, как только нажимает опцию меню Login (Вход в систему). Это простая форма, применяемая для ввода данных при входе в систему.

Следующий код — action-метод Login, который обрабатывает функциональность Login:

```
[HttpPost]
public IActionResult Login(LoginViewModel model)
{
    if (ModelState.IsValid)
    {
        var result = _authManager.Login(model);

        if (result)
        {
            return !string.IsNullOrEmpty(model.ReturnUrl) &&
                Url.IsLocalUrl(model.ReturnUrl)
                ? (IActionResult)Redirect(model.ReturnUrl)
                : RedirectToAction("Index", "Home");
        }
    }
    ModelState.AddModelError("", "Invalid login attempt");
    return View(model);
}
```

Предыдущий фрагмент кода — это запрос Post /Account/Login со страницы входа, который отправляет все полученные данные из класса LoginViewModel.

На рис. 7.9 приведен внешний вид представления Login.

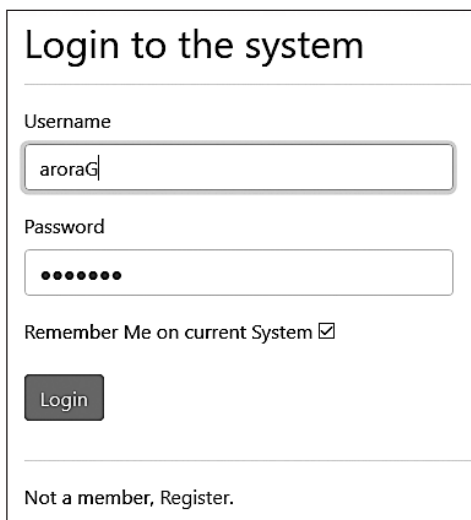
The image shows a web form titled "Login to the system". It contains a "Username" label followed by a text input field containing the text "aroraG". Below this is a "Password" label followed by a password input field represented by a series of dots. Under the password field is a checkbox labeled "Remember Me on current System" which is checked. Below the checkbox is a dark grey "Login" button. At the bottom of the form, there is a link that says "Not a member, Register."

Рис. 7.9

Здесь мы пытаемся войти, используя наши учетные данные пользователя по умолчанию (имя пользователя **AroraG** и пароль **test123**). Информация, связанная с этим входом, сохраняется в cookie-файлах, но только в том случае, если пользователь установил флажок **Remember Me** (Запомнить меня). Система помнит сеанс входа в систему на текущем компьютере до тех пор, пока пользователь не нажмет кнопку **Logout** (Выйти из системы).

После нажатия кнопки **Login** (Вход в систему) пользователем система аутентифицирует предоставленные им данные для входа в систему и перенаправляет его на главную страницу, как показано на рис. 7.10.

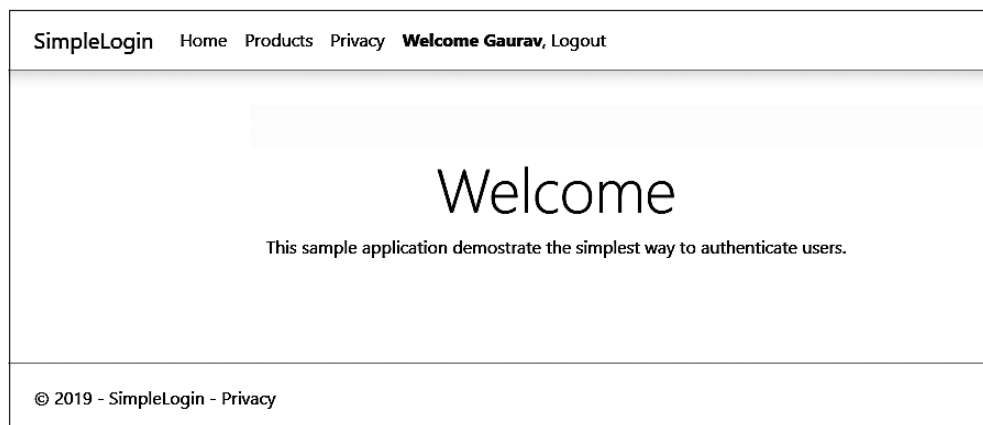


Рис. 7.10

Вы можете наблюдать текст в меню: **Welcome Gaurav**. Этот приветственный текст не отображается автоматически, но мы поручили нашей системе показать его, добавив несколько строк кода:

```
<li class="nav-item">
  @{
    if (AuthManager.IsAuthenticated)
    {
      <a class="nav-link text-dark" asp-area=""
        aspcontroller="Account" asp-action="Logout"><strong>Welcome
        @AuthManager.Name</strong>, Logout</a>

    }
    else
    {
      <a class="nav-link text-dark" asp-area=""
        aspcontroller="Account" asp-action="Login">Login</a>
    }
  }
</li>
```


Этот фрагмент кода взят из представления/страницы `_Layout.cshtml`. Мы проверяем, действительно ли `IsAuthenticated` возвращает `true`. Если да, то отображается приветственное сообщение. Оно сопровождается опцией `Logout` (Выход из системы), но отображает меню `Login`, когда `IsAuthenticated` возвращает значение `false`:

```
public bool IsAuthenticated
{
    get { return User.Identities.Any(u => u.IsAuthenticated); }
}
```

Элемент `IsAuthenticated` — это свойство `ReadOnly` класса `AuthManager`, которое проверяет, аутентифицирован ли запрос. Прежде чем двигаться дальше, вернемся к методу `Login`:

```
public IActionResult Login(LoginViewModel model)
{
    if (ModelState.IsValid)
    {
        var result = _authManager.Login(model);

        if (result)
        {
            return !string.IsNullOrEmpty(model.ReturnUrl) &&
                Url.IsLocalUrl(model.ReturnUrl)
                ? (IActionResult)Redirect(model.ReturnUrl)
                : RedirectToAction("Index", "Home");
        }
    }
    ModelState.AddModelError("", "Invalid login attempt");
    return View(model);
}
```

Метод `Login` просто проверяет пользователя. Взгляните на эту инструкцию: `var result = _authManager.Login(model);`. Она вызывает метод `Login` из `AuthManager` (рис. 7.11).

Если метод `Login` возвращает `true`, то перенаправляет со страницы `Login` (Вход в систему) на `Home` (Главная). В противном случае остается на странице `Login` (Вход в систему), выдавая сообщение `Invalid login attempt` (Неудачная попытка входа в систему). Ниже приведен код метода `Login`:

```
public bool Login(LoginViewModel model)
{
    var user = _userManager.FindBy(model);
    if (user == null) return false;
    SignInCookie(model, user);
    return true;
}
```

Login to the system

• Invalid login attempt

Username

aroraG

Password

Remember Me on current System ☒

Login

Not a member, Register.

Рис. 7.11

Метод `Login` типичен для класса `AuthManager`, который вызывает метод `FindBy(model)` из `UserManager` и проверяет, существует ли пользователь. Если да, то в дальнейшем вызывает метод `SignInCookie(model, user)` класса `AuthManager`, в противном случае просто возвращает `false`. Это значит, что вход неудачен:

```
private void SignInCookie(LoginViewModel model, User user)
{
    var claims = new List<Claim>
    {
        new Claim(ClaimTypes.Name, user.FirstName),
        new Claim(ClaimTypes.Email, user.EmailId),
        new Claim(ClaimTypes.NameIdentifier, user.Id.ToString())
    };

    var identity = new ClaimsIdentity(claims,
        CookieAuthenticationDefaults.AuthenticationScheme);
    var principal = new ClaimsPrincipal(identity);
    var props = new AuthenticationProperties { IsPersistent =
        model.RememberMe };
    _httpContext.SignInAsync(CookieAuthenticationDefaults.AuthenticationScheme,
        principal, props).Wait();
}
```

Следующий фрагмент кода гарантирует, что если пользователь аутентифицирован, то его данные должны быть сохранены в `HttpContext`, чтобы система могла аутентифицировать каждый входящий запрос от пользователей. Вы можете наблюдать выражение `_httpContext.SignInAsync(CookieAuthenticationDefaults.AuthenticationScheme, principal, props).Wait();`, которое фактически регистрирует пользователя и включает аутентификацию с помощью cookie-файлов:

```
// аутентификация через cookie-файлы
services.AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme).
AddCookie();
// для требований
services.AddSingleton<IHttpContextAccessor, HttpContextAccessor>();
services.AddTransient<IAuthManager, AuthManager>();
```

Предыдущие строки помогают нам включить аутентификацию через cookie и требования к входящим запросам. Наконец, `app.UseAuthentication();` добавляет в наше приложение возможность использования механизма аутентификации. Эти строки должны быть добавлены в класс `Startup.cs`.

Почему это имеет значение

Мы добавили много кода в веб-приложение, но действительно ли это помогает ограничить доступ к страницам? Страница **Products** (Продукты) все еще открыта, поэтому мы можем выполнить с нее любые доступные действия (рис. 7.12).

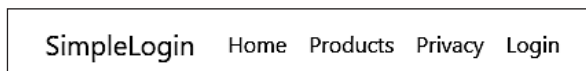


Рис. 7.12

Как пользователи, мы можем видеть опцию **Products** (Продукты) независимо от того, входили ли в систему (рис. 7.13).

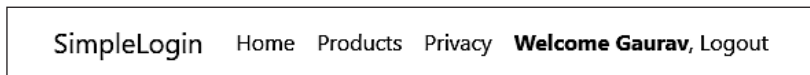


Рис. 7.13

На данном снимке экрана показан тот же пункт меню **Products** (Продукты) после входа в систему, что и до входа в нее.

Мы можем ограничить доступ к странице **Products** (Продукты) следующим образом:

```
<li class="nav-item">
  @{
    if (AuthManager.IsAuthenticated)
```

```
{
    <a class="nav-link text-dark" asp-area=""
      aspcontroller="Product" asp-action="Index">Products</a>
}
</li>
```

На рис. 7.14 приведен главный экран приложения.

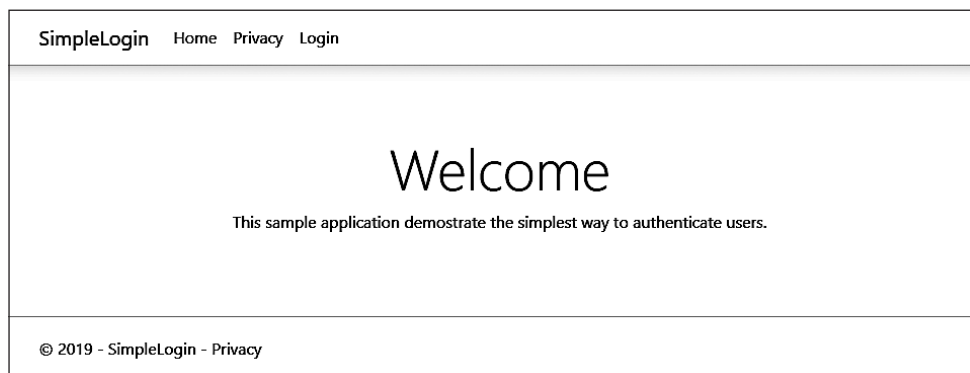


Рис. 7.14

Код выше помогает системам отображать меню **Products** (Продукты) только после входа пользователя в систему (аутентификации). Опции этого меню не будут отображаться на экране. Таким образом мы можем ограничить доступ. Однако у этого подхода есть свои недостатки. И вот главный из них: что, если кто-то знает URL-адрес страницы **Products** (Продукты), который приводит к `/Product/Index`? Тогда становится возможным выполнять операции ограниченного доступа. Доступ к ним ограничен, так как они не предназначены для применения пользователем, не вошедшим в систему.

Авторизация в действии

В предыдущем подразделе мы обсудили, как избежать неконтролируемого доступа к определенному или ограниченному экрану/странице. Мы видели, что **Login** (Вход в систему) аутентифицирует пользователя и позволяет ему сделать запрос в системе. С другой стороны, аутентификация не означает, что пользователь имеет право доступа к определенному разделу, странице или экрану.

На рис. 7.15 показан типичный процесс авторизации и аутентификации.

В этом процессе первый запрос/пользователь проходит аутентификацию (обычно это форма входа в систему), затем запрос авторизуется на выполнение определенной/запрошенной операции (операций). Возможно большое количество сценариев,

когда запрос аутентифицирован, но не авторизован для доступа к определенному ресурсу или выполнения определенной операции.



Рис. 7.15

В нашем приложении есть страница **Products** с операциями **CRUD**. Она не публична, то есть имеет ограниченный доступ.

Мы возвращаемся к следующей основной проблеме, которую не решили в предыдущем подразделе: «Что делать, если пользователь аутентифицирован, но не имеет права доступа к определенной странице/ресурсу? Не имеет значения, скрываем ли мы страницу от неавторизованного пользователя, поскольку он может легко получить к ней доступ или просмотреть ее, введя ее URL». Решить эту проблему мы можем с помощью шагов, представленных ниже.

1. Проверка авторизации при каждом доступе к скрытому ресурсу. То есть всякий раз, когда пользователь пытается получить доступ к ресурсу, вводя URL в браузер,

зере, система проверяет авторизацию, поэтому входящие запросы на доступ могут быть авторизованы. Если входящий запрос пользователя не авторизован, то не сможет выполнить указанную операцию.

2. Проверка авторизации на каждой операции ограниченного ресурса означает, что если пользователь аутентифицирован, то сможет получить доступ к скрытой странице, но операции этой страницы могут быть доступны только при авторизации пользователя.



Пространство имен `Microsoft.AspNetCore.Authorization` предоставляет встроенные функции для авторизации определенных ресурсов.

Чтобы ограничить доступ и избежать неуправляемого доступа к определенному ресурсу, мы можем использовать атрибут `Authorize` (рис. 7.16).

```

8 namespace SimpleLogin.Controllers
9 {
10     [Authorize]
11     public class ProductController : Controller
12     {
13         private readonly IInventoryRepository _inventoryRepository;
14
15         public ProductController(IInventoryRepository inventoryRepository) => _inventoryRepository = inventoryRepository;
16
17         public IActionResult Index() => View(_inventoryRepository.GetProducts().ToProductvm());
18
19         public IActionResult Details(Guid id) => View(_inventoryRepository.GetProduct(id).ToProductvm());
20
21         public IActionResult Create() => View();
22     }
23 }

```

Рис. 7.16

На данном снимке экрана видно, что мы помещаем атрибут `Authorize` в `ProductController`. Теперь нажмите F5 и запустите приложение.



Если пользователь не вошел в систему, то не сможет увидеть страницу `Products` (Продукты): мы уже добавили условие. Если пользователь проверен, то ссылка `Products` (Продукты) отображается в строке меню.

Не входите в систему и введите URL страницы `Products` (Продукты) — `http://localhost:56229/Product` — прямо в адресную строку браузера. Произойдет перенаправление пользователя на экран `Login` (Вход в систему). Пожалуйста, посмотрите рис. 7.17 и проверьте URL. Вы можете заметить, что URL содержит часть `ReturnUrl`, которая инструктирует систему о том, куда следует перенаправлять при успешной попытке входа.

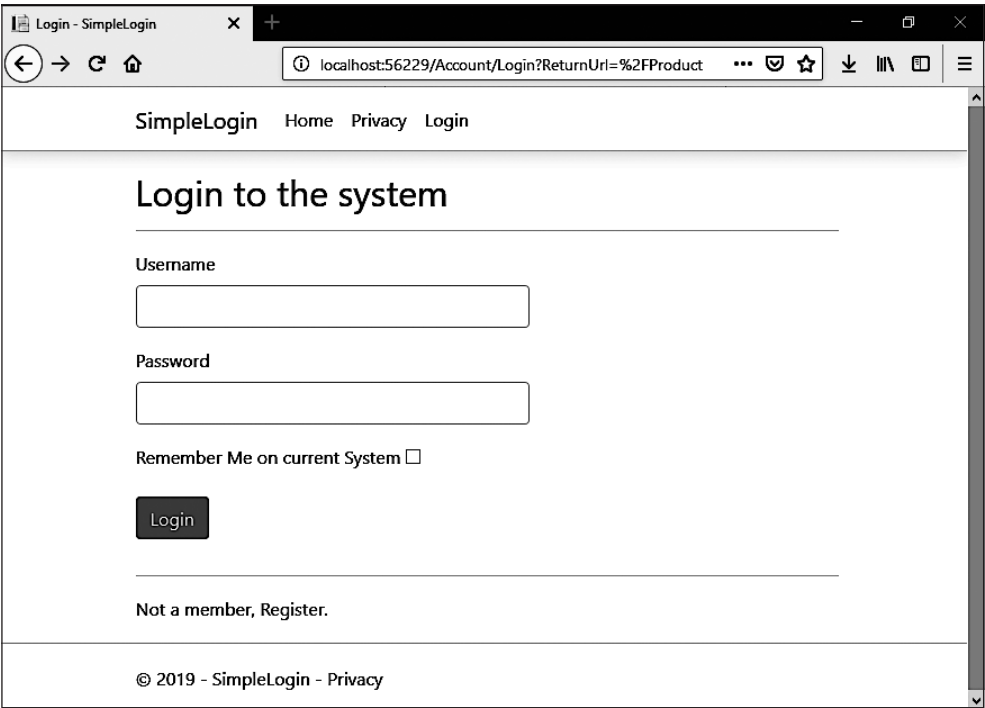


Рис. 7.17

На рис. 7.18 показан Product Listing (Список продуктов).

Product Listing					
Create New					
Product Name	Description	Image	Price (INR)	Category Name	Action
Mango	A juicy mango		40.00	Fruit	Edit Details Delete
Apple	Red apple		100.00	Fruit	Edit Details Delete
Orange	Fruity oranges		35.00	Fruit	Edit Details Delete

Рис. 7.18

Наш экран списка продуктов предоставляет такие операции, как создание, редактирование, удаление и детализация (Create New, Edit, Delete, Details). Сейчас приложение позволяет пользователю выполнять эти операции. Отсюда вопрос: имеет ли смысл, что любой посетивший и прошедший проверку подлинности пользователь может

создавать, обновлять и удалять продукт? Если мы дадим подобные права каждому пользователю, то это может привести к таким последствиям:

- ❑ мы можем получить много продуктов, которые уже были добавлены в систему;
- ❑ неизбежное удаление продуктов;
- ❑ неизбежное обновление продуктов.

Можем ли мы создать нечто подобное типу пользователя, отличающего всех пользователей типа **Admin** от обычных пользователей, позволяя выполнять эти операции только пользователям с правами администратора? Есть идея получше: добавить роли для пользователей. То есть нужно создать пользователя определенного типа.

Добавим новую сущность в наш проект и назовем ее **Role**:

```
public class Role
{
    public Guid Id { get; set; }
    public string Name { get; set; }
    public string ShortName { get; set; }
}
```

Предыдущий фрагмент кода, определяющий класс **Role** для пользователя, имеет свойства, описанные в следующем списке:

- ❑ **Id** — в качестве первичного ключа используется GUID;
- ❑ **Name** — имя **Role** типа **string**;
- ❑ **ShortName** — краткое название роли, имеющей тип **string**.

Нам нужно добавить наш новый класс в класс **Context**. Сделаем это так (рис. 7.19).

```
public class InventoryContext : DbContext
{
    public InventoryContext(DbContextOptions<InventoryContext> options)
        : base(options)
    {
    }

    public InventoryContext()
    {
    }

    public DbSet<Product> Products { get; set; }
    public DbSet<Category> Categories { get; set; }
    public DbSet<User> Users { get; set; }
    public DbSet<Role> Roles { get; set; }
}
```

Рис. 7.19

Предыдущий код позволяет работать с различными операциями базы данных с помощью EF:

```
public IEnumerable<Role> GetRoles() => _context.Roles.ToList();

public IEnumerable<Role> GetRolesBy(string userId) =>
    _context.Roles.Where(x => x.UserId.ToString().Equals(userId));

public string RoleNamesBy(string userId)
{
    var listOfRoleNames = GetRolesBy(userId).Select(x=>x.ShortName).ToList();
    return string.Join(", ", listOfRoleNames);
}
```

Три метода класса `UserManager`, которые появились в предыдущем фрагменте кода, позволяют получать `Roles` из базы данных:

```
private void SignInCookie(LoginViewModel model, User user)
{
    var claims = new List<Claim>
    {
        new Claim(ClaimTypes.Name, user.FirstName),
        new Claim(ClaimTypes.Email, user.EmailId),
        new Claim(ClaimTypes.NameIdentifier, user.Id.ToString())
    };

    if (user.Roles != null)
    {
        string[] roles = user.Roles.Split(",");
        claims.AddRange(roles.Select(role => new Claim(ClaimTypes.Role, role)));
    }

    var identity = new ClaimsIdentity(claims,
        CookieAuthenticationDefaults.AuthenticationScheme);

    var principal = new ClaimsPrincipal(identity);
    var props = new AuthenticationProperties { IsPersistent = model.RememberMe };
    _httpContext.SignInAsync(CookieAuthenticationDefaults.AuthenticationScheme,
        principal, props).Wait();
}
```

Мы добавили роли к нашим `Claims`, изменив метод `SigningCookie` класса `AuthManager` (рис. 7.20).

На снимке экрана выше видно, что у пользователя по имени Гаурав есть две роли: администратор (`Admin`) и менеджер (`Manager`) (рис. 7.21).

Мы предоставляем `ProductController` только пользователям с ролями `Admin` и `Manager`. Теперь попробуем войти в систему с помощью `aroraG` — и увидим `Product Listing`, как показано на рис. 7.22.

```
private void SignInCookie(LoginViewModel model, User user)
{
    model = {LoginViewModel}, user = {User}
    {
        var claims = new List<Claim>() { claims.Count = 5 }
        {
            new Claim(ClaimTypes.Name, user.FirstName),
            new Claim(ClaimTypes.Email, user.EmailId),
            new Claim(ClaimTypes.NameIdentifier, user.Id.ToString())
        };

        if (user.Roles != null)
        {
            string[] roles = user.Roles.Split(",");
            roles = {string[2]}

            claims.AddRange(roles.Select(role => new Claim(ClaimTypes.Role, role)));
        }

        var identity = new ClaimsIdentity(claims, "Cookie");
        var principal = new ClaimsPrincipal(identity);
        var props = new AuthenticationProperties { IsPersistent = model.RememberMe };

        _httpContext.SignInAsync(CookieAuthenticationDefaults.AuthenticationScheme, principal, props).Wait();
    }
}
```

Рис. 7.20

```
[Authorize(Roles = "Admin,Manager")]
```

Рис. 7.21

SimpleLogin Home Products Privacy **Welcome Gaurav, Logout**

Product Listing

Create New

Product Name	Description	Image	Price (INR)	Category Name	Action
Mango	A juicy mango		40.00	Fruit	Edit Details Delete
Apple	Red apple		100.00	Fruit	Edit Details Delete
Orange	Fruity oranges		35.00	Fruit	Edit Details Delete

Рис. 7.22

Попробуем войти в систему со вторым пользователем, **aroraG1**, который имеет роль редактора (**Editor**). Это приведет к ошибке **AccessDenied**, показанной на рис. 7.23.

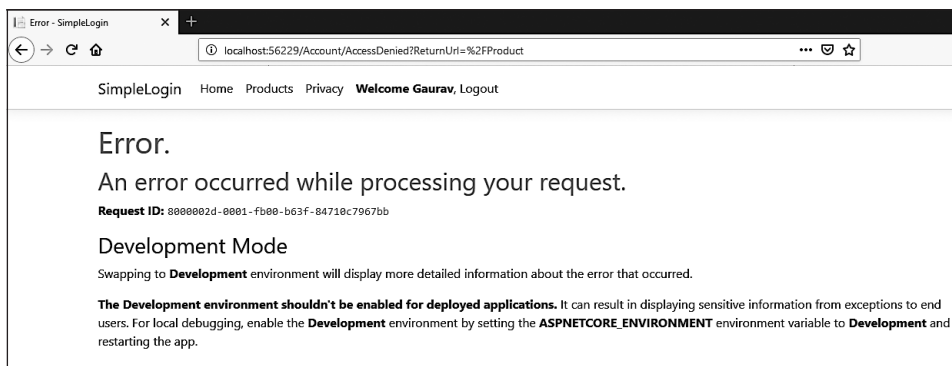


Рис. 7.23

Таким образом, мы сможем защитить наши скрытые ресурсы. Есть много способов добиться этого. .NET Core MVC предоставляет встроенный функционал для достижения данной цели; кроме того, вы можете настроить защиту. Если не хотите использовать встроенные функции, то можете легко разработать собственный функционал, добавив его в существующий код. В этом случае вам придется начинать с нуля. Более того, имея что-либо в доступе, нет смысла создавать нечто подобное снова. Если вы не нашли функциональность для доступных компонентов, то вам следует настроить существующую функциональность, а не писать весь код с нуля.



Разработчик должен реализовать механизм аутентификации, который не может быть взломан. В этом разделе мы много говорили об аутентификации и авторизации, а также о написании кода и создании веб-приложения. Касательно аутентификации мы должны использовать хороший механизм, чтобы никто не мог его взломать или обойти. Есть еще два подхода, с которых вы можете начать:

- фильтры аутентификации;
- аутентификация отдельных запросов/конечных точек.

После выполнения предыдущих шагов каждый запрос, приходящий в любом режиме, должен быть аутентифицирован и авторизован, прежде чем система ответит вызывающему пользователю или клиенту. Этот процесс в основном включает следующие элементы.

- ❑ *Конфиденциальность* — защищенная система гарантирует, что любые конфиденциальные данные не будут подвергаться неаутентифицированным и не-санкционированным запросам на доступ.
- ❑ *Доступность* — меры безопасности в системе обеспечивают доступность системы для проверенных пользователей, что подтверждается механизмом аутентификации и авторизации системы.
- ❑ *Целостность* — в защищенной системе взлом данных невозможен, поэтому данные защищены.

Создание тестового проекта веб-приложения

Модульное тестирование — это проверка корректности кода. То есть код, содержащий баги (некорректный), послужит причиной многих неизвестных и нежелательных проблем в приложении. Преодолеть их можно, следуя подходу TDD.



Вы можете попрактиковаться в TDD с Katas. Посетите страницу www.codeproject.com/Articles/886492/Learning-Test-Driven-Development-with-TDD-Katas, чтобы узнать больше. Если хотите потренироваться в этом подходе, то используйте репозиторий github.com/garora/TDD-Katas.

Мы уже много говорили о TDD в предыдущих главах, так что здесь не будем обсуждать его подробно. Вместо этого создадим тестовый проект следующим образом.

1. Откройте наше веб-приложение.
2. На панели Solution Explorer (Обозреватель решений) Visual Studio щелкните правой кнопкой мыши на пункте Solution (Решение) и выберите команду Add ▸ New Project (Добавить ▸ Новый проект) в контекстном меню, как показано на рис. 7.24.

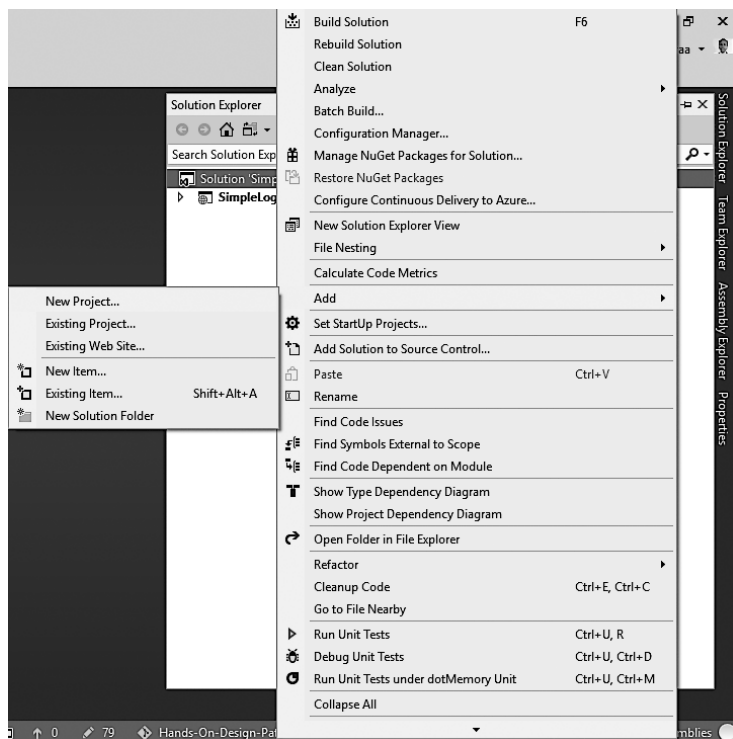


Рис. 7.24

3. В окне Add New Project (Добавить новый проект) приложений выберите .NET Core ► xUnit Test Project (.NET Core) и дайте проекту осмысленное название (рис. 7.25).

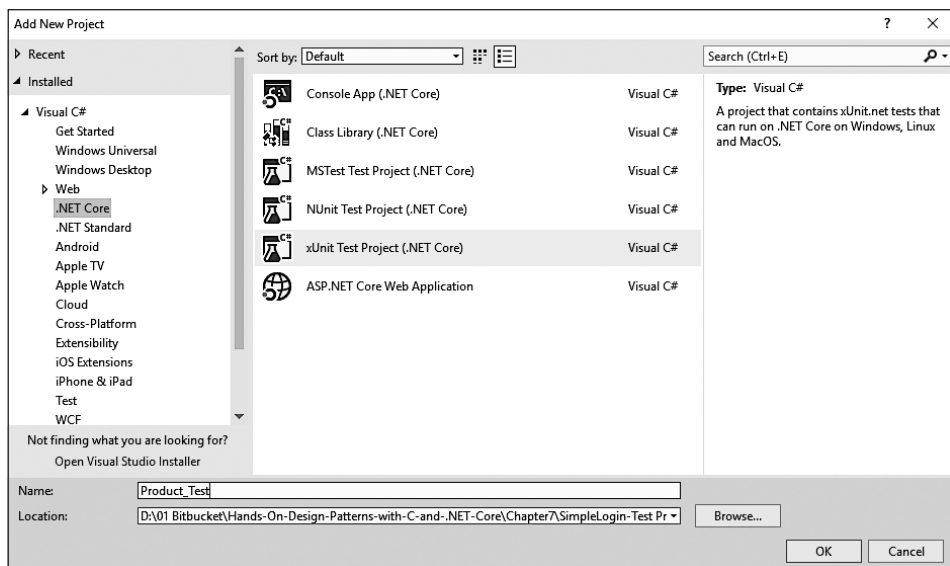


Рис. 7.25

Вы получите модульный класс `test` по умолчанию с пустым кодом, как показано в следующем фрагменте кода:

```
namespace Product_Test
{
    public class UnitTest1
    {
        [Fact]
        public void Test1()
        {
        }
    }
}
```

Вы можете изменить имя этого класса или отказаться от него, если хотите написать собственный класс `test`:

```
public class ProductData
{
    public IEnumerable<ProductViewModel> GetProducts()
    {
        var productVm = new List<ProductViewModel>
        {
            new ProductViewModel

```

```
        {
            CategoryId = Guid.NewGuid(),
            CategoryDescription = "Category Description",
            CategoryName = "Category Name",
            ProductDescription = "Product Description",
            ProductId = Guid.NewGuid(),
            ProductImage = "Image full path",
            ProductName = "Product Name",
            ProductPrice = 112M
        },
        ...
    };

    return productVm;
}
```

4. Код, представленный выше, относится к только что добавленному классу `ProductData`. Добавьте его в новую папку с именем `Fake`. Этот класс создает фиктивные данные, чтобы мы могли протестировать наше веб-приложение для продукта:

```
public class ProductTests
{
    [Fact]
    public void Get_Returns_ActionResults()
    {
        // подготовка
        var mockRepo = new Mock<IProductRepository>();
        mockRepo.Setup(repo => repo.GetAll()).Returns(new
            ProductData().GetProductList());
        var controller = new ProductController(mockRepo.Object);

        // действие
        var result = controller.GetList();

        // проверка
        var viewResult = Assert.IsType<OkObjectResult>(result);
        var model = Assert.IsAssignableFrom<IEnumerable<
            ProductViewModel>>(viewResult.Value);
        Assert.NotNull(model);
        Assert.Equal(2, model.Count());
    }
}
```

5. Добавьте новый файл под названием `ProductTests` в папку `Services`. Пожалуйста, обратите внимание, что в этом коде мы используем *заглушки* и *имитации*.

Предыдущий код будет указывать на ошибки, подчеркнув их красными волнистыми линиями, как показано на рис. 7.26.

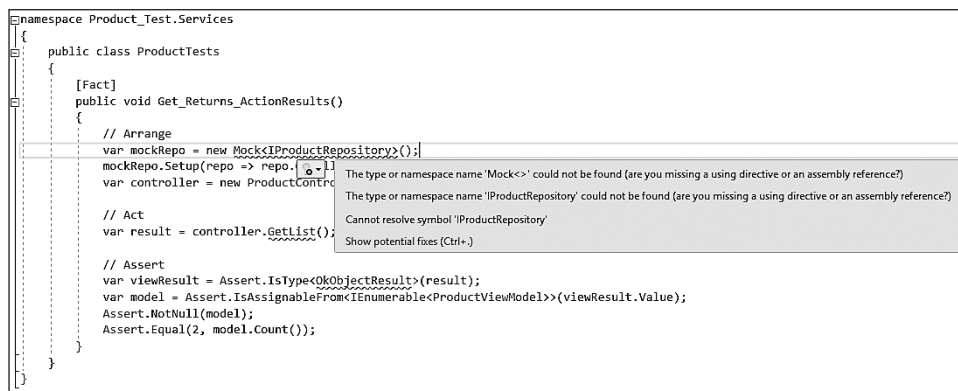


Рис. 7.26

6. Код выше содержит ошибки, так как мы не добавили некоторые пакеты, необходимые для выполнения тестов. Чтобы исправить эти ошибки, мы должны установить поддержку *moq* в наш проект *test*. Передайте следующую команду в консоль диспетчера пакетов (Package Manager Console):

install-package moq

7. Команда установит платформу *moq* в тестовом проекте. Обратите внимание: при ее выполнении вы должны выбрать тестовый проект, который мы создали (рис. 7.27).

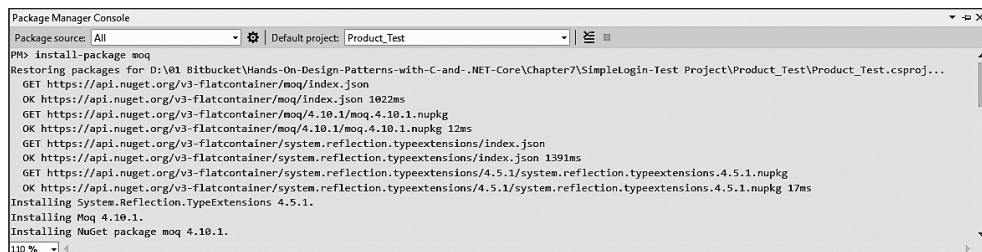


Рис. 7.27

Как только *moq* установлен, вы можете двигаться дальше и начать тестирование.



Во время работы с тестовыми проектами xUnit необходимо обратить внимание на следующие важные моменты:

- **[Fact]** — атрибут, применяемый для обычного метода тестирования, который не содержит параметров;
- **[Theory]** — атрибут, используемый для параметризованного метода тестирования.

8. Все установлено. Теперь перейдите к Test Explorer (Обозреватель тестов) и запустите тесты (рис. 7.28).

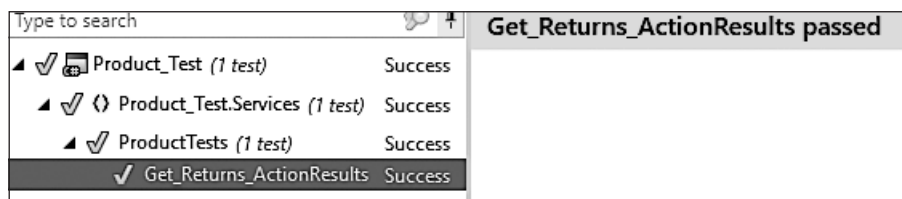


Рис. 7.28

Отлично, тесты завершаются успешно! Это значит, наши методы контроллера написаны качественно и в нашем коде нет никаких проблем или ошибок, которые могут нарушить функциональность приложения.

Резюме

Основная цель этой главы состояла в том, чтобы реализовать для нашего веб-приложения защиту от неконтролируемых запросов. В главе мы пошагово описали создание веб-приложения с помощью Visual Studio, а также аутентификацию и авторизацию. Кроме того, обсудили TDD и создали новый тестовый проект xUnit для веб-приложения, в котором использовали *заглушки* и *имитации*.

В следующей главе мы обсудим практические рекомендации и паттерны параллельного программирования в .NET Core.

Вопросы

Следующие вопросы позволят вам усвоить информацию, представленную в этой главе.

1. Что такое аутентификация и авторизация?
2. Безопасно ли использовать аутентификацию на первом уровне запроса, а затем разрешать входящие запросы для скрытых областей?
3. Как доказать, что авторизация всегда приходит после аутентификации?
4. Что такое разработка через тестирование и почему разработчики должны подходить к ней со всей тщательностью?
5. Что такое TDD Katas? Как он помогает улучшить подход к TDD?

Дальнейшее чтение

Чтобы узнать больше о рассмотренных в главе темах, обратитесь к следующим книгам:

- ❑ *Arora G., Dash T.* Building RESTful Web services with .NET Core. — Packt. www.packtpub.com/application-development/building-restful-web-services-net-core;
- ❑ *Adewole A.* C# and .NET Core Test Driven Development. — Packt. www.packtpub.com/application-development/c-and-net-core-test-driven-development.

Часть III

Функциональное программирование, реактивное программирование и кодирование для облака

Это самая важная часть книги. В ней читатели, уже знакомые с .NET Framework, могут соотнести свое обучение с .NET Core, а читатели, знакомые с .NET Core, могут расширить свои знания с помощью практических примеров. Мы будем использовать паттерны для разрешения некоторых наиболее сложных аспектов современной разработки программного обеспечения.

Часть III состоит из таких глав:

- ❑ глава 8 «Конкурентное программирование в .NET Core»;
- ❑ глава 9 «Функциональное программирование»;
- ❑ глава 10 «Модели и методы реактивного программирования»;
- ❑ глава 11 «Усовершенствованные методы проектирования и применения баз данных»;
- ❑ глава 12 «Разработка облачных приложений».

8

Конкурентное программирование в .NET Core

В предыдущей главе мы создали веб-приложение с помощью различных веб-паттернов. Мы настроили механизмы авторизации и проверки подлинности для защиты веб-приложения и обсудили *разработку через тестирование* (test-driven development, TDD), чтобы удостовериться, что наш код был проверен и работает.

В этой главе мы рассмотрим практические рекомендации, которые следует применять при конкурентном программировании в .NET Core. В следующих разделах данной главы мы узнаем о паттернах проектирования, актуальных для хорошо организованных параллельных приложений на C# и .NET Core.

В этой главе будут рассмотрены следующие темы:

- ❑ `async/await` — чем плоха блокировка;
- ❑ многопоточное и асинхронное программирование;
- ❑ конкурентные коллекции;
- ❑ паттерны и рекомендации — TDD и параллельный LINQ.

Технические требования

В этой главе содержатся примеры кода, объясняющие принципы качественной разработки. Код упрощен и предназначен лишь для демонстрации. Большинство примеров основаны на консольном приложении .NET Core, написанном на C#.



Полный исходный код доступен по ссылке <https://github.com/PacktPublishing/Hands-On-Design-Patterns-with-C-and-.NET-Core/tree/master/Chapter8>.

Чтобы запустить и выполнить код, вам потребуется следующее:

- ❑ Visual Studio 2019 (вы также можете использовать Visual Studio 2017);
- ❑ .NET Core;
- ❑ SQL Server (в этой главе применяется Express Edition).

Установка Visual Studio

Чтобы запустить примеры кода, вам необходимо установить Visual Studio. Для этого выполните следующие шаги.

1. Скачайте Visual Studio по ссылке, которая содержит инструкции по установке: docs.microsoft.com/ru-ru/visualstudio/install/install-visual-studio.
2. Следуйте инструкциям по установке. Доступны различные версии Visual Studio; в этой главе мы используем Visual Studio для Windows.

Вы также можете применять другую интегрированную среду разработки по вашему усмотрению.

Установка .NET Core

Если вы еще не установили .NET Core, то следуйте инструкции ниже.

1. Скачайте .NET Core для Windows по ссылке dotnet.microsoft.com/download.
2. Для получения нескольких версий и соответствующей библиотеки перейдите по ссылке dotnet.microsoft.com/download/dotnet-core/2.2.

Установка SQL Server

Если у вас не установлен SQL Server, то вам необходимо выполнить следующие действия.

1. Скачайте SQL Server по ссылке www.microsoft.com/ru-RU/download/details.aspx?id=1695.
2. Инструкции по установке можно найти по ссылке docs.microsoft.com/ru-ru/sql/ssms/download-sql-server-management-studio-ssms?view=sql-server-2017.



Для устранения неполадок и получения дополнительной информации см. www.blackbaud.com/files/support/infinityinstaller/content/installermaster/tkinstallsqlserver2008r2.htm.

Конкурентность в реальном мире

Конкурентность — это часть нашей жизни: она существует в реальном мире. Когда мы обсуждаем конкурентность, мы имеем в виду многозадачность.

В реальном мире многие из нас часто многозадачны. Например, мы можем писать программу, пока говорим по мобильному телефону, смотреть фильм, когда завтракаем, или петь, пока читаем ноты. Есть много примеров того, как мы, люди, можем быть многозадачными. Не вдаваясь в научные подробности, мы можем представить наш мозг: он пытается ухватить что-то новое, одновременно контролируя работу других органов тела.

То же применимо и к компьютерам. Современные компьютеры имеют несколько процессоров (или несколько ядер в процессорах). Это позволяет одновременно выполнять множество задач.

Настоящий параллелизм невозможен на компьютерах с одноядерным процессором, поскольку задачи не накладываются друг на друга из-за наличия лишь одного ядра процессора. То есть настоящий параллелизм возможен только при наличии в процессоре нескольких ядер. Если коротко, то конкурентное программирование включает в себя две вещи:

- ❑ *управление задачами* — распределение рабочих единиц по доступным потокам;
- ❑ *коммуникацию* — установку начальных параметров задачи и получение результата.



Всякий раз, когда задачи выполняются одновременно, мы называем это конкурентностью. В языке программирования каждый раз, когда какие-либо части нашей программы выполняются одновременно, это называется конкурентным программированием. В качестве синонима для него вы также можете использовать параллельное программирование.

Для примера представьте большую конференцию. Чтобы попасть на нее, вам нужен билет. У входа в конференц-зал нужно оплатить билет наличными или карточкой. Пока вы покупаете билет, ассистент кассира может ввести информацию о вас, распечатать счет и отдать билет. Теперь представим, что много людей хотят оплатить билет. Каждый человек выполняет все необходимые действия, чтобы забрать билет у кассира. В этом случае только один человек может обслуживаться одним кассиром. Предположим, что один человек занимает у кассира две минуты. То есть следующему человеку нужно ждать две минуты. Представим теперь, сколько времени будет ждать последний человек в очереди из 50 участников конференции. Здесь нужны изменения. Если бы было еще два кассира и каждый выполнял работу за две минуты, то каждые две минуты три человека могли бы забрать три

билета. То есть три кассира продают три билета каждые две минуты. Другими словами, каждый кассир выполняет ту же задачу (продает билеты) в один и тот же момент времени. Каждый кассир работает параллельно. Следовательно, кассиры конкурентны. Это показано на рис. 8.1.

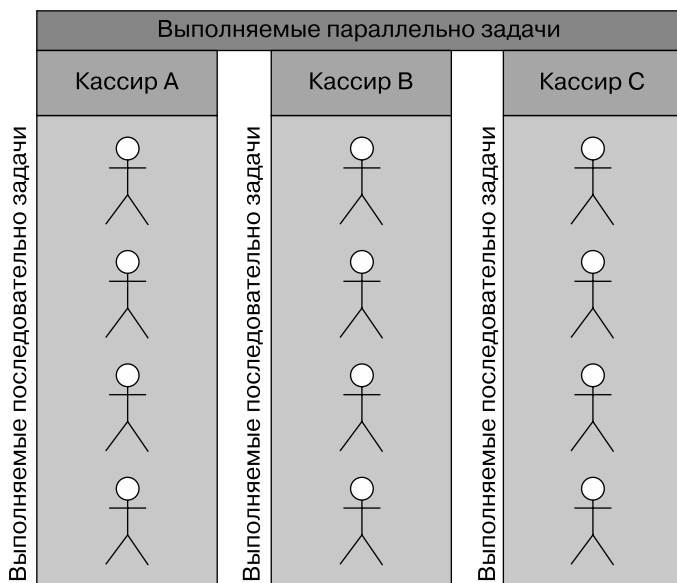


Рис. 8.1

На схеме ясно видно, что каждый человек в очереди либо ожидает, либо активен у кассира, и есть три очереди, в которых задачи выполняются последовательно. Все кассиры (`CounterA`, `CounterB` и `CounterC`) работают в одно и то же время — то есть параллельно.



Конкурентность — это когда две или более задачи запускаются, выполняются и завершаются в перекрывающихся временных интервалах.

Параллелизм возникает при одновременном выполнении двух и более задач.

Да, есть конкурентность, но подумайте о таком сценарии, когда в очереди стоит очень много людей (например, 10 000 человек). Здесь нет смысла в конкурентности: это не решает проблему вероятного узкого места операции. С другой стороны, вы можете увеличить количество кассиров до 50. Это решит проблему? Сложности подобного рода возникают во время работы с любым программным обеспечением. Это также вопрос, связанный с блокировкой. В следующем разделе обсудим конкурентное программирование более подробно.

Многопоточное и асинхронное программирование

Упрощая, можно сказать: многопоточность означает, что программа выполняется параллельно в несколько потоков. При асинхронном программировании единица работы запускается отдельно от основного потока приложения и сообщает вызывающему потоку, что задача выполнена, не выполнена или находится в процессе. В асинхронном программировании интересны вопросы о том, когда его использовать и каковы преимущества.



Потенциальная возможность получения доступа к одним и тем же общим данным и их обновление с непредсказуемыми результатами можно назвать состоянием гонки. Мы уже обсуждали это в главе 4.

Рассмотрим описанный выше сценарий, в котором люди в очереди забирают билеты. Попробуем отразить его в многопоточной программе:

```
internal class TicketCounter
{
    public static void CounterA() => Console.WriteLine("Person A
        is collecting ticket from Counter A");
    public static void CounterB() => Console.WriteLine("Person B
        is collecting ticket from Counter B");
    public static void CounterC() => Console.WriteLine("Person C
        is collecting ticket from Counter C");
}
```

Итак, есть класс `TicketCounter`, представляющий собой множество кассиров. Мы обсуждали их ранее. В данном классе есть три метода: `CounterA()`, `CounterB()` и `CounterC()`, представляющих отдельную стойку для получения билетов. Эти методы просто пишут сообщение в консоль, как показано ниже:

```
internal class Program
{
    private static void Main(string[] args)
    {
        var counterA = new Thread(TicketCounter.CounterA);
        var counterB = new Thread(TicketCounter.CounterB);
        var counterC = new Thread(TicketCounter.CounterC);
        Console.WriteLine("3-counters are serving...");
        counterA.Start();
        counterB.Start();
        counterC.Start();
        Console.WriteLine("Next person from row");
        Console.ReadLine();
    }
}
```

Код представляет класс `Program`, который начинает выполнение программы внутри метода `Main`. Мы объявили и запустили три потока для всех кассиров. Обратите внимание: мы запустили эти потоки в определенном порядке. Поскольку мы ожидаем, что они будут выполняться в той же последовательности, запустим программу и посмотрим выходные данные, как показано на рис. 8.2.

```
3-counters are serving...  
Next person from row  
Person A is collecting ticket from Counter A  
Person C is collecting ticket from Counter C  
Person B is collecting ticket from Counter B
```

Рис. 8.2

Программа не выполняется в соответствии с заданной в коде последовательностью. Согласно нашему коду последовательность выполнения должна быть такой:

```
3-counters are serving...  
Next person from row  
Person A is collecting ticket from Counter A  
Person B is collecting ticket from Counter B  
Person C is collecting ticket from Counter C
```

Это происходит из-за потоков. Они работают одновременно без гарантии того, что будут выполняться в определенной последовательности.

Еще раз запустим программу и посмотрим, получим ли мы тот же результат (рис. 8.3).

```
3-counters are serving...  
Person A is collecting ticket from Counter A  
Person B is collecting ticket from Counter B  
Next person from row  
Person C is collecting ticket from Counter C
```

Рис. 8.3

На данном рисунке показан другой результат по сравнению с предыдущими, поэтому теперь у нас есть вывод в таком порядке:

```
3-counters are serving...  
Person A is collecting ticket from Counter A  
Person B is collecting ticket from Counter B  
Next person from row  
Person C is collecting ticket from Counter C
```


Итак, потоки работают, но не в определенной нами последовательности.



Приоритеты потоков можно установить так: `counterC.Priority = ThreadPriority.Highest;`, `counterB.Priority = ThreadPriority.Normal;` и `counterA.Priority = ThreadPriority.Lowest;`.

Чтобы запустить потоки синхронизированно, изменим наш код следующим образом:

```
internal class SynchronizedTicketCounter
{
    public void ShowMessage()
    {
        int personsInQueue = 5;
        // Предположим, что это максимальное количество человек в очереди
        lock (this)
        {
            Thread thread = Thread.CurrentThread;
            for (int personCount = 0; personCount < personsInQueue;
                personCount++)
            {
                Console.WriteLine($"{thread.Name} {personCount + 1}
                    is collecting ticket from counter {thread.Name}.");
            }
        }
    }
}
```

Мы создали класс `SynchronizedTicketCounter` с методом `ShowMessage()`. Пожалуйста, обратите внимание на `lock(this){...}` в этом коде. Запустите программу и проверьте вывод (рис. 8.4).

Теперь, когда наши кассиры работают в правильном порядке, мы имеем ожидаемый результат.

```
Person 1 is collecting ticket from counter A.
Person 2 is collecting ticket from counter A.
Person 3 is collecting ticket from counter A.
Person 4 is collecting ticket from counter A.
Person 5 is collecting ticket from counter A.
Person 1 is collecting ticket from counter B.
Person 2 is collecting ticket from counter B.
Person 3 is collecting ticket from counter B.
Person 4 is collecting ticket from counter B.
Person 5 is collecting ticket from counter B.
Person 1 is collecting ticket from counter C.
Person 2 is collecting ticket from counter C.
Person 3 is collecting ticket from counter C.
Person 4 is collecting ticket from counter C.
Person 5 is collecting ticket from counter C.
```

Рис. 8.4

async/await — чем плоха блокировка

Асинхронное программирование очень полезно в случаях, когда мы ожидаем различных действий в один и тот же момент времени. С помощью ключевого слова `async` мы определяем наши метод/операцию как асинхронные. Посмотрим на фрагмент кода:

```
internal class AsyncAwait
{
    public async Task ShowMessage()
    {
        Console.WriteLine("\tServing messages!");
        await Task.Delay(1000);
    }
}
```

Здесь, у нас есть класс `AsyncAwait` с методом `async` под названием `ShowMessage()`. Этот метод просто печатает сообщение, отображаемое в окне консоли. Теперь всякий раз, когда мы вызываем его в другом коде, эта часть кода может ожидать/удерживать/блокировать операцию до тех пор, пока метод `ShowMessage()` не выполнится и не завершит свою задачу (рис. 8.5).

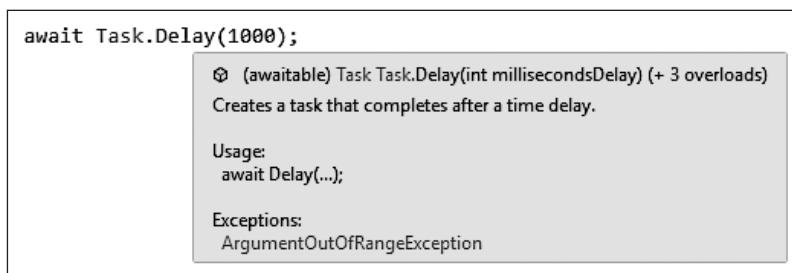


Рис. 8.5

Здесь видно, что для метода `ShowMessage()` мы установили задержку 1000 миллисекунд, то есть дали программе инструкцию завершить работу через 1000 миллисекунд. Если мы удалим `await`, то Visual Studio сразу обратит наше внимание на это, выдав предупреждение о возврате ключевого слова `await` (рис. 8.6).

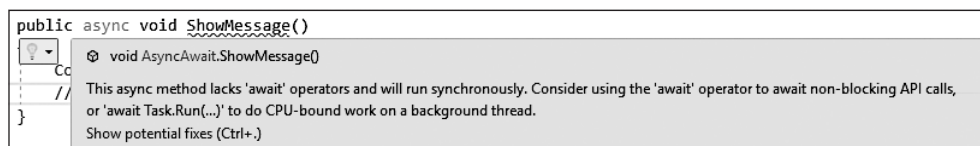


Рис. 8.6

С помощью оператора `await` мы применяем неблокирующие вызовы API. Запустим программу и посмотрим на вывод (рис. 8.7).

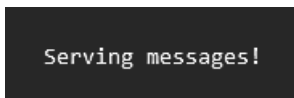


Рис. 8.7

Мы получим результат, показанный на предыдущем рисунке.

Конкурентная коллекция

Платформа .NET Core предоставляет множество коллекций, при работе с которыми мы можем использовать запросы LINQ. У разработчика есть гораздо меньше вариантов при поиске потокобезопасных коллекций. Без них разработчикам может быть трудно выполнять несколько операций. В таком случае мы столкнулись бы с состоянием гонки, которое уже обсуждали в главе 4. Чтобы решить эту проблему, нам нужно применить оператор `lock`, как мы уже делали в предыдущем разделе. Например, мы можем написать код упрощенной реализации оператора `lock` — обратитесь к следующему фрагменту кода, где мы использовали `lock` и класс коллекции `Dictionary`:

```
public bool UpdateQuantity(string name, int quantity)
{
    lock (_lock)
    {
        _books[name].Quantity += quantity;
    }

    return true;
}
```

Код взят из `InventoryContext`. Мы не позволяем другим потокам блокировать операцию, в которой пытаемся обновить количество.



Основная проблема класса `Dictionary` в том, что он не потокобезопасен. При работе с многопоточностью следует использовать оператор `lock`, когда мы применяем `Dictionary`. Чтобы сделать наш код потокобезопасным, мы можем использовать класс коллекции `ConcurrentDictionary`.

Класс `ConcurrentDictionary` — потокобезопасный класс коллекции, он хранит пары «ключ — значение». Использует `lock` и предоставляет потокобезопасный класс. Посмотрите на код ниже:

```
private readonly IDictionary<string, Book> _books;  
protected InventoryContext()  
{  
    _books = new ConcurrentDictionary<string, Book>();  
}
```

Это код класса `InventoryContext` нашего консольного приложения `FlixOne`. Здесь у нас есть поле `_books`, и оно инициализируется как класс коллекции `ConcurrentDictionary`.

Поскольку мы используем метод `UpdateQuantity()` класса `InventoryContext` многопоточно, существует вероятность того, что один поток добавляет количество, а другой сбрасывает его до начального уровня. Это происходит потому, что наш объект находится в одной коллекции и любые ее изменения в одном потоке не видны другим потокам. Все потоки ссылаются на исходную немодифицированную коллекцию. Если коротко, то наш метод не потокобезопасен, если только мы не используем оператор `lock` или класс коллекции `ConcurrentDictionary`.

Паттерны и рекомендации — TDD и параллельный LINQ

Работая с многопоточностью, мы должны следовать практическим рекомендациям, чтобы написать *гладкий код* — такой код, в котором разработчик не сталкивается со взаимными блокировками. Другими словами, многопоточность требует большой осторожности.



В то время как в классе/программе выполняется несколько потоков, взаимная блокировка возникает, когда каждый поток приближается к объекту или ресурсу, написанному в операторе `lock`. Точнее, она возникает, когда каждый поток приближается к блокировке объекта или ресурса, который уже заблокирован другим потоком.

Небольшая ошибка может привести к тому, что разработчикам придется решать неизвестные ошибки, возникающие из-за заблокированных потоков. Кроме того, неудачная реализация всего в несколько слов может плохо повлиять на сотни строк кода.

Вернемся к нашему примеру с билетами на конференцию, который мы обсуждали в начале этой главы. Что произойдет, если кассиры не смогут отдавать билеты? Тогда каждый человек будет пытаться добраться до кассы и получить билет, блокируя работу кассы. Та же логика применима и к нашей программе. Мы столкнулись бы с тупиковой ситуацией, в которой несколько потоков попытались бы заблокировать ресурс. Чтобы избежать такого состояния, рекомендуется использовать механизм

синхронизации доступа к ресурсу. Платформа .NET Core предоставляет для этого класс `Monitor`. Мы переписали наш старый код во избежание тупиковой ситуации:

```
private static void ProcessTickets()
{
    var ticketCounter = new TicketCounter();
    var counterA = new Thread(ticketCounter.ShowMessage);
    var counterB = new Thread(ticketCounter.ShowMessage);
    var counterC = new Thread(ticketCounter.ShowMessage);
    counterA.Name = "A";
    counterB.Name = "B";
    counterC.Name = "C";
    counterA.Start();
    counterB.Start();
    counterC.Start();
}
```

Здесь у нас есть метод `ProcessTickets`. Он запускает три потока (каждый из них представляет собой отдельного кассира). И каждый поток достигает метода `ShowMessage` класса `TicketCounter`. Возникнет проблема взаимоблокировки, если этот метод не написан достаточно хорошо для разрешения ситуации. Все три потока попытаются получить блокировку для соответствующего ресурса, связанного с данным методом.

Следующий код — реализация метода `ShowMessage`. Мы написали его, чтобы разрешить ситуацию взаимной блокировки:

```
private static readonly object Object = new object();
public void ShowMessage()
{
    const int personsInQueue = 5;
    if (Monitor.TryEnter(Object, 300))
    {
        try
        {
            var thread = Thread.CurrentThread;
            for (var personCount = 0; personCount < personsInQueue;
                personCount++)
                Console.WriteLine(
                    $"{thread.Name} {personCount + 1} is collecting ticket from
                    counter {thread.Name}.");
        }
        finally
        {
            Monitor.Exit(Object);
        }
    }
}
```

Это метод `ShowMessage()` нашего класса `TicketCounter`. В данном методе всякий раз, пытаясь заблокировать уже заблокированный объект, поток стремится сделать это

в течение 300 миллисекунд. Класс `Monitor` обрабатывает данную ситуацию автоматически. Его использование позволяет разработчику не беспокоиться о ситуации, в которой выполняется несколько потоков и каждый из них пытается получить блокировку. Запустите программу и посмотрите на вывод (рис. 8.8).

```
Person 1 is collecting ticket from counter A.
Person 2 is collecting ticket from counter A.
Person 3 is collecting ticket from counter A.
Person 4 is collecting ticket from counter A.
Person 5 is collecting ticket from counter A.
Person 1 is collecting ticket from counter C.
Person 2 is collecting ticket from counter C.
Person 3 is collecting ticket from counter C.
Person 4 is collecting ticket from counter C.
Person 5 is collecting ticket from counter C.
Person 1 is collecting ticket from counter B.
Person 2 is collecting ticket from counter B.
Person 3 is collecting ticket from counter B.
Person 4 is collecting ticket from counter B.
Person 5 is collecting ticket from counter B.
```

Рис. 8.8

Здесь вы заметите, что после `counterA` срабатывает `counterC`, а затем `counterB`. Это значит, что после `thread A` было начато выполнение `thread C`, а затем `thread B`. Другими словами, `thread A` сначала получает блокировку, а через 300 миллисекунд `thread C` пытается заблокировать объект. Затем `thread B` пытается заблокировать объект. Если вы хотите установить порядок или приоритеты потоков, то добавьте эти строки кода:

```
counterC.Priority = ThreadPriority.Highest;
counterB.Priority = ThreadPriority.Normal;
counterA.Priority = ThreadPriority.Lowest;
```

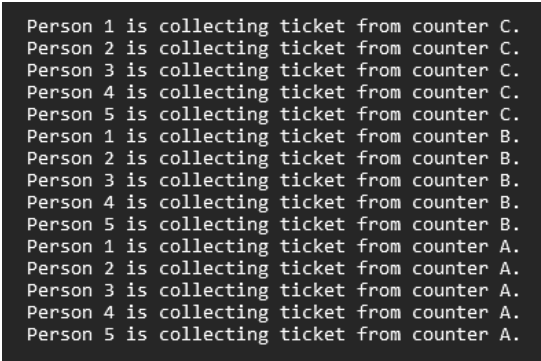
При добавлении этих строк в метод `ProcessTickets` будут работать все потоки: сначала `thread C`, затем `thread B` и, наконец, `thread A`.



Приоритеты потоков — это перечисление, указывающее на порядок выполнения потока `System.Threading.ThreadPriority`, для которого возможны следующие значения.

- `Lowest` — наименьший приоритет. Потоки с приоритетом `Lowest` выполняются после потоков с другими приоритетами.
- `BelowNormal` — выполняются после `Normal`, но до `Lowest`.
- `Normal` — приоритет по умолчанию. Эти потоки выполняются после `AboveNormal`, но до `BelowNormal`.
- `AboveNormal` — до `Normal`, но после `Highest`.
- `Highest` — самый высокий уровень приоритета. Выполняется первым. Потоки с таким приоритетом могут выполняться до потоков со всеми остальными приоритетами.

После установки уровня приоритета потоков выполните программу и посмотрите на вывод (рис. 8.9).



```
Person 1 is collecting ticket from counter C.
Person 2 is collecting ticket from counter C.
Person 3 is collecting ticket from counter C.
Person 4 is collecting ticket from counter C.
Person 5 is collecting ticket from counter C.
Person 1 is collecting ticket from counter B.
Person 2 is collecting ticket from counter B.
Person 3 is collecting ticket from counter B.
Person 4 is collecting ticket from counter B.
Person 5 is collecting ticket from counter B.
Person 1 is collecting ticket from counter A.
Person 2 is collecting ticket from counter A.
Person 3 is collecting ticket from counter A.
Person 4 is collecting ticket from counter A.
Person 5 is collecting ticket from counter A.
```

Рис. 8.9

Согласно этому рисунку после установки приоритета кассиры работают в порядке С, В и А. Соблюдая небольшую осторожность и с помощью простой реализации мы можем справиться с тупиковой ситуацией, а также запланировать наши потоки, которые будут работать в определенном порядке.

Фреймворк .NET Core предоставляет *библиотеку параллелизма по задачам* (Task Parallel Library, TPL). Это набор общедоступных API, которые относятся к пространствам имен `System.Threading` и `System.Threading.Tasks`. С помощью TPL разработчики могут сделать приложения параллельными, адаптировав их упрощенную реализацию.

Посмотрите на код ниже. Это простейшая реализация TPL:

```
public void ParallelVersion()
{
    var books = GetBooks();
    Parallel.ForEach(books, Process);
}
```

Здесь простой `ForEach` используется с ключевым словом `Parallel`. Мы просто итерируем коллекцию `books` и обрабатываем ее с помощью метода `Process`:

```
private void Process(Book book)
{
    Console.WriteLine($"{book.Id}\t{book.Name}\t{book.Quantity}");
}
```

Данный код представляет метод `Process` (опять же самый простой). Он печатает подробности о `books`. Пользователи могут выполнять столько действий, сколько захотят:

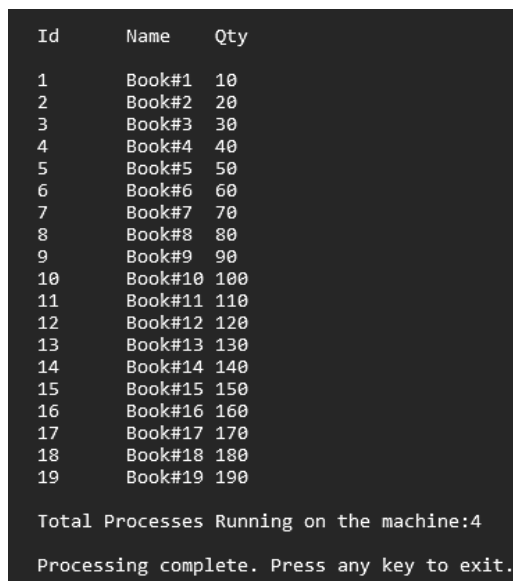
```
private static void ParallelismExample()
{
    var parallelism = new Parallelism();
    parallelism.GenerateBooks(19);
    Console.WriteLine("\n\tId\tName\tQty\n");
    parallelism.ParallelVersion();
    Console.WriteLine($" \n\tTotal Processes Running on the
        machine:{Environment.ProcessorCount}\n");
    Console.WriteLine("\tProcessing complete. Press any key to exit.");
    Console.ReadKey();
}
```

Как видите, у нас есть метод `ParallelismExample`. Он генерирует список книг, а также обрабатывает книги, выполняя метод `ParallelVersion`.

Перед выполнением программы, чтобы увидеть следующие выходные данные, сначала рассмотрим фрагмент кода последовательной реализации:

```
public void Sequential()
{
    var books = GetBooks();
    foreach (var book in books) { Process(book); }
}
```

Это последовательный метод `Sequential`. Он использует простой цикл `foreach` для обработки коллекций книг. Выполните программу и посмотрите на вывод (рис. 8.10).



```
Id      Name      Qty
1       Book#1    10
2       Book#2    20
3       Book#3    30
4       Book#4    40
5       Book#5    50
6       Book#6    60
7       Book#7    70
8       Book#8    80
9       Book#9    90
10      Book#10   100
11      Book#11   110
12      Book#12   120
13      Book#13   130
14      Book#14   140
15      Book#15   150
16      Book#16   160
17      Book#17   170
18      Book#18   180
19      Book#19   190

Total Processes Running on the machine:4

Processing complete. Press any key to exit.
```

Рис. 8.10

Обратите внимание на этот рисунок. Для начала в системе, на которой мы запускаем данную демонстрацию, запущено четыре процесса. Второе: коллекция итерируется в последовательности от 1 до 19. Программа не делит задачи на процессы, запущенные на компьютере. Нажмите любую клавишу для выхода из текущего процесса, выполните программу с методом `ParallelismVersion` и посмотрите на вывод (рис. 8.11).

```

Id      Name      Qty
1       Book#1    10
2       Book#2    20
3       Book#3    30
4       Book#4    40
5       Book#5    50
6       Book#6    60
7       Book#7    70
8       Book#8    80
9       Book#9    90
13      Book#13   130
10      Book#10   100
14      Book#14   140
15      Book#15   150
11      Book#11   110
16      Book#16   160
12      Book#12   120
17      Book#17   170
18      Book#18   180
19      Book#19   190

Total Processes Running on the machine:4
Processing complete. Press any key to exit.
```

Рис. 8.11

Здесь представлен вывод параллельного кода. Вы можете заметить, что код не обрабатывает данные последовательно и идентификаторы не идут по порядку. Как мы видим, `Id 13` идет после 9, но перед 10. Будь выполнение последовательным, порядок `Id` был бы таким: 9, 10 и 13.

LINQ появился в мире .NET задолго до рождения .NET Core. `LINQ-to-Objects` позволяет выполнять операции запросов в памяти с помощью произвольных последовательностей объектов. `LINQ-to-Objects` — это набор методов расширений поверх `IEnumerable<T>`.



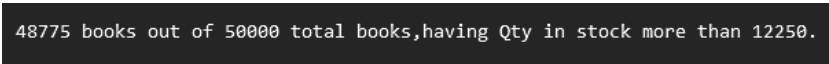
Отложенное выполнение означает, что выполнение происходит только тогда, когда происходит перечисление данных.

PLINQ может применяться в качестве альтернативы TPL. Это параллельная реализация LINQ. Запрос PLINQ работает с источниками данных в памяти `IEnumerable`

или `IEnumerable<T>`. Кроме того, возможно отложенное исполнение. Запрос LINQ выполняет операции последовательно, в то время как PLINQ делает это параллельно и полностью задействует все ядра процессора в компьютере. Посмотрите на код ниже. Вы увидите пример использования PLINQ:

```
public void Process()
{
    var bookCount = 50000;
    _parallelism.GenerateBooks(bookCount);
    var books = _parallelism.GetBooks();
    var query = from book in books.AsParallel()
                where book.Quantity > 12250
                select book;
    Console.WriteLine($"{query.Count()} books out of {bookCount}
        total books," + "having Qty in stock more than 12250.");
    Console.ReadKey();
}
```

В этом коде показан метод `Process` нашего PLINQ-класса. Здесь мы используем PLINQ для запроса любых книг с количеством больше чем 12250. Выполните код, чтобы увидеть вывод (рис. 8.12).



```
48775 books out of 50000 total books,having Qty in stock more than 12250.
```

Рис. 8.12

PLINQ использует все ядра процессора машины, но это регулируется с помощью метода `WithDegreeOfParallelism()`. Мы можем применить код ниже в методе `Process()` класса `Linq`:

```
var query = from book in books.AsParallel().WithDegreeOfParallelism(3)
            where book.Quantity > 12250
            select book;
return query;
```

Этот код использует только три ядра процессора. Выполните его — и обнаружите, что получаете такой же результат, как и в случае с предыдущим кодом.

Резюме

В этой главе мы обсудили конкурентное программирование и конкурентность в реальном мире. Мы увидели, как можем справиться с различными сценариями, связанными с конкурентностью в нашей повседневной жизни. Мы также увидели ситуацию с билетами на конференцию с точки зрения кассира и поняли, что такое конкурентное и параллельное программирование. Кроме того, мы коснулись многопоточности, `async/await`, коллекции `Concurrent` и PLINQ.

В следующей главе мы познакомимся с функциональным программированием на языке C#. Мы еще глубже погрузимся в концепции, которые показывают, как использовать C# в .NET Core в функциональном стиле.

Вопросы

Следующие вопросы позволят вам усвоить информацию, представленную в этой главе.

1. Что такое конкурентное программирование?
2. Когда возможен настоящий параллелизм?
3. Что такое состояние гонки?
4. Почему мы должны использовать параллельный словарь?

Дальнейшее чтение

Указанная ниже книга поможет узнать больше о вопросах, освещенных в этой главе:

- ❑ *Concurrent Patterns and Best Practices* Атула Хота (издательство Packt): www.packtpub.com/in/application-development/concurrent-patterns-and-best-practices.

9

Функциональное программирование

В предыдущей главе было описано понятие конкурентного программирования. Особое внимание мы обратили на преимущества использования `async/await` и параллелизма, позволяющие писать более производительные программы.

В этой главе мы познакомимся с функциональным программированием на языке C#. Мы углубимся в концепции, показывающие, как использовать C# и .NET Core в функциональном стиле. Цель этой главы — помочь вам понять, что такое функциональное программирование и как применять его с помощью языка C#.

Функциональное программирование основано на математике и решает проблемы через функции. В математике у нас есть формулы; в данном стиле программирования мы применяем математику в форме различных функций. Лучшее в функциональном программировании — то, что оно помогает реализовать гладкую конкурентность.

В этой главе будут рассмотрены следующие темы:

- ❑ понимание функционального программирования;
- ❑ приложение для инвентаризации;
- ❑ паттерн «Стратегия» и функциональное программирование.

Технические требования

В этой главе содержатся примеры кода, объясняющие принципы качественной разработки. Код упрощен и предназначен лишь для демонстрации. Большинство примеров основаны на консольном приложении .NET Core, написанном на C#.



Полный исходный код доступен по ссылке <https://github.com/PacktPublishing/Hands-On-Design-Patterns-with-C-and-.NET-Core/tree/master/Chapter9>.

Чтобы запустить и выполнить код, вам потребуется следующее:

- ❑ Visual Studio 2019 (вы также можете запустить приложение, используя Visual Studio 2017 версии 3 и новее);
- ❑ .NET Core;
- ❑ SQL Server (в этой главе используется Express Edition).

Установка Visual Studio

Чтобы запустить примеры кода, вам необходимо установить Visual Studio. Для этого выполните следующие шаги.

1. Скачайте Visual Studio по ссылке, которая содержит инструкции по установке: docs.microsoft.com/ru-ru/visualstudio/install/install-visual-studio.
2. Следуйте инструкциям по установке. Доступны различные версии Visual Studio; в этой главе мы используем Visual Studio для Windows.

Вы также можете применить другую интегрированную среду разработки по вашему усмотрению.

Установка .NET Core

Если вы еще не установили .NET Core, то следуйте инструкции ниже.

1. Скачайте .NET Core для Windows по ссылке dotnet.microsoft.com/download.
2. Для получения нескольких версий и соответствующей библиотеки перейдите по ссылке dotnet.microsoft.com/download/dotnet-core/2.2.

Установка SQL Server

Если у вас не установлен SQL Server, то вам необходимо выполнить следующие инструкции.

1. Скачайте SQL Server по ссылке www.microsoft.com/ru-RU/download/details.aspx?id=1695.
2. Инструкции по установке можно найти по ссылке docs.microsoft.com/ru-ru/sql/ssms/download-sql-server-management-studio-ssms?view=sql-server-2017.



Для устранения неполадок и получения дополнительной информации см. www.blackbaud.com/files/support/infinityinstaller/content/installermaster/tkinstallsqlserver2008r2.htm.

Основы функционального программирования

Если коротко, то *функциональное программирование* — это подход к символическим вычислениям, по сути, такой же, как решение математических задач. Функциональное программирование базируется на математических функциях и стиле написания кода. Любой поддерживающий его язык позволяет ответить на эти два вопроса:

- ❑ что решить;
- ❑ как решить?

Функциональное программирование не новое изобретение. Оно существует в индустрии много лет. Ниже представлен список известных языков, поддерживающих его:

- ❑ Haskell;
- ❑ Scala;
- ❑ Erlang;
- ❑ Clojure;
- ❑ Lisp;
- ❑ OCaml.



В 2005 году Microsoft выпустила первую версию языка F# (произносится как «эфшарп» — fsharp.org). Это функциональный язык программирования, имеющий много хороших функций, которыми должен располагать любой язык функционального программирования. В этой главе мы не будем подробно обсуждать F#, но обсудим функциональное программирование и его реализацию в C#.

Чистые функции — это функции, которые усиливают преимущества функционального стиля тем, что они собственно чистые, то есть работают на двух уровнях:

- ❑ конечный результат (вывод) такой функции всегда одинаков для одних и тех же параметров;
- ❑ такие функции не влияют на поведение программы или поток выполнения приложения, даже когда вызываются сотни раз.

Посмотрите на пример с FlixOne:

```
public static class PriceCalc
{
    public static decimal Discount(this decimal price, decimal discount) =>
        price * discount / 100;
```

```

    public static decimal PriceAfterDiscount(this decimal price,
        decimal discount) =>
        decimal.Round(price - Discount(price, discount));
}

```

Как видите, в классе `PriceCalc` два метода: `Discount` и `PriceAfterDiscount`. Их можно назвать чистыми функциями. Они соответствуют критериям чистоты функций (*Pure*); `Discount` рассчитает скидку, исходя из текущей цены и скидки. Вывод метода никогда не изменится для предоставленных значений параметров. Таким образом, скидка продукта с ценой **190.00** и скидкой **10.00** рассчитывается так: $190.00 * 10.00 / 100$, то есть возвращает **19.00**. Следующий метод — `PriceAfterDiscount` — при тех же значениях параметров рассчитает значение так: $190.00 - 19.00$, вернув значение **171.00**.

Есть еще один важный момент в функциональном программировании: функции являются чистыми и передают полную информацию. Это называется *функциональной честностью*. Посмотрите на `Discount` выше: данный метод честный. А если кто-то случайно предоставит ему отрицательную скидку или скидку больше фактической цены (более 100 %), то останется ли функция чистой и честной? Чтобы разобраться с этим сценарием, наша математическая функция должна быть написана таким образом: если кто-то введет значение `discount <= 0` или `discount > 100`, система его не примет. Рассмотрим код ниже с этим подходом:

```

public static decimal Discount(this decimal price,
    ValidDiscount validDiscount)
{
    return price * validDiscount.Discount / 100;
}

```

Как видите, функция `Discount` имеет параметр `ValidDiscount`, который проверяет входные данные. Таким образом, наша функция теперь честная.

Эти функции так же просты, как само функциональное программирование, но работа в функциональном стиле требует много практики. В следующих разделах мы обсудим другие понятия функционального программирования, включая его принципы. Рассмотрим код, где мы проверяем, корректно ли значение скидки:

```

private readonly Func<decimal, bool> _validDiscount = d =>
    d > 0 || d % 100 <= 1;

```

В этом фрагменте кода у нас есть поле с именем `_validDiscount`. Посмотрим, что происходит: `Func` принимает `decimal` как входное значение и возвращает `bool` как выходное. Из его названия видно, что `field` хранит только корректные скидки.



`Func` — это тип делегата, который указывает на метод одного или нескольких аргументов и возвращает значение. Обобщенное объявление `Func` — это `Func<TParameter, TOutput>`, где `TParameter` — входной параметр корректного типа данных, а `TOutput` — возвращаемое значение корректного типа данных.

Посмотрите на этот код, где в методе используется поле `_validDiscount`.

```
public IEnumerable<DiscountViewModel> FilterOutInvalidDiscountRates(
    IEnumerable<DiscountViewModel> discountViewModels)
{
    var viewModels = discountViewModels.ToList();
    var res = viewModels.Select(x => x.Discount).Where(_validDiscount);
    return viewModels.Where(x => res.Contains(x.Discount));
}
```

Здесь показан метод `FilterOutInvalidDiscountRates`. Он интуитивно понятен и указывает на то, что мы отфильтровываем недействительные скидки. Теперь проанализируем код.

Метод `FilterOutInvalidDiscountRates` возвращает коллекцию класса `DiscountViewModel` для продуктов с корректными скидками. Ниже показан код класса `DiscountViewModel`:

```
public class DiscountViewModel
{
    public Guid ProductId { get; set; }
    public string ProductName { get; set; }
    public decimal Price { get; set; }
    public decimal Discount { get; set; }
    public decimal Amount { get; set; }
}
```

Класс `DiscountViewModel` содержит следующие поля:

- ❑ `ProductId` — идентификатор продукта;
- ❑ `ProductName` — название продукта;
- ❑ `Price` — цена продукта. Настоящая цена до скидок, налогов и т. д.;
- ❑ `Discount` — скидка в процентах, например 10 или 3 %. Корректная скидка не должна быть отрицательной, равной нулю или превышать 100 % (другими словами, не должна быть больше фактической стоимости товара);
- ❑ `Amount` — стоимость продукта после скидок, налогов и т. д.

Теперь вернемся к методу `FilterOutInvalidDiscountRates` и посмотрим на `viewModels.Select(x => x.Discount).Where(_validDiscount)`. Вы можете заметить, что мы выбираем скидку из нашего списка `viewModels`. Он содержит корректные скидки в соответствии с полем `_validDiscount`. В следующей строке наш метод возвращает записи с корректными скидками.

В функциональном программировании такие функции называются *функциями первого класса*. Их значения можно использовать как входные или выходные данные для любой другой функции. Они также могут быть присвоены переменным или сохранены в коллекциях.

Запустите Visual Studio и откройте приложение FlixOne. Теперь запустите его, и вы увидите такое изображение (рис. 9.1).

FlixOne.Web

HomeProductsAboutContactPrivacy

Product Listing

Create New Product

Product Name	Description	Image	Price (INR)	Discount Rate	Discount (INR)	Net Price (INR)	Category Name	Action
Mango	A juicy mango		40.00	5.00	2.00	38.00	Fruit	Edit Details Delete
Apple	Red apple		100.00	10.00	10.00	90.00	Fruit	Edit Details Delete
Orange	Fruity oranges		35.00	3.00	1.05	33.95	Fruit	Edit Details Delete

© 2019 - FlixOne.Web - Privacy

Рис. 9.1

Это страница Product Listing (Список продуктов), показывающая доступные продукты. Она очень простая, и мы можем назвать ее Product Listing, поскольку на ней можно найти все продукты. Со страницы Create New Product (Создать новый продукт) добавляется новый продукт. С помощью команды Edit (Редактировать) происходит обновление существующего продукта. Наконец, страница Details (Подробности) показывает детальное описание определенного продукта. Выбрав команду Delete (Удалить), вы удалите существующий продукт из списка.

Пожалуйста, ознакомьтесь с классом DiscountViewModel1. У нас есть возможность иметь несколько скидок для продукта с бизнес-правилом, которое устанавливает, что только одна ставка дисконтирования активна в один период времени. Для просмотра всех скидок на продукт щелкните на скидке на предыдущем экране Product Listing. Появится следующий экран (рис. 9.2).

На данном экране, Product Discount Listing (Список скидок на продукты), показывается список скидок для манго. Здесь есть две скидки, но активна только Seasonal Discount (Сезонная скидка). В ней отмечается, что скидка является некорректной, поскольку она не отвечает критериям, описанным в поле _validDiscount (которое обсуждалось в предыдущем разделе).



Predicate — это тоже делегат, похожий на делегаты Func. Он представляет собой метод, который проверяет набор критериев. Другими словами, Predicate возвращает тип Predicate <T>, где T — допустимый тип данных. Он работает, когда критерии совпадают, и возвращает значение типа T.

FlixOne.Web Home Products About Contact Privacy					
Product Discount Listing					
Back to Product					
Product Name	Price (INR)	Discount Rate	Description	Active?	Remarks, if any
Mango	40.00	5%	Seasonal Discount	Yes	-
Mango	40.00	105%	Wrong discount, should be ignored	No	Discount rate is invalid, hence will not consider in price calculations.
Back to Product					
© 2019 - FlixOne.Web - Privacy					

Рис. 9.2

Рассмотрим код ниже, в котором проверяем, что название продукта корректно относительно регистра:

```
private static readonly TextInfo TextInfo =
    new CultureInfo("en-US", false).TextInfo;
private readonly Predicate<string> _isProductNameTitleCase = s =>
    s.Equals(TextInfo.ToTitleCase(s));
```

В коде используется `Predicate`, он анализирует условие для проверки `ProductName`, применяя ключевое слово `TitleCase`. Если критерии совпадают, то результат будет `true`. Если нет — то `false`. Посмотрите на фрагмент кода, где мы задействуем `_isProductNameTitleCase`:

```
public IEnumerable<ProductViewModel> FilterOutInvalidProductNames(
    IEnumerable<ProductViewModel> productViewModels) =>
    productViewModels.ToList()
        .Where(p => _isProductNameTitleCase(p.ProductName));
```

Здесь у нас есть метод `FilterOutInvalidProductNames`. Цель этого метода — выбрать продукты с допустимым названием (только `TitleCase`).

Совершенствование приложения FlixOne

Проект создан для гипотетической ситуации, когда компания FlixOne хочет улучшить приложение для управления растущей коллекцией продуктов. Это не новое приложение: мы уже начали его разработку и обсудили первый этап в главе 3, где разрабатывали консольное приложение. Время от времени заинтересованные лица пересматривают наше приложение, чтобы удовлетворить требования конечных

пользователей. Совершенствование важно, поскольку приложение будут применять как персонал для управления, так и клиенты для просмотра и создания новых заказов. Оно должно быть масштабируемым и является важной системой для бизнеса.

Поскольку это техническая книга, мы в основном обсуждаем различные технические особенности с точки зрения разработчиков и модели и методы, используемые для реализации приложения для управления инвентаризацией.

Требования

Существует потребность в улучшении приложения, и этого нельзя достичь за один день. Потребуется много встреч и обсуждений. В ходе нескольких совещаний представители бизнеса и группы разработчиков обсудили новые улучшения и требования к системе. Прогресс в определении четкого набора требований был медленным, а видение конечного продукта — неясным. Разработчики решили сократить огромный список требований до достаточного количества функциональных возможностей, чтобы назначенный человек мог начать записывать некую инвентаризационную информацию. Это позволяет обеспечить простое управление товарами и ту основу, на которой может развиваться бизнес. Мы будем работать над требованием и следовать подходу *продукта с минимальным функционалом* (Minimal Viable Product, MVP).



MVP — это продукт с минимальным, но достаточным набором функций, который имеет ценность для пользователей.

После нескольких встреч и обсуждений между менеджментом и бизнес-аналитиками был составлен список требований для улучшения веб-приложения FlixOne.

- ❑ *Реализация пагинации (разбиения на страницы)* — сейчас все списки в HTML-документах не разбиваются на отдельные страницы. Сложно просматривать элементы на больших страницах, прокручивая вниз или вверх по экрану.
- ❑ *Discount Rates* — сейчас нет возможности добавлять или просматривать различные ставки скидок продукта. Бизнес-правила для скидок таковы:
 - продукт может иметь несколько ставок скидок;
 - продукт может иметь только одну активную ставку скидок;
 - корректная ставка скидки не должна быть отрицательным значением и превышать цену продукта.

Вернемся к FlixOne

В предыдущем подразделе мы обсудили требования. Теперь реализуем их. Сначала вернемся к структуре файлов проекта. Взгляните на рис. 9.3.

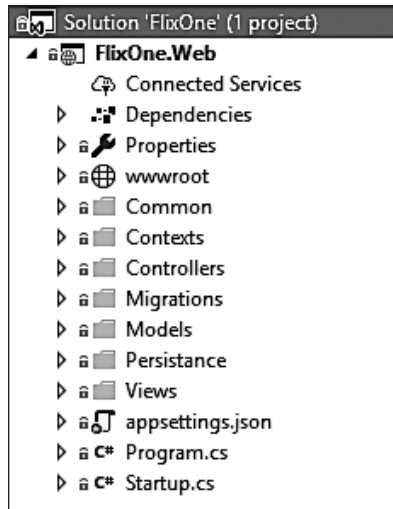


Рис. 9.3

Здесь изображено наше веб-приложение FlixOne со следующей структурой папок:

- ❑ `wwwroot` — папка со статическим содержимым, таким как CSS и файлы jQuery, которые требуются для проекта пользовательского интерфейса. Эта папка поставляется с шаблоном по умолчанию, предоставленным Visual Studio;
- ❑ `Common` — содержит все общие файлы и операции, связанные с бизнес-правилами, а также многое другое;
- ❑ `Contexts` — содержит `InventoryContext`, представляющий собой `DbContext`, который предоставляет возможности `Entity Framework Core`;
- ❑ `Controllers` — содержит все классы контроллеров приложения `FlixOne`;
- ❑ `Migration` — содержит снимок `InventoryModel` и первоначально созданные объекты;
- ❑ `Models` — содержит модели, `ViewModels`, которые нужны приложению;
- ❑ `Persistence` — содержит `InventoryRepository` и его операции;
- ❑ `Views` — содержит представления/экраны для приложения.

Посмотрите на этот код:

```
public interface IHelper
{
    IEnumerable<DiscountViewModel> FilterOutInvalidDiscountRates(
        IEnumerable<DiscountViewModel> discountViewModels);

    IEnumerable<ProductViewModel> FilterOutInvalidProductNames(
        IEnumerable<ProductViewModel> productViewModels);
}
```

Мы видим интерфейс `IHelper`, содержащий два метода. В коде ниже мы реализуем его:

```
public class Helper : IHelper
{
    private static readonly TextInfo TextInfo =
        new CultureInfo("en-US", false).TextInfo;
    private readonly Predicate<string> _isProductNameTitleCase = s =>
        s.Equals(TextInfo.ToTitleCase(s));
    private readonly Func<decimal, bool> _validDiscount = d =>
        d == 0 || d - 100 <= 1;

    public IEnumerable<DiscountViewModel> FilterOutInvalidDiscountRates(
        IEnumerable<DiscountViewModel> discountViewModels)
    {
        var viewModels = discountViewModels.ToList();
        var res = viewModels.Select(x =>
            x.ProductDiscountRate).Where(_validDiscount);
        return viewModels.Where(x => res.Contains(x.ProductDiscountRate));
    }

    public IEnumerable<ProductViewModel> FilterOutInvalidProductNames(
        IEnumerable<ProductViewModel> productViewModels) =>
        productViewModels.ToList()
            .Where(p => _isProductNameTitleCase(p.ProductName));
}
```

Класс `Helper` реализует интерфейс `IHelper`. В этом классе у нас есть два основных и важных метода: один проверяет корректность скидки, другой — корректность атрибута `ProductName`.

Прежде чем использовать эту функциональность в нашем приложении, мы должны добавить ее в `Startup.cs`, как показано ниже:

```
public void ConfigureServices(IServiceCollection services)
{
    services.AddTransient<IInventoryRepository, InventoryRepository>();
    services.AddTransient<IHelper, Helper>();
    services.AddDbContext<InventoryContext>(o =>
        o.UseSqlServer(Configuration.GetConnectionString("FlixOneDbConnection")));
}
```

```

services.Configure<CookiePolicyOptions>(options =>
{
    // Это лямбда-выражение определяет, требуется ли согласие
    // пользователя на применение несущественных файлов cookie
    // для данного запроса.
    options.CheckConsentNeeded = context => true;
    options.MinimumSameSitePolicy = SameSiteMode.None;
});
}

```

В коде выше мы видим написанный оператор `services.AddTransient<IHelper, Helper>()`. Таким образом добавляем временный сервис в наше приложение. Мы уже обсуждали инверсию управления в главе 5.

Рассмотрим код, где мы используем класс `IHelper` с инверсией управления:

```

public class InventoryRepository : IInventoryRepository
{
    private readonly IHelper _helper;
    private readonly InventoryContext _inventoryContext;

    public InventoryRepository(InventoryContext inventoryContext, IHelper helper)
    {
        _inventoryContext = inventoryContext;
        _helper = helper;
    }

    ...
}

```

Этот код содержит класс `InventoryRepository`, где мы можем видеть пример правильного *внедрения зависимости* (Dependency Injection, DI):

```

public IEnumerable<Discount> GetDiscountBy(Guid productId,
    bool activeOnly = false)
{
    var discounts = activeOnly ?
        GetDiscounts().Where(d => d.ProductId ==
            productId && d.Active) : GetDiscounts().Where(d =>
            d.ProductId == productId);
    var product = _inventoryContext.Products.FirstOrDefault(p =>
        p.Id == productId);
    var listDis = new List<Discount>();
    foreach (var discount in discounts)
    {
        if (product != null)
        {
            discount.ProductName = product.Name;
            discount.ProductPrice = product.Price;
        }
    }
}

```

```

        listDis.Add(discount);
    }

    return listDis;
}

```

Это метод `GetDiscountBy` класса `InventoryRepository`, представляющий собой возвращаемую коллекцию модели скидок для записей `active` или `de-active`. Рассмотрим фрагмент, используемый для коллекции `DiscountViewModel`:

```

public IEnumerable<DiscountViewModel> GetValidDiscountedProducts(
    IEnumerable<DiscountViewModel> discountViewModels)
{
    return _helper.FilterOutInvalidDiscountRates(discountViewModels);
}

```

Данный код, использующий коллекцию `DiscountViewModel`, отфильтровывает продукты с некорректными скидками в соответствии с бизнес-правилом, которое мы обсуждали ранее. Метод `GetValidDiscountProducts` возвращает коллекцию `DiscountViewModel`.

Если мы забудем определить `IHelper` в файле `Startup.cs`, то столкнемся с исключением, как показано на рис. 9.4.

An unhandled exception occurred while processing the request.

InvalidOperationException: Unable to resolve service for type 'FlixOne.Web.Common.IHelper' while attempting to activate 'FlixOne.Web.Persistence.InventoryRepository'.

Microsoft.Extensions.DependencyInjection.ServiceLookup.CallSiteFactory.CreateArgumentCallSites(Type serviceType, Type implementationType, CallSiteChain callSiteChain, ParameterInfo[] parameters, bool throwIfCallSiteNotFound)

Рис. 9.4

Здесь четко видно, что сервис `IHelper` не разрешен. В нашем случае мы не столкнемся с этим исключением, поскольку уже добавили `IHelper` в класс `Startup`.

До сих пор мы добавляли вспомогательные методы, чтобы реализовать новое требование к скидкам и проверить их. Теперь добавим контроллер и `action`-методы. Для этого добавьте новый контроллер `DiscountController` с помощью панели `Solution Explorer` (Обозреватель решений). После этого наше веб-решение будет выглядеть следующим образом (рис. 9.5).

Мы видим, что в папке `Controller` теперь есть новый контроллер, `DiscountController`. Ниже представлен код из него:

```

public class DiscountController : Controller
{

```

```

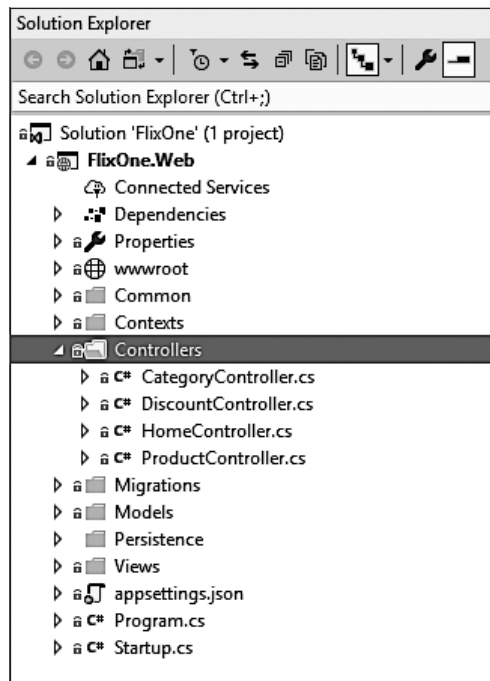
private readonly IInventoryRepository _repository;

public DiscountController(IInventoryRepository inventoryRepository)
{
    _repository = inventoryRepository;
}

public IActionResult Index()
{
    return View(_repository.GetDiscounts().ToDiscountViewModel());
}

public IActionResult Details(Guid id)
{
    return View("Index", _repository.GetDiscountBy(id).ToDiscountViewModel());
}
}

```

**Рис. 9.5**

Запустите приложение и на главном экране выберите Products (Продукты), а затем Product Discount Listing (Список скидок на продукты). На рис. 9.6 показан текущий экран.

FlixOne.Web

[Home](#) [Products](#) [About](#) [Contact](#) [Privacy](#)

Product Discount Listing

[Back to Product](#)

Product Name	Price (INR)	Discount Rate	Description	Active?	Remarks, if any
Mango	40.00	5%	Seasonal Discount	Yes	-
Mango	40.00	105%	Wrong discount, should be ignored	No	Discount rate is invalid, hence will not consider in price calculations.
Orange	35.00	3%	Fruit dhamaka	Yes	-
Orange	35.00	32%	Pitch Discount	No	-
Apple	100.00	10%	Special discount	Yes	-
Apple	100.00	2%	Seasonal Discount	No	-
Apple	100.00	125%	Discarded	No	Discount rate is invalid, hence will not consider in price calculations.

Рис. 9.6

На данном рисунке показан список скидок на все доступные продукты. В нем много записей, так что для просмотра элементов на экране нужно прокручивать вверх или вниз. Чтобы облегчить эту задачу, можно разбить список на страницы.

Паттерн «Стратегия» и функциональное программирование

В первых четырех главах этой книги мы много говорили о паттернах проектирования и практиках. «Стратегия» — один из важных паттернов «Банды четырех» (Gang of Four, GoF). Он относится к категории поведенческих паттернов и также известен как паттерн политики. Обычно реализуется с помощью классов. Кроме того, его проще реализовать, применяя функциональное программирование.

Вернитесь к разделу «Основы функционального программирования» этой главы и пересмотрите парадигму функционального программирования. Функции более высокого порядка — одна из важных его парадигм. Используя их, мы можем легко реализовать паттерн «Стратегия» функциональным способом.



Функции высокого порядка (High Order Functions, HOFs) принимают параметры в виде функций и могут возвращать другие функции.

Посмотрите на код, показывающий реализацию HOFs в функциональном программировании:

```
public static IEnumerable<T> Where<T>
    (this IEnumerable<T> source, Func<T, bool> criteria)
{
    foreach (var item in source)
        if (criteria(item))
            yield return item;
}
```

Это простая реализация `Where`, где мы использовали LINQ Query. В данном случае мы итерируем коллекцию и возвращаем элемент, если он соответствует критериям. Код выше может быть упрощен таким образом:

```
public static IEnumerable<T> SimplifiedWhere<T>
    (this IEnumerable<T> source, Func<T, bool> criteria) =>
    Enumerable.Where(source, criteria);
```

Как видите, метод `SimplifiedWhere` дает тот же результат, что и упомянутый выше метод `Where`. Он основан на критериях со стратегией возврата результатов. Мы можем легко вызвать предыдущую функцию в последующем методе, чтобы воспользоваться преимуществами функционального программирования. Рассмотрим следующий код:

```
public IEnumerable<ProductViewModel>
    GetProductsAbovePrice(IEnumerable<ProductViewModel>
        productViewModels, decimal price) =>
    productViewModels.SimplifiedWhere(p => p.ProductPrice > price);
```

У нас есть метод `GetProductsAbovePrice`. В нем мы предоставляем цену. Метод интуитивно понятен и работает на коллекции `ProductViewModel` с критерием для перечисления продуктов, цена которых превышает цену-параметр. В нашем приложении для инвентаризации `FlixOne` вы можете найти дополнительные возможности для реализации функционального программирования.

Резюме

Функциональное программирование касается функций, главным образом математических. Любой поддерживающий его язык всегда работает над решением двух основных вопросов: что нужно решить и как это можно сделать. Мы рассмотрели функциональное программирование и его простую реализацию с помощью языка программирования C#.

В следующей главе мы обсудим реактивное программирование, его модель и принципы. Кроме того, поговорим о *реактивном расширении*.

Вопросы

Следующие вопросы позволят вам усвоить информацию, представленную в этой главе.

1. Что такое функциональное программирование?
2. Что такое ссылочная прозрачность в функциональном программировании?
3. Что такое чистая функция?

10 Модели и методы реактивного программирования

В предыдущей главе мы углубились в функциональное программирование и узнали о *Func*, *Predicate*, *LINQ*, *лямбда-выражениях*, *анонимных функциях*, *дереве выражений* и *рекурсии*. Мы также рассмотрели реализацию паттерна «Стратегия» с помощью функционального программирования.

В этой главе мы рассмотрим реактивное программирование и увидим его практическую демонстрацию с помощью языка C#. Углубимся в принципы и модели реактивного программирования и обсудим провайдеров *IObservable* и *IObserver*.

Приложение для инвентаризации будет расширено двумя основными способами: мы сделаем так, чтобы оно реагировало на изменения, и реализуем в нем паттерн «Модель — представление — модель представления» (Model — View — View Model, MVVM).

В этой главе будут рассмотрены следующие темы:

- ❑ принципы реактивного программирования;
- ❑ реактивность и интерфейс *IObservable*;
- ❑ реактивные расширения — расширения .NET Rx;
- ❑ пример использования приложения для инвентаризации — получение инвентаризации с фильтром, подкачкой и сортировкой;
- ❑ паттерны и практики — MVVM.

Технические требования

В этой главе содержатся примеры кода, объясняющие принципы качественной разработки. Код упрощен и предназначен лишь для демонстрации. Большинство примеров основаны на консольном приложении .NET Core, написанном на C#.



Полный исходный код доступен по ссылке <https://github.com/PacktPublishing/Hands-On-Design-Patterns-with-C-and-.NET-Core/tree/master/Chapter10>.

Чтобы запустить и выполнить код, вам потребуется следующее:

- ❑ Visual Studio 2019 (вы также можете использовать Visual Studio 2017);
- ❑ .NET Core;
- ❑ SQL Server (в этой главе применяется Express Edition).

Установка Visual Studio

Чтобы запустить примеры кода, вам необходимо установить Visual Studio. Для этого выполните следующие шаги.

1. Скачайте Visual Studio 2017 или более позднюю версию (2019) по ссылке, которая содержит инструкции по установке: docs.microsoft.com/ru-ru/visualstudio/install/install-visual-studio.
2. Следуйте инструкциям по установке. Доступны различные версии Visual Studio; в этой главе мы используем Visual Studio для Windows.

Вы также можете применить другую интегрированную среду разработки по вашему усмотрению.

Установка .NET Core

Если вы еще не установили .NET Core, то следуйте инструкции ниже.

1. Скачайте .NET Core для Windows по ссылке dotnet.microsoft.com/download.
2. Для получения нескольких версий и соответствующей библиотеки перейдите по ссылке dotnet.microsoft.com/download/dotnet-core/2.2.

Установка SQL Server

Если у вас не установлен SQL Server, то вам необходимо выполнить следующие действия.

1. Скачайте SQL Server по ссылке www.microsoft.com/ru-RU/download/details.aspx?id=1695.
2. Инструкции по установке можно найти по ссылке docs.microsoft.com/ru-ru/sql/ssms/download-sql-server-management-studio-ssms?view=sql-server-2017.



Для устранения неполадок и получения дополнительной информации см. www.blackbaud.com/files/support/infinityinstaller/content/installermaster/tkinstallsqlserver2008r2.htm.

Принципы реактивного программирования

В наши дни все говорят об *асинхронном программировании*. Различные приложения построены на использующих его сервисах RESTful. Термин «асинхронный» имеет отношение к реактивному программированию. Реактивным называется все, что касается потоков данных, а реактивное программирование — это структура модели, построенная вокруг асинхронных потоков данных. Реактивное программирование также известно как *искусство программирования распространения изменений*. Вернемся к нашему примеру из главы 8, где мы обсуждали кассиров на большой конференции.

В дополнение к трем билетным кассирам у нас есть еще один, названный кассиром учета. Он концентрируется на подсчете суммы, то есть подсчитывает, сколько билетов распределяется каждым кассиром. Рассмотрим следующую схему (рис. 10.1).

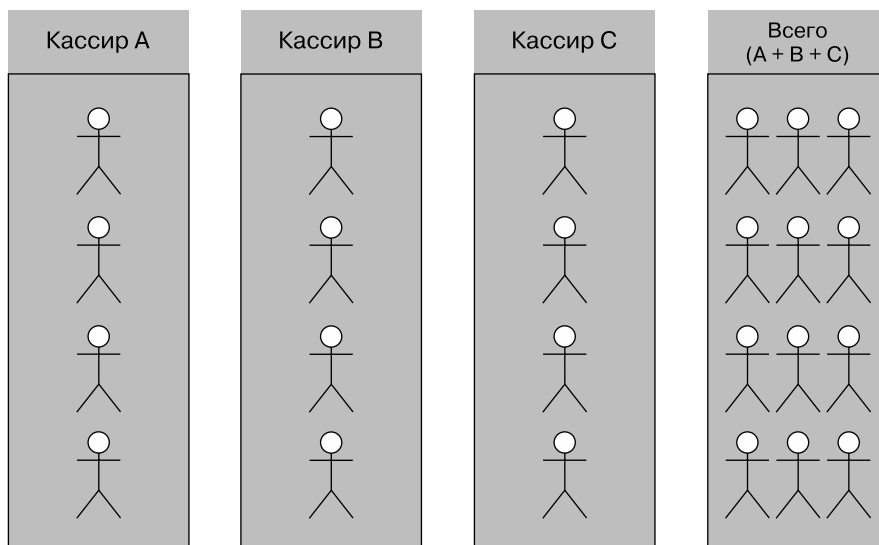


Рис. 10.1

Здесь сумма $A + B + C$ равна сумме оставшихся трех столбцов: $1 + 1 + 1 = 3$. Столбец *Всего* (Total) всегда показывает сумму остальных трех столбцов и никогда не покажет реального человека, который стоит в очереди, чтобы забрать билет. Значение *Всего* зависит от количества оставшихся столбцов. Если бы в очереди к кассиру А стояли два человека, то столбец *Всего* имел бы сумму $2 + 1 + 1 = 4$. Вы также можете ссылаться на столбец *Всего* как на вычисляемый столбец. Он вычисляет сумму, как только строки/столбцы меняют показания своих счетчиков (когда меняется

количество людей в очереди). Если бы мы записали столбец *Всего* на C#, то выбрали бы вычисляемое свойство и это выглядело бы следующим образом:

```
public int TotalColumn { get { return ColumnA + ColumnB + ColumnC; } }
```

На рис. 10.1 данные передаются от столбца к столбцу. Вы можете рассматривать это как поток данных. Вы можете создать поток для любых событий, таких как события щелчка и наведения указателя мыши. Поточковой переменной может быть что угодно: входные данные пользователя, свойства, кэши, структуры данных и др. В мире потоков вы можете слушать поток и реагировать соответственно.

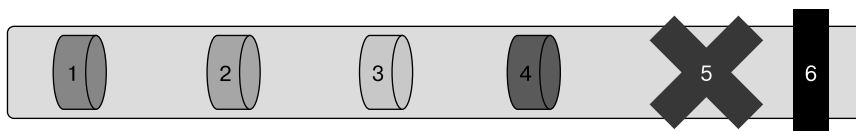


Последовательность событий называется потоком и может содержать значения, ошибки и сигнал о завершении потока.

Вы можете легко работать с потоком таким образом:

- ☐ один поток может быть входным для другого;
- ☐ несколько потоков могут быть входными данными для другого потока;
- ☐ потоки могут быть объединены;
- ☐ значения одного потока могут отображаться, используя значения другого;
- ☐ потоки можно фильтровать с помощью данных/событий, которые вам нужны.

Лучше понять потоки помогает рис. 10.2, представляющий поток (последовательность событий).



Где:
1, 2, 3, 4: события
5: ошибка
6: поток завершен

Рис. 10.2

На данной схеме представлен поток (последовательность событий), в котором мы имеем от одного до четырех событий. Любое из них может быть вызвано, или кто-то может нажать любое из них. Эти события могут быть представлены значениями, которые, в свою очередь, могут быть строками. Знак X показывает, что во время операции слияния потоков или сопоставления их данных произошла ошибка. Наконец, знак | уведомляет о завершении потока (или операции).

Будьте реактивны с реактивным программированием

Очевидно, что наше вычисляемое свойство (рассмотренное выше) не может быть реактивным или представлять реактивное программирование. Последнее имеет специфические конструкции и технологии. Чтобы опробовать его, вы можете начать с документов, доступных по адресу reactivex.io, и с манифеста реактивности (www.reactivemanoifesto.org).



Если коротко, то реактивными называются свойства привязки, которые реагируют при инициировании события.

В настоящее время, имея дело с различными большими системами/приложениями, мы обнаруживаем, что они слишком велики для огульной обработки. Эти большие системы делятся на более мелкие или состоят из более мелких, а те, в свою очередь, полагаются на реактивные свойства. Чтобы придерживаться реактивного программирования, реактивные системы применяют принципы проектирования, вследствие чего эти свойства могут применяться ко всем методам. С помощью данного дизайна/подхода мы можем создать составную систему.

Согласно манифесту реактивное программирование и реактивные системы — это разные вещи.

На основании реактивного манифеста мы можем заключить, что реактивные системы имеют следующие свойства.

- ❑ *Отзывчивость.* Реактивными являются системы проектирования, основанные на событиях. Они быстро реагируют на любой запрос в короткий промежуток времени.
- ❑ *Масштабируемость.* Реактивные системы имеют реактивную природу. Они могут реагировать на изменение коэффициента масштабируемости, расширяя или сокращая выделяемые ресурсы.
- ❑ *Устойчивость.* Эластичная система — та, которая не остановит работу, даже в случае неполадки/исключения. Реактивные системы сконструированы таким образом, что при любом исключении или отказе система никогда не выйдет из строя; она продолжает работать.
- ❑ *Основанность на сообщениях.* Любой элемент данных представляет собой сообщение, которое можно отправить в определенное место назначения. Когда сообщение или элемент данных достигают заданного состояния, событие посылает сигнал, уведомляющий подписчиков о том, что сообщение достигнуто. Реактивные системы полагаются на эту передачу сообщений.

На рис. 10.3 показано графическое представление реактивной системы.



Рис. 10.3

На этой схеме реактивная система состоит из небольших систем, которые являются устойчивыми, масштабируемыми, отзывчивыми и основанными на сообщениях.

Реактивные потоки в действии

До сих пор мы обсуждали тот факт, что реактивное программирование — это поток данных. В предыдущих разделах мы также обсуждали, как потоки работают и перемещаются своевременно. Мы рассмотрели пример событий и обсудили потоки данных в реактивной программе. Теперь возьмем один и тот же пример и проанализируем, как два потока работают с различными операциями.

В следующем примере у нас есть два наблюдаемых потока сбора целочисленных типов данных. Обратите внимание: в данной части текста мы используем псевдокод для объяснения поведения и способа работы этих коллекций потоков данных.

На рис. 10.4 представлены два наблюдаемых потока. Первый, `Observer1`, содержит числа 1, 2 и 4, в то время как `Observer2`, являющийся вторым потоком, — числа 3 и 5.

Слияние двух потоков предполагает объединение их элементов последовательности в новый поток. На рис. 10.5 показан новый поток — результат объединения `Observer1` и `Observer2`.

Данная схема является только представлением потока, но не фактическим представлением последовательности элементов в потоке. На ней мы видели, что эле-

менты (числа) находятся в последовательности 1, 2, 3, 4, 5, но в реалистическом примере это не так. Последовательность может меняться; это может быть 1, 2, 3, 4, 5 или любой другой порядок.

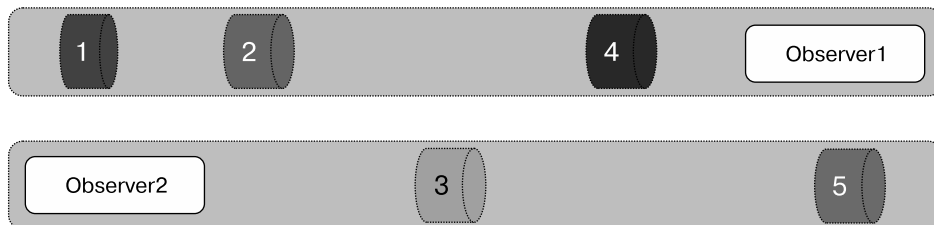


Рис. 10.4

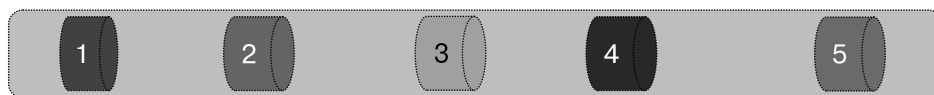


Рис. 10.5

Фильтрация потока подобна пропуску элементов/записей. Вы можете представить выражение с ключевым словом `Where` языка запросов LINQ таким образом: `myCollection.Where(num => num <= 3);`.

На рис. 10.6 показано наглядное представление критериев, где мы пытаемся выбрать только те элементы, которые соответствуют определенным критериям.

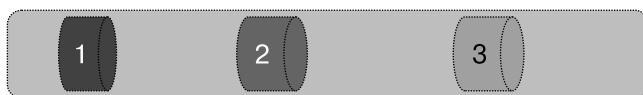


Рис. 10.6

Мы фильтруем наш поток и выбираем лишь те элементы, которые ≤ 3 . Это значит, что мы пропускаем элементы 4 и 5. В данном случае можно сказать: фильтр существует для пропуска элементов или соответствия критериям.

Чтобы понять отображающий поток, вы можете представить себе любую математическую операцию, в которой будете считать последовательности или увеличивать числа, добавляя некие постоянные значения. Например, если у нас целое значение 3 и наш отображающий поток $+3$, то это значит, что мы считаем последовательность как $3 + 3 = 6$. Вы также можете соотнести это с LINQ и выбрать и спроецировать результат следующим образом:

```
return myCollection.Select(num => num+3);
```

На рис. 10.7 представлено отображение потока.

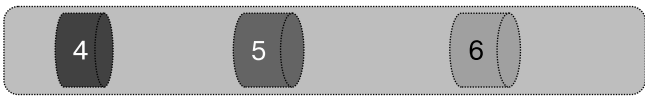


Рис. 10.7

После применения фильтров с условием ≤ 3 наш поток содержит элементы 1, 2 и 3. Кроме того, мы применили `Map (+3)` к отфильтрованному потоку с элементами 1, 2 и 3, и, наконец, наш поток содержит элементы 4, 5, 6 ($1 + 3$, $2 + 3$, $3 + 3$).

В реальном мире эти операции проводились бы последовательно или по требованию. Мы уже выполнили данную операцию с последовательностями, чтобы можно было одну за другой применять операции слияния, фильтрации и отображения. На рис. 10.8 представлен поток нашего воображаемого примера.

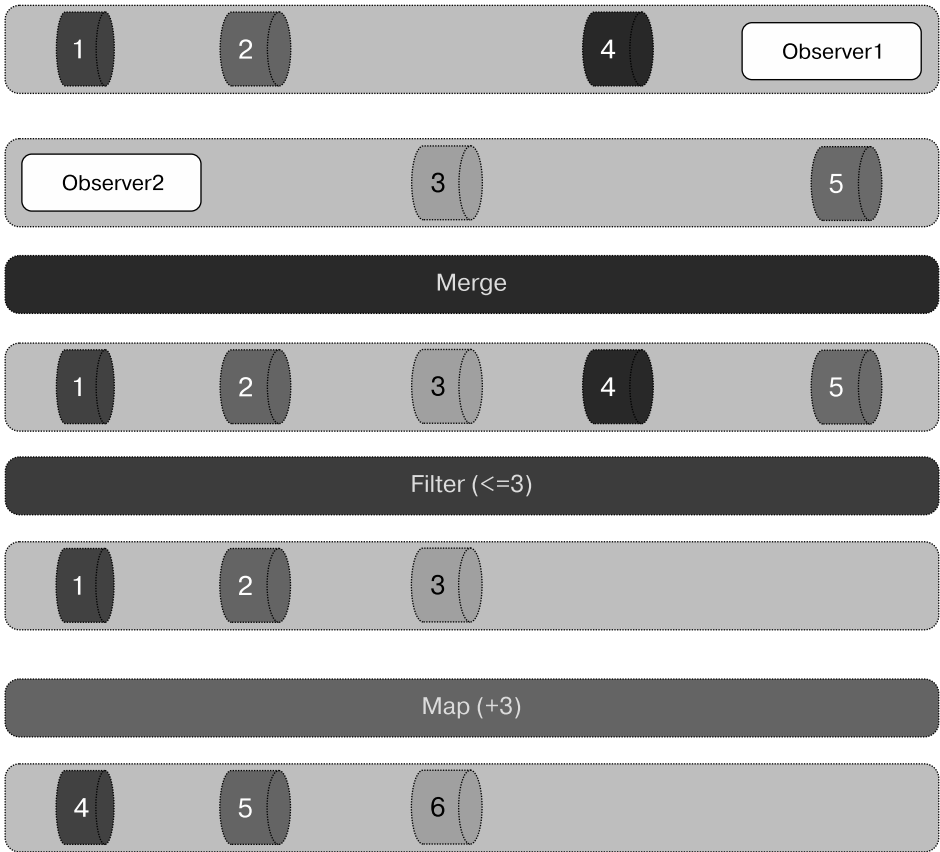


Рис. 10.8

Итак, мы попытались представить наши примеры через схемы, прошли через различные операции, где два потока общаются друг с другом, и получили новый поток, а затем отфильтровали поток и применили к нему отображение.



Чтобы лучше разобраться в теме, обратитесь к ресурсу rxmarbles.com.

Теперь создадим простой код, который дополнит этот пример в реальном мире. Сначала изучим код, реализующий этот пример, а затем обсудим вывод потока.

Рассмотрим следующий фрагмент кода в качестве примера интерфейса `IObservable`:

```
public static IObservable<T> From<T>(this T[] source) =>
    source.ToObservable();
```

Данный код представляет собой метод расширения массива типа `T`. Мы создали общий метод и назвали его `From`. Этот метод возвращает последовательность `Observable`.



Вы можете прочитать официальную документацию, чтобы узнать больше о методах расширения: docs.microsoft.com/ru-ru/dotnet/csharp/programming-guide/classes-and-structs/extension-methods.

В нашем коде есть класс `TicketCounter`. В данном классе — два наблюдателя, которые на самом деле являются массивами целочисленного типа данных. Следующий код показывает две наблюдаемые таблицы:

```
public IObservable<int> Observable1 => Counter1.From();
public IObservable<int> Observable2 => Counter2.From();
```

В данном коде мы применяем метод расширения `From()` к `Counter1` и `Counter2`. Они фактически представляют собой наши билетные кассы и напоминают наш пример из главы 8.

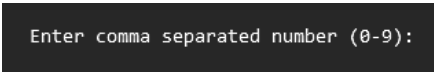
Следующий фрагмент кода представляет `Counter1` и `Counter2`:

```
internal class TicketCounter
{
    private IObservable<int> _observable;
    public int[] Counter1;
    public int[] Counter2;
    public TicketCounter(int[] counter1, int[] counter2)
    {
        Counter1 = counter1;
        Counter2 = counter2;
    }
    ...
}
```

В данном коде у нас есть два поля, `Counter1` и `Counter2`, и они инициализируются из конструктора. При инициализации класса `TicketCounter` эти поля получают значения из конструктора класса, как определено в следующем коде:

```
TicketCounter ticketCounter = new TicketCounter(new int[]{1,3,4},
    new int[]{2,5});
```

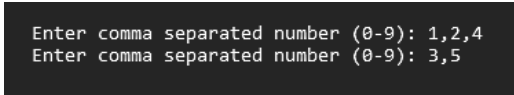
Чтобы понять весь код, выполните его в Visual Studio, нажав клавишу F5. Вы увидите следующее (рис. 10.9).



```
Enter comma separated number (0-9):
```

Рис. 10.9

Это вывод консоли, в котором пользователю предлагается ввести разделенное запятыми число с цифрами от 0 до 9. Введите число, разделенное запятыми (рис. 10.10). Обратите внимание: здесь мы пытаемся создать код, который отображает нашу схему представления потока данных. Она обсуждалась ранее в данном разделе.



```
Enter comma separated number (0-9): 1,2,4
Enter comma separated number (0-9): 3,5
```

Рис. 10.10

Как и на предыдущей схеме, мы ввели два разных числа, разделенных запятыми. Первое — 1,2,4, а второе — 3,5. Теперь рассмотрим метод `Merge`:

```
public IObservable<int> Merge() => _observable = Observable1.Merge(Observable2);
```

Метод `Merge` заключается в объединении двух последовательностей потока данных в одну `_observable`. Операция `Merge` иницируется следующим кодом:

```
Console.WriteLine("\n\tEnter comma separated number (0-9): ");
var num1 = Console.ReadLine();
Console.WriteLine("\tEnter comma separated number (0-9): ");
var num2 = Console.ReadLine();
var counter1 = num1.ToInts(',');
var counter2 = num2.ToInts(',');
TicketCounter ticketCounter = new TicketCounter(counter1, counter2);
```

В этом коде пользователю предлагается ввести числа, разделенные запятыми, а затем программа сохраняет их в `counter1` и `counter2`, применяя метод `ToInts`. Ниже приведен его код:

```
public static int[] ToInts(this string commaseparatedStringofInt,
    char separator) =>
    Array.ConvertAll(commaseparatedStringofInt.Split(separator), int.Parse);
```

Этот код является методом расширения для строки. Целевая переменная имеет тип `string`, содержащий целые числа, разделенные с помощью `separator`. В данном методе мы используем встроенный метод `ConvertAll`, предоставляемый .NET Core. Сначала выполняется разбиение строки и проверяется, имеет ли разделенное значение тип `integer`. Затем он возвращает массив целых чисел. Этот метод создает выходные данные, показанные на рис. 10.11.

```
Counter1:      1      2      4
Counter2:      3      5
```

Рис. 10.11

На рис. 10.12 приведены выходные данные операции `merge`.

```
Merge:          1      3      2      5      4
```

Рис. 10.12

Этот вывод показывает, что теперь у нас есть конечный объединенный поток наблюдателей с элементами в последовательности. Применим фильтр к этому потоку. Следующий код — метод `Filter`:

```
public IObservable<int> Filter() => _observable = from num in _observable
    where num <= 3
    select num;
```

У нас есть критерии фильтра для числа ≤ 3 , что означает выбор только элементов, значения которых меньше либо равны 3. Этот метод инициируется с помощью следующего кода:

```
ticketCounter.Print(ticketCounter.Filter());
```

При выполнении предыдущего кода он выдает такой результат (рис. 10.13).

```
Filter (<= 3):  1      3      2
```

Рис. 10.13

Наконец, мы имеем фильтрованный поток с элементами в последовательности 1, 3, 2. Теперь нам нужно составить отображение этого потока. Нам требуется

отображаемый элемент с `num + 3`, то есть следует вывести целое число, добавив к нему 3. Ниже приведен наш метод `Map`:

```
public IObservable<int> Map() => _observable = from num in _observable
    select num + 3;
```

Предыдущий метод будет инициализирован следующим кодом:

```
Console.WriteLine("\n\tMap (+ 3):");
ticketCounter.Print(ticketCounter.Map());
```

При выполнении предыдущего метода мы увидим следующий результат (рис. 10.14).

```
Map (+ 3):      4      6      5
```

Рис. 10.14

После применения метода `Map` мы получаем поток элементов в последовательности 4, 6, 5. Мы уже обсуждали, как работает реактивность, пусть и на воображаемом примере. Мы создали небольшое консольное приложение .NET Core, чтобы увидеть мощь операций `Merge`, `Filter` и `Map` над наблюдаемыми объектами. Ниже приведен вывод нашего консольного приложения (рис. 10.15).

```
Counter1:      1      2      4
Counter2:      3      5

Merge:         1      3      2      5      4

Filter (<= 3): 1      3      2

Map (+ 3):     4      6      5

Press any key...
```

Рис. 10.15

Здесь показана вся история выполнения нашего примера приложения; `Counter1` и `Counter2` являются данными потоками, которые содержат последовательности данных 1, 2, 4 и 3, 5. Мы имеем предыдущий вывод для `Merge` с результатом 1, 3, 2, 5, 4, `Filter (<=3)` с результатом 1, 3, 2 и `Map (+3)` с данными 4, 6, 5.

Реактивность и интерфейс IObservable

В предыдущем разделе мы обсудили реактивное программирование и рассмотрели его модель. Здесь мы обсудим его реализацию компанией Microsoft. В ответ на реактивное программирование в .NET Core у нас есть различные интерфейсы, которые обеспечивают способ реализации реактивного программирования в нашем приложении.

Интерфейс `IObservable<T>` — универсальный, определен в пространстве имен `System` и объявлен как `public interface IObservable<out T>`. Здесь `T` представляет универсальный тип параметра, который дает информацию об уведомлении. Если коротко, то этот интерфейс помогает определить поставщика уведомлений и они могут быть отправлены для получения информации. Паттерн «Наблюдатель» можно использовать при реализации интерфейса `IObservable<T>` в приложении.

Паттерн «Наблюдатель» — реализация с помощью IObservable<T>

Говоря коротко, подписчик регистрируется у издателя (поставщика), чтобы получать уведомления, связанные с информацией о сообщении. Они уведомляют поставщика о том, что сообщения были доставлены подписчикам. Вдобавок данная информация может быть связана с изменениями в операциях или любыми другими изменениями в самом методе или объекте. Это также известно как *изменения состояния*.



Паттерн «Наблюдатель» определяет два термина: наблюдатель и наблюдаемое. Второе — это поставщик, также известный как субъект. Наблюдатель регистрируется в типах `Observable/Subject/Provider` и будет автоматически уведомлен поставщиком о любых изменениях, вызванных заранее определенными критериями/условиями, изменением или событием и т. д.

На рис. 10.16 изображено простое представление модели наблюдателя, где субъект уведомляет двух разных наблюдателей.

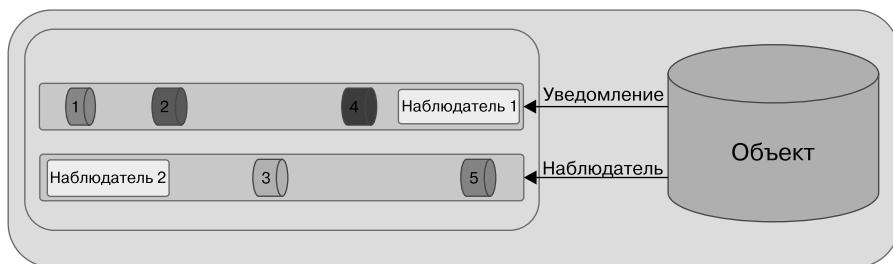


Рис. 10.16

Вернитесь к веб-приложению из главы 9, запустите программу Visual Studio и откройте файл `FlixOne.sln`.

Содержимое панели Solution Explorer (Обозреватель решений) должно выглядеть примерно так, как показано на рис. 10.17.

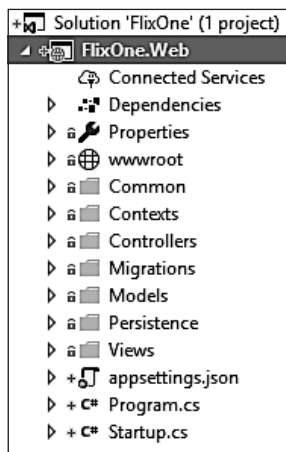


Рис. 10.17

Разверните папку `Common` на панели Solution Explorer (Обозреватель решений) и добавьте два файла: `ProductRecorder.cs` и `ProductReporter.cs`. Они являются реализацией интерфейсов `IObservable<T>` и `IObserver<T>`. Нам также нужно добавить новую модель `ViewModel`, чтобы мы могли сообщать пользователям об актуальных сообщениях. Для этого необходимо открыть папку `Models` и добавить файл `MessageViewModel.cs`.

Следующий код показывает наш класс `MessageViewModel`:

```
public class MessageViewModel
{
    public string MsgId { get; set; }
    public bool IsSuccess { get; set; }
    public string Message { get; set; }

    public override string ToString() => $"Id:{MsgId},
        Success:{IsSuccess}, Message:{Message}";
}
```

Класс `MessageViewModel` содержит следующее:

- ❑ `MsgId` — уникальный идентификатор;
- ❑ `IsSuccess` — показывает, была ли операция успешной;

- ❑ `Message` — сообщение об успехе или об ошибке, которое зависит от значения `IsSuccess`;
- ❑ `ToString()` — переопределенный метод, который возвращает строку после объединения всех сведений.

Теперь обсудим два наших класса; следующий код взят из класса `ProductRecorder`:

```
public class ProductRecorder : IObservable<Product>
{
    private readonly List<IObserver<Product>> _observers;

    public ProductRecorder() => _observers = new List<IObserver<Product>>();

    public IDisposable Subscribe(IObserver<Product> observer)
    {
        if (!_observers.Contains(observer))
            _observers.Add(observer);
        return new Unsubscriber(_observers, observer);
    }
    ...
}
```

Наш класс `ProductRecorder` реализует интерфейс `IObservable<Product>`. Если вы вспомните нашу дискуссию о наблюдателе, то узнаете, что этот класс на самом деле является поставщиком, субъектом или наблюдаемым. Интерфейс `IObservable<T>` имеет метод `Subscribe`, который мы должны использовать для подписки наших подписчиков или наблюдателей (мы обсудим наблюдателя позже в этом разделе).

Должен существовать критерий или условие, чтобы подписчик мог получать уведомления. В нашем случае метод `Record` служит этой цели. Рассмотрим следующий код:

```
public void Record(Product product)
{
    var discountRate = product.Discount.FirstOrDefault(x => x.ProductId ==
        product.Id)?.DiscountRate;
    foreach (var observer in _observers)
    {
        if (discountRate < 0 || discountRate - 100 > 0)
            observer.OnError(
                new Exception($"Product:{product.Name} has invalid discount
                    rate {discountRate}"));
        else
            observer.OnNext(product);
    }
}
```

Здесь мы видим метод `Record`. Мы создали его, чтобы продемонстрировать мощь паттерна. Этот метод просто проверяет корректность скидки. Если значение

`discount rate` недопустимо в соответствии с критерием/условием, то данный метод вызовет исключение и передаст название продукта с этим недопустимым значением.

Предыдущий метод проверяет значение `discount rate` в соответствии с критериями и отправляет уведомление о возникшем исключении подписчику при невыполнении критериев. Взгляните на блок итераций (цикл `foreach`) и вообразите ситуацию, когда нам нечего повторять, а все подписчики были уведомлены. Можно ли представить, что произойдет в этом случае? Такая же ситуация может возникнуть и для бесконечного цикла. Остановить это поможет нечто завершающее цикл. Для подобных случаев у нас есть метод `EndRecording`:

```
public void EndRecording()
{
    foreach (var observer in _observers.ToArray())
        if (_observers.Contains(observer))
            observer.OnCompleted();
    _observers.Clear();
}
```

Наш метод `EndRecording` пропускает через цикл коллекцию `_observers` и запускает метод `OnCompleted()`. Наконец, она очистила коллекцию `_observers`.

Теперь обсудим класс `ProductReporter`. Он является примером реализации интерфейса `IObserver<T>`. Рассмотрим код:

```
public void OnCompleted()
{
    PrepReportData(true, $"Report has completed: {Name}");
    Unsubscribe();
}

public void OnError(Exception error) => PrepReportData(false, $"Error
    occurred with instance: {Name}");

public void OnNext(Product value)
{
    var msg =
        $"Reporter:{Name}. Product - Name: {value.Name},
        Price:{value.Price},Desc: {value.Description}";
    PrepReportData(true, msg);
}
```

В интерфейсе `IObserver<T>` есть методы `OnComplete`, `OnError`, и `OnNext`, которые мы должны реализовать в классе `ProductReporter`. Цель метода `OnCompleted` — уведомить подписчика о выполнении работы и затем отменить его подписку. Кроме того, `OnError` вызывается при возникновении ошибки во время выполнения, в то время

как `OnNext` предоставляет информацию об очередном элементе в последовательности потока.

В следующем коде метод `PrepReportData` — дополнение значения, которое предоставляет пользователю форматированный отчет обо всех операциях процесса:

```
private void PrepReportData(bool isSuccess, string message)
{
    var model = new MessageViewModel
    {
        MsgId = Guid.NewGuid().ToString(),
        IsSuccess = isSuccess,
        Message = message
    };

    Reporter.Add(model);
}
```

Описанный метод — простое добавление модели к нашей коллекции `Reporter`, которая представляет собой коллекцию классов `MessageViewModel`. Обратите внимание: в целях упрощения вы также можете использовать метод `ToString()` в `MessageViewModel`.

Следующий фрагмент кода показывает методы `Subscribe` и `Unsubscribe`:

```
public virtual void Subscribe(IObservable<Product> provider)
{
    if (provider != null)
        _unsubscriber = provider.Subscribe(this);
}

private void Unsubscribe() => _unsubscriber.Dispose();
```

Два предыдущих метода сообщают системе, что провайдер существует. Подписчики могут подписаться на провайдера или отказаться от подписки/избавиться от него после завершения операций.

Теперь пришло время продемонстрировать нашу реализацию и увидеть некоторые результаты. Чтобы сделать это, нам нужно внести некоторые изменения в существующую страницу `Product Listing` и добавить новую *страницу представления* в проект. Добавьте ссылку на представление `Audit Report` (Аудиторский отчет) на нашу страницу `Index.cshtml`:

```
<a asp-action="Report">Audit Report</a>
```

В предыдущем фрагменте кода мы добавили новую ссылку на страницу `Audit Report` (Аудиторский отчет), основанную на нашей реализации метода `Report Action`, который мы определили в нашем классе `ProductConstrroller`.

После добавления этого кода наша страница Product Listing (Список продуктов) будет выглядеть следующим образом (рис. 10.18).

FlixOne.Web Home Products About Contact Privacy								
Product Listing								
Create New Product Products Discount Listing Audit Report								
Product Name	Description	Image	Price (INR)	Discount Rate	Discount (INR)	Net Price (INR)	Category Name	Action
Mango	A juicy mango		40.00	5.00	2.00	38.00	Fruit	Edit Details Delete
Apple	Red apple		100.00	10.00	10.00	90.00	Fruit	Edit Details Delete
Orange	Fruity oranges		35.00	3.00	1.05	33.95	Fruit	Edit Details Delete
© 2019 - FlixOne.Web - Privacy								

Рис. 10.18

Для начала обсудим метод Report action. Для этого рассмотрим следующий код:

```
var mango = _repository.GetProduct(new Guid
    ("09C2599E-652A-4807-A0F8-390A146F459B"));
var apple = _repository.GetProduct(new Guid
    ("7AF8C5C2-FA98-42A0-B4E0-6D6A22FC3D52"));
var orange = _repository.GetProduct(new Guid
    ("E2A8D6B3-A1F9-46DD-90BD-7F797E5C3986"));
var model = new List<MessageViewModel>();
// провайдер
ProductRecorder productProvider = new ProductRecorder();
// наблюдатель 1
ProductReporter productObserver1 = new ProductReporter(nameof(mango));
// наблюдатель 2
ProductReporter productObserver2 = new ProductReporter(nameof(apple));
// наблюдатель 3
ProductReporter productObserver3 = new ProductReporter(nameof(orange));
```

В данном коде мы берем только первые три продукта для демонстрационных целей. Обратите внимание: вы можете изменить код в соответствии с собственной реализацией. В коде мы создали класс `productProvider` и трех наблюдателей для подписки на наш класс `productProvider`.

На рис. 10.19 наглядно представлены все действия для демонстрации интерфейсов `IObservable<T>` и `IObserver<T>`, которые мы обсудили.

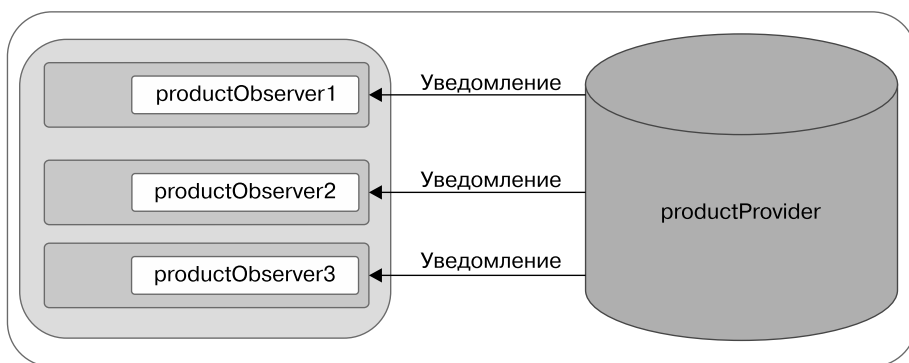


Рис. 10.19

Следующий код используется для подписки на `productProvider`:

```
// подписка
productObserver1.Subscribe(productProvider);
productObserver2.Subscribe(productProvider);
productObserver3.Subscribe(productProvider);
```

Наконец, нам нужно записать отчет в журнал и затем отказаться от подписки:

```
// отчет и отмена подписки
productProvider.Record(mango);
model.AddRange(productObserver1.Reporter);
productObserver1.Unsubscribe();
productProvider.Record(apple);
model.AddRange(productObserver2.Reporter);
productObserver2.Unsubscribe();
productProvider.Record(orange);
model.AddRange(productObserver3.Reporter);
productObserver3.Unsubscribe();
```

Вернемся к нашему экрану и добавим файл `Report.cshtml` в `Views ► Product` (Представления ► Продукт). Следующий код — часть нашей страницы `Report` (Отчет). Полный код можно найти в папке `Product`:

```
@model IEnumerable<MessageViewModel>

<thead>
<tr>
    <th>
        @Html.DisplayNameFor(model => model.IsSuccess)
    </th>
    <th>
        @Html.DisplayNameFor(model => model.Message)
    </th>
</tr>
</thead>
```

Этот код создаст заголовок для столбцов нашей таблицы, в котором будет показан аудиторский отчет.

Следующий код завершит таблицу и добавит значения в `IsSuccess` и столбцы `Message`:

```
<tbody>
@foreach (var item in Model)
{
    <tr>
        <td>
            @Html.HiddenFor(modelItem => item.MsgId)
            @Html.DisplayFor(modelItem => item.IsSuccess)
        </td>
        <td>
            @Html.DisplayFor(modelItem => item.Message)
        </td>
    </tr>
}
</tbody>
</table>
```

На этом мы закончили реализацию паттерна «Наблюдатель» с помощью интерфейсов `IObservable<T>` и `IObserver<T>`. Запустите проект в Visual Studio, нажав клавишу F5, щелкните кнопкой мыши на ссылке **Product** (Продукт) на главной странице, а затем по ссылке **Audit Report** (Аудиторский отчет). Вы увидите аудиторский отчет по выбранным нами продуктам, как показано на рис. 10.20.

Здесь представлена простая страница листинга, на которой показаны данные из класса `MessageViewModel`. Вы можете вносить изменения и модифицировать их в соответствии с вашими требованиями. В целом аудиторские отчеты поступают от многих видов оперативной деятельности, которые мы видим на данном экране. Вы также можете сохранить проверенные данные в базе данных, а затем обслуживать их соответствующим образом для различных целей, например для составления отчетов администратору и не только.

Реактивные расширения: .NET Rx Extensions

Обсуждение в предыдущем разделе было направлено на реактивное программирование и его реализацию с помощью интерфейсов `IObservable<T>` и `IObserver<T>` в качестве паттерна «Наблюдатель». В этом разделе мы будем развивать наше обучение с помощью *Rx Extensions*. Если вы хотите узнать больше о данной разработке, то вам следует обратиться к официальному репозиторию по адресу github.com/dotnet/reactive.

FlixOne.Web Home Products About Contact Privacy	
Product Audit Report Product Listing	
IsSuccess	Message
☒	Reporter:mango. Product - Name: Mango, Price:40.00,Desc: A juicy mango
☒	Reporter:apple. Product - Name: Mango, Price:40.00,Desc: A juicy mango
☒	Reporter:apple. Product - Name: Apple, Price:100.00,Desc: Red apple
☒	Reporter:orange. Product - Name: Mango, Price:40.00,Desc: A juicy mango
☒	Reporter:orange. Product - Name: Apple, Price:100.00,Desc: Red apple
☒	Reporter:orange. Product - Name: Orange, Price:35.00,Desc: Fruity oranges
© 2019 - FlixOne.Web - Privacy	

Рис. 10.20

Обратите внимание: Rx Extensions теперь объединены с пространством имен `System` и вы можете найти их все в пространстве имен `System.Reactive`. Если у вас есть опыт работы с Rx Extensions, то вы должны знать, что пространство имен этих расширений было изменено следующим образом:

- ☐ `Rx.Main` изменено на `System.Reactive`;
- ☐ `Rx.Core` изменено на `System.Reactive.Core`;
- ☐ интерфейсы `Rx.Interfaces` изменены на `System.Reactive.Interfaces`;
- ☐ `Rx.Linq` изменено на `System.Reactive.Linq`;
- ☐ `Rx.PlatformServices` изменен на `System.Reactive.PlatformServices`;
- ☐ `Rx.Testing` изменено на `Microsoft.Reactive.Testing`.

Для начала откройте в Visual Studio проект `SimplyReactive` (обсуждался в предыдущем разделе) и панель `NuGet Package Manager` (Менеджер пакетов NuGet). Нажмите кнопку `Browse` (Поиск) и введите поисковый запрос `System.Reactive`. Вы увидите следующие результаты (рис. 10.21).

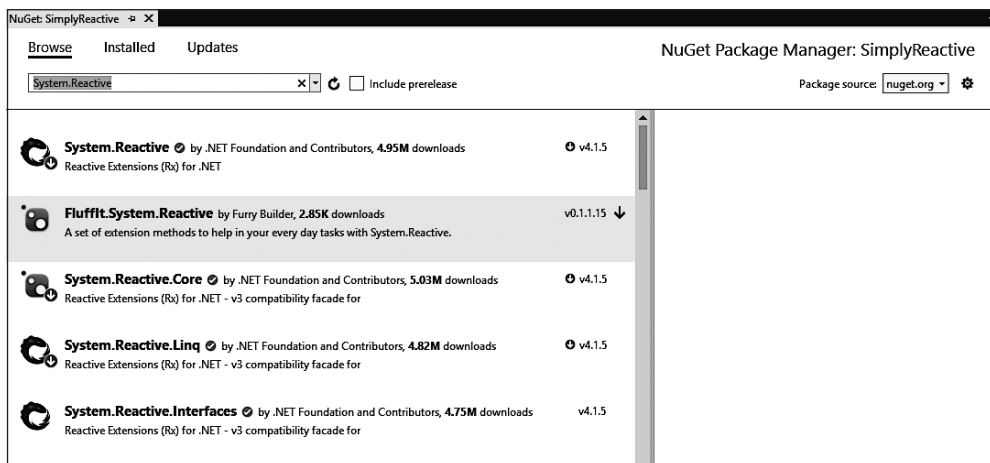


Рис. 10.21

Цель данного раздела — познакомить вас с реактивными расширениями, но не углубляться в их внутреннее развитие. Они находятся под лицензией Apache2.0 и поддерживаются .NET Foundation. Мы уже реализовали реактивные расширения в приложении *SimplyReactive*.

Настройка приложения FlixOne

В этом разделе мы продолжим работу с нашим приложением для инвентаризации под названием *FlixOne*. Здесь мы обсудим паттерн веб-приложения и допишем веб-приложение, разработанное в главе 4.



В этой главе мы продолжаем рассматривать веб-приложения, которые обсуждались в предыдущей. Если вы пропустили эту главу, то, пожалуйста, вернитесь к ней, прежде чем читать эту.

В данном разделе мы пройдем процесс сбора требований, а затем обсудим различные проблемы развития и бизнеса с веб-приложением, которое мы разработали ранее.

Начало проекта

В главе 7 мы добавили функции нашего веб-приложения *FlixOne* для инвентаризации. Мы расширили заявку после рассмотрения следующих пунктов:

- ☐ бизнесу нужен многофункциональный пользовательский интерфейс;
- ☐ новые возможности требуют от веб-приложения гибкости.

Требования

После нескольких встреч и обсуждений с руководством, *бизнес-аналитиком* (business analyst, BA), а также предпродажной подготовки, руководство организации приняло решение работать над следующими требованиями высокого уровня.

Бизнес-требования

Бизнес-пользователи перечислили такие требования.

- ❑ *Фильтрация элементов* — в настоящее время пользователи не могут фильтровать элементы по категориям. Для расширения функции просмотра списка пользователь должен иметь возможность отфильтровать товар по его соответствующей категории.
- ❑ *Сортировка элементов* — в настоящее время элементы появляются в порядке их добавления в базу данных. Отсутствует механизм, с помощью которого пользователь может отсортировать элементы по имени, цене и т. п.



Веб-приложение FlixOne для управления инвентаризацией — это воображаемый продукт. Мы создаем его, чтобы обсудить различные паттерны проектирования, необходимые/используемые в веб-проекте.

Применение в приложении FlixOne фильтрации, пагинации и сортировки

В соответствии с бизнес-требованиями мы должны применить фильтрацию, пагинацию и сортировку в нашем приложении для инвентаризации FlixOne. Сначала начнем сортировку. Для этого мы создали проект и поместили его в папку FlixOneWebExtended. Запустите приложение Visual Studio и откройте решение FlixOne. Мы применим сортировку в нашем списке продукции для этих столбцов: `Category`, `productName`, `Description` и `Price`. Пожалуйста, обратите внимание, что мы не будем использовать какой-либо внешний компонент для сортировки, но создадим собственный логин.

Откройте панель Solution Explorer (Обозреватель решений) и `ProductController`, который доступен в папке `Controllers`. Добавьте параметр `[FromQuery] Sort sort` к методу `Index`. Обратите внимание: атрибут `[FromQuery]` указывает на то, что этот параметр является параметром запроса. Мы будем использовать его для поддержания порядка сортировки.

Следующий код показывает класс `Sort`:

```
public class Sort
{
```

```

public SortOrder Order { get; set; } = SortOrder.A;
public string ColName { get; set; }
public ColumnType ColType { get; set; } = ColumnType.Text;
}

```

Класс `Sort` содержит три общедоступных свойства:

- ❑ `Order` — определяет порядок сортировки. `SortOrder` — перечисление, определенное таким образом: `public enum SortOrder { D, A, N };`
- ❑ `ColName` — имя колонки;
- ❑ `ColType` — тип колонки; `ColumnType` — это перечисление: `public enum ColumnType { Text, Date, Number }.`

Откройте интерфейс `IInventoryRepository` и добавьте метод `IEnumerable<Product> GetProducts(Sort sort)`. Он отвечает за сортировку результатов. Пожалуйста, обратите внимание: мы будем использовать запросы LINQ для сортировки. Реализуйте метод класса `InventoryRepository` и добавьте следующий код:

```

public IEnumerable<Product> GetProducts(Sort sort)
{
    if(sort.ColName == null)
        sort.ColName = "";
    switch (sort.ColName.ToLower())
    {
        case "categoryname":
        {
            var products = sort.Order == SortOrder.A
                ? ListProducts().OrderBy(x => x.Category.Name)
                : ListProducts().OrderByDescending(x => x.Category.Name);
            return PDiscounts(products);
        }
    }
}

```

Код ниже обрабатывает случай, когда `sort.ColName` — это `productname`:

```

        case "productname":
        {
            var products = sort.Order == SortOrder.A
                ? ListProducts().OrderBy(x => x.Name)
                : ListProducts().OrderByDescending(x => x.Name);
            return PDiscounts(products);
        }
    }
}

```

Следующий код обрабатывает случай, когда `sort.ColName` — это `productprice`:

```

        case "productprice":
        {
            var products = sort.Order == SortOrder.A
                ? ListProducts().OrderBy(x => x.Price)
                : ListProducts().OrderByDescending(x => x.Price);
        }
    }
}

```

```

        return PDiscounts(products);
    }
    default:
        return PDiscounts(ListProducts().OrderBy(x => x.Name));
    }
}

```

В данном коде мы установили значение параметра `sort` как пустое, если оно содержит нулевое значение, а затем обработали его с помощью оператора `switch...case` в `sort.ColName.ToLower()`.

Ниже приведен наш метод `ListProducts()`, дающий результат типа `IIncludeQueryable<Product, Category>`:

```

private IIncludeQueryable<Product, Category> ListProducts() =>
    _inventoryContext.Products.Include(c => c.Category);

```

Этот код просто дает нам `Products`, включая `Categories` для каждого продукта. Порядок сортировки будет определяться пользователем, поэтому нам нужно изменить страницу `Index.cshtml`. Нам также нужно добавить тег привязки к столбцам заголовка таблицы. Для этого рассмотрим код ниже:

```

<thead>
  <tr>
    <th>
      @Html.ActionLink(Html.DisplayNameFor(model =>
        model.CategoryName), "Index", new Sort { ColName =
        "CategoryName", ColType = ColumnType.Text, Order = SortOrder.A })
    </th>
    <th>
      @Html.ActionLink(Html.DisplayNameFor(model =>
        model.ProductName), "Index", new Sort { ColName =
        "ProductName", ColType = ColumnType.Text, Order = SortOrder.A })
    </th>
    <th>
      @Html.ActionLink(Html.DisplayNameFor(model =>
        model.ProductDescription), "Index", new Sort { ColName =
        "ProductDescription", ColType = ColumnType.Text,
        Order = SortOrder.A })
    </th>
  </tr>
</thead>

```

В данном коде показаны столбцы заголовков таблицы; `new Sort { ColName = "ProductName", ColType = ColumnType.Text, Order = SortOrder.A }` это основной способ, которым мы реализуем `SortOrder`.

Запустите приложение, и вы увидите страницу `Product Listing` (Список продуктов) с функцией сортировки (рис. 10.22).

FlixOne.Web

[Home](#) [Products](#) [About](#) [Contact](#) [Privacy](#)

Product Listing

[Create New Product](#) | [Products Discount Listing](#) | [Audit Report](#)

Cat Name	Name	Description	Image	Price	Discount Rate	Discount	Net Price	
Fruit	Orange	Fruity oranges		35.00	3.00	1.05	33.95	Edit Details Delete
Fruit	Mango	A juicy mango		40.00	5.00	2.00	38.00	Edit Details Delete
Fruit	Apple	Red apple		100.00	10.00	10.00	90.00	Edit Details Delete

Рис. 10.22

Теперь откройте страницу `Index.cshtml` и добавьте следующий код:

```
@using (Html.BeginForm())
{
    <p>
        Search by: @Html.TextBox("searchTerm")
        <input type="submit" value="Search" class="btn-sm btn-success" />
    </p>
}
```

В коде мы добавляем текстовое поле под `Form`. Здесь пользователь вводит данные/значение, и они отправляются на сервер, как только он нажимает кнопку отправки. На стороне сервера отфильтрованные данные вернутся и покажут список продуктов. После реализации предыдущего кода наша страница `Product Listing` (Список продуктов) будет выглядеть так (рис. 10.23).

Product Listing

[Create New Product](#) | [Products Discount Listing](#) | [Audit Report](#)

Search by:

Cat Name	Name	Description	Image	Price	Discount Rate	Discount	Net Price	
Fruit	Mango	A juicy mango		40.00	5.00	2.00	38.00	Edit Details Delete
Fruit	Apple	Red apple		100.00	10.00	10.00	90.00	Edit Details Delete
Fruit	Orange	Fruity oranges		35.00	3.00	1.05	33.95	Edit Details Delete

Рис. 10.23

Перейдите к методу `Index` в `ProductController` и измените параметры. Теперь метод `Index` выглядит так:

```
public IActionResult Index([FromQuery]Sort sort, string searchTerm)
{
    var products = _repository.GetProducts(sort, searchTerm);
    return View(products.ToProductvm());
}
```

Аналогично нам необходимо обновить параметры метода `GetProducts()` в классе `InventoryRepository`. Ниже приведен код для данного класса:

```
private IEnumerable<Product> ListProducts(string searchTerm = "")
{
    var includableQueryable = _inventoryContext.Products.Include(c =>
        c.Category).ToList();
    if (!string.IsNullOrEmpty(searchTerm))
    {
        includableQueryable = includableQueryable.Where(x =>
            x.Name.Contains(searchTerm) ||
            x.Description.Contains(searchTerm) ||
            x.Category.Name.Contains(searchTerm)).ToList();
    }

    return includableQueryable;
}
```

Теперь запустите проект, нажав клавишу `F5` в `Visual Studio`, и перейдите к опции фильтрации/поиска на странице `Product Listing` (Список продуктов). Для этого посмотрите на рис. 10.24.

FlixOne.Web Home Products About Contact Privacy								
Product Listing								
Create New Product Products Discount Listing Audit Report								
Search by: mango Search								
Cat Name	Name	Description	Image	Price	Discount Rate	Discount	Net Price	
Fruit	Apple	Red apple		100.00	10.00	10.00	90.00	Edit Details Delete
Fruit	Mango	A juicy mango		40.00	5.00	2.00	38.00	Edit Details Delete
Fruit	Orange	Fruity oranges		35.00	3.00	1.05	33.95	Edit Details Delete

Рис. 10.24

После ввода поискового слова нажмите кнопку Search (Поиск), и вы получите результаты, показанные на рис. 10.25.

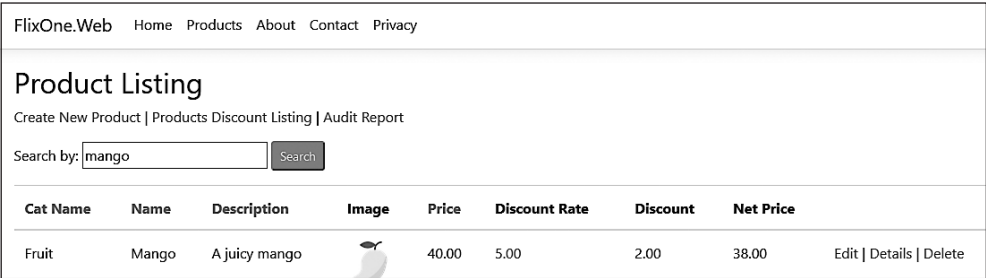


Рис. 10.25

На данном снимке экрана показано, что на странице Product Listing (Список продуктов) мы фильтруем записи Product (Продукт) с помощью `searchTerm mango` и получаем один результат. У этого подхода есть одна проблема, связанная с поиском данных: добавить `fruit` в качестве поискового запроса и посмотреть, что произойдет. Результат будет нулевым. Это показано на рис. 10.26.

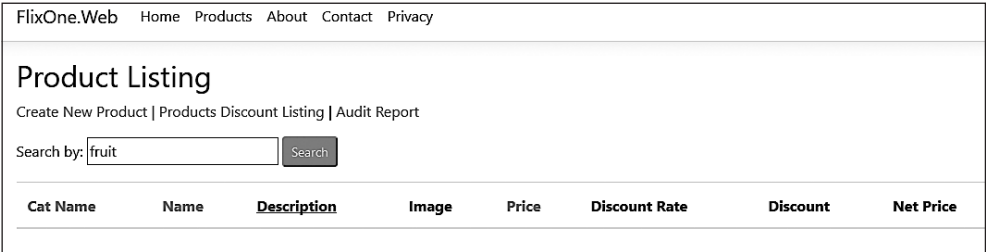


Рис. 10.26

Мы не получаем никакого результата. Это значит, наш поиск не работает, когда мы пишем `searchTerm` в нижнем регистре. Это также говорит о том, что поиск чувствителен к регистру. Нам нужно изменить код, чтобы он начал работать.

Ниже представлен наш измененный код:

```
var includableQueryable = _inventoryContext.Products.Include(c =>
    c.Category).ToList();
if (!string.IsNullOrEmpty(searchTerm))
{
    includableQueryable = includableQueryable.Where(x =>
        x.Name.Contains(searchTerm,
            StringComparison.InvariantCultureIgnoreCase) ||
        x.Description.Contains(searchTerm,
```

```

StringComparison.InvariantCultureIgnoreCase) ||
    x.Category.Name.Contains(searchTerm,
        StringComparison.InvariantCultureIgnoreCase)).ToList();
}

```

Мы игнорируем регистр, чтобы сделать наш поиск нечувствительным к нему, используя `StringComparison.InvariantCultureIgnoreCase`. Теперь наш поиск будет работать как с заглавными, так и со строчными буквами. На рис. 10.27 показаны результаты для запроса `fruit` в нижнем регистре.

Product Listing								
Create New Product Products Discount Listing Audit Report								
Search by: fruit Search								
Cat Name	Name	Description	Image	Price	Discount Rate	Discount	Net Price	
Fruit	Apple	Red apple		100.00	10.00	10.00	90.00	Edit Details Delete
Fruit	Mango	A juicy mango		40.00	5.00	2.00	38.00	Edit Details Delete
Fruit	Orange	Fruity oranges		35.00	3.00	1.05	33.95	Edit Details Delete

Рис. 10.27

В предыдущем обсуждении расширения приложения FlixOne мы применили `Sort` и `Filter`. Теперь нам нужно добавить `paging` (пагинацию, разбиение на страницы). Для этого мы добавили новый класс с именем `PageList`, как показано ниже:

```

public class PagedList<T> : List<T>
{
    public PagedList(List<T> list, int totalRecords, int currentPage,
        int recordPerPage)
    {
        CurrentPage = currentPage;
        TotalPages = (int) Math.Ceiling(totalRecords / (double) recordPerPage);

        AddRange(list);
    }
}

```

Теперь изменим параметры `Index` метода `ProductController` следующим образом:

```

public IActionResult Index([FromQuery] Sort sort, string searchTerm,
    string currentSearchTerm,
    int? pageNumber,
    int? pagesize)

```


Добавим следующий код на страницу `Index.cshtml`:

```
@{
    var prevDisabled = !Model.HasPreviousPage ? "disabled" : "";
    var nextDisabled = !Model.HasNextPage ? "disabled" : "";
}

<a asp-action="Index"
    asp-route-sortOrder="@ViewData["CurrentSort"]"
    asp-route-pageNumber="@((Model.CurrentPage - 1))"
    asp-route-currentFilter="@ViewData["currentSearchTerm"]"
    class="btn btn-sm btn-success @prevDisabled">
    Previous
</a>
<a asp-action="Index"
    asp-route-sortOrder="@ViewData["CurrentSort"]"
    asp-route-pageNumber="@((Model.CurrentPage + 1))"
    asp-route-currentFilter="@ViewData["currentSearchTerm"]"
    class="btn btn-sm btn-success @nextDisabled">
    Next
</a>
```

Данный код позволяет переместиться на следующую или предыдущую страницы. Наш последний экран будет выглядеть вот так (рис. 10.28).

Product Listing							
Create New Product Products Discount Listing Audit Report							
Search by:		Search Text	Search	Clear			
CategoryName	ProductName	ProductDescription	ProductImage	ProductPrice	ProductDiscountRate	ProductDiscount	ProductNetPrice
Fruit	Apple	Red apple		100.00	10.00	10.00	90.00
Fruit	Mango	A juicy mango		40.00	5.00	2.00	38.00
Fruit	Orange	Fruity oranges		35.00	3.00	1.05	33.95
Previous		Next					

Рис. 10.28

В этом подразделе мы обсудили и расширили возможности нашего приложения FlixOne, реализовав сортировку, пагинацию и фильтрацию. Цель этого подраздела — дать вам практический опыт работы с приложением. Мы написали наше приложение таким образом, что оно будет непосредственно соответствовать реальным приложениям. Благодаря предыдущему улучшению приложение способно

выдавать список продуктов, который можно отсортировать, разбить на страницы и отфильтровать.

Паттерны и практики — MVVM

В главе 6 мы обсудили паттерн *MVC* и создали приложение на его основе.

Кен Купер и Тед Питерс разработали паттерн *MVVM*. Во время этого изобретения и Кен, и Тед занимались проектированием в корпорации Microsoft. Они сделали данный паттерн, чтобы упростить пользовательский интерфейс событийного программирования. Позже он был реализован в *Windows Presentation Foundation (WPF)* и *Silverlight*.



Паттерн *MVVM* был анонсирован в 2005 году Джоном Госсманом. Джон вел блог об этом паттерне в контексте создания WPF-приложений. Узнать о *MVVM* можно, пройдя по ссылке blogs.msdn.microsoft.com/johngossman/2005/10/08/introduction-to-modelviewviewmodel-pattern-for-building-wpf-apps/.

MVVM считается одной из вариаций *MVC*, отвечающей современному подходу создания *пользовательского интерфейса (UI)*, в котором UI-разработка — основная обязанность дизайнеров/UI-программистов, а не разработчиков приложений. При таком подходе дизайнер, который является графическим энтузиастом и сосредоточен на том, чтобы сделать пользовательский интерфейс более привлекательным, может беспокоиться или не беспокоиться о технической стороне разработки приложения. Как правило, дизайнеры (пользователи UI) применяют различные инструменты, чтобы сделать его более привлекательным. Он может быть написан с помощью простого HTML-кода, CSS и т. д., а также многофункциональных элементов управления WPF или Silverlight.



Microsoft Silverlight — фреймворк, который помогает разрабатывать приложения с многофункциональным пользовательским интерфейсом. Многие разработчики называют его альтернативой Flash компании Adobe. В июле 2015 года Microsoft сообщила, что больше не поддерживает Silverlight. Microsoft объявила о поддержке WPF в .NET Core 3.0 во время сборки (developer.microsoft.com/en-us/events/build). Существует также блог с более подробной информацией о поддержке WPF: devblogs.microsoft.com/dotnet/net-core-3-and-support-for-windows-desktop-applications/.

Модель *MVVM* можно доработать следующим образом.

- ❑ *Модель* — хранит данные и не заботится о какой-либо бизнес-логике в приложении. Я предпочитаю называть это доменным объектом, поскольку он содержит

фактические данные приложения, с которым мы работаем. Другими словами, мы можем сказать: модель не несет ответственности за то, чтобы данные были красивыми. Например, в продукте FlixOne модель содержит значения различных свойств, характеризующих продукт по названию, описанию, категории, цене и т. д. Эти свойства содержат фактические данные продукта, но модель не несет ответственности за внесение поведенческих изменений в какие-либо данные. Например, модель продукта не отвечает за форматирование описания продукта, чтобы оно выглядело идеально в пользовательском интерфейсе. С другой стороны, многие из наших моделей содержат проверки и другие вычисляемые свойства. Главная задача — поддерживать чистую и ясную модель, а это значит, что она должна напоминать модель реального мира. В нашем случае модель **Product** называется *чистой*. Таковая напоминает реальные свойства реальных продуктов. Например, если модель **Product** хранит данные фруктов, то должна показывать такие свойства, как цвет фруктов и т. д. Следующий код взят из модели нашего воображаемого приложения:

```
export class Product {  
  name: string;  
  cat: string;  
  desc: string;  
}
```

Обратите внимание: этот код написан на Angular. Мы подробно обсудим его чуть ниже, в подразделе «Реализация MVVM».

- ❑ *Представление* — это представление данных для доступа конечного пользователя через UI. Оно просто отображает значение данных, и это значение может быть отформатировано или нет. Например, мы можем показать скидку как 18 % в пользовательском интерфейсе, а она будет сохранена как 18,00 в модели. Представление также может отвечать за поведенческие изменения. Оно принимает вводимые пользователем данные. Например, существует представление формы/экрана для добавления нового продукта. Кроме того, представление может управлять вводом данных пользователем, таким как нажатая клавиша, обнаружение ключевого слова и др. Оно может быть активным или пассивным. Представление, которое принимает вводимые пользователем данные и управляет их моделью (свойствами) в соответствии с этими данными, является активным. Пассивное представление ничего не делает. Другими словами, представление, не связанное с моделью, является пассивным. Этим видом представления управляет контроллер.
- ❑ *Модель представления* — работает как посредник между представлением и моделью. Его обязанность заключается в том, чтобы сделать представление лучше. В нашем предыдущем примере, где представление показывает ставку скидки как 18 %, но у модели скидка 18,00, модель представления несет ответственность за форматирование 18,00 в 18 %.

Если мы объединим все рассмотренные моменты, то сможем визуализировать весь паттерн MVVM. Он будет выглядеть так (рис. 10.29).



Рис. 10.29

На этом рисунке дано наглядное представление MVVM. Здесь показано, что *модель представления* разделяет *представление* и *модель*. Кроме того, *модель представления* сопровождает операции *state* и *perform*. Это помогает *представлению* представить конечный результат пользователю. Представление — это UI, который получает данные и представляет их пользователю. В следующем разделе мы реализуем паттерн MVVM с помощью Angular.

Реализация MVVM

В предыдущем разделе мы поняли, что такое паттерн MVVM и как он работает. В этом подразделе мы используем нашу программу FlixOne и разработаем приложение с помощью Angular. Чтобы продемонстрировать паттерн MVVM, мы применяем API, построенный на ASP.NET Core 2.2.

Запустите приложение Visual Studio и откройте FlixOne Solution (Решение FlixOne) из папки FlixOneMVVM. Запустите проект FlixOne.API, в котором увидите следующую страницу документации Swagger (рис. 10.30).

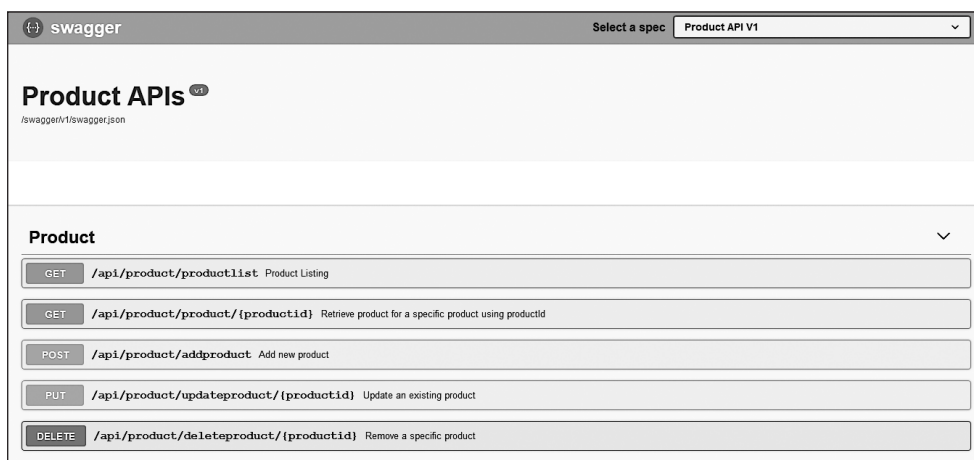


Рис. 10.30

Здесь показана страница документации Product API, на которой мы добавили Swagger для документации API. При желании вы можете протестировать API с помощью этого экрана. Если API возвращают результаты, то ваш проект настроен правильно. В противном случае, пожалуйста, проверьте предварительные требования для этого проекта, а также файл `README.md` из репозитория Git для этой главы. У нас есть все необходимое для создания нового пользовательского интерфейса. Как обсуждалось ранее, мы создадим приложение Angular, которое будет применять API продукта. Для начала выполните следующие действия.

1. Откройте панель Solution Explorer (Обозреватель решений).
2. Щелкните правой кнопкой мыши на пункте FlixOne Solution (Решение FlixOne).
3. Выберите пункт Add New Project (Добавить новый проект).
4. В окне Add New Project (Добавить новый проект) выберите пункт .NET Core Web Application (Веб-приложение .NET Core). Назовите проект `FlixOne.Web` и нажмите кнопку OK (рис. 10.31).

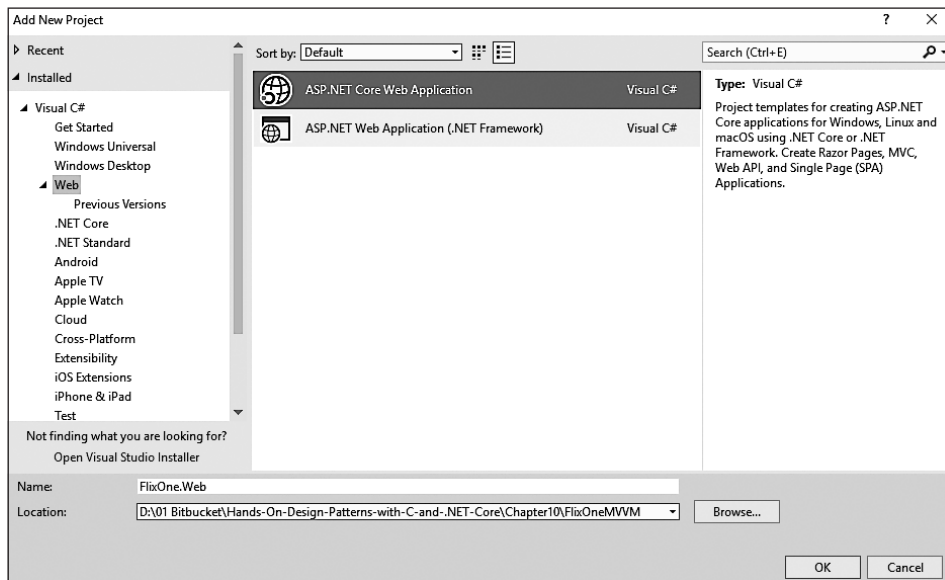


Рис. 10.31

5. В следующем окне щелкните кнопкой мыши на значке Angular, выберите в раскрывающемся списке пункт ASP.NET Core 2.2, нажмите кнопку OK (рис. 10.32).
6. Откройте панель Solution Explorer (Обозреватель решений), и вы увидите новый проект `FlixOne.Web`, а также иерархию папок, похожую на ту, что показана на рис. 10.33.

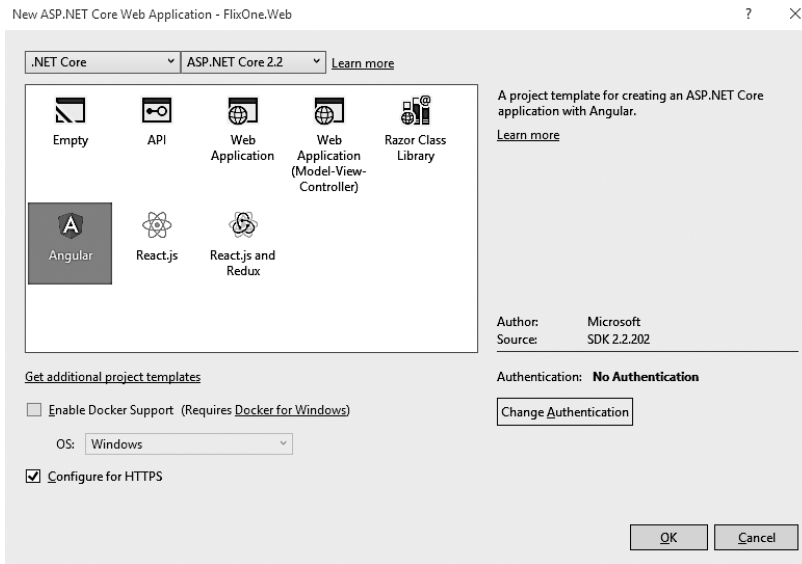


Рис. 10.32

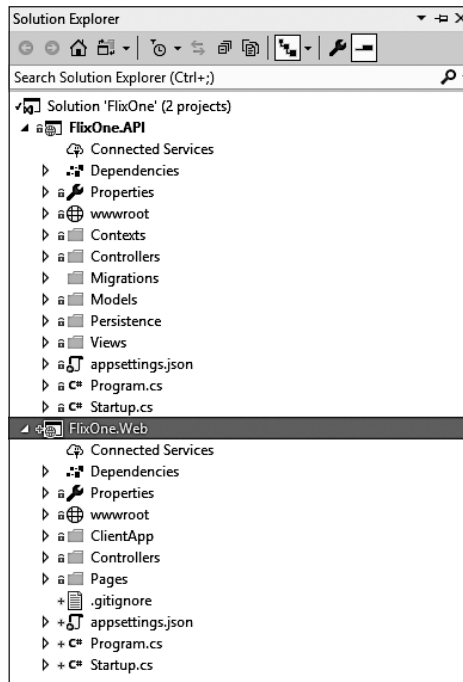


Рис. 10.33

7. На панели Solution Explorer (Обозреватель решений) щелкните правой кнопкой мыши на проекте FlixOne.Web, а затем выберите пункт Set as Startup (Установить как стартовый) (рис. 10.34).

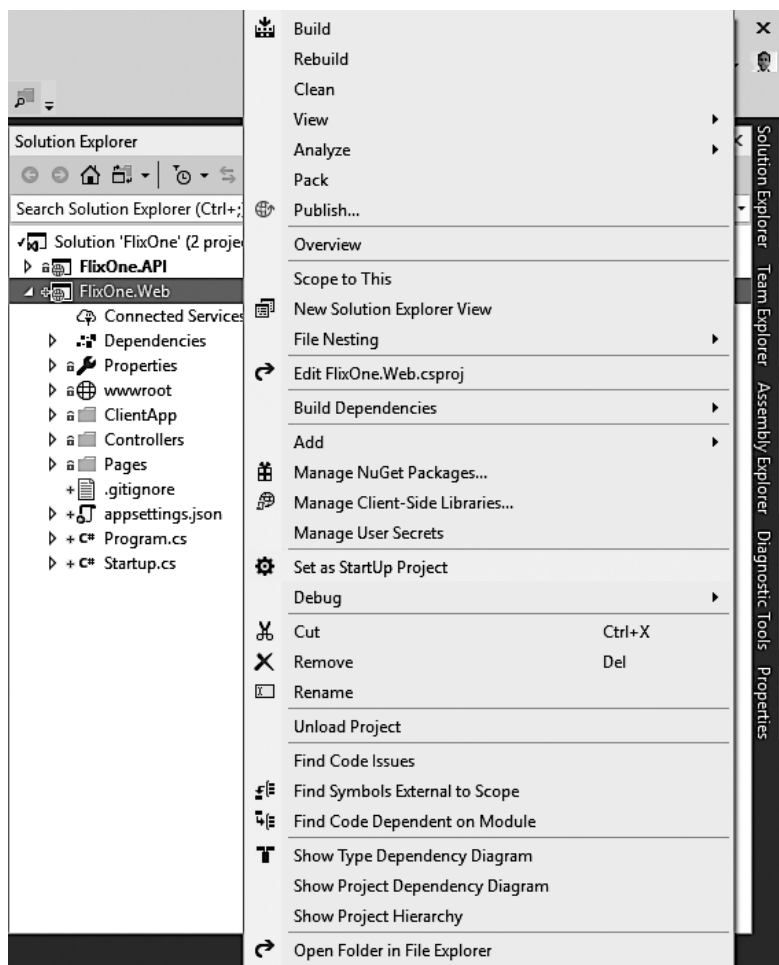


Рис. 10.34

8. Запустите FlixOne.Web, и вы увидите следующий экран (рис. 10.35).

Мы успешно настроили наше Angular-приложение. Вернитесь в Visual Studio и откройте окно Output (Вывод). Посмотрите на рис. 10.36.

Вы увидите строку `ng serve "--port" "60672"` в окне Output (Вывод). Это команда, которая указывает приложению Angular слушать и выдавать данные. Откройте

файл `package.json` на панели Solution Explorer (Обозреватель решений). Данный файл находится в папке `ClientApp`. Вы заметите `"@angular/core": "6.1.10"`, а это значит, что наше приложение построено на `angular6`.

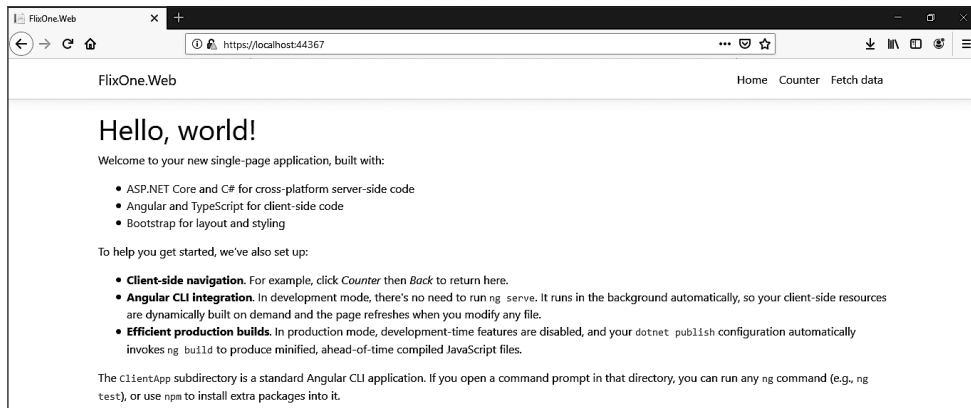


Рис. 10.35

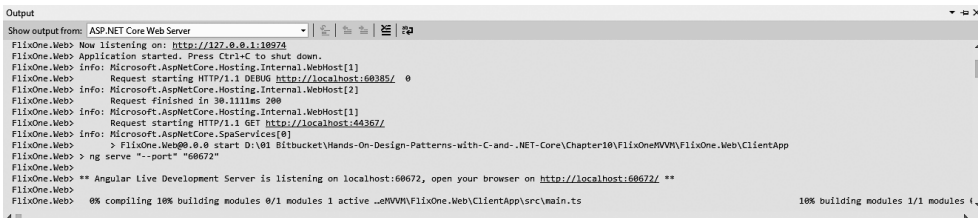


Рис. 10.36

Ниже приведен код файла `product.component.html` (это представление):

```
<table class='table table-striped' *ngIf="forecasts">
  <thead>
    <tr>
      <th>Name</th>
      <th>Cat. Name (C)</th>
      <th>Price(F)</th>
      <th>Desc</th>
    </tr>
  </thead>
  <tbody>
    <tr *ngFor="let forecast of forecasts">
      <td>{{ forecast.productName }}</td>
      <td>{{ forecast.categoryName }}</td>
      <td>{{ forecast.productPrice }}</td>
```



```

        <td>{{ forecast.productDescription }}</td>
      </tr>
    </tbody>
  </table>

```

Запустите приложение из Visual Studio и щелкните кнопкой мыши на ссылке **Product** (Продукт). Вы увидите страницу **Product Listing** (Список продуктов), похожую на эту (рис. 10.37).

FlixOne.Web

HomeProduct

Product Listing

Product Listing...

Name	Cat. Name	Price	Desc
Microservices Book		655	Building Microservices with .NET Core2.0
Mango		135	A juicy mango

Рис. 10.37

В данном подразделе мы создали небольшое приложение на Angular.

Резюме

Целью данной главы было дать представление о реактивном программировании через обсуждение его принципов и модели. Реактивность — это о потоках данных, которые мы рассмотрели на примерах. Мы расширили пример из главы 8, в котором обсуждали вариант использования кассира на конференции.

Мы исследовали реактивную систему во время нашего обсуждения манифеста реактивного программирования. Обсудили реактивную систему путем демонстрации операций **merge**, **filter** и **map**, а также с помощью примеров поняли, как работают потоки. Кроме того, изучили паттерн **IObservable** и расширения Rx, используя примеры.

Мы продолжили разрабатывать наше приложение для инвентаризации FlixOne и обсудили варианты реализации функций пагинации и сортировки данных продуктов. Наконец, обсудили паттерн **MVVM** и создали небольшое приложение на архитектуре **MVVM**.

В следующей главе мы рассмотрим расширенные методы проектирования и применения баз данных, включая применение *разделения ответственности команд и запросов* (command query responsibility segregation, CQRS) и базы данных в стиле бухгалтерской книги.

Вопросы

Следующие вопросы позволят вам усвоить информацию, представленную в этой главе.

1. Что такое поток?
2. Что такое реактивные свойства?
3. Что такое реактивная система?
4. Что подразумевается под слиянием двух реактивных потоков?
5. Что такое паттерн MVVM?

Дальнейшее чтение

Чтобы узнать больше о темах, рассмотренных в этой главе, обратитесь к книге, указанной ниже. В ней представлены различные углубленные и практические упражнения по реактивному программированию:

- *Esposito A., Ciceri M.* Reactive Programming for .NET Developers. — Packt. www.packtpub.com/web-development/reactive-programming-net-developers.

11

Усовершенствованные методы проектирования и применения баз данных

В предыдущей главе мы узнали о реактивном программировании, обсудив его принципы и модели. Мы также обсудили и рассмотрели примеры того, как оно связано с потоками данных.

Проектирование баз данных — сложная задача и требует большого терпения. В этой главе мы обсудим лучшие практики работы с базами данных и приложениями, включая применение *разделения ответственности команд и запросов* (command query responsibility segregation, CQRS) и базы данных в стиле бухгалтерской книги.

Как и в предыдущих главах, будет показана сессия сбора требований для определения *продукта с минимальным функционалом* (minimum viable product, MVP). В этой главе мы обсудим несколько факторов, которые приведут к проектированию CQRS. Мы будем использовать подход в стиле бухгалтерской книги, который состоит в усиленном отслеживании изменений в ассортименте товаров, а также в предоставлении публичных API для извлечения данных о них. Мы расскажем, почему разработчики используют базы данных в стиле бухгалтерской книги и почему мы должны сосредоточиться на реализации CQRS. В этой главе мы увидим, зачем нужен паттерн CQRS.

В этой главе будут рассмотрены следующие темы:

- ❑ обсуждение примера использования;
- ❑ дискуссия о базах данных;
- ❑ базы данных в стиле бухгалтерской книги для инвентаризации;
- ❑ реализация паттерна CQRS.

Технические требования

В этой главе содержатся примеры кода, объясняющие принципы качественной разработки. Код упрощен и предназначен лишь для демонстрации. Большинство примеров основаны на консольном приложении .NET Core, написанном на C#.

Чтобы запустить и выполнить код, вам потребуется следующее:

- ❑ Visual Studio 2019 (вы также можете запустить приложение, используя Visual Studio 2017 версии 3 и новее);
- ❑ .NET Core;
- ❑ SQL Server (в этой главе используется Express Edition).

Установка Visual Studio

Чтобы запустить примеры кода, вам необходимо установить Visual Studio. Для этого выполните следующие шаги.

1. Скачайте Visual Studio по ссылке docs.microsoft.com/ru-ru/visualstudio/install/install-visual-studio.
2. Следуйте инструкциям по установке. Доступны различные версии Visual Studio; в этой главе мы используем Visual Studio для Windows.

Вы также можете применить другую интегрированную среду разработки по вашему усмотрению.

Установка .NET Core

Если вы еще не установили .NET Core, то следуйте инструкции ниже.

1. Скачайте .NET Core для Windows по ссылке dotnet.microsoft.com/download.
2. Для получения нескольких версий и соответствующей библиотеки перейдите по ссылке dotnet.microsoft.com/download/dotnet-core/2.2.

Установка SQL Server

Если у вас не установлен SQL Server, то вам необходимо выполнить следующие действия.

1. Скачайте SQL Server по ссылке www.microsoft.com/ru-RU/download/details.aspx?id=1695.
2. Инструкции по установке можно найти по ссылке docs.microsoft.com/ru-ru/sql/ssms/download-sql-server-management-studio-ssms?view=sql-server-2017.



Для устранения неполадок и получения дополнительной информации см. www.blackbaud.com/files/support/infinityinstaller/content/installer-master/tkinstallsqlserver2008r2.htm.



Полная версия исходного кода доступна на GitHub. Код, представленный в данной главе, может быть неполным, поэтому мы рекомендуем вам скачать полный код для запуска примеров (github.com/PacktPublishing/Hands-On-Design-Patterns-with-C-and-.NET-Core/tree/master/Chapter11).

Обсуждение примера использования

В этой главе мы продолжим работу с нашим приложением для инвентаризации FlixOne. Мы обсудим модели CQRS и расширим веб-приложение, разработанное в предыдущих главах.



Данная глава продолжает тему разработки веб-приложения, описанную в предыдущей главе. Если вы ее пропустили, то, пожалуйста, вернитесь к ней и прочитайте, чтобы понять материал текущей главы.

В этом разделе мы соберем требования, а затем обсудим различные проблемы веб-приложения.

Начало проекта

В главе 7 мы расширили наше приложение, реализовав аутентификацию и авторизацию. Мы сделали это после рассмотрения следующих пунктов:

- ❑ текущее приложение открыто для всех; то есть любой пользователь может посетить любую страницу, даже ту, доступ к которой ограничен;
- ❑ пользователи не должны получать доступ к страницам, требующим доступа или специальных прав доступа. Эти страницы также известны как ограниченные или страницы с ограниченным доступом;
- ❑ пользователи должны иметь доступ к страницам/ресурсам в соответствии со своими ролями.

В главе 10 мы дополнительно расширили наше приложение FlixOne и добавили пагинацию, фильтрацию и сортировку для всех страниц, отображающих перечисление продуктов. При этом были учтены следующие моменты:

- ❑ *фильтрация элементов* — в настоящее время пользователи не могут отфильтровывать элементы по их категориям. Мы решили, что пользователи должны иметь возможность делать это;
- ❑ *сортировка элементов* — сейчас элементы отображаются в порядке их добавления в базу данных. Отсутствует механизм, позволяющий пользователю сортировать товары по таким категориям, как название товара или цена.

Требования

После нескольких встреч и обсуждений с руководством, *бизнес-аналитиком* (business analyst, BA) и предпродажным персоналом руководство решило работать над бизнес- и техническими требованиями.

Бизнес-требования

На основе обсуждений с заинтересованными сторонами и конечными пользователями, а также в соответствии с результатами исследования рынка наша бизнес-команда перечислила следующие требования.

- ❑ *Распространение продукта* — продукт доставляется различным пользователям. Сейчас самое время расширить приложение. Впоследствии оно будет поддерживать дальнейшее расширение.
- ❑ *Модель продукта* — в приложении для инвентаризации пользователи должны чувствовать себя свободно (это означает отсутствие ограничений на уровне модели, без сложных валидаций), и не должно быть никаких ограничений, пока пользователи взаимодействуют с приложением. Все экраны и страницы нужно сделать интуитивно понятными.
- ❑ *Проектирование базы данных* — база данных приложения должна быть спроектирована так, чтобы расширение не занимало много времени.

Технические требования

Наконец, отвечающие запросам бизнеса требования готовы к разработке. После нескольких обсуждений с бизнес-сотрудниками мы пришли к выводу, что эти требования выглядят следующим образом.

- ❑ *Лендингу* или *главной странице* следует:
 - иметь панель мониторинга, содержащую различные виджеты;
 - показывать общее состояние магазина.
- ❑ Страница продукта должна давать возможность:
 - добавлять, обновлять и удалять продукты;
 - добавлять, обновлять и удалять категории продуктов.



Веб-приложение FlixOne для инвентаризации — вымышленный продукт. Мы создаем это приложение, чтобы обсудить различные паттерны проектирования, которые используются в веб-проектах.

Проблемы

Мы расширили наше существующее веб-приложение, однако оно имеет различные проблемы как для разработчиков, так и для бизнеса. В данной части текста мы обсудим эти проблемы, а затем найдем их решения.

Проблемы разработки

Ниже перечислены проблемы, возникшие в связи с большими изменениями в приложении. Они стали результатом существенных расширений, связанных с обновлением консольного приложения до веб-приложения:

- ❑ *нет поддержки для RESTful* — в настоящее время отсутствует поддержка RESTful, так как не был разработан соответствующий API;
- ❑ *ограниченная безопасность* — в текущем приложении существует только один механизм, который может ограничивать/разрешать пользователю доступ к определенному экрану или модулю приложения, то есть по логину.

Проблемы бизнеса

При адаптации нового стека технологий и внесении в код множества изменений возникают проблемы. Поэтому для достижения конечного результата требуется время, которое задерживает продукт, что приводит к убыткам для бизнеса. Проблемы следующие:

- ❑ *потеря клиентов* — мы еще находимся в стадии разработки, но спрос на наши услуги или продукты очень высок. Тем не менее разработка занимает больше времени, чем ожидалось;
- ❑ *для развертывания производственных обновлений* требуется больше времени — усилия по разработке в настоящее время отнимают много времени, и это задерживает последующую деятельность, что приводит к задержке производства в целом.

Решение проблем

После нескольких встреч и мозговых штурмов разработчики решили, что веб-приложение нуждается в стабилизации. Чтобы преодолеть описанные выше проблемы, разработчики и бизнес-пользователи объединились и определили решения и ключевые точки.

Ниже перечислена функциональность, поддерживаемая этим решением:

- ❑ развитие веб-сервисов RESTful — должна быть одна панель управления API;
- ❑ строгое следование *разработке через тестирование* (TDD);

- ❑ перепроектирование *пользовательского интерфейса* (UI) в соответствии с ожиданиями пользователей.

Поговорим о базах данных

Прежде чем начать обсуждение базы данных, мы должны рассмотреть общую картину веб-приложения FlixOne.

- ❑ Одна часть нашего приложения — это управление товарами, но другая — веб-приложение электронной коммерции.
- ❑ Самое сложное заключается в том, что наше приложение также будет служить *точкой продажи* (Point Of Sale, POS). В этой части/в модуле пользователь может оплачивать товары, приобретенные им на автономных прилавках/в торговых точках.
- ❑ Что касается товаров, нам необходимо решить, какой подход мы будем использовать для расчета и ведения счетов и операций, а также для определения стоимости любого проданного товара.
- ❑ Сопровождение управления отпуском товаров доступно в различных вариантах, причем двумя наиболее часто используемыми вариантами являются «*первым пришел — первым ушел*» (First In First Out, FIFO) и «*последним пришел — первым ушел*» (Last In First Out, LIFO).
- ❑ Большая часть операций связана с финансовыми данными, вследствие чего эти операции требуют наличия полных сведений. Каждая запись должна содержать следующую информацию: текущее значение, значение до текущих изменений и внесенные изменения.
- ❑ Мы учитываем имеющиеся товары, но также обязаны вести учет приобретенных.

Есть еще несколько моментов, которые важны при разработке базы данных для любого веб-приложения электронной коммерции. Мы ограничиваем область применения FlixOne управлением ассортиментом компании и отпуском товаров.

Обработка баз данных

Подобно другим темам, которые мы рассмотрели в этой книге, существует большое количество баз данных для многих задач. Начиная от базовых паттернов, касающихся схемы БД, и заканчивая паттернами, которые определяют архитектуру систем хранения. В этом разделе мы рассмотрим два системных паттерна: «*Обработку транзакций в режиме онлайн*» (Online Transaction Processing, OLTP) и «*Аналитическую обработку в режиме онлайн*» (Online Analytical Processing,

OLAP). Чтобы глубже понять паттерны проектирования баз данных, мы изучим конкретный паттерн — базы данных в стиле бухгалтерской книги — более подробно.



Схема базы данных — еще одно слово для обозначения коллекции таблиц, представлений, хранимых процедур и других компонентов, из которых состоит база. Считайте, что это чертеж БД.

OLTP

База данных OLTP была создана для работы с большим количеством операторов, изменяющих БД. По сути, все операторы `INSERT`, `UPDATE` и `DELETE` вызывают изменения и ведут себя совсем не так, как выражение `SELECT`. БД OLTP были разработаны с учетом этого. Указанные базы записывают изменения, поэтому обычно являются *основной* (main), или *главной* (master), базой данных — то есть хранилищами текущих данных.



Оператор `MERGE` также определяется как вызывающий изменения. Это связано с тем, что данный подход предоставляет удобный синтаксис для вставки записи при отсутствии строки и для вставки обновления при ее наличии. Данные будут обновлены при условии, что строка действительно существует. Оператор `MERGE` поддерживают не все поставщики или версии БД.

Базы данных OLTP, как правило, предназначены для быстрой обработки операторов, вызывающих изменения. Обычно это обеспечивается тщательным планированием структур таблиц. Простой способ увидеть это — рассмотреть таблицу базы данных. Она может иметь поля для хранения данных, ключи для эффективного поиска данных, индексы к другим таблицам, а также триггеры для реагирования на конкретные ситуации и другие конструкции таблиц. Каждая из указанных конструкций отрицательно сказывается на производительности. Таким образом, проектирование баз данных OLTP — это баланс между использованием минимального количества конструкций в таблице и желаемым поведением.

Рассмотрим таблицу, которая записывает книги в нашей системе инвентаризации. Применительно к каждой книге могут быть записаны название, количество, дата публикации, а также ссылки на информацию об авторах, издателях и другие соответствующие таблицы. Мы могли бы поставить индекс на все столбцы и даже добавить индексы для данных в соответствующие таблицы. Проблема такого подхода в том, что каждый индекс должен храниться и сопровождаться для каждого оператора, вызывающего изменения. Проектировщикам баз данных следует тщательно планировать и анализировать БД, чтобы определить оптимальную комбинацию добавления и, что не менее важно, не добавлять индексы и другие конструкции в таблицы.



Индекс таблицы можно представить как виртуальную таблицу поиска, которая обеспечивает реляционную базу данных быстрым инструментом поиска данных.

OLAP

Ожидается, что базы данных, разработанные с использованием OLAP, будут содержать больше операторов `SELECT`, чем вызывающих изменения операторов. Эти базы обычно имеют консолидированное представление данных из одной или нескольких баз. Поэтому эти БД обычно не являются главными, а используются для предоставления отчетности и анализа отдельно от главной базы. В некоторых ситуациях это обеспечивается инфраструктурой, изолированной таким образом, чтобы не влиять на работу оперативных баз. Данный тип развертывания часто называют *хранилищем данных*.

Хранилище данных может использоваться для предоставления консолидированного представления о системе или о наборе систем внутри предприятия. Данные традиционно подаются с помощью более медленных периодических задач обновления данных из других систем, но в современных системах баз данных есть тенденция к консолидации в режиме, близком к реальному времени.

Основное различие между OLTP и OLAP в том, как данные хранятся и организуются. Во многих ситуациях это потребует таблиц или постоянных представлений — в зависимости от используемой технологии, — созданных в базе данных OLAP, поддерживающей определенные сценарии отчетности и дублирующей данные. В базах OLTP дублирование данных нежелательно, поскольку при этом вводится несколько таблиц, которые необходимо сопровождать для одного оператора, вызывающего изменения.

Базы данных в стиле бухгалтерской книги

Мы более подробно рассмотрим проектирование базы данных в стиле бухгалтерской книги (*ledger-style*), так как это одновременно и паттерн, который использовался во многих финансовых БД десятилетиями, и он может быть неизвестен некоторым разработчикам. Эта база, как видно из названия, подобна бухгалтерской книге, где операции добавляются в документ, а количество и/или суммы подсчитываются, чтобы получить окончательное количество или сумму. В таблице ниже приведена бухгалтерская книга с примером продажи яблок (рис. 11.1).

В этом примере есть пара вещей, на которые стоит обратить внимание. Информация о *Purchaser* записывается отдельными строками вместо стирания цены и ввода новой суммы. Возьмите две покупки и один кредит для *West Country Produce*. Обычно это отличается от многих баз данных, в которых одна строка содержит информацию о *Purchaser* с отдельными полями для *Amount* и *Price*.

Date	Purchaser	Amount	Price
5/05/2019	West Country Produce	100	\$ 200.00
5/05/2019	Jim's Freshest	30	\$ 60.00
5/07/2019	West Country Produce	-25	-\$ 50.00
5/07/2019	Marlborough Fair	70	\$ 140.00
5/07/2019	West Country Produce	12	\$ 30.00
5/07/2019	Marlborough Fair	15	\$ 30.00
6/07/2019	Coopers Collective	80	\$ 160.00
		282	\$ 570.00

Рис. 11.1

База данных в стиле бухгалтерской книги использует эту концепцию, имея отдельную строку для каждой транзакции, таким образом не применяя операторы **UPDATE** и **DELETE** и полагаясь только на операторы **INSERT**. Описанный подход имеет несколько преимуществ. Как и в случае с бухгалтерской книгой, после записи каждой проводки ее нельзя удалить или изменить. Если происходит ошибка или изменение, то для достижения желаемого состояния необходимо написать новую транзакцию. Интересным преимуществом такого подхода является то, что исходная таблица теперь ценна сама по себе для предоставления подробного журнала деятельности. Если бы мы добавили столбец **modified by**, то имели бы подробный журнал о том, кто или что внесло изменения, а также каким оно было.



Этот пример предназначен для бухгалтерской книги с одинарной записью, но в реальном мире будет использоваться бухгалтерская книга с двойной записью. Разница в том, что в такой книге каждая операция отражается как кредит в одной таблице и дебет в другой.

Следующая задача — запечатлеть окончательную или свернутую версию таблицы. В данном примере это купленное количество яблок и их цена. Первый подход может использовать оператор **SELECT**, просто выполняющий **GROUP BY** для покупателя, как показано ниже:

```
SELECT Purchaser, SUM(Amount), SUM(Price)
FROM Apples
GROUP BY Purchaser
```

Хотя это было бы хорошо для данных небольших размеров, проблема здесь заключается в том, что производительность запроса снижается с течением времени по мере увеличения количества строк. Альтернативой было бы объединить данные в другую форму. Есть два основных способа сделать это. Первый заключается в выполнении данного действия одновременно с записью информации из таблицы бухгалтерской книги в другую (или материализованное представление, если поддерживается), которая содержит данные в агрегированной форме.



Персистентное или материализованное представление похоже на представление базы данных, но его результаты кэшируются. Это дает нам преимущество: мы не требуем, чтобы представление пересчитывалось при каждом запросе, и оно обновляется либо периодически, либо при изменении базы.

Второй подход основан на том, что мы не привязываемся к оператору `INSERT` для получения агрегированного представления, когда это необходимо. В некоторых системах основной сценарий записи изменений в таблицу и получения результата выполняется реже. В этом случае было бы разумнее оптимизировать базу таким образом, чтобы операции записи были быстрее операций чтения, и тем самым ограничить объем обработки, необходимый при вставке новых записей.

Следующий раздел посвящен интересному паттерну CQRS, который может применяться на уровне базы данных. Это может использоваться при проектировании БД в стиле книги.

Реализация паттерна CQRS

CQRS работает над разделением запросов (для чтения) и команд (для изменения). *Разделение команд и запросов* (command-query separation, CQS) — это подход к объектно-ориентированному проектированию (Object-Oriented Design, OOD).



CQRS впервые был представлен Бертраном Мейером (ru.wikipedia.org/wiki/Мейер,_Бертран). Он упомянул этот термин в своей книге *Object-Oriented Software Construction* в конце 1980-х годов: www.amazon.in/Object-Oriented-Software-Construction-Prentice-hall-International/dp/0136291554.

CQRS хорошо подходит в ряде случаев и обладает некоторыми полезными свойствами.

- ❑ *Разделение модели* — в терминах моделирования мы можем иметь несколько представлений для нашей модели данных. Четкое разделение позволяет выбирать одни фреймворки или методы для запросов, а другие — для команд. Возможно, это достижимо с помощью сущностей типа *Create, Read, Update* и *Delete* (CRUD), хотя в этом случае часто возникает единственная сборка уровня данных.
- ❑ *Взаимодействие* — на некоторых предприятиях разделение между запросом и командой будет полезно для разработчиков, участвующих в создании сложных систем, особенно когда некоторые сотрудники больше сосредоточены на различных аспектах некой сущности. Например, команда, которая больше

заботится о представлении, может сфокусироваться на модели запроса, в то время как другая команда, больше сосредоточенная на целостности данных, может сопровождать модель команды.

- ❑ *Независимая масштабируемость* — многие решения, как правило, требуют либо большего количества операций чтения, по сравнению с моделью, либо большего количества операций записи, в зависимости от требований бизнеса.



Применяя CQRS, помните, что команды обновляют данные, а запросы считывают их.

При работе над CQRS следует отметить некоторые важные вещи:

- ❑ команды должны быть асинхронными, а не синхронными операциями;
- ❑ никогда не следует изменять базы данных с помощью запросов.

CQRS упрощает проектирование с помощью отдельных команд и запросов. Кроме того, мы можем физически отделить операции чтения данных от операций их записи. В этом случае база данных чтения может использовать отдельную схему базы. Другими словами, применять оптимизированную для запросов БД только для чтения.

Поскольку база данных использует подход физического разделения, мы можем визуализировать поток CQRS приложения, как показано на рис. 11.2.

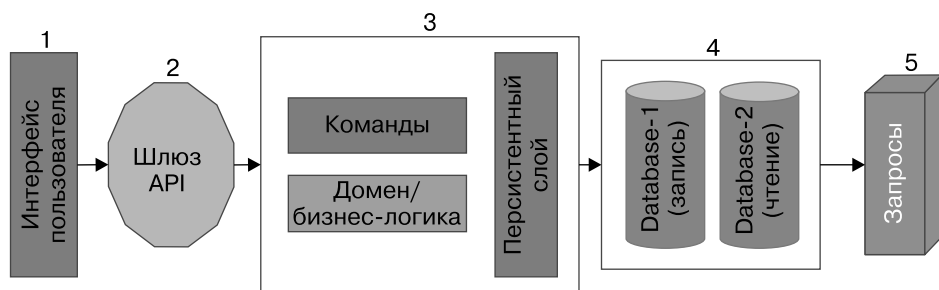


Рис. 11.2

Здесь показан воображаемый рабочий процесс приложения CQRS, в котором оно имеет физически отдельные базы данных для операций записи и чтения. Это воображаемое приложение на основе веб-сервисов RESTful (.NET Core API). Ни один API не был представлен непосредственно клиенту/конечному пользователю, применяющему эти API. Существует шлюз API, открытый для пользователей. Любые запросы к приложению будут поступать через него.



Шлюз API предоставляет точку входа для групп с аналогичными типами сервисов. Вы также можете смоделировать его с помощью паттерна «Фасад», который является частью распределенной системы.

На рис. 11.2 показано следующее:

- ❑ *пользовательский интерфейс* — любой клиент (который использует API), веб-приложение, настольное приложение, мобильное приложение или любое другое;
- ❑ *шлюз API* — любой запрос от пользовательского интерфейса и ответ на него поставляется из данного шлюза. Это основная часть CQRS, поскольку бизнес-логика может быть включена с помощью команд и уровней персистентности;
- ❑ *базы данных* — на диаграмме показаны две физически разделенные БД. В реальных приложениях это зависит от требований продукта. Вы можете использовать базу для операций как записи, так и чтения;
- ❑ запросы генерируются с помощью операций *Read*, которые являются объектами переноса данных (data transfer objects, DTOs).

Теперь вы можете вернуться к разделу «Обсуждение примера использования», в котором мы обсуждали новые функции/расширения нашего приложения для инвентаризации FlixOne. Здесь же мы создадим новое приложение FlixOne с функциями, описанными ранее, с помощью паттерна CQRS. Пожалуйста, обратите внимание: сначала мы будем разрабатывать API. Если вы не установили предварительные требования, то я предлагаю вернуться к разделу «Технические требования», собрать все необходимое программное обеспечение и установить его на свой компьютер. Если вы выполнили предварительные условия, то начнем со следующих шагов.

1. Запустите программу Visual Studio.
2. Выберите команду меню **File ► New Project** (Файл ► Новый проект), чтобы создать новый проект.
3. В окне **New Project** (Новый проект) выберите пункт **Web** и затем **ASP.NET Core Web Application** (Веб-приложение ASP.NET Core).
4. Присвойте название проекту. Мы назвали его **FlixOne.API** и указали *название решения* — **FlixOne**.
5. Выберите расположение (Location) папки **Solution**, нажмите кнопку **OK**, как показано на рис. 11.3.
6. Теперь вы увидите экран **New ASP.NET Web Core Application - FlixOne.API**. Выберите в раскрывающемся списке пункт **ASP.NET Core 2.2**. В числе доступных шаблонов выберите **Web Application (Model-View-Controller)** и снимите флажок **Configure for HTTPS**, как показано на рис. 11.4.

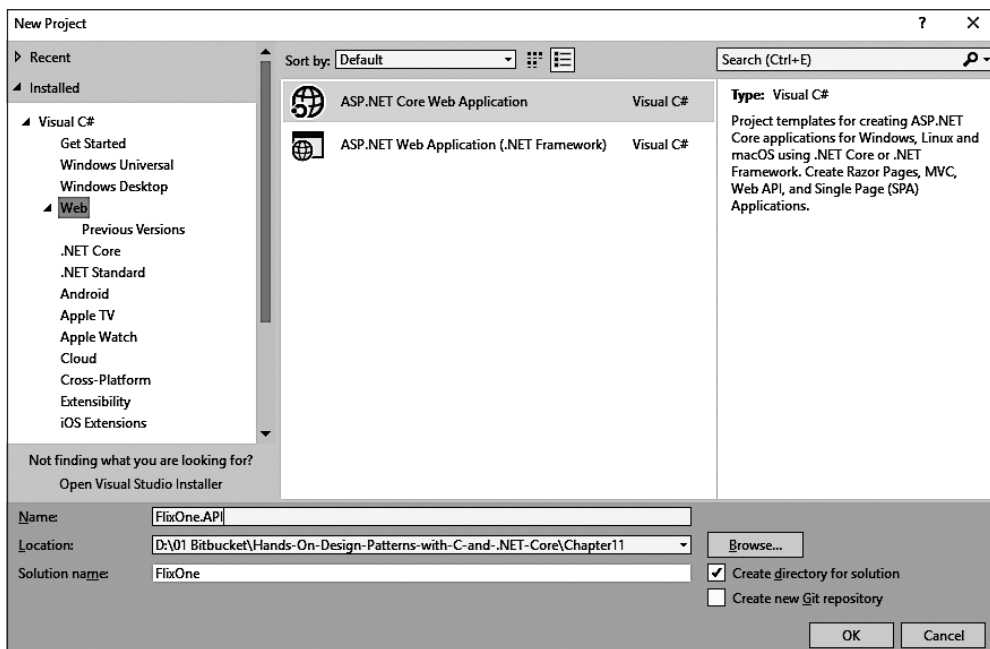


Рис. 11.3

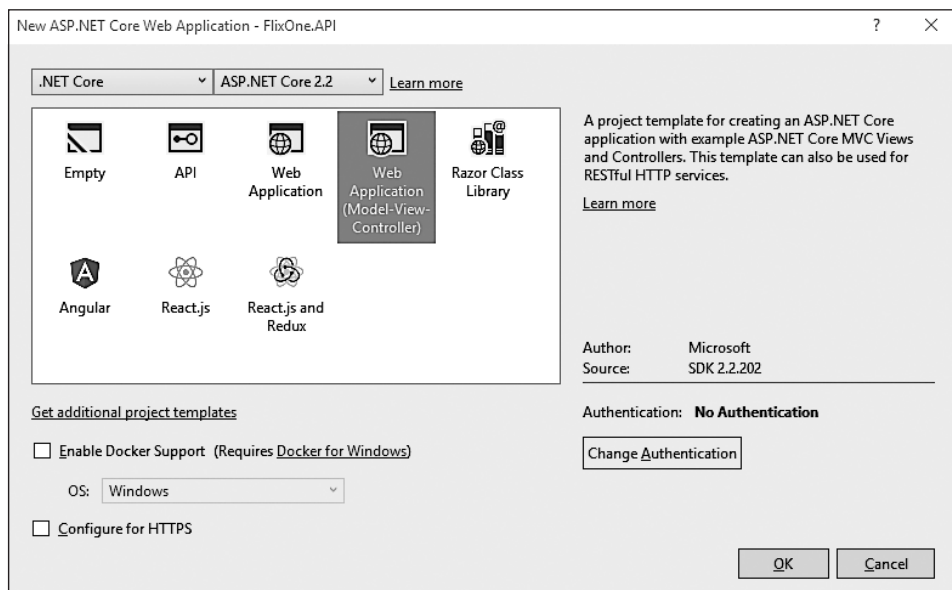


Рис. 11.4

7. Вы увидите страницу по умолчанию, как показано на рис. 11.5.

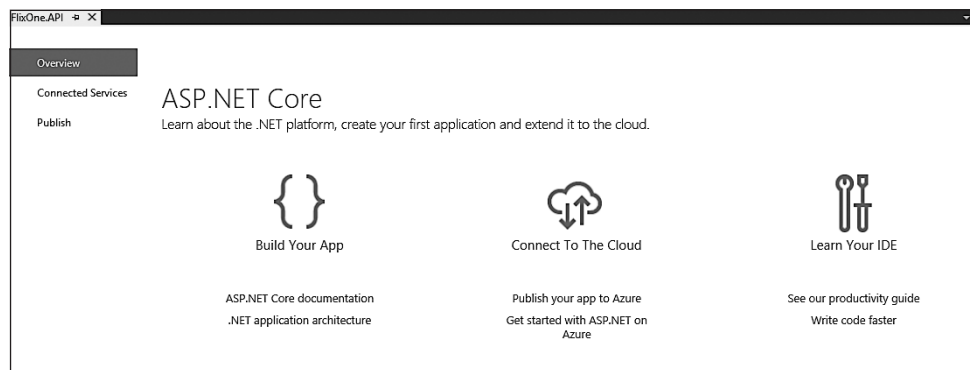


Рис. 11.5

8. Разверните панель Solution Explorer (Обозреватель решений) и нажмите кнопку Show All files (Показать все файлы). Вы увидите папки/файлы по умолчанию, созданные Visual Studio. Посмотрите на рис. 11.6.

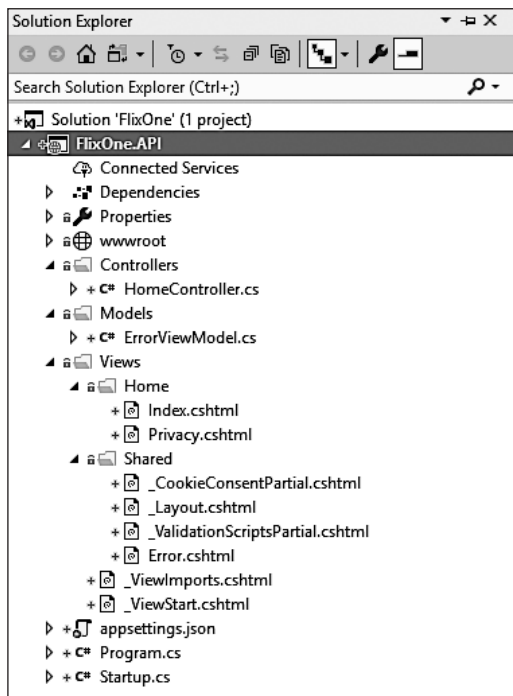


Рис. 11.6

Мы выбрали шаблон *ASP.NET Core Web (Model-View-Controller)*. Как следствие, у нас есть папки по умолчанию, *Controllers*, *Models* и *Views*. Это шаблон по умолчанию, предоставляемый Visual Studio. Чтобы проверить его, нажмите клавишу F5 и запустите проект. Вы увидите следующую страницу по умолчанию (рис. 11.7).

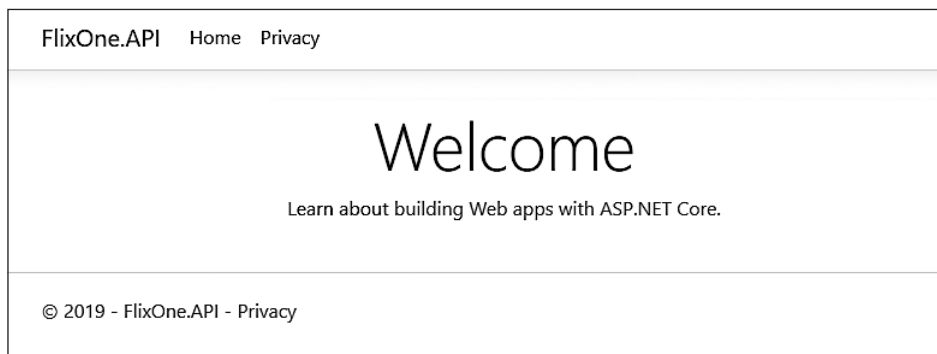


Рис. 11.7

Здесь показан экран *Home* (Главная) по умолчанию нашего веб-приложения. Возможно, вы думаете, что это сайт, и ожидаете здесь страницу документации API вместо веб-страницы? Так происходит потому, что при выборе шаблона Visual Studio по умолчанию добавляет контроллер MVC вместо контроллера API. Пожалуйста, обратите внимание: в ASP.NET Core оба контроллера — MVC и API — используют один и тот же конвейер контроллера (см. класс контроллера: docs.microsoft.com/ru-ru/dotnet/api/microsoft.aspnetcore.mvc.controller?view=aspnetcore-2.2).

Прежде чем подробно обсуждать проекты API, сначала добавим новый проект в решение FlixOne. Для этого разверните панель *Solution Explorer* (Обозреватель решений), щелкните правой кнопкой мыши на имени решения и выберите *Add ► New Project* (Добавить ► Новый проект) (рис. 11.8).

В окне *New Project* (Новый проект) добавьте новый проект *FlixOne.CQRS* и нажмите кнопку *OK* (рис. 11.9).

На данном рисунке показано диалоговое окно *Add New Project* (Добавить новый проект). В нем выберите вариант *.NET Core*, а затем проект *Class Library(.NET Core)*. Введите имя *FlixOne.CQRS* и нажмите кнопку *OK*. В решение будет добавлен новый проект. Затем вы можете добавить папки в новое решение, как показано на рис. 11.10.

На данном рисунке видно, что я добавил четыре новые папки: *Commands*, *Queries*, *Domain* и *Helper*. В папке *Commands* есть подпапки *Command* и *Handler*. Аналогично в папку *Queries* я добавил вложенные папки *Handler* и *Query*.

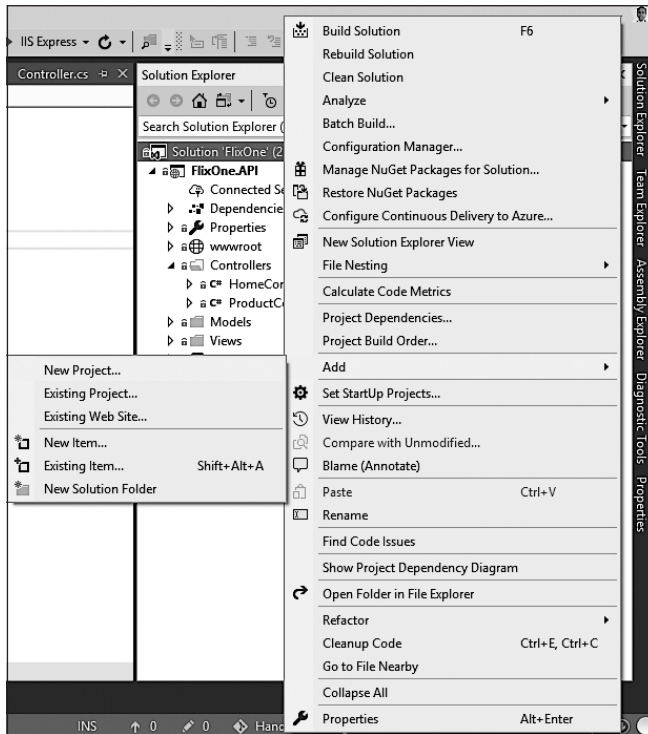


Рис. 11.8

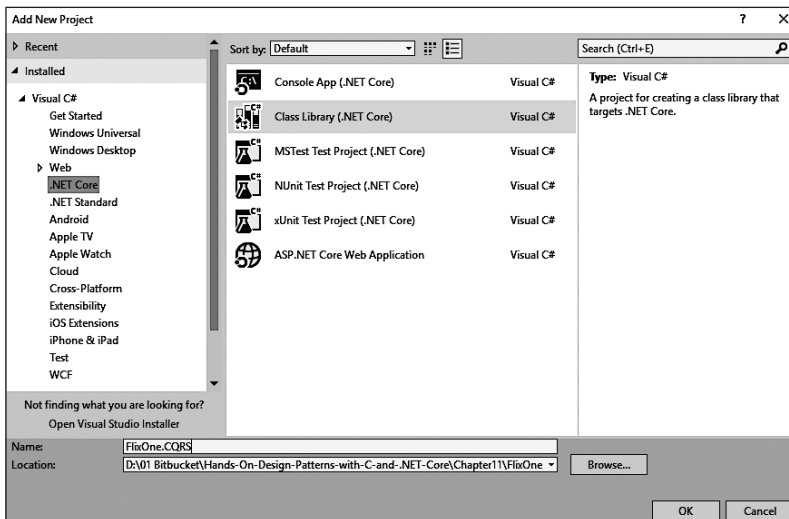


Рис. 11.9

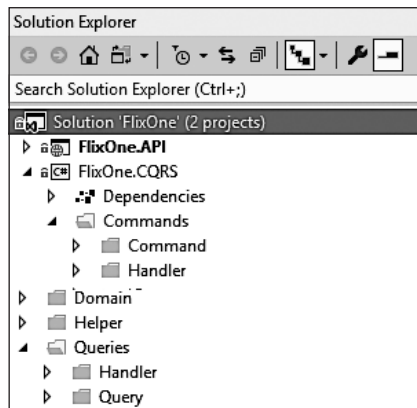


Рис. 11.10

Чтобы начать работу с проектом, сначала добавим в него две доменные сущности. Ниже приведен необходимый код:

```
public class Product
{
    public Guid Id { get; set; }
    public string Name { get; set; }
    public string Description { get; set; }
    public string Image { get; set; }
    public decimal Price { get; set; }
}
```

Предыдущий код — это доменная сущность **Product** со следующими свойствами:

- ❑ **Id** — уникальный идентификатор;
- ❑ **Name** — название продукта;
- ❑ **Description** — описание продукта;
- ❑ **Image** — изображение изделия;
- ❑ **Price** — цена продукта.

Нам также нужно добавить базу данных **CommandResponse**. Добавление играет важную роль при взаимодействии с базой данных/репозиторием, поскольку гарантирует, что система получит ответ. Ниже приведен код-фрагмент сущностной модели **CommandResponse**:

```
public class CommandResponse
{
    public Guid Id { get; set; }
    public bool Success { get; set; }
    public string Message { get; set; }
}
```

Класс `CommandResponse` содержит такие свойства:

- ❑ `Id` — уникальный идентификатор;
- ❑ `Success` — с помощью значений `True` или `False` класс сообщает, успешна ли операция;
- ❑ `Message` — сообщение в ответ на операцию. Если `Success` возвращает `false`, то это сообщение содержит ошибку.

Теперь пришло время добавить интерфейсы для запроса. Для этого выполните следующие действия.

1. На панели **Solution Explorer** (Обозреватель решений) щелкните правой кнопкой мыши на папке **Queries**, раскройте меню **Add** (Добавить), а затем выберите пункт **New Item** (Новый элемент), как показано на рис. 11.11.

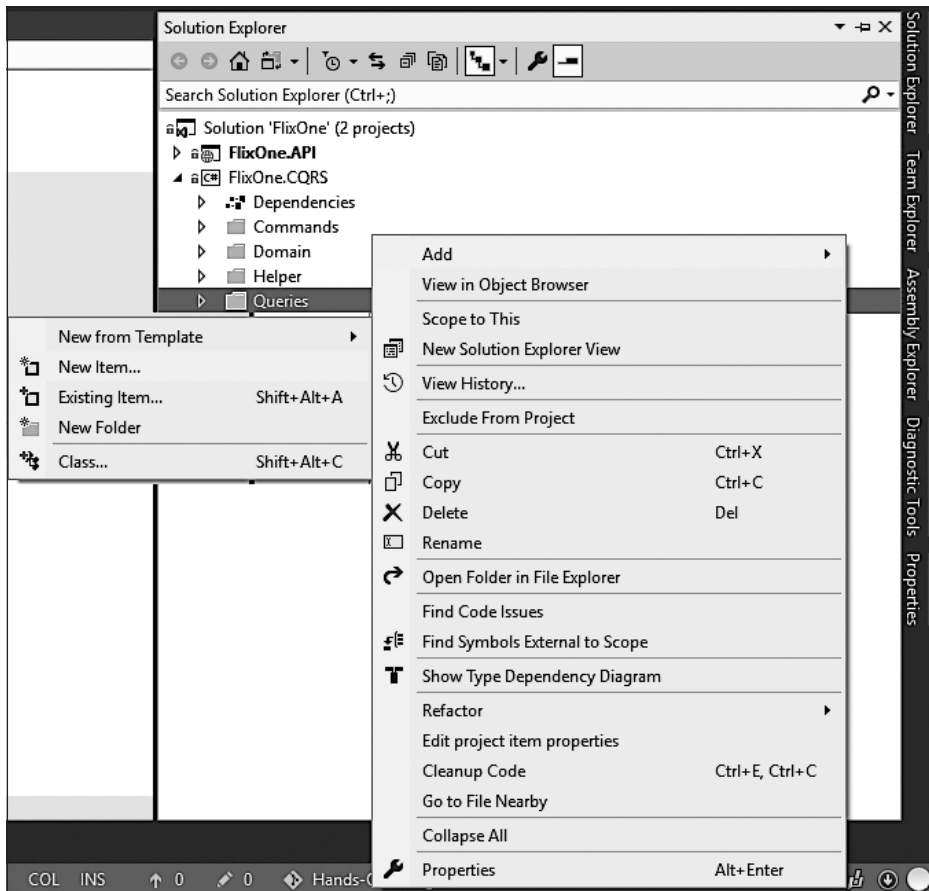


Рис. 11.11

2. В окне Add New Item (Добавить новый элемент) выберите пункт Interface (Интерфейс), присвойте ему имя IQuery и нажмите кнопку Add (Добавить) (рис. 11.12).

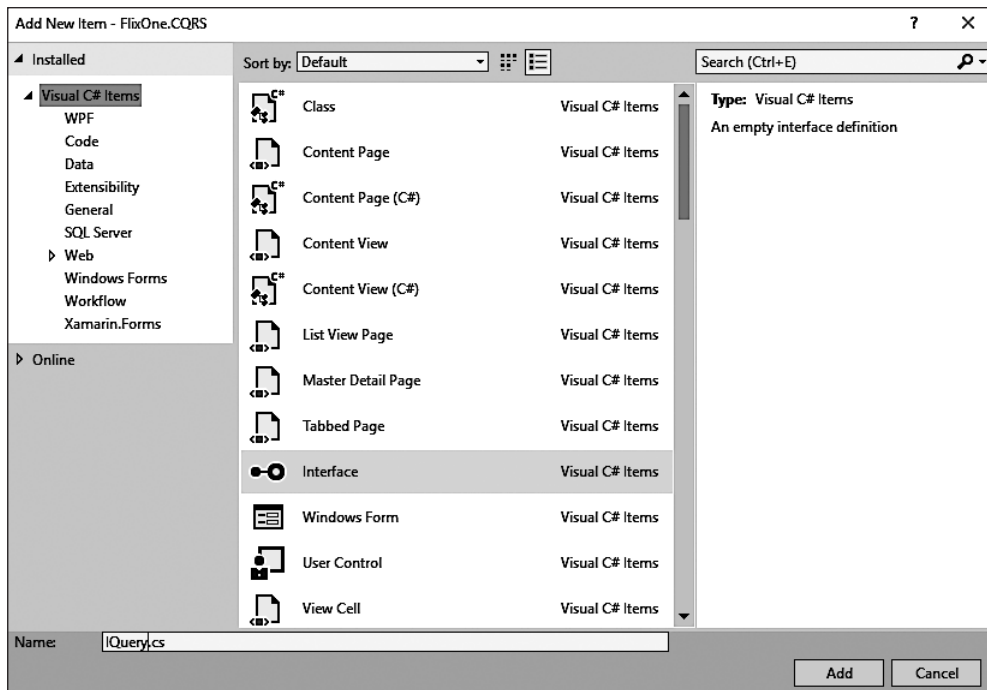


Рис. 11.12

3. Повторите предыдущие шаги и добавьте интерфейс IQueryHandler. Ниже приведен код из интерфейса IQuery:

```
public interface IQuery<out TResponse>
{
}
```

4. Данный интерфейс играет роль каркаса запроса любой операции. Это универсальный интерфейс, использующий параметр типа TResponse.

Ниже приведен код класса ProductQuery:

```
public class ProductQuery : IQuery<IEnumerable<Product>>
{
}

public class SingleProductQuery : IQuery<Product>
{
    public SingleProductQuery(Guid id)
```

```

    {
        Id = id;
    }

    public Guid Id { get; }
}

```

Далее приведен код класса `ProductQueryHandler`:

```

public class ProductQueryHandler : IQueryHandler<ProductQuery,
IEnumerable<Product>>
{
    public IEnumerable<Product> Get()
    {
        // вызов репозитория
        throw new NotImplementedException();
    }
}

public class SingleProductQueryHandler :
    IQueryHandler<SingleProductQuery, Product>
{
    private SingleProductQuery _productQuery;
    public SingleProductQueryHandler(SingleProductQuery productQuery)
    {
        _productQuery = productQuery;
    }

    public Product Get()
    {
        // вызов репозитория
        throw new NotImplementedException();
    }
}

```

Ниже приведен код класса `ProductQueryHandlerFactory`:

```

public static class ProductQueryHandlerFactory
{
    public static IQueryHandler<ProductQuery, IEnumerable<Product>>
        Build(ProductQuery productQuery)
    {
        return new ProductQueryHandler();
    }

    public static IQueryHandler<SingleProductQuery, Product>
        Build(SingleProductQuery singleProductQuery)
    {
        return new SingleProductQueryHandler(singleProductQuery);
    }
}

```

Как и в случае с интерфейсами *Query* и классами *Query*, мы должны добавить интерфейсы для команд и их классов.

Теперь, после создания CQRS для доменной сущности продукта, вы можете следовать этому рабочему процессу и добавлять новые сущности столько раз, сколько захотите. Теперь перейдем к нашему проекту *FlixOne.API* и добавим новый контроллер API, выполнив следующие шаги.

1. На панели *Solution Explorer* (Обозреватель решений) щелкните правой кнопкой мыши по папке *Controllers*.
2. Выберите команду меню *Add ► New Item* (Добавить ► Новый элемент).
3. Выберите пункт *API Controller Class* и присвойте название *ProductController* (рис. 11.13).

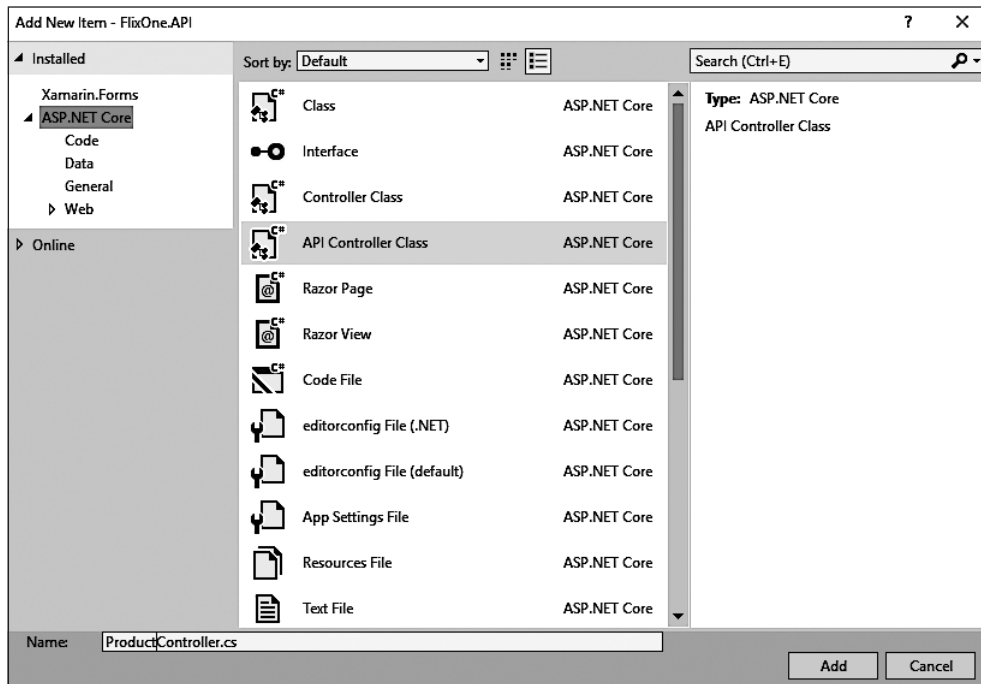


Рис. 11.13

4. Добавьте код, показанный ниже, в контроллер API:

```
[Route("api/[controller]")]
public class ProductController : Controller
{
```

```
// GET: api/<controller>
[HttpGet]
public IEnumerable<Product> Get()
{
    var query = new ProductQuery();
    var handler = ProductQueryHandlerFactory.Build(query);
    return handler.Get();
}

// GET api/<controller>/5
[HttpGet("{id}")]
public Product Get(string id)
{
    var query = new SingleProductQuery(id.ToValidGuid());
    var handler = ProductQueryHandlerFactory.Build(query);
    return handler.Get();
}
```

Следующий код предназначен для сохранения продукта:

```
// POST api/<controller>
[HttpPost]
public IActionResult Post([FromBody] Product product)
{
    var command = new SaveProductCommand(product);
    var handler = ProductCommandHandlerFactory.Build(command);
    var response = handler.Execute();
    if (!response.Success) return StatusCode(500, response);
    product.Id = response.Id;
    return Ok(product);
}
```

Этот код написан для удаления продуктов:

```
// DELETE api/<controller>/5
[HttpDelete("{id}")]
public IActionResult Delete(string id)
{
    var command = new DeleteProductCommand(id.ToValidGuid());
    var handler = ProductCommandHandlerFactory.Build(command);
    var response = handler.Execute();
    if (!response.Success) return StatusCode(500, response);
    return Ok(response);
}
```

Мы создали API продукта. В этом разделе мы не собираемся создавать пользовательский интерфейс. Чтобы посмотреть на сделанное нами, добавим поддержку *Swagger* в проект API.

Swagger — это инструмент, который может использоваться для документирования, он предоставляет всю информацию о конечных точках API на одном экране, где можно визуализировать API и протестировать его с нужными параметрами.

Чтобы начать внедрение Swagger в проект API, выполните следующие шаги.

1. Откройте менеджер пакетов Nuget.
2. Выберите команду Nuget Package Manager ► Browse (Менеджер пакетов Nuget ► Поиск) и выполните поиск по запросу Swashbuckle.AspNetCore (рис. 11.14).

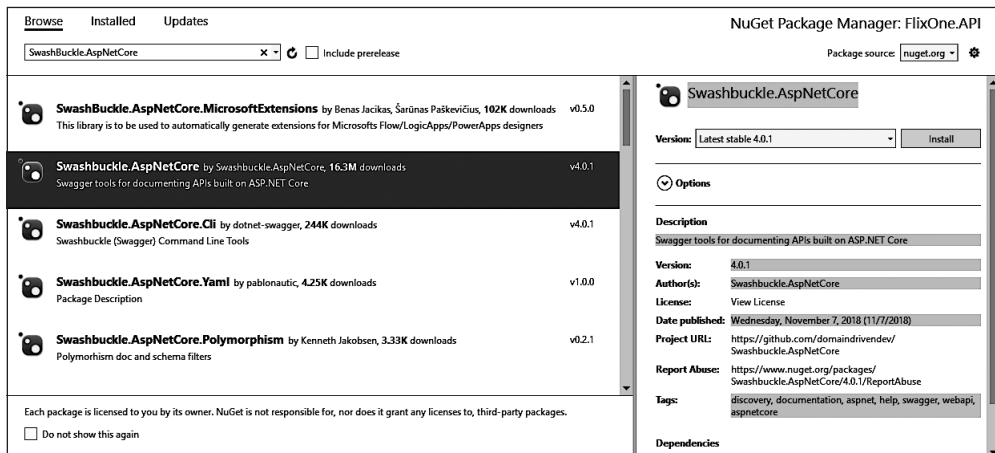


Рис. 11.14

3. Откройте файл Startup.cs и добавьте код, показанный ниже, в метод ConfigureServices:

```
// регистрация Swagger
services.AddSwaggerGen(swagger =>
{
    swagger.SwaggerDoc("v1", new Info { Title = "Product APIs",
        Version = "v1" });
});
```

4. Теперь добавьте код, показанный ниже, в метод Configure:

```
// Включение middleware для выдачи
// сгенерированного Swagger JSON в качестве конечной точки
app.UseSwagger();

// Включение middleware для выдачи
// swagger-ui (HTML, JS, CSS и т. д.),
// определение конечной точки Swagger JSON
```

```
app.UseSwaggerUI(c =>
{
    c.SwaggerEndpoint("/swagger/v1/swagger.json", "Product API V1");
});
```

Теперь мы внесли все изменения, служащие для демонстрации возможностей CQRS в приложении. В Visual Studio нажмите клавишу F5 и откройте страницу документации Swagger, перейдя по адресу `localhost:52932/swagger/`. Обратите внимание: номер порта 52932 может варьироваться в зависимости от ваших настроек проекта. Вы увидите следующую страницу документации *Swagger* (рис. 11.15).

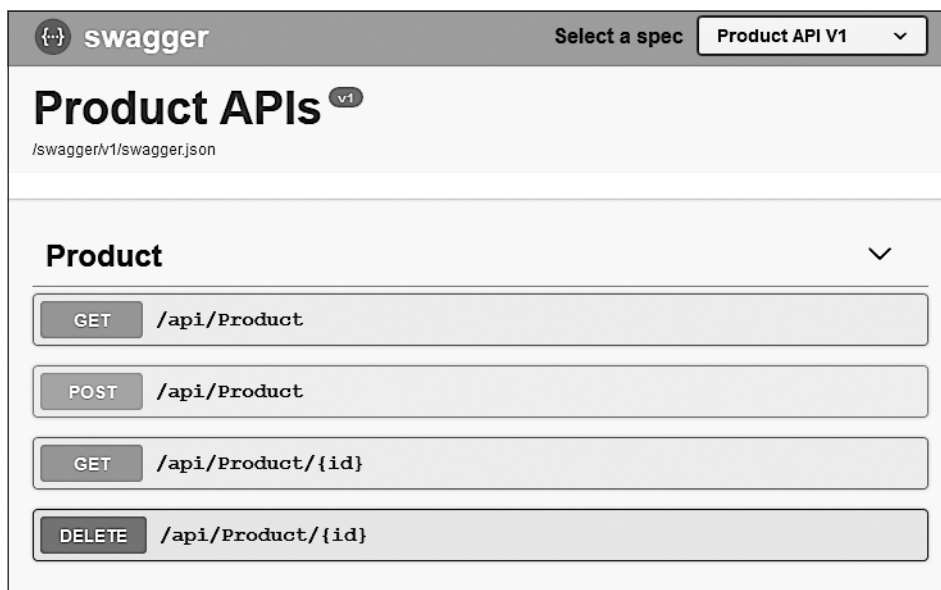


Рис. 11.15

Здесь вы можете протестировать API продукта.

Резюме

В этой главе представлен паттерн CQRS, который мы внедрили в приложение. Цель главы состояла в изучении методов работы с базами данных и того, как работают базы в стиле бухгалтерской книги для систем инвентаризации. Чтобы продемонстрировать мощь CQRS, мы создали API продукта и добавили поддержку документации Swagger.

В следующей главе мы обсудим облачные сервисы, а также подробно рассмотрим микросервисы и бессерверные технологии.

Вопросы

Следующие вопросы позволят вам усвоить информацию, представленную в этой главе.

1. Что такое база данных в стиле бухгалтерской книги?
2. Что такое CQRS?
3. Когда мы должны использовать CQRS?

12

Разработка облачных приложений

В предыдущих главах мы исследовали различные паттерны, начиная от низкоуровневых концепций, таких как «Одиночка» и «Фабрика», и заканчивая теми, которые предназначены для определенных технологий вроде баз данных и веб-приложений. Эти паттерны являются незаменимыми в проектировании решений, обеспечивая их сопровождаемость и эффективную реализацию. Они предоставляют прочный фундамент, который позволяет улучшать и модифицировать ваши приложения по мере изменения требований и добавления новой функциональности.

В этой главе рассмотрим программные решения в более общем виде, обращая внимание на разные аспекты проектирования и реализации надежных, масштабируемых и безопасных приложений. Паттерны, с которыми вы здесь познакомитесь, применяются в средах с множеством приложений, хранилищем данных и целым рядом потенциальных инфраструктурных конфигураций.

Индустрия программного обеспечения постоянно развивается, и эти изменения несут с собой как новые возможности, так и новые вызовы. В этой главе мы обсудим разные паттерны проектирования облачного ПО. Многие из них не новы и изначально применялись в локальных корпоративных средах. И теперь, когда облачно-ориентированные решения становятся нормой, эти паттерны приобретают еще более широкое распространение, позволяя создавать системы, которые не полагаются на внутреннюю инфраструктуру организаций.



Облачно-ориентированные решения предназначены для использования облачных вычислительных ресурсов. Но существуют также гибридные системы, способные работать одновременно и в облаке, и в частном центре обработки данных.

В этой главе определяется пять ключевых аспектов разработки облачных решений:

- ☐ масштабируемость;
- ☐ доступность;
- ☐ безопасность;

- ❑ архитектура приложений;
- ❑ DevOps.

Мы обсудим каждый из них и объясним, какую роль они играют в построении облачных систем. По ходу этого обсуждения будут описаны разные паттерны, которые можно применять для решения соответствующих проблем.

Технические требования

Эта глава в основном носит теоретический характер, поэтому у нее нет никаких специальных технических требований или примеров с исходным кодом.

Ключевые моменты, которые следует учитывать при разработке облачных решений

Планирование перехода в облако сопряжено с рядом проблем и вызовов. В этом разделе мы рассмотрим пять ключевых аспектов, которые следует учитывать при создании облачных решений. С ними можно столкнуться и в других областях, но при переходе в облако они требуют к себе особого внимания ввиду широкого диапазона доступных технологий и продуктов.

Вот эти ключевые аспекты.

- ❑ *Масштабируемость*. Позволяет развивающейся компании справляться с растущими нагрузками или объемами трафика.
- ❑ *Устойчивость/доступность*. Обеспечивают надлежащую обработку сбоев в системе с минимальным воздействием на пользователей.
- ❑ *Безопасность*. Гарантирует сохранность частных и коммерческих данных, а также их защиту от взломов и атак.
- ❑ *Архитектура*. Этот пункт относится к особым архитектурным требованиям, которые свойственны облачно-ориентированным решениям.
- ❑ *DevOps*. Это набор инструментов и рекомендованных методик для разработки и обслуживания облачно-ориентированных решений.

То, какими из этих аспектов вам нужно заниматься, зависит от ваших бизнес-требований. Кроме того, преследуя интересы своей компании, вы должны позаботиться о непредвиденных ситуациях, воспользовавшись услугами подходящих провайдеров.

В следующих разделах мы подробно обсудим эти вопросы и предложим для их решения доступные паттерны.



Мы рассмотрели разные паттерны: как специфичные для различных технологий, так и архитектурные и бизнес-процессные. Кроме того, один и тот же паттерн может покрывать сразу несколько проблем.

Масштабируемость

Масштабируемость — это способность выделять и администрировать ресурсы, используемые приложением, для поддержания приемлемого качества обслуживания в условиях имеющихся рабочих нагрузок. Большинство облачных сервисов предоставляют механизмы для увеличения качества и объема ресурсов приложения. Например, Azure App Service позволяет изменять как размер системы, так и количество ее экземпляров.

Масштабируемость можно рассматривать как спрос на ограниченные ресурсы. Речь идет о дисковом пространстве, оперативной памяти, пропускной способности сети и других аспектах программного обеспечения, которые поддаются измерению. Спрос может определяться количеством пользователей, параллельных соединений и другими характеристиками, требующими определенного объема ресурсов. По мере увеличения спроса растет и нагрузка на приложение. И, если это влияет на его производительность, мы можем говорить о нехватке ресурсов (то есть какой-то ресурс является узким местом приложения).

Об этом, к примеру, можно судить по тому, скольких пользователей способно обслужить приложение, прежде чем его производительность начнет ухудшаться. Мы можем решить, что среднее время ожидания запроса не должно превышать две секунды. Таким образом при наплыве пользователей мы можем анализировать нагрузку на систему и определять отдельные узкие места, которые сказываются на ее производительности.

Рабочая нагрузка

Чтобы найти эффективное решение для проблем с масштабируемостью, необходимо понимать, с какими рабочими нагрузками будет сталкиваться ваша система. Существует четыре основных типа рабочих нагрузок: статические, периодические, уникальные и непредсказуемые.

Статическая рабочая нагрузка представляет постоянный уровень активности в системе. Поскольку активность не колеблется, такого рода системам не требуется слишком уж эластичная инфраструктура.

Периодическая рабочая нагрузка характерна для систем, активность которых меняется предсказуемым образом. В качестве примера можно привести систему, которая испытывает всплеск активности на выходных или в период оформления

налоговых деклараций. Когда нагрузка увеличивается, можно выделять дополнительные ресурсы, чтобы поддерживать желаемый уровень качества обслуживания; при снижении нагрузки ресурсы можно сокращать.

Уникальные рабочие нагрузки свойственны системам, созданным под какое-то определенное событие. Они развертываются для выполнения конкретных задач и сворачиваются, когда в них больше нет нужды.

Системы с непредсказуемыми рабочими нагрузками часто выигрывают от автоматического масштабирования, о котором упоминалось ранее. Они демонстрируют большие колебания активности, которые либо все еще плохо понятны руководству компании, либо обусловлены другими факторами.

Понимание и проектирование облачно-ориентированного приложения с учетом подходящей рабочей нагрузки играет ключевую роль как в поддержании высокого уровня производительности, так и в оптимизации расходов.

Паттерны

Мы можем предложить три паттерна проектирования и один архитектурный паттерн для обеспечения масштабируемости ваших систем:

- ☐ вертикальное масштабирование;
- ☐ горизонтальное масштабирование;
- ☐ автоматическое масштабирование;
- ☐ микросервисы.

Давайте подробно рассмотрим каждый из них.

Вертикальное масштабирование

Конечно, мы можем вручную расширять свои корпоративные серверы за счет дополнительной оперативной памяти или дисковых накопителей, но большинство облачных провайдеров позволяют с легкостью увеличивать и уменьшать вычислительную мощность системы. И зачастую это можно делать с минимальными простоями или вовсе без таковых. Это так называемое вертикальное масштабирование, которое относится к изменению таких ресурсов, как тип центрального процессора, объем и качество оперативной памяти или размер/скорость диска.



Вертикальное масштабирование относится к объему ресурсов, а горизонтальное — к количеству серверов.

Горизонтальное масштабирование

Горизонтальное масштабирование отличается от вертикального тем, что вместо изменения размера системы мы меняем количество ее экземпляров. Например, веб-приложение может работать на одном сервере с 4 Гбайт оперативной памяти и двумя процессорами. Если обновить этот сервер до 8 Гбайт памяти и четырех процессоров, это будет вертикальное масштабирование. Но, если вместо этого добавить еще два сервера с той же конфигурацией (4 Гбайт оперативной памяти и два процессора), такое масштабирование будет горизонтальным.

Для выполнения горизонтального масштабирования можно использовать ту или иную форму балансировки нагрузки, которая распределяет запросы между экземплярами системы. Это проиллюстрировано на рис. 12.1.

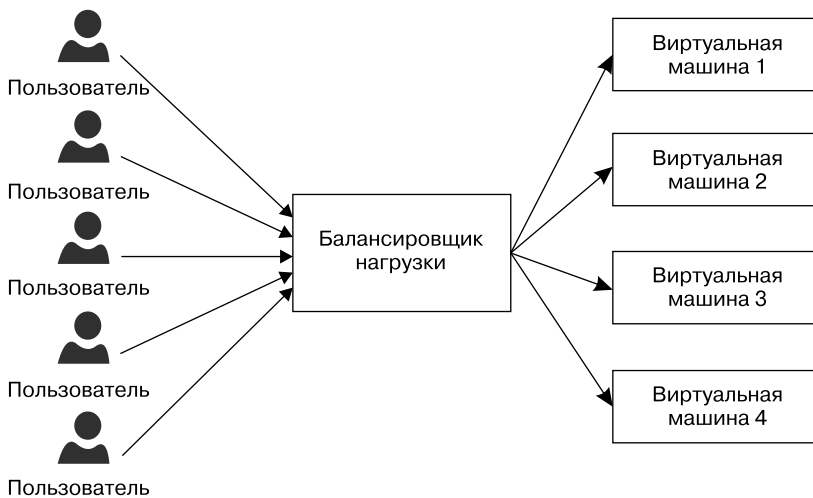


Рис. 12.1

В облачных решениях предпочтение обычно отдается горизонтальному масштабированию. Это объясняется тем, что определенный уровень производительности дешевле обеспечить за счет нескольких небольших виртуальных машин, чем с помощью одного крупного сервера.

Чтобы горизонтальное масштабирование было как можно более эффективным, ваша система должна иметь подходящую архитектуру. Например, веб-приложения без так называемых «липких» сессий (sticky sessions) и/или хранения состояния на сервере лучше подходят для этого типа масштабирования. Это вызвано тем, что из-за «липких» сессий запросы пользователя перенаправляются к одной и той же виртуальной машине и со временем нагрузка начинает распределяться неравномерно (и, следовательно, не оптимальным образом).



Приложения с сохранением состояния

Приложение с сохранением состояния (stateful) хранит информацию об активном сеансе на сервере или в базе данных.

Приложения, не хранящие состояние

Приложения, не хранящие состояние (stateless), не нуждаются в сохранении информации об активном сеансе на сервере или в базе данных. Благодаря этому каждый следующий запрос в рамках одного сеанса может передаваться для обработки разным серверам.

Веб-приложениям с сохранением состояния необходимо хранить сеансы или другую информацию в общем хранилище данных. Веб-приложения, состояние которых не сохраняется, имеют более устойчивую архитектуру, в которой запрос может обработать любой сервер в веб-саду или веб-ферме. Таким образом, информация о сеансе не теряется, даже если выходит из строя один из узлов веб-приложения.



Веб-сад — это когда несколько копий одного веб-приложения размещаются на одном сервере. Для сравнения, в веб-ферме копии веб-приложения распределяются по разным серверам. В обоих случаях для предоставления доступа к этим копиям используется маршрутизация, что позволяет им вести себя как единое приложение.

Автоматическое масштабирование

Одним из преимуществ использования облачного провайдера по сравнению с локальными решениями является встроенная поддержка автоматического масштабирования. Эта возможность как одно из побочных свойств горизонтального масштабирования зачастую присутствует в конфигурации облачного сервиса. Например, Azure App Service позволяет настраивать профили автомасштабирования, с помощью которых приложение может реагировать на внешние условия. Пример такого профиля показан на рис. 12.2.

Этот профиль, рассчитанный на выходные дни, будет увеличивать и уменьшать количество экземпляров приложения в зависимости от нагрузки на серверы. Нагрузка определяется тем, насколько занят процессор. Если процессор занят на 60 %, количество экземпляров увеличивается вплоть до 10. Если же загруженность процессора падает ниже 30 %, количество экземпляров может уменьшиться до 2 (но не ниже).



Эластичная инфраструктура дает возможность масштабировать ресурсы вертикально и горизонтально без повторного развертывания или простоя. На самом деле под этим термином понимают степень эластичности, а не ее наличие или отсутствие. Например, эластичный сервис может предусматривать как вертикальное, так и горизонтальное масштабирование, не требуя перезагрузки серверов. В менее эластичном сервисе отсутствие перезагрузки может поддерживаться только при горизонтальном масштабировании, но не при изменении размеров сервера.

Weekday Autoscale

Scale mode ☒ Scale based on a metric ☐ Scale to a specific instance count

Scale out

When (Average) CpuPercentage > 60 Increase count by 2

Scale in

When (Average) CpuPercentage < 30 Decrease count by 1

[+ Add a rule](#)

Instance limits Minimum Maximum Default

Schedule ☐ Specify start/end dates ☒ Repeat specific days

Repeat every ☒ Monday ☒ Tuesday ☒ Wednesday ☒ Thursday ☒ Friday
☐ Saturday ☐ Sunday

Timezone

Start time

End time

Рис. 12.2

Микросервисы

Существуют разные мнения насчет того, что такое микросервисы и какое отношение они имеют к *сервис-ориентированной архитектуре* (service-oriented architecture, или SOA). В этом разделе мы будем считать их разновидностью SOA, а не отдельным архитектурным паттерном. Микросервисная архитектура расширяет SOA за счет введения некоторых ключевых принципов, согласно которым сервис должен:

- ☐ быть небольшим (отсюда и приставка *микро*);
- ☐ разрабатываться вокруг какой-то бизнес-возможности;
- ☐ быть слабо связанным с другими сервисами;
- ☐ поддерживать независимое обслуживание;
- ☐ иметь изолированное состояние.

Малый размер

Микросервисы — это развитие концепции SOA. Они должны иметь как можно меньший размер. Это хорошо сочетается с некоторыми другими паттернами, которые нам уже встречались: *KISS* (Keep It Simple Stupid — «Делай проще, тупица»)

и *YAGNI* (You aren't Gonna Need It — «Вам это не понадобится») из главы 2. Единственная задача микросервиса — выполнять стоящие перед ним требования.

Бизнес-возможность

Разрабатывая сервис вокруг бизнес-возможности, мы придаем нашей реализации такие свойства, благодаря которым ее можно будет обновлять в ответ на изменение бизнес-требований. В результате изменение в одной бизнес-области с меньшей долей вероятности повлияет на другие.

Слабая связанность

Микросервис должен взаимодействовать с другими сервисами, которые его окружают, используя технологически нейтральный протокол, такой как HTTP. Это упрощает интеграцию микросервисов и, что еще важнее, не требует их пересборки при изменении другого сервиса. Но для этого должен существовать общепринятый *сервисный контракт*.



Сервисный контракт — это определение сервиса, доступное другим командам разработки. Оно часто выполнено на языке WSDL (Web Services Description Language — язык описания веб-сервисов), основанном на XML, но существуют и другие популярные языки вроде Swagger.

При реализации микросервиса важно иметь стратегию управления изменениями. Чтобы четко информировать клиентов сервиса об изменениях в контракте, последний должен поддерживать ведение версий.

Например, стратегия микросервиса для хранения каталога книг может выглядеть так.

- ❑ Каждый сервис должен поддерживать ведение версий и содержать определение Swagger.
- ❑ Каждый сервис начинается с версии 1.
- ❑ При обновлении, которое требует изменения контракта, версия увеличивается на 1.
- ❑ Сервис будет поддерживать до трех версий.
- ❑ При внесении изменения в сервис необходимо убедиться в том, что все текущие версии ведут себя как следует.

Несмотря на свою простоту, эта стратегия имеет интересные последствия. Прежде всего команда, сопровождающая сервис, должна заботиться о том, чтобы его изменение не нарушало работу других сервисов. Это делается для того, чтобы разрывание новых возможностей не приводило к отказу существующих. Контракт предусматривает наличие до трех параллельных версий, что позволяет обновлять зависимые сервисы по отдельности.

Независимое обслуживание

Это одна из самых характерных особенностей микросервисов. Возможность сопровождать микросервисы по отдельности дает возможность компании управлять отдельными сервисами без каких-либо последствий для других частей системы. Под обслуживанием и сопровождением мы понимаем как разработку, так и развертывание. Благодаря этому принципу микросервисы можно обновлять и развертывать, не опасаясь, что это повлияет на другие сервисы; к тому же это дает возможность изменять разные части системы с разной интенсивностью.

Изолированное состояние

Изолированное состояние включает в себя как данные, так и другие разделяемые ресурсы, такие как базы данных и файлы. Это еще одна отличительная черта микросервисов. Независимое состояние уменьшает вероятность того, что изменение модели данных в ходе работы над одним сервисом повлияет на другие.

На рис. 12.3 проиллюстрирован более традиционный подход, применяемый в SOA, где разные сервисы используют общую базу данных.

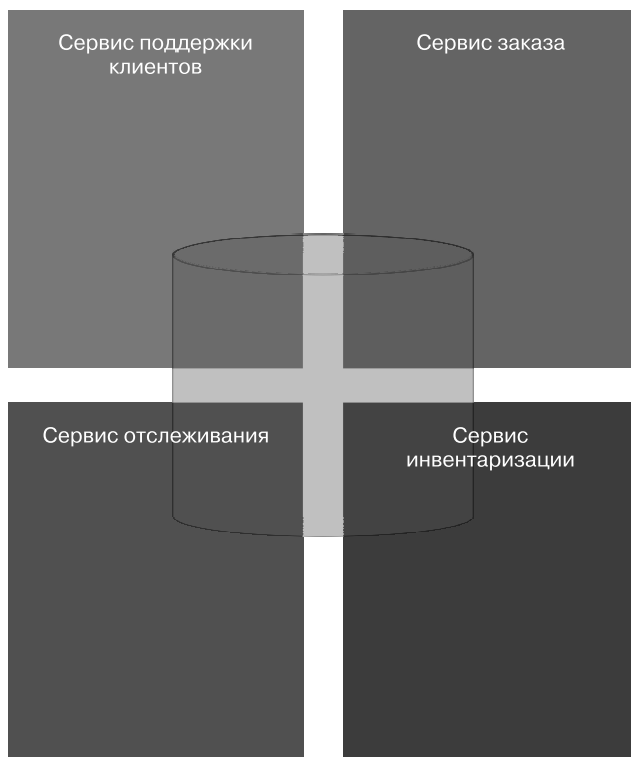


Рис. 12.3

Если в число требований входит изолированное состояние, у каждого микросервиса должна быть своя БД. Это показано на рис. 12.4.

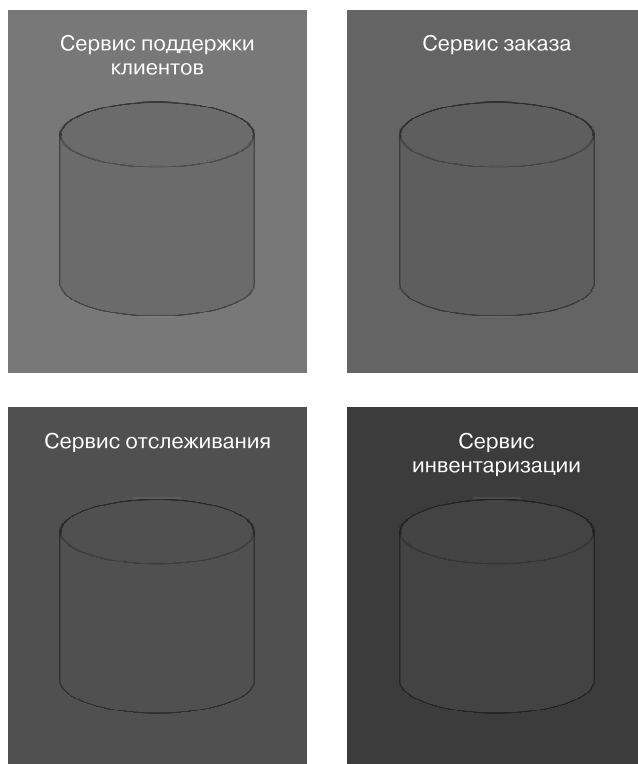


Рис. 12.4

Преимущество этого подхода в том, что для каждого сервиса можно выбрать такие технологии, которые лучше всего соответствуют его требованиям.

Преимущества

Микросервисная архитектура воплощает в себе отказ от традиционного подхода к проектированию сервисов, и она отлично вписывается в облачно-ориентированные решения. Преимущества микросервисов и причина, по которой они набирают популярность, могут показаться не совсем очевидными. Мы поверхностно объяснили, за счет чего эта архитектура позволяет более элегантно управлять изменениями. С технической точки зрения, микросервисы могут масштабироваться независимо друг от друга как сами по себе, так и на уровне базы данных.

Не совсем очевидным может быть то, какую пользу микросервисная архитектура приносит бизнесу. Наличие мелких независимых сервисов позволяет компании применять разные подходы к их обслуживанию и разработке. Каждый отдельный сервис можно развертывать наиболее оптимальным для него образом, возможно даже с использованием разных облачных провайдеров. С другой стороны, изолированность сервисов позволяет сделать процесс их разработки более динамичным. При внесении изменений человеческие ресурсы (то есть разработчиков) можно распределять между сервисами по мере необходимости, и чем меньше сервис, тем меньше предметная область, знание которой требуется для его написания.

Устойчивость/доступность

Устойчивость — это способность приложения как следует справляться с перебоями в работе, а доступность — это количество времени, которое приложение находится в рабочем состоянии. Доступ к приложению может сохраняться, даже если один из его ресурсов становится недееспособным или недоступным.



Если приложение спроектировано таким образом, чтобы продолжать работу при выходе из строя одного или нескольких ресурсов, это называется постепенной деградацией.

Паттерны применяются как для изоляции элементов приложения, так и для обеспечения их взаимодействия, поэтому, когда происходит сбой, его последствия ограничены. Многие паттерны, относящиеся к устойчивости, предназначены для обмена сообщениями между компонентами одного или разных приложений. Паттерн Bulkhead, к примеру, распределяет трафик по изолированным пулам, чтобы переполнение одного пула не имело отрицательного влияния на другие. Существуют паттерны, которые реализуют обмен сообщениями с помощью специальных методик, таких как политика повторных попыток или компенсирующие транзакции.

Доступность играет важную роль во многих облачно-ориентированных приложениях, и обычно ее измеряют согласно *соглашению об уровне обслуживания* (service level agreement, SLA). В большинстве случаев SLA определяет, какую часть времени приложение должно оставаться в рабочем состоянии. Паттерны здесь используются как для резервирования компонентов, так и для ограничения последствий повышения активности. Например, паттерн балансировки нагрузки на основе очередей (Queue-Based Load Leveling) использует очередь, чтобы ограничить потенциальное влияние всплесков активности на приложение, выступая буфером между вызывающей стороной (клиентом) и приложением или сервисом.



В контексте облачных решений устойчивость и доступность являются взаимосвязанными факторами, так как устойчивое приложение позволяет удовлетворить жесткие требования к SLA в отношении доступности.

Паттерны решений

Чтобы гарантировать наличие системы, обладающей устойчивостью и доступностью, лучше всего искать поставщика с определенной архитектурой. Добавьте *событийно-ориентированную архитектуру* (Event-Driven Architecture, EDA).

EDA представляет собой архитектурный паттерн, который использует *события* для управления поведением и активностью системы. Модели решений, имеющиеся в его рамках, помогут нам достичь намеченных решений.

EDA

EDA продвигает концепцию наличия слабо связанных между собой производителей и потребителей в тех случаях, когда производители не имеют точных данных о потребителях. Событие в этом смысле — любое изменение, начиная от входа пользователя в систему для размещения заказа и заканчивая неудачным завершением процесса. EDA хорошо вписывается в распределенные системы и позволяет создавать высокомасштабируемые решения.

Существует много взаимосвязанных схем и подходов к EDA, и схемы, указанные ниже, представлены в этом разделе как имеющие непосредственное отношение к EDA:

- ☐ выравнивание нагрузки на основе очереди;
- ☐ «Издатель — подписчик»;
- ☐ приоритизированная очередь;
- ☐ компенсационная транзакция.

Выравнивание нагрузки на основе очереди

Выравнивание нагрузки на основе очереди — эффективный способ минимизации влияния случаев высокой востребованности на доступность. Вводя очередь между клиентом и сервисом, мы можем ограничивать количество запросов, которые обрабатываются сервисом одновременно. Это позволяет системе работать эффективнее. Возьмем для примера следующую схему (рис. 12.5).

Здесь показан клиент, подавший запрос в очередь на обработку, а результат сохранен в таблице. Очередь работает, чтобы предотвратить внезапный всплеск активности.

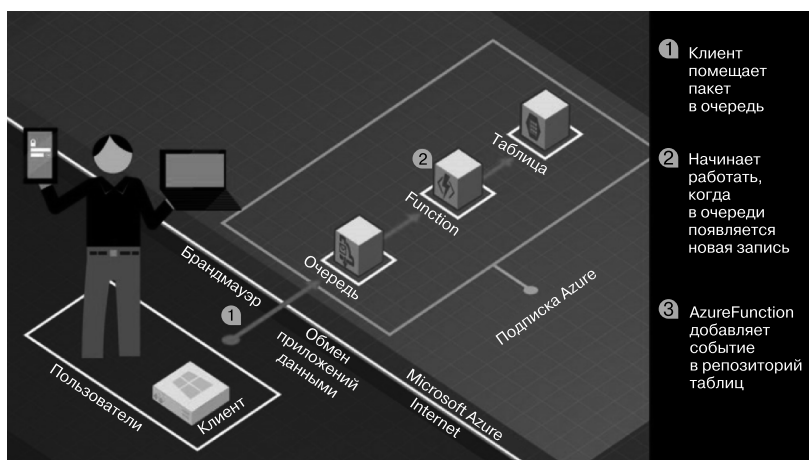


Рис. 12.5

Издатель — подписчик

Название паттерна «Издатель — подписчик» говорит о том, что есть поставщики (издатели) и потребители событий. По сути, это сердце EDA: издатели отделены от потребителей и не обеспокоены доставкой событий потребителям, а только их публикацией. События будут содержать информацию, используемую для их направления заинтересованным потребителям. После этого потребитель регистрируется или подписывается на интересующие его конкретные события (рис. 12.6).

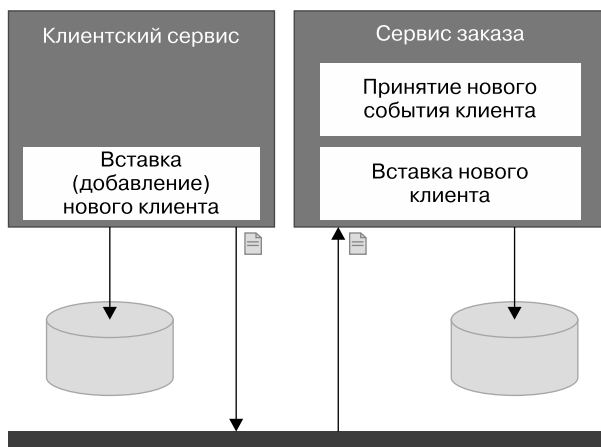


Рис. 12.6

На данной схеме показаны служба поддержки клиентов и заказов. Первый выступает в качестве издателя и отправляет событие при добавлении клиента. Сервис заказов, в свою очередь, подписался на новые клиентские события. При поступлении нового клиентского события он вставляет информацию о клиенте в свое локальное хранилище.

С вводом в архитектуру паттерна «Издатель — подписчик» сервис заказов отделяется от сервиса поддержки клиентов. Такой подход обеспечивает более гибкую в смысле изменений архитектуру. Например, можно внедрить новый сервис для добавления новых клиентов в решения, не требующие добавления в репозиторий, который уже используется сервисом поддержки клиентов. Кроме того, на события для новых клиентов могут подписаться несколько сервисов. Функцию отправки приветственного электронного письма проще привязать к регистрации нового подписчика, чем встраивать ее в единое монолитное решение.

Приоритезированная очередь

Другой связанный паттерн — приоритезированная очередь, которая обеспечивает механизм, позволяющий по-разному рассматривать похожие события. На примере нового клиента из предыдущего подпункта можно было бы иметь двух подписчиков на событие от нового клиента. Один подписчик будет заинтересован в большинстве новых клиентов, в то время как другой определяет подмножество клиентов, которые должны быть обработаны по-другому. Например, новые абоненты из сельских районов могут получать по электронной почте дополнительную информацию о специализированных поставщиках услуг доставки.

Компенсационная транзакция

В распределенных системах не всегда практично или желательно выдавать команду в виде транзакции. Транзакция в этом контексте относится к программной конструкции более низкого уровня, управляющей одной или несколькими командами как единым действием, которое полностью завершается либо успешно, либо нет. В некоторых ситуациях распределенная транзакция не поддерживается или накладные расходы, связанные с ее использованием, перевешивают преимущества. Чтобы справиться с этой ситуацией, был разработан паттерн «Компенсационная транзакция». Рассмотрим следующий пример на основе оркестрации BizTalk (рис. 12.7).

Здесь представлены два этапа процесса: создание заказа в сервисе заказов и списание средств из сервиса поддержки клиентов. Показано, как сначала создается заказ, а затем удаляются средства. Если списание денежных средств не удастся, то заказ удаляется из сервиса заказов.

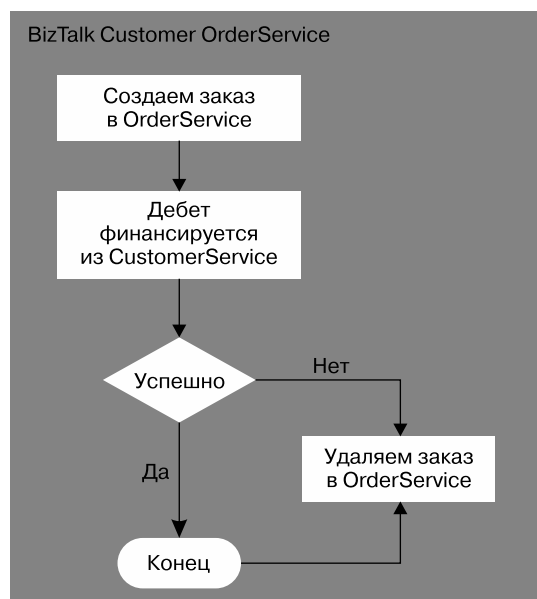


Рис. 12.7

Безопасность

Безопасность гарантирует, что приложение не раскроет информацию по ошибке и не позволит использовать себя не по назначению. Это распространяется как на вредоносные, так и на ошибочные действия. Предоставление доступа к облачным приложениям ограниченному кругу проверенных пользователей нередко вызывает затруднения, особенно учитывая растущую популярность широкого спектра провайдеров идентификации.

Аутентификация и авторизация конечных пользователей требуют разработки и планирования, поскольку все меньше приложений работают изолированно, и обычно задействованы несколько провайдеров идентификации, таких как Facebook, Google и Microsoft. В некоторых случаях паттерны применяются в целях обеспечения прямого доступа к ресурсам для повышения производительности и масштабируемости. Кроме того, другие паттерны связаны с созданием виртуальной стены между клиентами и приложениями.

Паттерны решений

По мере того как отрасль становится все более взаимосвязанной, растет распространение схемы применения внешней стороны для аутентификации пользователей. Мы выбрали для обсуждения паттерн «Федеративная безопасность», поскольку это один из лучших способов обеспечить безопасность в наших системах. Большинство

платформ типа «*приложение как сервис*» (software-as-a-service, SaaS) предлагают данную функцию.

Федеративная безопасность

Федеративная безопасность (federated security) делегирует проверку подлинности пользователя или сервиса (потребителя) внешней стороне, известной как провайдер идентификации (identity provider, IdP). Приложение, задействующее федеративную безопасность, будет доверять IdP правильную аутентификацию потребителя и выдачу точной информации о потребителе или приложениях. Эта информация представлена в виде токена. Распространенный пример в данном случае — веб-приложение с идентификацией через сети Google, Facebook и Microsoft.

Федеративная безопасность может работать в различных сценариях: от интерактивных сессий до внутренних сервисов проверки подлинности или неинтерактивных сессий. Еще один распространенный сценарий — возможность обеспечить единую проверку подлинности, или *единый вход* (single sign-on, SSO), в наборе отдельно размещенных приложений. Этот сценарий позволяет получить один токен из *сервиса токенов безопасности* (Security Token Service, STS) и тот же токен, используемый для представления нескольким приложениям, не требуя повторения процедуры входа в систему (рис. 12.8).

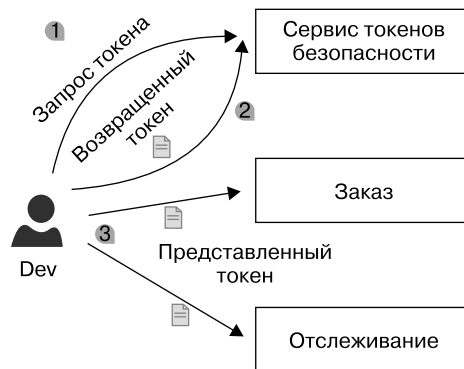


Рис. 12.8

У федеративной безопасности две основные цели. Во-первых, она упрощает управление идентификационными данными за счет использования единого хранилища удостоверений. Это позволяет централизованно и унифицированно управлять идентификационными данными, что облегчает выполнение таких задач управления, как обеспечение возможности входа в систему, управление забытыми паролями, а также последовательное аннулирование паролей. Во-вторых, федеративная безопасность улучшает опыт взаимодействия за счет применения похожих приемов работы с пользователями и наличия единой формы аутентификации, что освобождает от необходимости помнить несколько паролей.

Существует несколько стандартов федеративной безопасности, и два широко используемых из них — это *язык разметки утверждений безопасности* (Security Assertion Markup Language, SAML) и *OpenId Connect* (OIDC). SAML старше OIDC и позволяет обмениваться сообщениями с помощью формата XML SAML. OIDC построен на базе протокола OAuth 2.0 и, как правило, использует *веб-токены JSON* (JSON Web Token, JWT) для описания токенов безопасности. Оба формата поддерживают федеративную безопасность, SSO. И многие приложения, такие как Facebook, Google и Microsoft, поддерживают оба стандарта.

Проектирование приложения

Проектирование приложения может зависеть от многих факторов, не только технических. На эти факторы влияют команды, участвующие в создании, управлении и сопровождении приложений. Некоторые паттерны, например, лучше всего работают с небольшими выделенными группами, но не с большим количеством географически рассредоточенных групп. Другие паттерны, связанные с проектированием, лучше справляются с различными типами рабочей нагрузки и используются в определенных сценариях. Кроме того, есть паттерны, разработанные для решения проблемы частоты изменений и того, как ограничить негативное влияние изменений на приложение после релиза.

Паттерны решений

Поскольку почти все локальные паттерны применимы к облачным решениям, объем паттернов, которые можно описывать, ошеломляет. Паттерны «Кэш» и CQRS были выбраны потому, что первый очень распространен, используется в большинстве веб-приложений, а второй изменяет способ мышления о решениях и хорошо подходит для других архитектурных паттернов, таких как SOA и микросервисы.

Кэш

Хранение информации, полученной из более медленных форм хранения в более быстрые, или кэширование, было технологией, которая использовалась в программировании в течение десятилетий и заметна в таком ПО, как кэш браузера, и аппаратном обеспечении (оперативной памяти). В этой главе мы рассмотрим три примера: «Кэш на стороне», «Кэш со сквозной записью» и «Хостинг статического контента».

Кэш на стороне

Данный паттерн может использоваться для повышения производительности путем загрузки часто применяемых данных в локальную или более быструю форму хранения. При такой схеме ответственность за сопровождение состояния кэша лежит на приложении. Это показано на рис. 12.9.

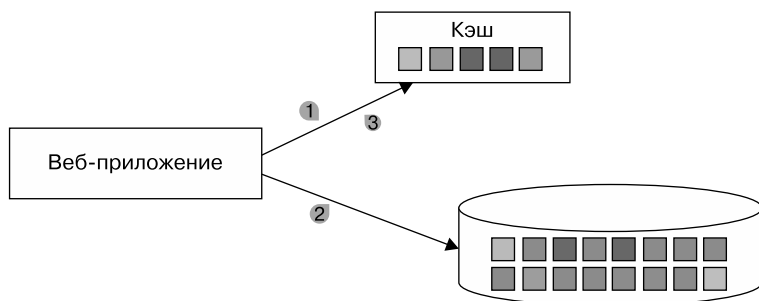


Рис. 12.9

Сначала приложение запрашивает информацию из кэша. Если она отсутствует, то запрашивается из хранилища данных. Затем приложение обновляет кэш, используя эту информацию. После этого информация будет извлекаться из кэша и использоваться без ссылки на более медленное хранилище данных. Этот паттерн подразумевает, что приложение несет ответственность за сопровождение кэша как при его промахах, так и при обновлении данных.



Термин «промах кэша» (cache miss) относится к случаям, когда данные не найдены в кэше. Другими словами, они в нем отсутствуют.

Кэш со сквозной записью

Как и «Кэш на стороне», паттерн «Кэш со сквозной записью» тоже может использоваться для повышения производительности. Его подход отличается тем, что управление содержимым кэша перемещается из приложения в сам кэш, как показано на рис. 12.10.

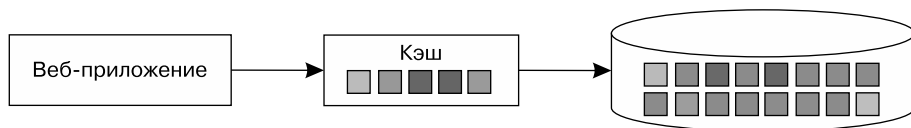


Рис. 12.10

Запрос делается для получения конкретной информации в кэше. Если данные еще не загружены, то информация извлекается из хранилища данных, помещается в кэш, а затем возвращается. Если данные уже были сохранены, то возвращаются немедленно. Этот паттерн поддерживает обновление кэша путем передачи информации на запись через сервис кэша. Затем тот обновляет информацию, хранящуюся как в кэше, так и в хранилище данных.

Размещение статического контента

Данный паттерн перемещает статический контент, такой как мультимедийные изображения, фильмы и другие нединамические файлы, в систему, предназначенную для их быстрого извлечения. Специализированная служба для этого называется *сетью доставки контента* (content delivery network, CDN), управляет распределением контента между несколькими центрами обработки данных и направляет запросы в центр, ближайший к вызывающему абоненту, как показано на рис. 12.11.

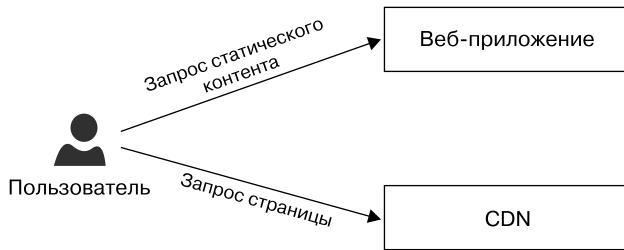


Рис. 12.11

Размещение статического контента — это общий паттерн веб-приложений, где динамическая страница запрашивается из веб-приложения и содержит коллекцию статического контента, такого как JavaScript и изображения, которые браузер затем извлекает непосредственно из CDN. Это эффективный способ уменьшить трафик самого веб-приложения.

Разделение ответственности команд и запросов (CQRS)

Данный паттерн прекрасно подходит для более детального обсуждения, поскольку концептуально прост и относительно легок в реализации, но влечет значительные последствия как для приложения, так и для участвующих разработчиков. Этот паттерн четко отделяет команды, влияющие на состояние приложения, от запросов, которые только извлекают данные. Проще говоря, такие команды, как обновление, добавление и удаление, предоставляются в сервисах, отличных от сервиса запросов, не изменяющих никаких данных.

Вы можете воскликнуть: «*Опять CQRS!*» — и мы признаем, что использовали пример CQRS в ООП и проектировании баз данных. Тот же принцип применим ко многим областям разработки программного обеспечения. CQRS в данной рубрике представляется в качестве паттерна проектирования сервисов, поскольку это приводит к некоторым интересным преимуществам и хорошо вписывается в современные паттерны, такие как микросервисы и проектирование реактивных приложений.



CQRS основан на объектно-ориентированном проектировании, представленном в конце 1980 годов книгой Бертрана Мейера Object-Oriented Software Construction: se.ethz.ch/~meyer/publications/.

Если мы вернемся к главе 5, то покажем этот паттерн, разделив наш контекст инвентаризации на два интерфейса: `IInventoryReadContext` и `IInventoryWriteContext`. Напомним, что здесь представлены интерфейсы:

```
public interface IInventoryContext : IInventoryReadContext,
    IInventoryWriteContext { }

public interface IInventoryReadContext
{
    Book[] GetBooks();
}

public interface IInventoryWriteContext
{
    bool AddBook(string name);
    bool UpdateQuantity(string name, int quantity);
}
```

Как видим, метод `GetBooks` отделен от двух методов, `AddBook` и `UpdateQuantity`, которые изменяют состояние товара. Это иллюстрирует CQRS в коде.

Такой же подход можно применить и на уровне сервиса. Если мы используем сервис для инвентаризации в качестве примера, то разобьем его на сервис обновления информации о товарах и сервис ее извлечения. Это показано на рис. 12.12.

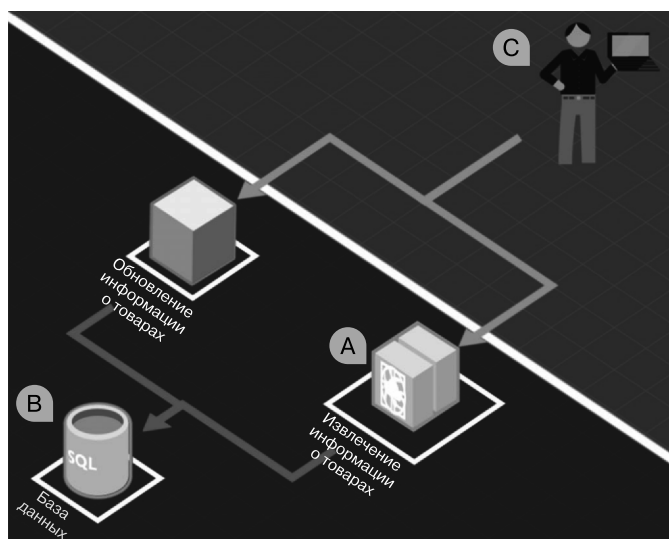


Рис. 12.12

Сначала исследуем CQRS, рассмотрев проблемы, связанные с его применением в облачных решениях.

Проблемы CQRS

Использование паттерна CQRS с сервисами сопряжено со значительными трудностями, такими как:

- ❑ согласованность;
- ❑ внедрение.

Свежесть — это мера того, насколько данные отражают ту их версию, которая была передана на запись¹. Данные в большинстве систем могут потенциально изменяться, что приводит к несоответствию между данными при чтении и данными в источнике данных. Это проблема всех распределенных систем, где нецелесообразно гарантировать, что показанное пользователю значение отражает исходное. Когда данные непосредственно отражают то, что хранится, мы можем назвать их согласованными; когда же этого не происходит, данные рассматриваются как несогласованные.



Общий термин, используемый в распределенных системах, — согласованность в конечном счете. Применяется с целью сказать, что приложение будет согласовано с течением времени. Другими словами, в конечном счете оно станет согласованным.

Другая, более тонкая проблема — внедрение. Реализация перехода на CQRS в уже прочно сработавшейся команде разработчиков может вызвать сопротивление со стороны как разработчиков, так и проектировщиков, которые не знакомы с этим паттерном. Более того, инициатива может не найти поддержки со стороны бизнеса из-за отклонения от текущих паттернов проектирования.

Итак, каковы преимущества?

Зачем нужен CQRS?

Ниже приведены три убедительные причины для использования CQRS:

- ❑ *совместная работа*;
- ❑ *разделение моделей*;
- ❑ *независимая масштабируемость*.

¹ В англоязычной литературе используется противоположный по значению термин *staleness*, который можно перевести как «черствость».

Отдельные сервисы позволяют сопровождать, развертывать и масштабировать их независимо друг от друга. Это повышает качество совместной работы команд разработчиков.

Имея отдельные сервисы, мы можем использовать паттерн, наилучшим образом соответствующий сервису. Командный сервис может применять простые инструкции SQL непосредственно рядом с базой данных, поскольку это наиболее знакомая технология для ответственной команды. В то же время команда, которая создает сервис запросов, может задействовать фреймворк для обработки сложных инструкций.

Большинство решений, как правило, имеют более высокий уровень чтения данных, чем их записи (или наоборот), поэтому разделение сервисов по данному критерию имеет смысл во многих сценариях.

DevOps

При использовании облачных решений центр обработки данных размещается удаленно и вы часто не имеете полного контроля или доступа ко всем аспектам приложения. В некоторых случаях, таких как бессерверные сервисы, инфраструктура абстрагируется. Какое-то приложение по-прежнему должно предоставлять информацию о запущенном приложении, которая может применяться для управления приложением и мониторинга. Паттерны, применяемые с этой целью, имеют важное значение для успеха приложения, обеспечивая возможность поддерживать его работоспособность и предоставлять стратегическую информацию для бизнеса.

Паттерны решений

С появлением коммерческих пакетов, связанных с решениями для мониторинга и управления, многие предприятия получили более полный контроль и понимание своих распределенных систем. Мы выбрали для более детального обсуждения телеметрию и непрерывную доставку/интеграцию, поскольку они имеют особое значение в облачных решениях.

Телеметрия

По мере развития индустрии программного обеспечения, роста количества сервисов и приложений в распределенных системах возможность иметь целостное представление о системе стало огромным преимуществом. Популяризированные такими сервисами, как New Relic и Microsoft Application Insights, *системы управления производительностью приложений* (application performance management, АРМ) используют записанную информацию о приложениях и инфраструктуре (телеметрия) для мониторинга, управления производительностью и обзора доступности системы. В облачных решениях, где часто практически невозможно получить прямой доступ к инфраструктуре системы, АРМ позволяет отправить

телеметрию в центральный сервис, переработать ее, а затем предоставить команде эксплуатации и бизнесу (рис. 12.13).

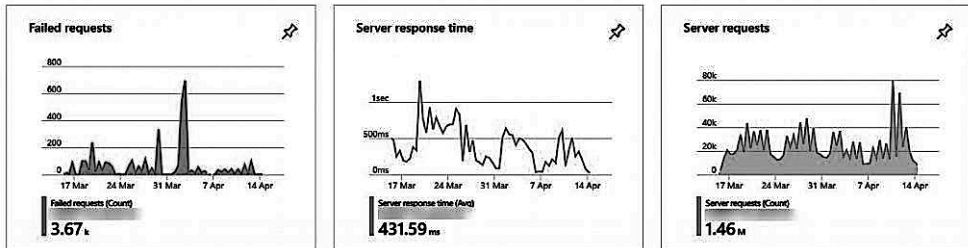


Рис. 12.13

Диаграммы взяты из Microsoft Application Insights и отражают работу запущенного веб-приложения. Администраторы могут легко заметить изменения в поведении системы и принять соответствующие меры.

Непрерывная интеграция/непрерывное развертывание

Непрерывная интеграция/непрерывное развертывание (continuous integration/continuous deployment, CI/CD) представляет собой современный процесс в разработке, предназначенный для *оптимизации жизненного цикла программного обеспечения* (software delivery product life cycle, SDLC) за счет частого внедрения и развертывания изменений. CI решает проблемы, возникающие при разработке корпоративного ПО, когда несколько программистов работают в одной и той же кодовой базе или один продукт управляется несколькими ветвями кода.

Взгляните на рис. 12.14.

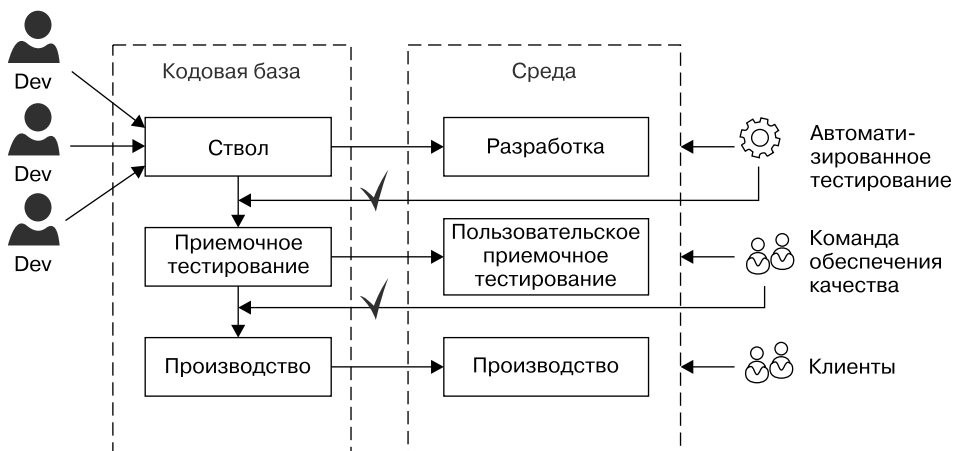


Рис. 12.14

В приведенном примере существует три целевые среды: разработка, *пользовательское приемочное тестирование* (User Acceptance Testing, UAT) и производство. Среда разработки — это начальная среда, в которой все изменения, внесенные в приложение, тестируются вместе. Среда UAT используется *командой обеспечения качества* (Quality Assurance, QA) для проверки того, что система работает должным образом, прежде чем изменения будут перенесены в среду, ориентированную на клиента (на рисунке называемую производством). Кодовая база была разбита на три соответствующие ветви: ствол (trunk), в который объединяются все изменения от команды разработчиков; UAT, используемую для развертывания в среде UAT; и, наконец, производственную кодовую базу (Production), применяемую для развертывания в производственной среде.

Паттерн CI применяется путем создания новой сборки при изменении кодовой базы. После успешной сборки выполняется набор модульных тестов для проверки того, что существующая функциональность не была нарушена. Если сборка не увенчалась успехом, то команда разработчиков исследует и исправляет либо кодовую базу, либо модульный тест, чтобы сборка прошла.

Затем успешные сборки переносятся в целевую среду. Ствол может быть настроен на автоматическую отдачу новой сборки в среду интеграции один раз в день, в то время как команда контроля качества запросила уменьшение помех в своей среде, поэтому новая сборка для них отдается только один раз в неделю после рабочего дня. Производство может потребовать ручной запуск для координации новых релизов, чтобы анонсировать новые возможности и исправление ошибок в официальном релизе.



Существует путаница с терминами «непрерывное развертывание» и «непрерывная поставка». Во многих источниках эти термины различаются в зависимости от того, является развертывание автоматизированным или ручным процессом. Другими словами, непрерывное развертывание требует автоматической непрерывной доставки.

Триггер, приводящий к слиянию сред и, следовательно, к переносу сборки в среду или ее выпуску, может отличаться. На рис. 12.14, изображающем среду разработки, видно, что у нас есть набор автоматизированных тестов, которые автоматически запускаются вместе с новыми сборками. Если тесты успешны, то слияние выполняется автоматически из ствола в кодовую базу UAT. Слияние между UAT и производственной кодовой базой выполняется только после того, как команда контроля качества утвердила или приняла изменения в среде UAT.

Каждое предприятие будет адаптировать CI/CD под свой жизненный цикл и бизнес-требования. Например, публичный сайт может потребовать быстрого SDLC, чтобы оставаться конкурентоспособным на рынке, в то время как внутреннему приложению будет необходим более консервативный подход для ограничения сбоев, вызванных изменением функциональности без подготовки персонала.

Так или иначе, были разработаны наборы инструментов для управления процессом CI/CD в рамках организации. Например, Azure DevOps помогает управлять этим процессом, позволяя создавать конвейер для обработки при создании сборок и при их выпуске в среду, включая как ручные, так и автоматические триггеры.

Резюме

Облачная разработка требует тщательного планирования, сопровождения и мониторинга, а паттерны могут помочь в создании высокомасштабируемых, надежных и безопасных решений. Многие паттерны, описанные в этой главе, применимы к локальным приложениям и имеют большое значение в облачных решениях. Разработка облачного приложения должна учитывать множество факторов, включая масштабируемость, доступность, сопровождение, мониторинг и безопасность.

Масштабируемое приложение позволяет учитывать колебания нагрузки на систему при сохранении приемлемого уровня производительности. Нагрузку можно измерить в количестве пользователей, параллельных процессах, объеме данных и других факторах. Возможность масштабировать решение горизонтально требует определенного типа разработки приложений и является парадигмой, которая особенно важна для облачных вычислений. Такие паттерны, как выравнивание нагрузки на основе очередей, — отличный метод обеспечения того, чтобы решения оставались отзывчивыми при повышенной нагрузке.

Многие из паттернов, рассмотренных в этой главе, дополняют друг друга. Например, приложение, в котором применяется CQRS, может использовать федеративную безопасность для обеспечения единого входа в систему, а управляемую событиями архитектуру — для обеспечения согласованности между компонентами приложения.

В облачных решениях существует почти бесконечный набор применимых паттернов, которые решают различные проблемы в распределенных системах. Паттерны, представленные в этой главе, выбраны по их широте, а также по тому, как они дополняют друг друга. Пожалуйста, ознакомьтесь со ссылками, чтобы изучить другие паттерны, подходящие для облачных решений.

Вот это путешествие! Мы рассмотрели паттерны, начиная с паттернов проектирования ПО, используемых в объектно-ориентированном программировании, и архитектурных паттернов в облачных решениях и заканчивая бизнес-паттернами для более эффективных команд, а также паттернами для создания успешных приложений. Мы попытались охватить широкий спектр паттернов, однако обязательно найдутся те, которые могли бы и должны были быть добавлены.

Наконец, спасибо вам от Гаурава и Джеффри. Мы надеемся, вы получили удовольствие и пользу от чтения этой книги. Пожалуйста, дайте нам знать, что вы думаете, и поделитесь с нами своими любимыми паттернами.

Вопросы

Следующие вопросы позволят вам усвоить информацию, представленную в этой главе.

1. Правда ли, что большинство моделей были разработаны в последнее время и применимы только к облачным приложениям?
2. Что означает ESB и в каком типе архитектуры она может быть использована: EDA, SOA или монолитной?
3. Выравнивание нагрузки на основе очередей применяется в основном для DevOps, масштабируемости или доступности?
4. Каковы преимущества CI/CD? Что выгоднее: большое количество глобально рассредоточенных команд или одна небольшая объединенная команда разработчиков?
5. На сайте после размещения статического контента браузер получает изображения и статический контент непосредственно через CDN или веб-приложение получает информацию от имени браузера?

Дальнейшее чтение

Чтобы узнать больше о темах из этой главы, обратитесь к книгам, указанным ниже. Они содержат углубленные примеры по данным темам:

- ❑ *Sreeram P. K.* Azure Serverless Computing Cookbook. — Packt. www.packtpub.com/in/virtualization-and-cloud/azure-serverless-computing-cookbook;
- ❑ *Tanasseri N., Rai R.* Microservices with Azure. — Packt. www.packtpub.com/in/virtualization-and-cloud/microservices-azure;
- ❑ *Mrzygłód K.* Hands-On Azure for Developers. — Packt. www.packtpub.com/virtualization-and-cloud/hands-azure-developers;
- ❑ *Arora G.* Building Microservices with .NET Core 2.0. Second Edition. — Packt. www.packtpub.com/application-development/building-microservices-net-core-20-second-edition.

Приложения



Дополнительные практические рекомендации

До сих пор в книге мы рассматривали различные паттерны, стили и код. Цель обсуждения состояла в том, чтобы понять паттерны и методы написания аккуратного, чистого и надежного кода. В этом приложении основное внимание будет уделено практике. Она очень важна, когда речь заходит о соблюдении любого правила или любого стиля кодирования. Как разработчик, вы должны отрабатывать кодирование каждый день. Старая пословица гласит: *практика — путь к совершенству*.

Доказано, что такие навыки, как умение играть в игры, вождение автомобиля, чтение или письмо, не приходят мгновенно. Мы должны совершенствовать их с помощью практики. Например, когда вы начинаете водить машину, заводите ее медленно. Нужно помнить, когда нажимать на сцепление, когда — на тормоз, как далеко поворачивать руль и т. д. Однако как только водитель осваивает вождение, ему уже нет нужды напоминать эти шаги. Все происходит естественно. Это возможно благодаря практике.

В данном приложении будут рассмотрены следующие темы:

- ☐ обсуждение примера использования;
- ☐ практические рекомендации;
- ☐ другие паттерны проектирования.

Технические требования

В этом приложении содержатся примеры кода, объясняющие принципы качественной разработки. Код упрощен и предназначен лишь для демонстрации. Большинство примеров основаны на консольном приложении .NET Core, написанном на C#.

Чтобы запустить и выполнить код, вам потребуется следующее:

- ☐ Visual Studio 2019 (однако вы также можете запустить приложение, используя Visual Studio 2017).

Установка Visual Studio. Чтобы запустить примеры кода, вам необходимо установить Visual Studio. Для этого выполните следующие шаги.

1. Скачайте Visual Studio по ссылке docs.microsoft.com/ru-ru/visualstudio/install/install-visual-studio.
2. Следуйте инструкциям по установке. Доступны различные версии Visual Studio; в этой главе мы используем Visual Studio для Windows.

Вы также можете применить другую интегрированную среду разработки по вашему усмотрению.



Примеры файлов кода для этой главы доступны по ссылке <https://github.com/PacktPublishing/Hands-On-Design-Patterns-with-C-and-.NET-Core/tree/master/Appendix>.

Обсуждение примера использования

Проще говоря, пример использования — это предварительное создание или символическое представление бизнес-сценария. Например, мы можем представить наш вариант применения страницы входа в систему в графическом/символическом представлении. В нашем примере пользователи пытаются войти в систему. Если логин успешен, они смогут сделать это. Если нет, то система проинформирует о неудаче. На рис. А.1 показан пример использования *логина*.

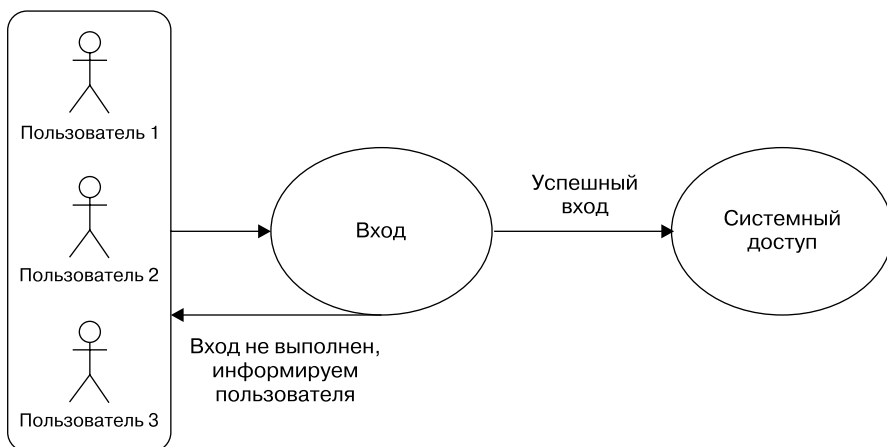


Рис. А.1

В данной схеме пользователи 1, 2 и 3 пытаются войти в систему с помощью функциональности приложения для логина. Если попытка входа в систему успешна,

то пользователь сможет получить доступ к ней. В противном случае приложение уведомляет пользователя о том, что вход в систему не был успешным, и пользователь не может получить доступ к ней. Эта схема гораздо понятнее, чем наше ее подробное описание, и не требует пояснений.

UML-диаграмма

В предыдущем подразделе мы обсуждали функциональность входа в систему с помощью символического представления. Обозначения или символы на схеме — часть символического языка, называемого *унифицированным языком моделирования* (Unified Modeling Language, UML). Это способ визуализации программы, ПО или даже класса.



Символ или обозначение, используемые в UML, начинались с работ Грейди Буча, Джеймса Рамбо, Ивара Якобсона и корпорации Rational Software Corporation.

Типы диаграмм UML

Эти диаграммы делятся на две основные группы.

- ❑ *Структурные UML-диаграммы* — подчеркивают то, что должно присутствовать в моделируемой системе. Эта группа далее делится на такие типы, как:
 - диаграмма классов;
 - диаграмма пакета;
 - диаграмма объекта;
 - диаграмма компонента;
 - диаграмма структуры композиции;
 - диаграмма развертывания.
- ❑ *Поведенческие UML-диаграммы* — используются для отображения функциональности системы и включают диаграммы примеров использования, последовательности, совместной работы, машины состояний и действия. Эта группа делится на такие типы, как:
 - диаграмма действий;
 - диаграмма последовательности;
 - диаграмма примеров использования;
 - диаграмма состояний;
 - диаграмма коммуникаций;
 - диаграмма обзора взаимодействия;
 - временная диаграмма.

Практические рекомендации

Как мы уже установили, практика — это привычка, возникающая в нашей повседневной деятельности. В программной инженерии, где ПО создается инженерами, а не поставлено на поток, мы должны практиковаться, чтобы написать код хорошего качества. Приведу практические рекомендации, связанные с разработкой программного обеспечения.

- ❑ *Короткий, упрощенный код* — очень простая вещь, которая действительно требует практики. Разработчики должны ежедневно использовать короткий, упрощенный код. Он должен быть чистым и без повторов. Чистый код и его упрощение рассматривались в предыдущих главах; если вы пропустили эту тему, пожалуйста, вернитесь к главе 2. Взгляните на пример короткого кода:

```
public class Math
{
    public int Add(int a, int b) => a + b;
    public float Add(float a, float b) => a + b;
    public decimal Add(decimal a, decimal b) => a + b;
}
```

В данном фрагменте кода — класс `Math` с тремя методами `Add`. Они написаны для вычисления суммы двух целых чисел и суммы двух чисел с плавающей запятой, а также десятичных чисел. Методы `Add(float a, float b)` и `Add(decimal a, decimal b)` — это перегруженные методы `Add(int a, int b)`. Данный пример кода представляет сценарий, в котором требования заключаются в создании одного метода с выводом типа данных `int`, `float` или `decimal`.

- ❑ *Модульное тестирование* — неотъемлемая часть разработки, когда мы хотим протестировать наш код, написав другой. *Разработка через тестирование (TDD)* — одна из практических рекомендаций, которой следует придерживаться. Мы обсуждали TDD в главе 7.
- ❑ *Согласованность кода* — в настоящее время разработчик очень редко действует в одиночку. Он в основном трудится в команде, а это значит, что унифицированность кода очень важна. Согласованность может относиться к стилю кода. Существует несколько рекомендуемых конвенций кодирования, которые разработчики должны регулярно использовать при написании программ.

Например, есть множество способов объявить переменную. Вот один из лучших примеров объявления переменных:

```
namespace Implement
{
    public class Consume
    {
        BestPractices.Math math = new BestPractices.Math();
    }
}
```

В этом коде мы объявили переменную `math` типа `BestPractices.Math`. Здесь `BestPractices` — пространство имен, `Math` — это класс. Если мы не применяем директиву `using` в коде, то это хороший способ иметь полностью квалифицированные переменные пространства имен.



Официальная документация языка C# очень подробно описывает эти соглашения. Вы можете обратиться к ней, перейдя по следующей ссылке: docs.microsoft.com/ru-ru/dotnet/csharp/programming-guide/inside-a-program/coding-conventions.

- ❑ *Обзор кода* — делать ошибки свойственно человеческой природе. Это происходит и в разработке. Обзор (ревью) кода — первый шаг в практике написания кода без ошибок и выявления непредсказуемых ошибок.

Другие паттерны проектирования

До сих пор мы рассматривали различные паттерны и принципы проектирования, включая практические рекомендации по написанию кода. В этом разделе мы опишем следующую партию паттернов и приведем рекомендации по написанию качественного и надежного кода. Детали и реализация этих паттернов выходят за рамки данной книги.

Мы уже рассмотрели следующие паттерны:

- ❑ паттерны «Банды четырех»;
- ❑ принципы проектирования;
- ❑ жизненный цикл разработки ПО;
- ❑ разработку через тестирование.

В данной книге мы рассмотрели множество тем и разработали пример приложения (консольное и веб-приложение). Это еще не конец. В мире есть еще много вещей, которым нужно научиться.

- ❑ *Паттерн «Архитектура, базирующаяся на пространстве» (Space-based Pattern, SBP)* — один из паттернов, которые помогают с масштабируемостью приложения, минимизируя факторы, ограничивающие масштабирование приложения. Эти паттерны также известны как *паттерны облачной архитектуры*. Мы рассмотрели многие из них в главе 12.
- ❑ *Паттерны обмена сообщениями* используются для соединения двух приложений сообщениями (отправляемыми в виде пакетов). Эти пакеты или сообщения передаются по логическому пути, по которому соединяются различные приложения (такие логические пути известны как каналы). Могут возникать сценарии, когда

одно приложение имеет несколько сообщений; в таком случае не все могут быть отправлены сразу. В сценарии с несколькими сообщениями канал можно назвать очередью, а несколько сообщений могут быть поставлены в очередь в канале и быть доступны из различных приложений в одно и то же время.

❑ *Дополнительный паттерн предметно-ориентированного проектирования (Domain Driven Design) — многоуровневая архитектура* — отображает разделение проблем, в которых проявляется концепция многоуровневой архитектуры. Основная идея разработки приложения в том, что оно должно структурироваться в концептуальные слои. В целом приложения имеют четыре концептуальных уровня:

- *пользовательский интерфейс* — на этом слое есть все, с чем взаимодействует конечный пользователь; слой принимает команды, а затем предоставляет соответствующую информацию;
- *слой приложения* — этот слой ближе к управлению транзакциями, переводу данных и т. д.;
- *слой домена* — этот слой касается поведения и состояния домена;
- *слой инфраструктуры* — здесь происходит все, что связано с репозиториями, адаптерами и фреймворками.

❑ *Паттерны контейнеризованных приложений* — прежде чем углубляться в это, мы должны узнать, что такое контейнеры. Контейнер — легкое, портативное программное обеспечение. Оно определяет окружение, с которым может работать ПО. Как правило, программное обеспечение, работающее внутри контейнера, разрабатывается как приложение с единственной задачей. В контейнерах наиболее важен следующий паттерн:

- *паттерн построения образов Docker* — основан на «Строителе», паттерне «Банды четырех». Мы обсуждали его в главе 3. Он описывает установку так, чтобы ее можно было использовать для построения контейнера. Кроме того, существует многоступенчатый паттерн построения образов, который обеспечивает построение нескольких образов из одного файла Docker.

Резюме

Цель этого приложения — подчеркнуть важность практики. Мы обсуждали, как можно практиковаться и совершенствовать навыки. Освоив навыки, нет необходимости запоминать шаги, чтобы выполнить конкретную задачу. Мы рассмотрели несколько примеров использования из реального мира, обсудили практические рекомендации для кода и другие паттерны проектирования, применимые в нашей повседневной практике для повышения квалификации. Наконец, мы узнали, что с помощью практики и внедрения различных паттернов разработчики могут улучшить качество своего кода.

Вопросы

Следующие вопросы позволят вам усвоить информацию, представленную в этом приложении.

1. Что такое практика? Приведите несколько примеров из нашей повседневной жизни.
2. С помощью практики мы можем освоить конкретные навыки написания кода. Объясните это.
3. Что такое разработка через тестирование и как она помогает разработчикам практиковаться?

Дальнейшее чтение

Мы почти дошли до конца книги! В данном приложении мы рассмотрели много вещей, связанных с практиками. Это не конец обучения, а только начало. Есть еще несколько книг, из которых вы можете почерпнуть знания:

- ❑ *Zimarev A.* Hands-On Domain-Driven Design with .NET Core. — Packt. www.packtpub.com/in/application-development/hands-domain-driven-design-net-core;
- ❑ *Adewole A.* C# and .NET Core Test-Driven Development. — Packt. www.packtpub.com/in/application-development/c-and-net-core-test-driven-development;
- ❑ *Raj P., Subramanian H. et al.* Architectural Patterns. — Packt. www.packtpub.com/in/application-development/architectural-patterns;
- ❑ *Khot A. S.* Concurrent Patterns and Best Practices. — Packt. www.packtpub.com/in/application-development/concurrent-patterns-and-best-practices.



Ответы на вопросы

Глава 1. Обзор ООП в .NET Core и C#

1. Что означают термины «позднее» и «раннее» связывание?

Раннее связывание происходит при компиляции исходного кода, а позднее — при запуске компонента.

2. Поддерживает ли C# множественное наследование?

Нет. Причина — множественное наследование приводит к усложнению исходного кода.

3. Применительно к C# какой уровень инкапсуляции можно использовать для предотвращения доступа к классу вне библиотеки?

Модификатор доступа `internal` может использоваться для ограничения видимости класса только границами библиотеки.

4. Чем различаются агрегация и композиция?

Оба понятия — разновидности ассоциаций. Самый простой способ различить их — это определить, могут ли соответствующие классы существовать, не будучи связанными друг с другом. В композиционной ассоциации соответствующие классы сильно зависят от жизненного цикла. Это значит, что при удалении одного класса удаляются и связанные с ним классы.

5. Могут ли интерфейсы содержать свойства? (Вопрос с подвохом.)

Интерфейс может определять свойства, но так как в нем нет реализаций...

6. Едят ли собаки рыбу?

Собаки прекрасны, но едят практически все, что попадает им в рот.

Глава 2. Современные паттерны и принципы проектирования ПО

1. Что означает буква S в аббревиатуре SOLID? Что подразумевается под ответственностью?

Принцип единой ответственности. Ответственность можно рассматривать как причину изменений.

2. *Какой метод основан на циклах — Waterfall или Agile?*

Agile построен на концепции процесса разработки в виде набора циклов.

3. *Паттерн «Декоратор» порождает ли структурный?*

«Декоратор» — структурный паттерн, позволяющий разделить функциональность между классами. Он особенно полезен в расширении классов во время выполнения.

4. *За что отвечает паттерн интеграции «Издатель — подписчик»?*

«Издатель — подписчик» — это полезный паттерн, когда одни процессы публикуют сообщения, а другие подписываются на их получение.

Глава 3. Реализация паттернов проектирования — основы (часть 1)

1. *Почему иногда затруднительно определить требования при разработке программного обеспечения для организации?*

Разработка ПО для организации связана с многочисленными трудностями. Примером может служить то, что изменения в отрасли организации могут привести к необходимости изменить текущие требования.

2. *В чем два главных преимущества и недостатка разработки ПО методами Waterfall и Agile?*

Разработка программного обеспечения методом Waterfall проигрывает гибкой разработке, так как последняя проще в понимании и реализации. В некоторых ситуациях, когда сложность и размер проекта меньше, разработка ПО с помощью Waterfall может быть лучшим вариантом по сравнению с Agile. Waterfall, однако, не справляется с изменениями и, ввиду более широкого охвата, имеет больше шансов на изменение требований до завершения проекта.

3. *Как внедрение зависимостей помогает при написании модульных тестов?*

При введении зависимостей класс становится легче тестировать, поскольку зависимости четко известны и легко доступны.

4. *Почему следующее утверждение ложное? Используя TDD, вы не нуждаетесь в людях, тестирующих новое программное обеспечение.*

Разработка через тестирование помогает повысить качество решения путем построения четкой стратегии тестирования в жизненном цикле разработки программного обеспечения. Однако такие тесты могут быть неполными, поэтому все еще существует потребность в дополнительных ресурсах для проверки предоставляемого ПО.

Глава 4. Реализация паттернов проектирования — основы (часть 2)

1. *Приведите пример, показывающий, почему использование «Одиночки» не подходит для ограничения доступа к общему ресурсу.*

«Одиночка» намеренно создает узкое место в приложении. Это также один из первых паттернов, который учатся применять разработчики, и поэтому он часто используется в ситуациях, когда ограничение доступа к общему ресурсу не требуется.

2. *Верно ли следующее утверждение? Почему да или почему нет? `ConcurrentDictionary` предотвращает обновление элементов коллекции несколькими потоками одновременно.*

Для многих разработчиков на C# осознание того, что `ConcurrentDictionary` не препятствует обновлению элементов коллекции несколькими потоками одновременно — болезненный урок. `ConcurrentDictionary` защищает общий словарь от одновременного доступа и изменения.

3. *Что такое состояние гонки и почему его следует избегать?*

Состояние гонки возникает, когда порядок обработки нескольких потоков может привести к различным результатам.

4. *Как паттерн «Фабрика» помогает упростить код?*

«Фабрика» — это эффективный способ разделить создание объектов внутри приложения.

5. *Требуют ли приложения .NET Core контейнеров IoC от сторонних производителей?*

В .NET Core есть мощная инверсия управления, встроенная в фреймворк. Его можно усилить другими контейнерами, но это не требуется.

Глава 5. Реализация паттернов проектирования в .NET Core

1. *Если вы не уверены, какой тип срока жизни сервиса использовать, то с каким типом лучше всего зарегистрировать класс? Почему?*

Сервисы со временным сроком службы создаются каждый раз, когда запрашиваются. Большинство классов должны быть облегченными сервисами без состояния, так что это лучший для использования срок службы.

2. *В решениях .NET Core ASP .NET область действия определяется для каждого веб-запроса или каждой сессии?*

Область применения — каждый веб-запрос (соединение).

3. *Сделает ли класс потокобезопасным его регистрация в качестве «Одиночки» во фреймворке DI в .NET Core?*

Нет, фреймворк предоставляет один и тот же экземпляр для последующих запросов, но не делает класс потокобезопасным.

4. *Правда ли, что фреймворк DI в .NET Core можно заменить только другими фреймворками DI, поставляемыми компанией Microsoft?*

Да, есть много DI-фреймворков, которые можно использовать вместо родного DI-фреймворка.

Глава 6. Реализация паттернов проектирования для веб-приложений (часть 1)

1. *Что такое веб-приложение?*

Это программа, которая использует веб-браузер и может быть доступна из любого места, если доступна в публичной сети. Она работает на архитектуре «клиент — сервер» и обслуживает клиента, принимая HTTP-запрос и предоставляя HTTP-ответ.

2. *Создайте веб-приложение по вашему выбору и покажите, как оно работает.*

Соплнитесь в рассказе на приложение FlixOne.

3. *Что такое инверсия контроля?*

Инверсия контроля (Inversion of Control, IoC) — это контейнер для инверсии или делегирования управления. Он базируется на платформе DI. .NET Core имеет встроенный контейнер IoC.

4. *Какие паттерны проектирования мы рассмотрели в этой главе? Какой из них вам нравится и почему?*

Архитектурный паттерн пользовательского интерфейса предназначен для создания надежного UI, чтобы дать пользователю лучший опыт работы с приложением. С точки зрения разработчика, MVC, MVP и MVVM — популярные паттерны.

Глава 7. Реализация паттернов проектирования для веб-приложений (часть 2)

1. *Что такое аутентификация и авторизация?*

Аутентификация — процесс, в ходе которого система проверяет или идентифицирует входящие запросы с помощью учетных данных (обычно это идентификатор пользователя и пароль). Обнаружив, что предоставленные учетные данные неверны, система уведомляет пользователя (обычно через сообщение на экране GUI) и завершает процесс авторизации.

Авторизация всегда происходит после аутентификации. Это процесс, который позволяет аутентифицированному пользователю, создавшему запрос, получить доступ к ресурсам или данным после проверки того, что у него есть доступ к определенным ресурсам или данным.

2. *Безопасно ли использовать аутентификацию на первом уровне запроса, а затем разрешать входящие запросы для скрытых областей?*

Это не всегда безопасно. Как разработчики, мы должны предпринять все необходимые шаги, чтобы сделать наше приложение более безопасным. После запроса первого уровня проверки подлинности система также должна проверить разрешения на уровне ресурсов.

3. *Как доказать, что авторизация всегда происходит после аутентификации?*

В простом сценарии веб-приложение сначала проверяет пользователя, запрашивая учетные данные для входа, а затем авторизует его в соответствии с ролью для доступа к определенному ресурсу.

4. *Что такое разработка через тестирование и почему разработчики должны подходить к ней со всей тщательностью?*

Разработка через тестирование — способ убедиться, что код работает; это похоже на тестирование кода путем написания кода. TDD также известен как принцип «красный/синий/зеленый». Разработчики должны следовать ему, чтобы их программа работала без каких-либо ошибок.

5. *Что такое TDD Katas? Как он помогает улучшить подход к TDD?*

TDD Katas — это небольшие сценарии или проблемы, помогающие научиться программировать на практике. Можно взять пример Fizz Buzz Kata, где разработчики должны писать код, чтобы на практике научиться TDD. Если вы хотите попрактиковаться с TDD Katas, то обратитесь к этому репозиторию: github.com/garora/TDD-Katas.

Глава 8. Конкурентное программирование в .NET Core

1. *Что такое конкурентное программирование?*

Всякий раз, когда задачи выполняются одновременно, мы говорим, что они выполняются конкурентно. В нашем языке программирования каждый раз, когда какие-либо части нашей программы выполняются одновременно, это называется конкурентным программированием.

2. *Когда возможен настоящий параллелизм?*

Настоящий параллелизм невозможен на машине с одним процессором: задачи не переключаются, поскольку она имеет одно ядро. Параллелизм может быть только на машине с несколькими процессорами (несколькими ядрами).

3. *Что такое состояние гонки?*

Потенциальная возможность получения доступа к одним и тем же общим данным и их обновление с непредсказуемыми результатами можно назвать состоянием гонки.

4. *Почему мы должны использовать параллельный словарь?*

Конкурентный словарь представляет собой безопасный для потоков класс коллекций и хранит пары «ключ — значение». Этот класс использует оператор блокировки, то есть является потокобезопасным.

Глава 9. Функциональное программирование

1. *Что такое функциональное программирование?*

Функциональное программирование — это подход к символьным вычислениям как к математическим задачам. Функциональное программирование основано на математических функциях. Любой язык программирования в функциональном стиле подходит к решению задачи с двух сторон: что решить и как это сделать?

2. *Что такое ссылочная прозрачность в функциональном программировании?*

В функциональных программах, как только мы определяем переменные, они не меняют своего значения на протяжении всей программы. Функциональные программы не имеют операторов присваивания, так что если нам нужно сохранить значение, то мы не можем сделать это. Вместо этого мы определяем новые переменные.

3. *Что такое чистая функция?*

Чистые функции подчеркивают функциональность своим определением. Такие функции соответствуют двум критериям:

- конечный результат/вывод всегда остается одним и тем же для одних и тех же параметров;
- они не влияют на поведение программы или поток выполнения приложения, даже если вызываются сотни раз.

Глава 10. Модели и методы реактивного программирования

1. *Что такое поток?*

Потоком называется последовательность событий. Поток может содержать три вещи: значение, ошибку и сигнал для завершения.

2. *Что такое реактивные свойства?*

Реактивные свойства — это привязанные свойства, реагирующие на событие.

3. Что такое реактивная система?

На основании реактивного манифеста можно сделать вывод, что реактивные системы таковы.

- *Отзывчивы* — в основе реактивных систем лежит событийная модель, что позволяет им быстро реагировать на любые запросы.
- *Масштабируемы* — реактивные системы реактивны по своей природе. Они могут реагировать на изменение, расширяя или сокращая выделенные ресурсы.
- *Эластичны* — эластичная система — такая, которая не остановится, даже если есть какой-либо сбой/исключение. Реактивная система сконструирована таким образом, что продолжает работать, несмотря на любое исключение или сбой.
- *Основаны на сообщениях* — любые данные элемента представляют собой сообщение и могут быть отправлены в определенное место. Когда сообщение или данные достигают заданного состояния, происходит событие-сигнал, уведомляющее о том, что сообщение получено. Реактивные системы полагаются на эту передачу сообщений.

4. Что подразумевается под слиянием двух реактивных потоков?

Слияние двух реактивных потоков фактически означает объединение элементов двух одинаковых или отличных потоков в новый реактивный поток. Например, если у вас есть `stream1`, `stream2`, а также `stream3 = stream1.merge(stream2)`, то последовательность `stream3` была бы не по порядку.

5. Что такое паттерн MVVM?

Данный паттерн — одна из вариаций паттерна «Модель — представление — контроллер» (Model — View — Controller, MVC), соответствующая современному подходу к разработке пользовательского интерфейса, в котором разработка UI — основная ответственность дизайнера, а не разработчиков приложения. При таком подходе к разработке дизайнер, который в большей степени является графическим энтузиастом и сосредоточен на том, чтобы сделать пользовательский интерфейс более привлекательным, может беспокоиться или не беспокоиться о разработке приложения. Как правило, дизайнеры применяют различные инструменты, чтобы сделать UI более привлекательным.

MVVM определяется следующим образом:

- *модель* — также называется доменным объектом и содержит только данные. В ней нет никакой бизнес-логики, валидаций и т. д.;
- *представление* — представление данных для конечного пользователя;
- *модель представления* — разделяет представление и модель. Основная ответственность состоит в том, чтобы лучше обслуживать конечных пользователей.

Глава 11. Усовершенствованные методы проектирования и применения баз данных

1. *Что такое база данных в стиле бухгалтерской книги?*

Эта база данных предназначена только для вставки операций. Нет никаких обновлений. Затем вы создаете представление, объединяющее вставки.

2. *Что такое CQRS?*

Разделение ответственности команд и запросов — это паттерн, разделяющий ответственность между запросом (для вставок) и командами (для обновлений).

3. *Когда мы должны использовать CQRS?*

CQRS может быть паттерном, подходящим для применения в системах, основанных на задачах или событиях, особенно когда решение состоит из нескольких приложений, а не монолитного сайта или приложения. Это *паттерн*, а не *архитектура*, поэтому должен применяться в конкретных случаях, а не во всех бизнес-сценариях.

Глава 12. Разработка облачных приложений

1. *Правда ли, что большинство моделей были разработаны в последнее время и применимы только к облачным приложениям?*

Нет, это неправда. Паттерны развивались по мере изменений в разработке программного обеспечения, но многие из основных паттернов существовали десятилетиями.

2. *Что такое ESB и в каком типе архитектуры она может быть использована: EDA, SOA или монолитной?*

Аббревиатура расшифровывается как «сервисная шина предприятия». Может эффективно использоваться в событийной и сервисно-ориентированной архитектуре.

3. *Выравнивание нагрузки на основе очереди применяется в основном для DevOps, масштабируемости или доступности?*

Для доступности. В основном используется для обработки больших колебаний нагрузки, действуя в качестве буфера, чтобы уменьшить вероятность недоступности приложения.

4. *Каковы преимущества CI/CD? Что выгоднее: большое количество глобально рассредоточенных команд или одна небольшая объединенная команда разработчиков?*

В целом CI/CD помогает выявлять проблемы на ранних этапах жизненного цикла разработки с помощью частого слияния кода и развертывания. Большие,

более сложные решения, как правило, показывают преимущества CI/CD лучше, чем меньшие, более простые решения.

5. *На сайте после размещения статического контента браузер получает изображения и статический контент непосредственно через CDN или веб-приложение получает информацию от имени браузера?*

Сеть доставки контента может использоваться для повышения производительности и доступности путем кэширования статических ресурсов в нескольких центрах обработки данных, что позволяет браузеру получать контент непосредственно из ближайшего центра.

Приложение А. Практические рекомендации

1. *Что такое практика? Приведите несколько примеров из нашей повседневной жизни.*

Практика может быть одним или несколькими повседневными действиями. Чтобы научиться водить машину, мы должны практиковаться в вождении. Практика — это деятельность, которую не требуется заучивать.

2. *С помощью практики мы можем освоить конкретные навыки написания кода. Объясните это.*

Да, практика позволяет закрепить определенные навыки в кодировании. Она требует внимания и последовательности. Например, вы хотите научиться разработке через тестирование. Чтобы выполнять разработку, вам нужно сначала научиться ей. И для этого вы можете практиковаться с TDD-Katas.

3. *Что такое разработка через тестирование и как она помогает разработчикам практиковаться?*

Разработка через тестирование — способ убедиться, что код работает. Это похоже на тестирование кода путем написания кода. TDD также известна как принцип «красный/синий/зеленый». Разработчики должны следовать ему, чтобы их программа работала без каких-либо ошибок.

Гаурав Арораа, Джеффри Чилберто
Паттерны проектирования для C# и платформы .NET Core
Перевел с английского *С. Черников*

Заведующая редакцией	<i>Ю. Сергиенко</i>
Руководитель проекта	<i>С. Давид</i>
Ведущий редактор	<i>Н. Гринчик</i>
Научный редактор	<i>М. Сагалович</i>
Литературный редактор	<i>Н. Хлебина</i>
Художественный редактор	<i>В. Мостипан</i>
Корректоры	<i>Е. Павлович, Е. Рафалюк-Бузовская</i>
Верстка	<i>Г. Блинов</i>

Изготовлено в России. Изготовитель: ООО «Прогресс книга».
Место нахождения и фактический адрес: 194044, Россия, г. Санкт-Петербург,
Б. Сапсониевский пр., д. 29А, пом. 52. Тел.: +78127037373.

Дата изготовления: 01.2021. Наименование: книжная продукция. Срок годности: не ограничен.

Налоговая льгота — общероссийский классификатор продукции ОК 034-2014, 58.11.12 —
Книги печатные профессиональные, технические и научные.

Импортер в Беларусь: ООО «ПИТЕР М», 220020, РБ, г. Минск, ул. Тимирязева, д. 121/3, к. 214, тел./факс: 208 80 01.

Подписано в печать 15.01.21. Формат 70×100/16. Бумага офсетная. Усл. п. л. 28,380. Тираж 500. Заказ 0000.